目录

1 GPU Vs CPU 18
1.1 ALU 与核心的对比 18
1.2 顺序执行与并行执行 19
1.3 硬件架构 20
1.4 GPU 内部组件 21
1.5 A100 GPU 实例 22
2 NVIDIA 历史 24
2.1 RIVA 128 25
2.2 GeForce 世代 25
3 GPU 架构与代系关系 26
3.1 架构的定义 26
3.2 架构与世代的关系 28
3.3 GPU 世代分类 29
3.4 GeForce 与 Tesla 在超级计算机中的应用 31
4 如何识别 GPU 架构与代际 32
4.1 使用 TechPowerUp 查询规格 32
4.2 A100 与 RTX 3090 架构对比 34
4.3 视觉区分 GeForce 与 Tesla 34
5 GPU 和 GPU 芯片 35
5.1 GPU 芯片的定义 35
5.2 GeForce GTX 480 与 GF100 芯片 35
5.3 A100 与 GA100 芯片 36
6 各架构对应的芯片型号 37
6.1 GA100 芯片与高性能计算 37
6.2 GA102 芯片与消费级市场 39
6.3 芯片的多用途适配 40
7 NVIDIA GPU 架构历史 40

7.1 Fermi 架构 41
7.2 Kepler 架构 42
7.3 Maxwell 架构 43
7.4 Pascal 架构 44
8 不同架构间的核心对比参数 44
8.1 内存带宽 45
8.2 总线宽度 46
8.3 指令吞吐量 49
8.4 能耗权衡 51
8.5 新特性 52
9 半精度、单精度与双精度运算 54
9.1 整数与浮点类型 55
9.2 半精度、单精度与双精度的差异 55
9.3 PI 的精度对比 56
10 GPU 计算能力 57
10.1 计算能力编号 57
10.2 功能支持 58
10.3 软件兼容性 59
11 如何阅读 GPU 技术白皮书 62
11.1 搜索白皮书 62
11.2 白皮书的结构 64
11.3 SM 内部结构 65
12 Volta 架构白皮书阅读 65
12.1 Volta 关键特性 66
12.2 SM 增强 68
12.3 深入硬件架构 70
12.4 架构对比与 NVLink 72
13 CUDA 工具包概述 75
13.1 nvcc 编译器 75

13.2 优化库 76
13.3 运行时 API 76
13.4 调试与分析工具 77
13.5 示例代码 77
14 在 Linux 系统上安装 CUDA 工具包 78
14.1 检查 nvcc 78
14.2 安装 CUDA 工具包 79
15 NVIDIA GPU 硬件与软件层级结构及 CUDA 介绍 79
15.1 硬件四级层级 81
15.2 软件层级：块与 GigaThread Engine 82
15.3 Warp 与调度 83
16 第一个 CUDA 项目：核函数与执行配置 84
16.1 项目创建与核函数 85
16.2 执行配置参数 85
16.3 逐步增加线程数与块数 86
16.4 硬件限制 87
17 在 Linux 系统上编译和运行 CUDA 程序 88
17.1 使用 nvcc 编译 89
17.2 运行程序与输出问题 89
17.3 cudaDeviceSynchronize 89
17.4 编译错误排查 90
18 CUDA 课程项目二：线程束 ID（Warp ID） 90
18.1 Warp ID 概念 91
18.2 计算 Warp ID 91
18.3 多块场景下的 Warp ID 92
19 CUDA 课程项目三：向量加法（Vector Addition）讲解 . . 93
19.1 CPU 上的向量加法 93
19.2 CUDA 核函数实现 94
19.3 完整代码解析 96

20 CUDA 编程教程：向量加法扩展与 GPU 利用率优化 100
20.1 问题提出与硬件限制 100
20.2 解决方案：调整网格配置 101
20.3 索引映射原理 101
20.4 代码实现 102
20.5 GPU 利用率分析 106
20.5.1 问题一：SM 的使用数量 106
20.5.2 问题二：提高 GPU 利用率的措施 107
20.6 执行时间测量 108
20.7 性能对比与总结 109
21 大规模向量加法：内存限制与分块处理策略 109
21.1 引言：从中小规模到超大规模向量 109
21.2 挑战一：超出主机内存限制 110
21.3 调试策略：使用 printf 定位问题 111
21.4 根本原因分析 111
21.5 解决方案：数据分块（Chunking）处理 112
21.6 分块代码实现逻辑 112
21.7 验证与结果 113
22 CUDA 运行时 API 详解与设备属性查询 113
22.1 核心函数：cudaGetDeviceProperties 114
22.2 项目创建与代码实现 115
22.3 代码解析 115
22.4 编译与运行验证 118
22.5 关键要点总结 119
23 NVIDIA 系统管理接口（nvidia-smi）详解 119
23.1 nvidia-smi 的三大关键领域 120
23.2 基础命令与输出解读 121
23.3 第一个表格：性能概览 121
23.4 第二个表格：进程列表 121

23.5 持续监控与数据导出 122
23.6 循环查询模式 122
23.7 定向查询与 CSV 导出 123
23.8 设置管理：功耗与时钟控制 124
23.9 功耗限制 124
23.10 持久模式（Persistence Mode） 125
23.11 时钟速度查询与控制 125
23.12 查询时钟 125
23.13 查询支持的时钟频率 127
23.14 设置应用时钟 128
23.15 重置为默认值 128
23.16 帮助与总结 128
24 GPU 占用率与延迟隐藏技术详解 144
24.1 引言：占用率的概念区分 144
24.2 获取硬件限制：运行时 API 示例 151
24.3 理论占用率计算示例 153
24.4 向量加法案例研究 153
24.5 理论占用率总结 155
24.6 达到的占用率与延迟隐藏 155
24.7 简化示例设定 156
24.8 场景一：无内存请求无指令依赖 157
24.9 延迟隐藏的概念 158
24.10 场景二：内存请求与指令依赖 159
24.11 Nsight Compute 实测验证 160
24.12 为什么 GPU 占用率很重要? 161
25 每个 SM 分配的活跃线程块详解 (阅) 162
25.1 引言：核心概念与硬件限制表 162
25.2 四个关键硬件限制 162
25.3 每个 SM 的最大线程块数 162

25.4 每个 SM 的最大 Warp 数 163
25.5 块限制 Warp 数（Block Limit Warp） 163
25.6 全局块分配与并发块的区别 164
25.7 寄存器限制与寄存器溢出 164
25.8 寄存器限制计算示例 164
25.9 四重限制取最小值原则 165
25.10 Nsight Compute 实测验证：RTX 3090 166
25.11 理论占用率 100% 的验证 167
26 掌握 CUDA GPU 并行编程（硬件与软件）Nsight Compute 入门 167
26.1 问题分析：ncu 工具报错 168
26.2 GPU 性能计数器权限设置 168
26.3 重启与验证 169
27 NVIDIA 全系性能分析工具（Nsight Systems/Compute/nvprof） 171
27.1 CUDA-MEMCHECK 与 Visual Profiler 171
27.2 Nsight 套件三大工具 171
28 错误检查 API 详解 173
28.1 内存不足问题分析 173
28.2 同步错误检查实现 175
28.3 异步错误检查 178
28.4 错误检查宏封装 178
29 命令行模式下的 Nsight Compute 性能分析 180
29.1 文档与基础命令 181
29.2 分段 (Sections) 181
29.3 指标 (Metrics) 182
29.4 指标命名结构 182
29.5 硬件单元前缀 183
29.6 聚合方式后缀 183

29.7 常用指标示例 184
29.8 实战应用与导出 184
29.9 收集特定指标 184
29.10 使用正则表达式过滤 185
29.11 导出结果 185
30 图形化 Nsight Compute 使用 186
30.1 安装与项目创建 186
30.2 附加进程与指标选择 188
30.3 分析结果概览 190
30.4 内存瓶颈分析 192
30.5 源代码与汇编分析 193
30.6 指令依赖与停滞分析 197
30.7 占用率计算器与优化 201
31 性能分析方法论 204
31.1 Python 自动化分析脚本 205
31.2 分析结果图表 207
31.3 波次与执行时间分析 209
31.4 块大小对性能的影响 212
32 非 2 的幂次方向量加法实现 213
32.1 Warp 与线程数的约束 213
32.2 块大小与线程利用率 215
32.3 波次与执行效率 216
33 基于二维线程块与线程的矩阵加法 217
33.1 二维线程块配置 218
33.2 行主序内存存储 219
33.3 二维索引映射 221
33.4 二维矩阵配置实例 223
33.5 一维与二维代码对比 227
33.6 性能分析与优化 229

34 L1 缓存命中率为零的原因分析 231
34.1 引言：矩阵加法回顾 231
34.2 NVIDIA GPU 内存层次结构回顾 231
34.3 全局内存与 HBM 231
34.4 L2 缓存 232
34.5 L1 数据缓存与共享内存 232
34.6 数据访问流程与 L1 命中率分析 232
34.7 数据加载路径 232
34.8 为何矩阵加法的 L1 命中率为零? 233
34.9 L1 缓存技术细节与合并访问 233
34.10 线程束（Warp）与内存合并 233
34.11 L2 缓存命中率的”秘密”：写入操作被视为命中 . . 234
34.12 观测到的异常现象 234
34.13 原因解析 234
34.14 验证实验 234
34.15 关键结论与建议 235
35 CUDA GPU 并行编程：共享内存技术详解 236
35.1 引言 236
35.2 GPU 内存层次结构概述 236
35.3 共享内存的架构特性 237
35.4 硬件与软件的融合 237
35.5 Ampere 架构配置示例 237
35.6 可编程性与数据重用 238
35.7 共享内存的物理组织 238
35.8 存储体冲突（Bank Conflict）详解 238
35.9 冲突定义与计算 238
35.10 典型访问模式分析 239
35.11 双精度数据类型 240
35.12 总结 240

36 Visual Studio 调试技巧 241
36.1 Nsight Visual Studio Edition 入门与安装 241
36.1.1 课程前言与工具回顾 241
36.1.2 本章目标：向量归约与调试需求 241
36.1.3 NVIDIA Nsight Visual Studio Edition 简介 242
36.1.4 安装方法 242
36.1.5 安装步骤 242
36.1.6 工具界面与功能概览 243
36.1.7 集成工具入口 243
36.1.8 核心调试选项 243
36.2 调试实战：变量观察与内存分析 244
36.2.1 调试器的核心功能与界面概览 244
36.2.2 准备工作与文档资源 244
36.2.3 变量窗口与汇编级调试 245
36.2.4 实战演示：向量加法调试 245
36.2.5 验证程序正确性 245
36.2.6 设置断点与启动调试 245
36.2.7 Nsight Compute 调试界面概述 246
36.2.8 线程视图与任务选择 246
36.2.9 启动配置参数解读 246
36.2.10 单步执行与变量观察 247
36.2.11 访问内存窗口 247
36.2.12 定位变量与地址计算 247
36.2.13 整数类型的内存布局 248
36.2.14 数据验证示例 248
36.2.15 调试暂停与部分计算结果 248
36.3 SASS 汇编分析与 Nsight 工具集成 249
36.3.1 访问反汇编视图 249
36.3.2 高级代码与 SASS 指令映射 249

36.3.3 汇编分析的价值 250
36.3.4 Visual Studio 集成 Nsight 工具 250
36.3.5 VS 扩展一键启动 250
36.3.6 Nsight Compute 集成分析 251
36.3.7 交互式分析 vs 文件分析 251
36.3.8 Nsight Systems 时间线分析 251
36.3.9 时间线核心解读 252
36.3.10 交互操作 252
37 仅使用全局内存的向量归约（基准版本） 253
37.1 向量归约的定义 253
37.2 归约的执行流程 254
37.3 多块归约与跨块同步问题 256
37.4 多内核解决方案 256
37.5 内核代码实现 258
37.6 部分和的收集 260
38 CUDA GPU 并行编程：向量归约代码解析与性能剖析 . . . 261
38.1 CPU 端归约函数 261
38.2 CUDA 应用程序的七个步骤 261
38.3 精度差异说明 267
38.4 Nsight Compute 性能分析 268
39 CUDA GPU 并行编程：向量归约优化 271
39.1 使用 while 循环自动化多步归约 271
39.2 移除取模过滤器的优化 272
39.2.1 移除过滤器的可行性分析 272
39.2.2 取模运算的性能开销 276
39.2.3 性能对比验证 276
39.3 汇编级分析 276
39.4 调试器验证 277
40 CUDA GPU 并行编程：竞态条件问题与调试方案 277

40.1 引言：调试选项引发的输出差异 277
40.2 核心概念：竞态条件 (Race Condition) 278
40.3 什么是竞态条件 278
40.4 向量归约中的竞态场景 278
40.5 为什么调试版本反而正确? 279
40.6 -G 选项与编译器优化的关系 280
40.7 -G 选项的作用 280
40.8 优化开关如何影响竞态表现 280
40.9 跨平台编译配置对比 281
40.10 Linux 命令行 (NVCC) 281
40.11 Windows Visual Studio 281
40.12 实践任务与验证方法 281
40.13 四象限测试矩阵 281
40.14 预期观察 282
40.15 总结 283
41 CUDA GPU 并行编程：向量归约中的线程利用率优化 . . . 283
41.1 引言 283
41.2 问题分析：线程利用不足 284
41.3 优化策略：增加每线程工作量 284
41.4 核心思想 284
41.5 两阶段执行流程 284
41.6 网格与索引调整 285
41.7 性能分析与实测结果 285
41.8 执行时间对比 285
41.9 占用率与内存行为 286
41.10 进阶优化探讨 286
41.11 循环遍历顺序的影响 286
41.12 位运算替代除法 286
41.13 总结与展望 287

42 NVIDIA GPU 内存层级回顾 291
42.1 共享内存优化实现 292
42.2 核心改动 292
42.3 边界检查的重要性 293
42.4 性能实测与瓶颈分析 293
42.5 循环展开（Loop Unrolling）优化 298
42.6 优化动机 298
42.7 设备函数封装 298
42.8 块大小调优与综合优化 303
42.9 总结与进阶方向 303
43 CUDA GPU 并行编程：洗牌操作（Shuffle）优化技巧 . . . 304
43.1 引言与回顾 304
43.2 GPU 内存层级与寄存器的局限性 305
43.3 内存层级回顾 305
43.4 寄存器通信的瓶颈 305
43.5 洗牌操作（Shuffle Operations）原理 306
43.6 基本概念 306
43.7 关键术语解析 306
43.8 规约中的工作流 307
43.9 代码实现详解 308
43.10 核函数结构 308
43.11 关键点说明 313
43.12 性能分析与实测结果 313
43.13 编译与环境配置 313
43.14 性能对比 313
43.15 总结 314
44 CUDA GPU 并行编程：屋顶线模型分析 314
44.1 引言：性能评估的核心目标 314
44.2 关键性能指标与算术强度 315

44.3 峰值性能参数 315
44.4 算术强度（Arithmetic Intensity, AI） 315
44.5 可达性能公式 315
44.6 屋顶线图解读与转折点 316
44.7 坐标轴定义 316
44.8 转折点（Ridge Point） 316
44.9 多级存储屋顶线 316
44.10 实战案例：向量规约的屋顶线分析 317
44.11 基准版本分析 317
44.12 优化版本对比 317
44.13 总结与优化策略指引 318
45 CUDA GPU 并行编程：矩阵乘法基础实现 318
45.1 引言：矩阵乘法的重要性 318
45.2 软件生态：专用库与硬件单元 319
45.3 软件库 319
45.4 专用硬件单元 319
45.5 算法原理：从 CPU 到 GPU 320
45.6 数学定义 320
45.7 CPU 串行实现 320
45.8 GPU 朴素实现（Naive GEMM） 320
45.9 CUDA 代码实现步骤 320
45.10 关键代码片段 321
45.11 验证与性能分析 324
45.12 总结与展望 325
46 使用共享内存优化矩阵乘法：分块技术 325
46.1 引言：全局内存的重复读取问题 325
46.2 全局内存与共享内存的延迟对比 327
46.3 L1 缓存与共享内存的本质差异 327
46.4 分块与瓦片的基本概念 328

46.4.1 瓦片的定义 328
46.4.2 网格与线程块配置 328
46.4.3 K 维循环累加 329
46.5 共享内存实现 330
46.5.1 共享内存数组的声明 330
46.5.2 共享内存容量核算 330
46.5.3 数据加载与索引计算 330
46.5.4 同步与计算 331
46.5.5 完整内核代码 332
46.6 常见错误与调试 336
46.7 性能评估 336
46.7.1 与朴素实现的对比 336
46.7.2 瓦片大小的选择 337
46.8 总结 337
47 CUDA GPU 并行编程：Float4 向量化优化矩阵乘法 338
47.1 前言与背景回顾 338
47.2 核心概念辨析 339
47.3 优化动机：计算与内存受限 339
47.4 优化原理 339
47.5 循环展开（Loop Unrolling） 339
47.6 向量化加载与预取（Vectorized Load & Prefetching） 340
47.7 线程配置调整 340
47.8 内存布局约束 341
47.9 代码实现要点 341
47.10 累加器声明 341
47.11 全局内存到共享内存的向量化加载 342
47.12 计算阶段的向量化访问 342
47.13 性能评估 350
47.14 总结 350

48 CUDA GPU 并行编程：矩阵乘法中的分块技术 351
48.1 引言 351
48.2 关键参数与约束 351
48.3 分块策略案例分析 352
48.4 案例 1：理想方阵分块 352
48.5 案例 2：非方阵与边界填充 353
48.6 案例 3：分块数与 SM 不匹配 353
48.7 案例 4：最差场景 354
48.8 非方形分块的灵活性 354
48.9 学习资源推荐 355
48.10 总结与展望 355
49 CUDA GPU 并行编程：张量核心（Tensor Core）技术详解 355
49.1 引言：张量核心的重要性 355
49.2 张量核心的架构演进与性能飞跃 356
49.3 代际性能对比 356
49.4 数据类型与精度支持 357
49.5 软件生态与编程接口 358
49.6 课程预告：实测与编程实践 359
50 CUDA GPU 并行编程：张量核心实战评估 360
50.1 引言 360
50.2 实验环境与配置 360
50.3 矩阵参数与精度 360
50.4 编译注意事项 360
50.5 朴素版本编译 360
50.6 张量核心版本编译陷阱 366
50.7 解决方案：指定计算能力 366
50.8 性能评估结果 367
50.9 基准测试 (1024×1024) 367
50.10 规模扩展测试 (2048×2048) 368

50.11 讨论与总结 368
50.12 无优化下的纯硬件收益 368
50.13 后续内容预告 368
51 CUDA GPU 并行编程：使用 WMMA API 编程张量核心 . 369
51.1 引言 369
51.2 矩阵乘法的核心步骤回顾 369
51.3 WMMA API 核心概念 370
51.4 定义与适用架构 370
51.5 碎片（Fragments）与数据布局 370
51.6 支持的维度与类型 371
51.7 内存布局：行主序 vs 列主序 371
51.8 WMMA 关键指令 372
51.9 Warp ID 的计算方法 373
51.10 一维网格/线程块 373
51.11 二维网格扩展 373
51.12 WMMA 矩阵乘法执行流程 373
51.13 重要提示 374
52 CUDA GPU 并行编程：WMMA API 实战案例 374
52.1 概述 374
52.2 核函数实现细节 375
52.3 基本参数与 Warp 划分 381
52.4 数据加载与计算流程 382
52.5 主机端验证与同步 382
52.6 编译配置的关键注意事项 382
52.7 架构指定（至关重要） 382
52.8 性能分析与对比 383
52.9 Profiling 工具使用 383
52.10 性能对比结果 383
52.11 总结与展望 383

53 CUDA GPU 并行编程：使用 cuBLAS 库调用张量核心 . . 384
53.1 概述：cuBLAS 与 WMMA API 的区别 384
53.2 环境配置与安装 385
53.3 头文件引用 385
53.4 库的安装 385
53.5 代码实现详解 385
53.6 基本流程 385
53.7 GEMM 公式 386
53.8 函数调用解析：cublasGemmEx 386
53.9 编译与性能对比 390
53.10 Makefile 编译配置 390
53.11 性能测试结果 391
53.12 学习建议 391

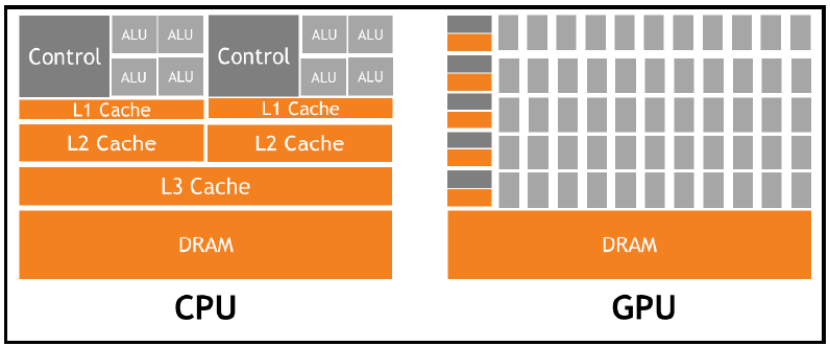


图 1.1: GPU Vs CPU

1 GPU Vs CPU

从这两张图（图 1.1）来看，左侧代表中央处理器（CPU），右侧代表图形处理器（GPU）。两者似乎都包含相似的组件或单元。例如，两者都有动态随机存取存储器（DRAM）和不同级别的缓存。具体来说，GPU 配备了两个缓存级别，其功能与 CPU 中的缓存类似；而 CPU 包含三个缓存级别——L1、L2 和 L3。接下来来看算术逻辑单元（ALU）。

这个单元负责执行应用程序的指令。CPU 中只有数量有限的 ALU，而 GPU 则包含数百甚至数千个类似单元，称为核心（Core）。鉴于以上相似性——两者都有全局内存、都有多级缓存——我们自然会问：它们之间到底有什么区别?

1.1 ALU 与核心的对比

CPU 中有 ALU，而 GPU 中有核心。为了理解它们的区别，以及为什么大多数系统同时需要 CPU 和 GPU，让我们来仔细分析。在一对一的比较中，CPU 中的 ALU 明显比 GPU 中的核心更强大、更复杂。这里的” 一对一” 指将 CPU 中的一个 ALU 与 GPU 中的一个核心进行单独对比。当我们说一个 ALU” 更强大” 时，指的是速度等属性——例如现代 CPU 的运行频率可达 3 到 4 GHz。而作为对比，一款 2020 年发布的 GPU——英伟

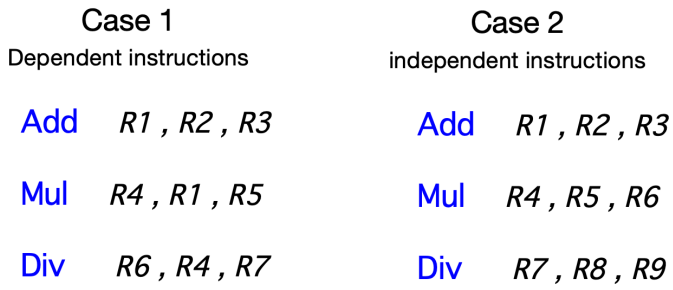


图 1.2: 两种计算场景

达 A100——其频率范围在 765 到 1200 MHz 之间。

需要明确的是，1200 MHz 意味着 1.2 GHz，大约只是主流 CPU 速度的四分之一。另一方面，GPU 核心更为专精，通常设计用于高效执行特定类型的任务。CPU 中的 ALU 是为通用任务设计的，专注于快速执行单个线程。而 GPU 核心则是为并行性设计的——它们经过优化，可以同时处理大量线程，这使它们非常适合像渲染图形这类需要对大量元素并行执行相同操作的任务。

理解 CPU 和 GPU 之间的这些区别非常重要，因为它们直接影响了应用程序的性能。毕竟本课程的核心目标，就是利用 GPU 和并行编程来优化应用程序性能。那么，核心数量的差异具体如何影响性能呢?

1.2 顺序执行与并行执行

我们将研究两种场景。在第一种场景中，假设大多数应用程序指令是相互依赖的——每条指令都依赖于前一条指令的结果。为此，我们来看三条指令，每条执行不同的操作：第一条将两个值相加，第二条将它们相乘，最后一条进行除法运算。具体来说，第一条指令从寄存器 R2 和 R3 中取值相加，并将结果存储在 R1 中。

接着将 R1 中的值与 R5 相乘，将结果存储到 R4 中。这里的关键是这些指令之间的相互依赖性：第二条指令使用了 R1，这个寄存器正是前一条

加法指令的目标寄存器。这意味着在执行乘法之前，加法操作必须先完成。

第二条指令依赖于第一条的结果。同样，除法操作必须等待乘法操作的结果，因为它涉及 R4，而乘法结果就存储在那里。这种处理方式被称为顺序执行——指令以串行方式一条接一条地执行。你可能会想：为什么不利用多个核心来并行执行这些指令，一步完成呢？在这个例子中，我们需要三步来执行这三条指令。即使有三个独立的核心分别处理加法、乘法和除法，它们也无法同时运行——第二条指令仍需等待第一条，第三条等待第二条。

这引出了一个问题：在这种场景下，CPU 中少量的 ALU 反而是理想的选择。如果 CPU 最适合此类任务，那么我们何时需要 GPU？答案在于指令的性质。虽然我们讨论的场景最适合 CPU，但 GPU 在应用程序内指令相互独立的情况下表现得最为出色。为了说明这一点，我们来看另一个例子——这次使用相同的加法、乘法和除法指令，但改为每条指令相互独立。

指令之间是独立的，因此如果我们有多个可用核心，就可以将每条指令分配给一个独立的核心，同时并行执行它们。这还可以大规模地发生。在相互独立的情况下，1000 条指令可以在 1000 个核心上并行执行，总共只需一个周期即可完成。例如，假设我们有 1000 条指令，每条执行只需一个周期。而在相互依赖的情况下，将需要 1000 个周期，因为每条指令必须等待前一条指令完成。

从以上讨论中我们可以推断出：CPU 是专门为顺序操作设计的；而当处理充满大量独立指令的任务时，GPU 则处于领先地位——因为它们能够跨多个核心同时运行多条指令。这里的”顺序”指的是每条指令都依赖于前一条指令的结果。

1.3 硬件架构

有一种常见的误解是 GPU 在所有情况下都比 CPU 更好，但这并非事实。接下来我们看看 CPU 和 GPU 在大多数系统中是如何共存的。下图展示了大多数电脑中的通用主板。你可以看到 CPU 插槽旁装有风扇，专门用于保持 CPU 冷却、防止过热。

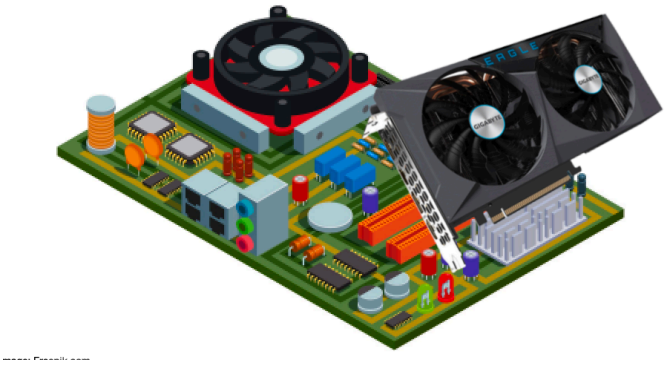


图 1.3: 硬件架构

主板上还有 DRAM 插槽、电源接口，以及关键的外围组件互连（PCIe）接口。将 GPU 插入 PCIe 插槽后，它就通过主板总线与 CPU 建立了直接连接。PCIe 接口至关重要，因为它允许连接多种扩展卡，包括 GPU、网卡和声卡。当我们把 GPU 插入 PCIe 插槽时，它就与 CPU 连接起来了。下图展示了 CPU 与 GPU 之间的这种典型连接方式。

同样，GPU 也有其专用的全局内存。回顾上图，CPU 的全局内存（DRAM）位于内存插槽中，通过总线直接连接到 CPU。

而 GPU 的全局内存则集成在 GPU 扩展卡本身的电路板上。图中绿色方框代表的就是 GPU 扩展卡。本课程的目标正是从最基础的知识开始，清晰地讲解每一个部分。

1.4 GPU 内部组件

现在让我们深入分析 GPU 中的关键组件，特别聚焦于英伟达的 GPU。在 GPU 内部有一个关键单元，称为流多处理器（Streaming Multiprocessor，简称 SM）。这个术语在本课程中将频繁出现，因此从一开始认识到它的重要性至关重要。此外，诸如调度器和分发器等组件也存在，我们将在后续章节中更深入地探讨。

除了流多处理器之外，GPU 中还存在多种内存级别，例如 L2 缓存和全局内存。深入来看，每个流多处理器配备了多种针对特定任务设计的核

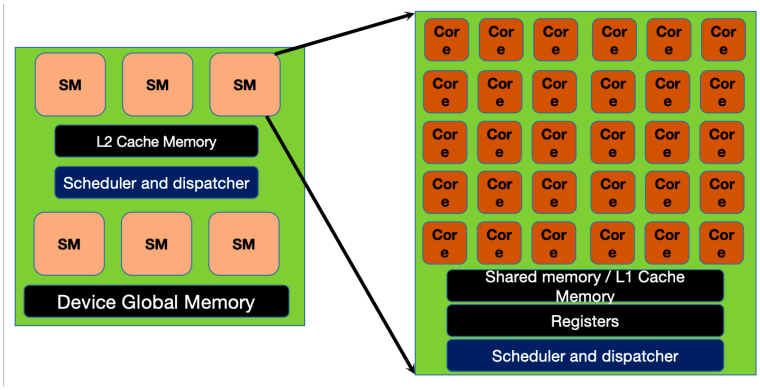


图 1.4: GPU 架构

心：其中一些核心专门用于执行浮点运算，另一些则执行整数运算。

1.5 A100 GPU 实例

让我们以英伟达 A100 GPU 为例来看一个真实 GPU 及其流多处理器。A100 于 2020 年发布。下图展示了单个流多处理器的一部分（并非整个 SM，更不是整个 GPU）。在 SM 内部有 L1 缓存，而 L2 缓存则位于整个 GPU 内部（SM 外部）。需要记住的关键点是：L1 缓存位于 SM 内部，而 L2 缓存是更广泛的 GPU 层级的组成部分。

在执行或功能单元方面，流多处理器非常多样化。我们发现了专门为矩阵乘法操作设计的张量核心（Tensor Core）、用于处理浮点运算的浮点核心，以及整数核心。此外，如前所述，SM 包含一个调度器单元、一个分发器单元和若干个寄存器。当涉及内存交互时，SM 配备了加载/存储单元和一些特殊功能单元（SFU）——这些单元负责从内存中读取以及向内存写入数据，并处理诸如对数计算等特定任务。

这张详细的图片仅展示了 GPU 架构白皮书中一个 SM 四分之一部分的快照——这里指的是整个 GPU 的结构图，而非单个 SM。这张特定的图片详细展示了英伟达 A100 GPU 的规格。在接下来的章节中，我们将更深入地探讨白皮书是什么、在哪里可以访问它们，以及其他相关细节。

图 1.5: A100 GPU 架构

The **NVIDIA A100 Tensor Core GPU implementation** of the GA100 GPU includes the following units:

- 7 GPCs, 7 or 8 TPCs/GPC, 2 SMs/TPC, up to 16 SMs/GPC, 108 SMs
- 64 FP32 CUDA Cores/SM, 6912 FP32 CUDA Cores per GPU
- 4 Third-generation Tensor Cores/SM, 432 Third-generation Tensor Cores per GPU
- 5 HBM2 stacks, 10 512-bit Memory Controllers

图 1.6: A100 特性

每个 SM 包含 64 个单精度核心。在 A100 GPU 内部，约有 108 个 SM，这意味着整个 A100 GPU 拥有约 7000 个单精度核心——这是一个非常大的数量。该 GPU 还拥有另外 7000 个专用于整数运算的核心。此外，深入研究 A100 的内部结构，你还会发现特殊功能单元（SFU）以及张量核心。正是这些核心凭借其庞大的数量所提供的巨大计算能力，使 A100 GPU 成为全球众多高性能超级计算机的核心组件。例如，META 公司宣布了他们雄心勃勃的计划——建造世界上最强大的超级计算机。如果你查阅 META 官方网站的文章，就会清楚地看到他们的超级计算机架构严重依赖于 A100 GPU。

在最近的发展中，有推测称他们可能会整合 2022 年发布的新款 H100 GPU——这是在 A100（基于 Ampere 架构）发布两年后推出的。与 A100 相比，H100 拥有进一步的进步和增强，我们将在本课程中讨论这两者。

你可以在 TOP500 网站 https://top500.org 上找到超级计算机排名列表。为了强调英伟达 GPU——特别是 A100 GPU——的重要性，让我们看看全球前 500 名超级计算机。该网站提供了关于世界领先超级计算机的详细信息。从数据中可以看到，A100 GPU 存在于前十名超级计算机中的三台里：它被集成到排名第四的莱昂纳多（Leonardo）超级计算机中，以及排名第八和第九的超级计算机中。

感谢你与我们一同深入 GPU 的世界，我们下章再见。

2 NVIDIA 历史

在上个世纪的最后二十年里，几家如今众所周知的巨头公司相继成立，其中最伟大之一就是英伟达（NVIDIA）。英伟达由黄仁勋于 1993 年创立。成立仅两年后，英伟达推出了其第一款产品——NV1。该设备设计相当简单，其 GPU 仅有 2 MB 内存、64 位总线宽度和单个核心。要了解如今的差距，可以将其与最新的 GPU GeForce RTX 4090 Ti 进行比较：RTX 4090 Ti 拥有约 18,000 个核心，而 NV1 只有 1 个核心。

核心数量方面，增长了约 18,000 倍。而内存容量从 NV1 的 2 MB 增

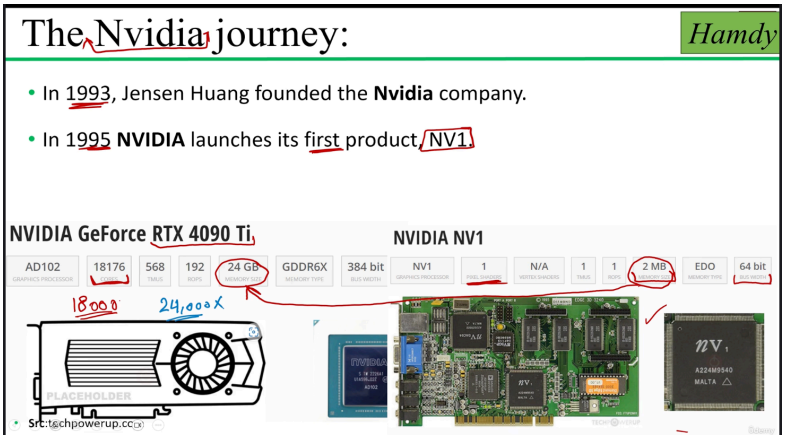


图 2.1: NVIDIA 历史

加到 RTX 4090 Ti 的 48 GB，增长约 24,000 倍。此外，再看看对应用程序性能至关重要的运行频率——已从 75 MHz 大幅提升至约 2400 MHz，带来 32 倍的速度提升。这充分展示了 GPU 技术在过去 25 年间的巨大演变。

2.1 RIVA 128

暂时放下这些数据，我们继续往下看。1997 年，英伟达推出了 RIVA 128，也称为 NV3。该型号是首批为消费级市场带来三维加速的 GPU 之一，这也是为什么它比其前身 NV1 更成功。值得注意的是，RIVA 128 是英伟达首次取得重大市场成功的产品，在巩固其市场地位方面发挥了关键作用——四个月内销量即达到约 100 万台。

2.2 GeForce 世代

GeForce 这一品牌为许多人所熟悉，因为这些 GPU 在问世一两年后就广泛出现在台式机和笔记本电脑中。具体来说，1998 年英伟达揭开了后来成为其标志性 GPU 世代的 GeForce 系列的序幕。此次发布标志着 GPU 技术领域的一次革命性转变。从图中可以看到，GeForce 系列的首款 GPU 与 NV1 相比差异巨大——它拥有四个核心，相对于英伟达第一款 GPU NV1

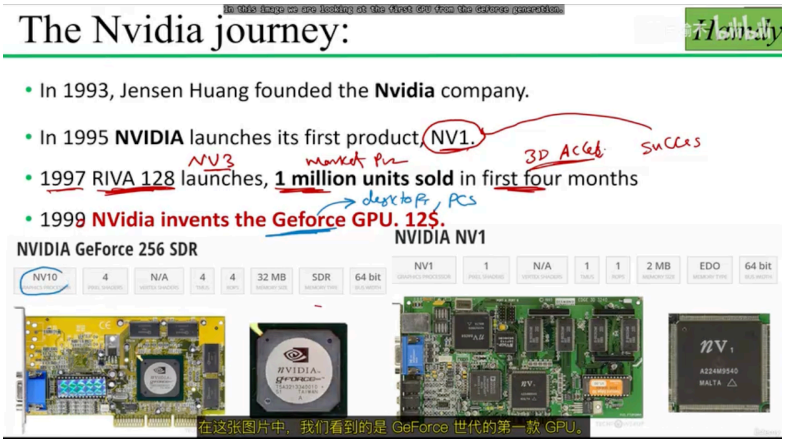


图 2.2: GeForce

的单核心来说是一个巨大的飞跃。别忘了，那是在 1998 年，一台设备中有四个核心本身就是一场革命。内存容量也从 NV1 的 2 MB 增加到 32 MB，增长了 16 倍。

从那时起，英伟达就成为全球 GPU 领域最知名的公司。如今，英伟达既是推动计算加速的关键力量，也主导着游戏产业。以上便是英伟达发展历程的简要概述。

3 GPU 架构与代系关系

3.1 架构的定义

在英伟达 GPU 的语境中，有两个关键概念需要解释：世代与架构。先说架构——它指的是 GPU 的基本设计和结构，涵盖了 GPU 在最基础层面的构建方式：核心的排列与管理、数据处理和传输的技术，以及渲染图形和计算任务的方法。每种新架构通常都会在处理能力、能效以及对新功能的支持方面带来改进，例如光线追踪或 AI 驱动的任务。

英伟达的 GPU 架构决定了 GPU 的性能与能力。英伟达为每种架构赋予不同的代号，通常使用著名科学家或发明家的名字，如 Turing 和 Pascal。

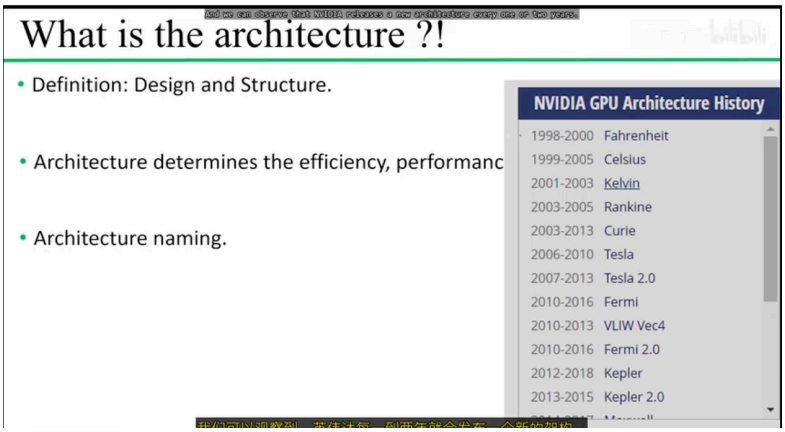


图 3.1: GPU 架构

例如，Turing 架构引入了实时光线追踪以及用于 AI 计算的核心单元。而在 Pascal 架构中，我们可以看到对其前身的一些改进，例如更多的核心和更高的时钟频率。总而言之，GPU 架构至关重要，因为它为 GPU 的能力奠定了基础，影响着从游戏图形到科学计算的方方面面。

下图展示了英伟达的若干 GPU 架构，可以看到英伟达每 1 到 2 年就会发布一个新架构。在后续章节中，我将带你了解从 Fermi 架构开始的各个英伟达架构。

接下来谈谈英伟达的不同世代。英伟达将其产品分为两个不同的类别：第一类面向普通消费者，涵盖我们在笔记本电脑、台式机和工作站中使用的 GPU；第二类服务于高性能计算（HPC）领域，这在云计算等场景中至关重要。META、亚马逊、微软和谷歌都是 HPC 提供商的典型代表。

第一类别包括 Tegra、GeForce 和 Quadro 等世代系列，其中 GeForce 和 Quadro 也是游戏 GPU 的热门选择——我想大多数人都熟悉它们，因为它们常见于许多笔记本电脑中。

而第二类别（HPC 类别）中有一个世代被称为 Tesla。需要指出的是，这个 Tesla 与埃隆 · 马斯克的公司没有任何关联——它只是为高性能计算设计的一系列 GPU 的统称。

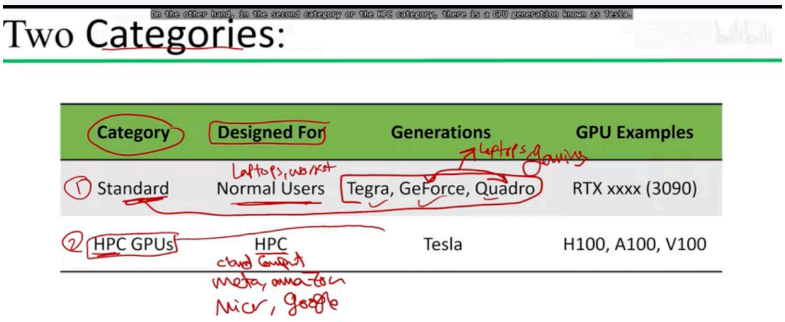


图 3.2: GPU 架构

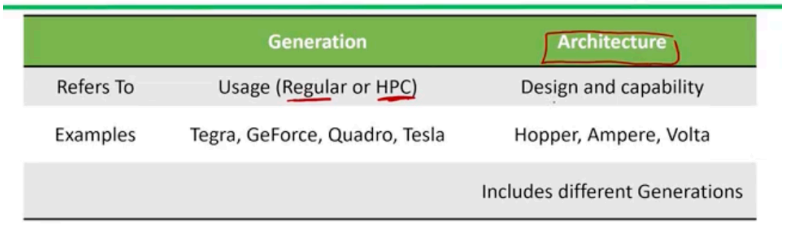


图 3.3: GPU 架构

3.2 架构与世代的关系

下表旨在总结这两个术语之间的区别：世代指的是 GPU 的具体用途，例如是用于 HPC 服务器还是标准台式机。而架构，如前所述，指的是 GPU 芯片的设计与能力。每个架构可能包含多个不同的世代。在世代一栏，我们看到了 Tegra、GeForce、Quadro 和 Tesla 这些名称；而 Hopper、Ampere 和 Volta 则是英伟达提供的不同架构。

为了说明这一点，考虑一下 2020 年发布的 Ampere 架构。这个架构包含了多个世代的 GPU。例如，在 Ampere 架构内，你既可以找到 GeForce 世代的 RTX 3090，也可以找到 Tesla 世代的 A100——这两款 GPU 基于相同的 Ampere 架构芯片，但面向的应用截然不同：RTX 3090 属于 GeForce

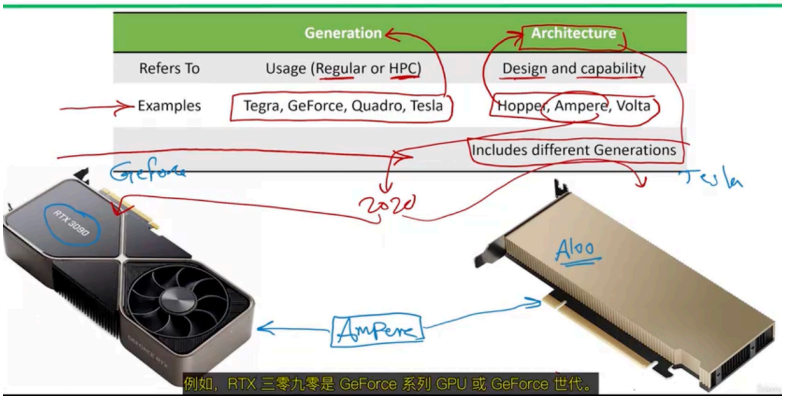


图 3.4: GPU 架构

系列，主要设计用于游戏等消费级用途；而 A100 属于 Tesla 世代，专为超级计算机和云服务器等 HPC 环境设计。

总之，我们可以这样理解：每当英伟达整合了新的软件或硬件增强功能，就会宣布一个新架构；然后基于这个新架构，再为普通消费者和 HPC 领域分别开发不同世代的 GPU。

3.3 GPU 世代分类

现在让我们深入探讨不同类型的世代。下表展示了英伟达最知名的 GPU 世代之间的区别：Tegra、GeForce、Quadro 和 Tesla。

首先来看 Tegra。Tegra 是英伟达为智能手机、平板电脑和移动互联网设备等移动平台开发的片上系统（SoC），将 GPU 和 CPU 集成在单一芯片中。第一款使用 Tegra 的产品是微软的 Zune HD 媒体播放器，于 2009 年 9 月发布。2017 年 3 月 15 日，Tech Insights 披露了任天堂 Switch，它同样由 Tegra 芯片驱动。

接下来是 GeForce GPU——请注意，这一世代的 GPU 专为个人计算机打造，面向消费级市场，尤其是游戏玩家。这里说的是三维图形渲染和游戏中的高性能，而非 HPC 级别的计算。它们也被普通消费者用于视频编辑和其他图形密集型任务。GeForce 系列中一个突出的例子是 RTX 系列，如

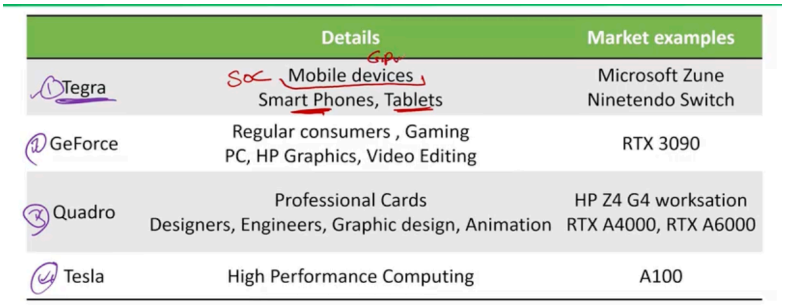


图 3.5: GPU 架构

GeForce RTX 3090、3080、3070 和 3060，这些是当前市场上该世代最知名的 GPU。接下来是 Quadro GPU——专业工作站显卡，专为需要高精度和计算能力的设计师和工程师打造。Quadro GPU 经过优化，适用于高级图形设计、视频编辑和动画等专业软件。它在许多笔记本电脑中广泛使用，以提供增强的性能。此外，最新工作站（如 HP Z 系列）通常也使用 Quadro GPU。

在离开 Quadro GPU 这个话题之前，让我们花点时间讨论一些最近的更新。例如 HP 最近推出的一款工作站——HP Z4 G4，这款工作站可以配备高端的英伟达 Quadro RTX 显卡。但在未来的 GPU 中，我们将不再看到 Quadro 品牌。正如下图所示（来源见链接所示），Quadro 命名已被 RTX 标签取代，我们只会看到 RTX 标签。

此外，同一网页包含一个表格，展示了各种 Quadro 系列与其对应英伟达架构之间的关系，以及它们在 RTX 系列中的新命名等价物。表格的排列方式为：第一列列出了不同的 Quadro 系列，如 8000 和 6000 等。以 6000 系列为例，这一行展示了不同架构下的命名惯例——在 Maxwell 架构中前缀是 M，在 Pascal 架构中前缀是 P，命名惯例为 Quadro 后跟系列数字（如 6000），前面加上架构的首字母。

在 Pascal 中前缀是 P。但在 Turing 架构中，RTX 标签被插入在 Quadro 和系列数字之间。正如我们所见，如果你打算在并行编程或 GPU 领域工

Series	Pre-Kepler	Kepler	Maxwell	Pascal	Volta	Turing	Today's Equivalent
Specialty		Quadro K6000 (for FP64)		Quadro GP100	Quadro GV100		Quadro GV100
8000						Quadro RTX 8000	RTX A6000
6000	Older 6000-series		Quadro M6000 (12GB), Quadro M6000 24GB	Quadro P6000		Quadro RTX 6000	RTX A6000
5000	Older 5000-series	Quadro K5000, Quadro K5200	Quadro M5000	Quadro P5000		Quadro RTX 5000	RTX A5000, RTX A5500
4000	Older 4000-series	Quadro K4000, Quadro K4200	Quadro M4000	Quadro P4000		Quadro RTX 4000	RTX A4000, RTX A4500
2000	Older 2000-series	Quadro K2000, Quadro K2000D, Quadro K2200	Quadro M2000	Quadro P2000, Quadro P2200			RTX A2000, RTX A2000 12GB

图 3.6: GPU 架构

作，了解 GPU 的命名规则非常重要。然而，从 Ampere 架构开始，英伟达已经弃用了 Quadro 品牌，只使用 RTX 前缀。

还有一点需要注意：在弃用 Quadro 术语之前，英伟达将 RTX 名称同时用于 Quadro 和 GeForce 两条产品线。例如 RTX 30XX 系列，包括 RTX 3090、3080、3070 和 3060 等型号，都属于面向消费者（尤其是游戏玩家）的 GeForce 家族。

在专业领域（如设计和内容创作），正如表格所描绘的，曾存在 Quadro RTX 这样的型号。我们有 GeForce RTX 和 Quadro RTX 两个系列，例如 Quadro RTX 4000、Quadro RTX 5000 和 Quadro RTX 8000。RTX 标签同时出现在 GeForce 世代和 Quadro 世代中。但需要记住的是，在未来的 GPU 中，我们将不再看到 Quadro 品牌，只会看到 RTX。虽然这些信息乍一看可能让人应接不暇，但这完全正常——当你更深入地研究 GPU 行业并开始使用这些产品时，命名和术语会变得更加熟悉，也更容易理解。

总而言之，我们在最顶层有 GPU 架构，每个架构可用于生产不同的世代。

3.4 GeForce 与 Tesla 在超级计算机中的应用

现在让我们来看两种来自不同世代的 GPU 的真实案例，每个世代针对不同的市场细分——从 GeForce 这样的普通消费级 GPU 到为 HPC 设

计的 Tesla 系列。正是这种区别，使得在全球前 500 名超级计算机中看到 GeForce 系列的 GPU 相当罕见，因为它并非为超级计算机设计，而是为普通台式机和工作站设计的。

让我们深入 TOP500 网站进一步探索。如下所示，这是当年六月发布的前 500 名超级计算机列表。如果我们搜索 A100、V100 和 P100 等 GPU——这些都属于 Tesla 世代——可以观察到许多超级计算机采用了它们，因为 Tesla 正是为超级计算机等用途设计的。我们将在接下来的章节中详细而深入地讨论这些 GPU。

另一方面，如果我们搜索 RTX 系列，结果显示在排名靠前的超级计算机中没有此类应用。我想现在你已经清楚了 GeForce 与 Tesla 世代之间的区别。理解所有这些区别对于为你的特定应用选择正确的 GPU 至关重要。我们将在接下来的章节中更深入地介绍 GeForce 和 Tesla 等不同世代。

4 如何识别 GPU 架构与代际

4.1 使用 TechPowerUp 查询规格

下面我们以 RTX 3090 和 A100 GPU 为例，介绍如何快速获取这些规格参数。

方法很简单：推荐使用 TechPowerUp 网站（A100 页面），它收录了各大厂商详尽的 GPU 数据库。你只需在搜索引擎中输入”A100 GPU TechPowerUp” 并回车，点击搜索结果中的对应链接，即可查看该显卡的完整规格与详细信息。

同样，你可以在 TechPowerUp（RTX 3090 页面）查 RTX 3090 的规格。现在对这两款 GPU 做一个简单的比较：首先是 GPU 芯片代号——A100 的代号为 GA100。关于 GPU 芯片的具体含义我们将在后续章节中解释，这里只需关注它的名称。然后看核心数量：A100 拥有约 7000 个核心，而 RTX 3090 拥有超过一万个核心。

你在这里也可以看到许多规格参数，解释了每款 GPU 的不同能力。但

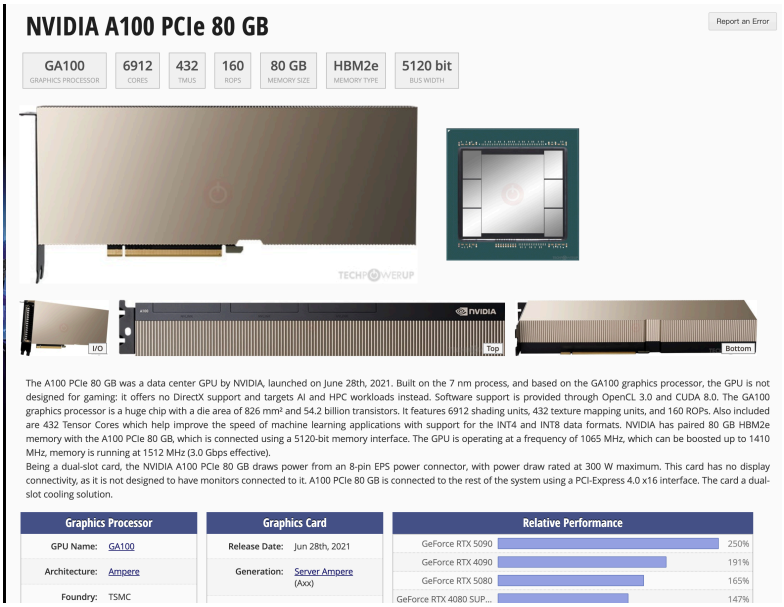


图 4.1: A100 GPU 架构

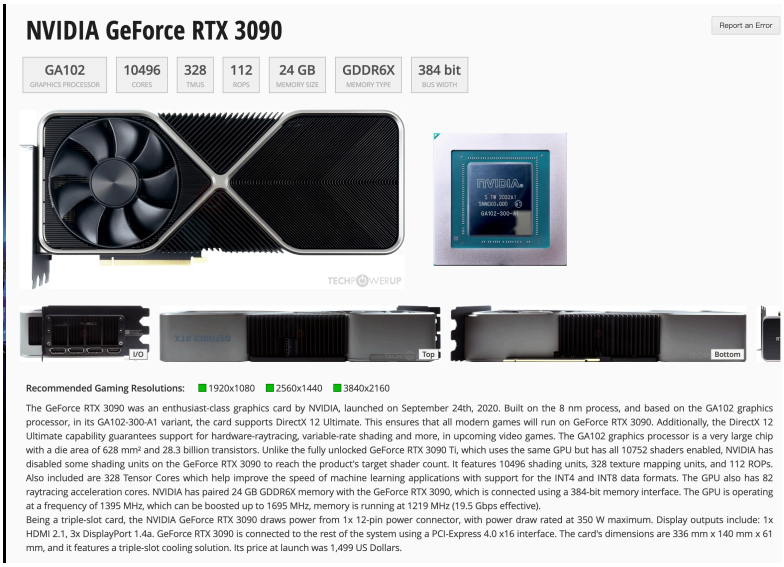


图 4.2: RTX 3090 GPU 架构

本文先跳过这些，只关注世代和架构，后续章节将详细讨论更多规格。不过请记住 6192 这个数字——这是专用于处理单精度运算的核心数量，并不代表用于各种操作的总核心数。举例来说，A100 GPU 还包含用于处理整数运算和双精度运算的核心。这些将在后续章节中从官方白皮书和数据表中深入探讨，本文暂不展开。

4.2 A100 与 RTX 3090 架构对比

现在让我们聚焦于这些 GPU 的世代和架构。以 RTX 3090 为例，其世代为 GeForce 30，架构为 Ampere。你还记得 GeForce 是为谁设计的吗？正如上一章所述，它是为普通台式机、笔记本电脑和工作站设计的。

再来看 A100 GPU，其世代是 Tesla，正如前文所述，Tesla 世代专用于 HPC 服务器和超级计算机。值得关注的是，A100 与 RTX 3090 共享相同的 Ampere 架构，尽管它们是为不同的应用场景而设计的。

这里引出一个重要问题：我们如何从外观上确认 GeForce 与 Tesla 世代之间的区别？

4.3 视觉区分 GeForce 与 Tesla

检查这些 GPU 的图片时，你会注意到 Tesla GPU 通常没有板载冷却风扇。如果你看 A100 GPU 的图片，找不到任何冷却风扇。同样，V100 GPU 也属于 Tesla 世代，搜索它的图片，可以看到它也不包含冷却风扇。其原因是这些 GPU 设计用于 HPC 环境，而这些环境中配备了外部且更强大的冷却系统——如果你搜索全球任何超级计算中心的照片，就能看到其冷却系统的复杂性。

另一方面，GeForce GPU（例如我们电脑中的 RTX 系列）通常都配备内置风扇。这些风扇是标准台式机或工作站环境中散热所必需的。这就是我们从外观上确认 GeForce 和 Tesla 世代之间区别的方法：Tesla GPU 无冷却风扇，GeForce GPU 有冷却风扇——原因就在于它们所使用的目标环境不同。

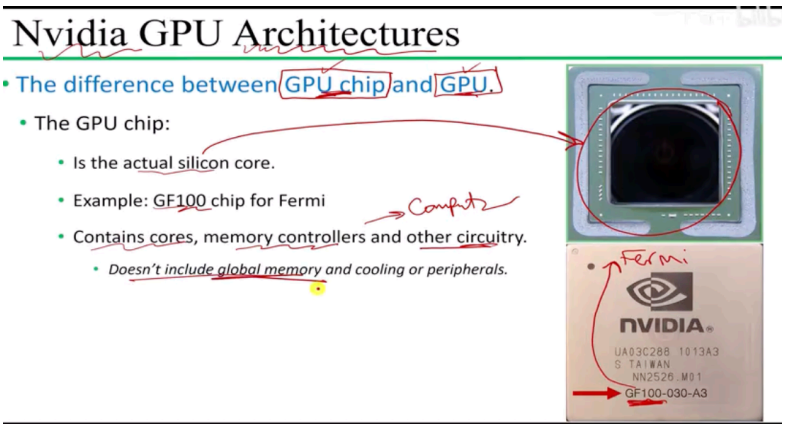


图 5.1: GPU 和 GPU 芯片

5 GPU 和 GPU 芯片

5.1 GPU 芯片的定义

本章我们将比较两个关键术语：GPU 和 GPU 芯片。本章非常重要，因为如果不理解 GPU 与 GPU 芯片之间的区别，就无法解释英伟达生产的各种架构。

首先来说明这两个术语的区别，从 GPU 芯片开始。GPU 芯片代表着 GPU 的心脏——它是实际的硅核心，不附带任何外围组件。例如，一款名为 GF100 的 GPU 芯片，其中的 F 指 Fermi 架构。在该芯片内部，集成了各种核心、内存控制器和其他负责 GPU 计算的电子元件。而 GPU 则是消费者购买的最终产品，它集成了芯片以及必要的系统和接口，包括许多 GPU 中都存在的冷却机制。

5.2 GeForce GTX 480 与 GF100 芯片

这款 GPU 由上文讨论过的 GF100 芯片驱动。下图展示的是 GeForce GTX 480 GPU——除了 GF100 芯片之外，它还包含全局内存单元、显示端口（如 HDMI 端口）和一个冷却风扇。之所以需要冷却风扇，是因为它

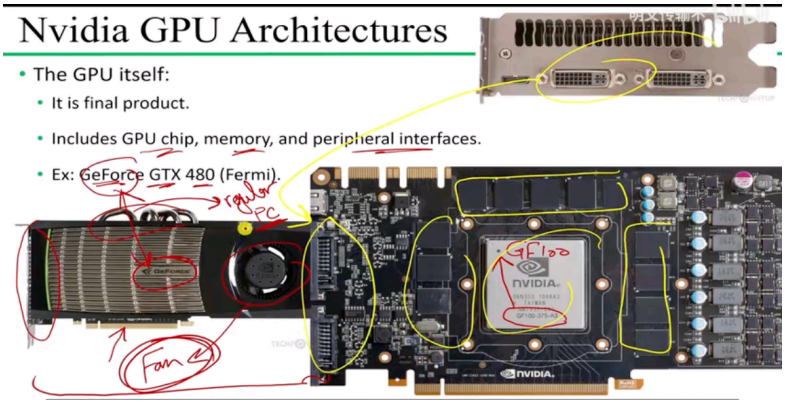


图 5.2: GPU 和 GPU 芯片

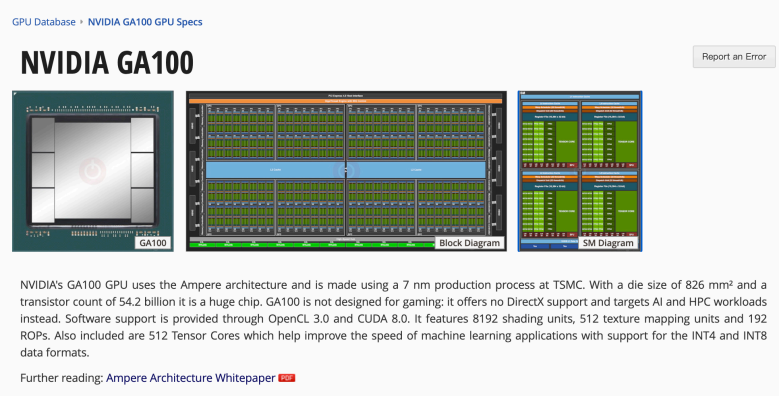


图 5.3: GA100 芯片

属于 GeForce 世代，这类环境需要风扇来散发计算产生的热量。

根据之前的讨论，GeForce 世代是面向普通消费者的标准台式机和工作站设计的。

5.3 A100 与 GA100 芯片

这两个术语之间区别的另一个例子可以从 A100 GPU 及其芯片中看到。基于 Ampere 架构的 A100 GPU，其名称中的字母 A 即体现了这一点。与 Fermi 芯片类似的命名方式，驱动 A100 GPU 的芯片为 GA100。

在 Fermi 架构中，芯片命名为 GF100。在 Ampere 架构中，芯片命名为 GA100。访问 TechPowerUp 网站，可以看到 A100 GPU 的众多规格，网页上首先展示的就是芯片名称——GA100。通过搜索芯片名称 GA100，可以找到对应的链接，点击即可看到 GA100 芯片：如图所示，它仅是一块电子芯片，没有连接任何其他组件。

回到 A100 GPU 的网页，可以看到 A100 GPU 包含芯片、内存单元和其他外围组件。总而言之，GPU 芯片是 GPU 的核心，而 GPU 本身是包含芯片、内存接口和冷却系统的完整产品，可以直接供消费者使用。

6 各架构对应的芯片型号

本章将深入探讨不同 GPU 架构与其特定芯片之间的关联。需要指出的是，在每个 NVIDIA GPU 架构内部都会发布一系列芯片，每颗芯片可能具有独特的规格，但它们都共享相同的底层架构。以 Ampere 架构为例，如图所示，Ampere 架构包含多款芯片，如 GA100、GA102 等。这些芯片名称中的字母 A 代表 Ampere 架构。因此，每一颗芯片都可用于构建面向不同市场细分的 GPU。

6.1 GA100 芯片与高性能计算

例如，GA100 芯片用于构建 A100 GPU，这是超级计算机和 HPC 服务器中最知名的 GPU 之一。你可以在 TechPowerUp 网站上验证这一信息：打开 A100 GPU 的页面，这里列出的芯片名称就是 GA100。如果你搜索该 GPU 的产品系列，会发现它被归类于 Tesla 系列。正如前几章多次讨论的，Tesla 系列专用于 HPC 服务器和超级计算机，这一分类清楚地说明了为何 GA100 芯片常被用于构建 HPC GPU。

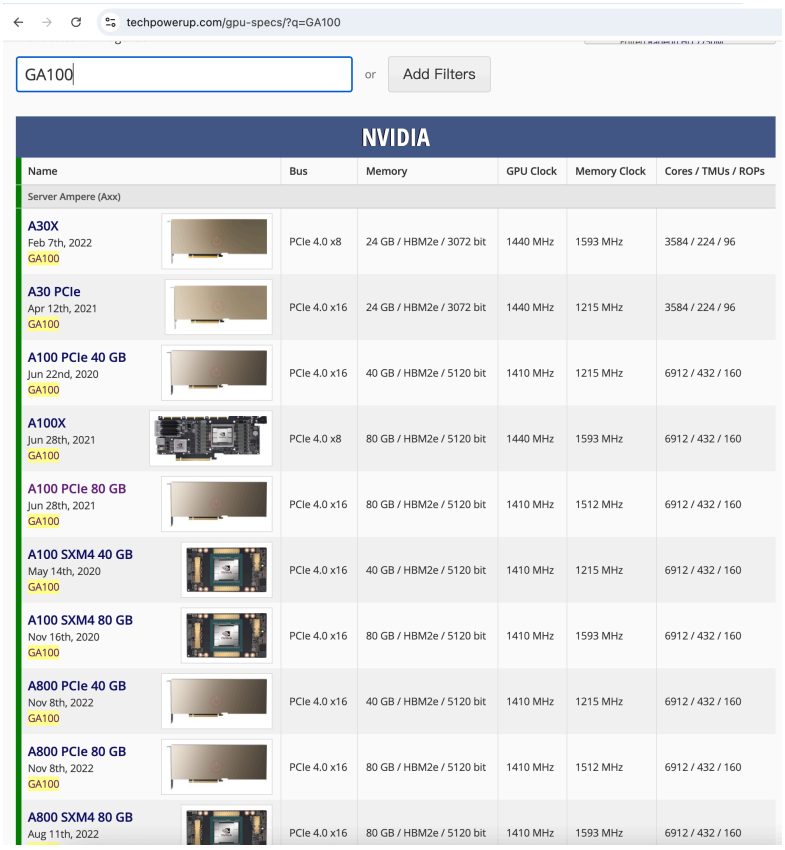

techpowerup.com/gpu-specs/?q=GA100
GA100
or
Add Filters
NVIDIA
Name
Bus
Memory
GPU Clock
Memory Clock
Cores / TMUs / ROPs
Server Ampere (Axx)
A30X
Feb 7th, 2022
GA100
PCIe 4.0 x8
24 GB / HBM2e / 3072 bit
1440 MHz
1593 MHz
3584 / 224 / 96
A30 PCIe
Apr 12th, 2021
GA100
PCIe 4.0 x16
24 GB / HBM2e / 3072 bit
1440 MHz
1215 MHz
3584 / 224 / 96
A100 PCIe 40 GB
Jun 22nd, 2020
GA100
PCIe 4.0 x16
40 GB / HBM2e / 5120 bit
1410 MHz
1215 MHz
6912 / 432 / 160
A100X
Jun 28th, 2021
GA100
PCIe 4.0 x8
80 GB / HBM2e / 5120 bit
1440 MHz
1593 MHz
6912 / 432 / 160
A100 PCIe 80 GB
Jun 28th, 2021
GA100
PCIe 4.0 x16
80 GB / HBM2e / 5120 bit
1410 MHz
1512 MHz
6912 / 432 / 160
A100 SXM4 40 GB
May 14th, 2020
GA100
PCIe 4.0 x16
40 GB / HBM2e / 5120 bit
1410 MHz
1215 MHz
6912 / 432 / 160
A100 SXM4 80 GB
Nov 16th, 2020
GA100
PCIe 4.0 x16
80 GB / HBM2e / 5120 bit
1410 MHz
1593 MHz
6912 / 432 / 160
A800 PCIe 40 GB
Nov 8th, 2022
GA100
PCIe 4.0 x16
40 GB / HBM2e / 5120 bit
1410 MHz
1215 MHz
6912 / 432 / 160
A800 PCIe 80 GB
Nov 8th, 2022
GA100
PCIe 4.0 x16
80 GB / HBM2e / 5120 bit
1410 MHz
1512 MHz
6912 / 432 / 160
A800 SXM4 80 GB
Aug 11th, 2022
PCIe 4.0 x16
80 GB / HBM2e / 5120 bit
1410 MHz
1593 MHz
6912 / 432 / 160

图 6.1: GA100

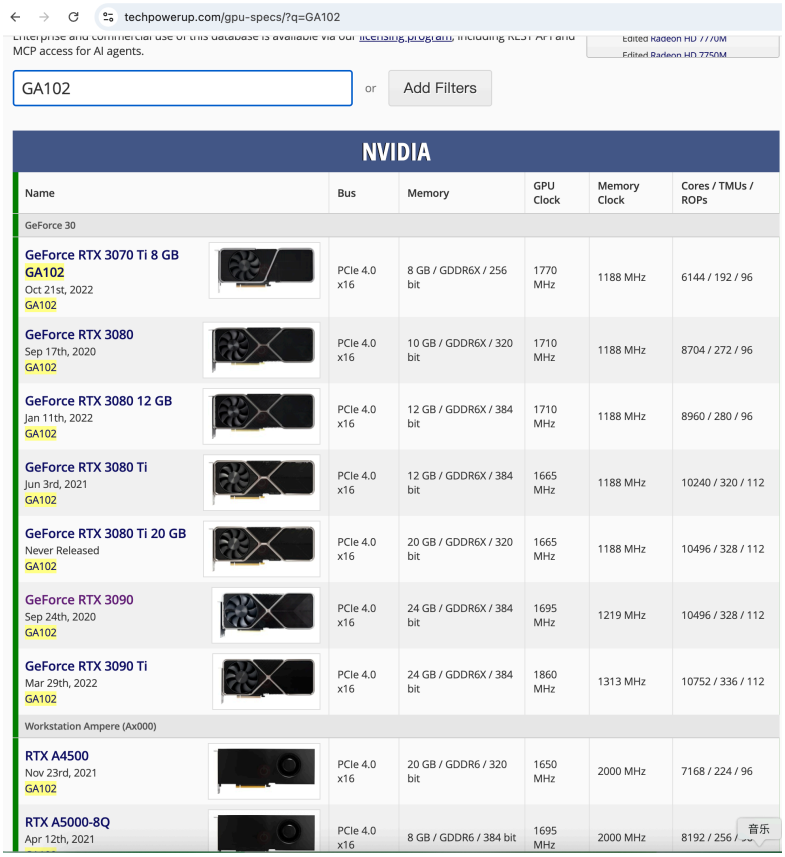


图 6.2: GA102

6.2 GA102 芯片与消费级市场

第二个例子是 GA102 芯片，它更常用于面向普通用户或游戏玩家的 GPU。例如，该芯片出现在 GeForce RTX 系列中，专为普通 PC 或工作站设计。此外，从 TechPowerUp 网站可以看到，采用该芯片的大多数 GPU 都带有散热风扇——这是一个视觉标志，表明这些 GPU 是为日常使用设计的，而非用于 HPC 或超级计算机。

同样，回到 GA100 芯片的网页，你会发现它通常用于不带散热风扇的 GPU，表明它被应用于超级计算机和 HPC 服务器——这些场景通常采用更强大的集中式散热系统，不过这并非绝对。总结而言，现在已经很清楚

了：某些芯片专门为 HPC 应用设计，而另一些则是为日常消费级用途设计的。

6.3 芯片的多用途适配

NVIDIA 有时会发布同一芯片的不同版本。以 Ampere 架构中的 GA102 为例，在其网页上我们注意到它常用于游戏市场——因为大多数采用该芯片的 GPU 都带有散热风扇，产品系列也显示为 GeForce。然而，如果查看该表格的前几款 GPU，会发现这款芯片也被用于制造 HPC 服务器用的 GPU。这是 GA102 芯片的一个不同版本，即面向 HPC 的版本，其编号为 890，而面向游戏市场的版本编号则在 100 到 350 之间。

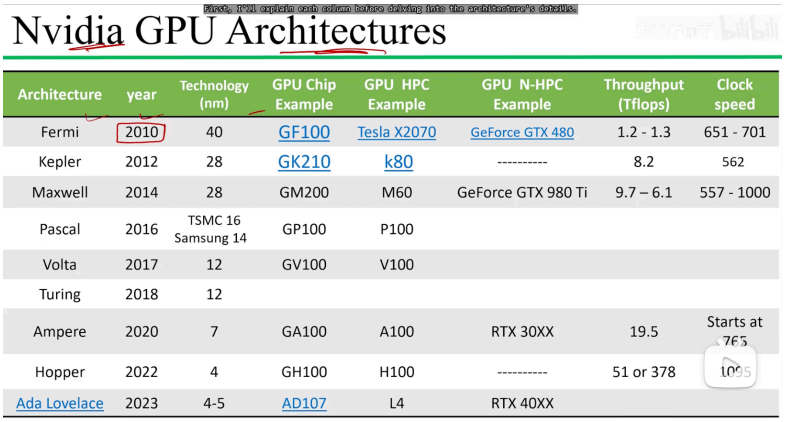
这展示了 NVIDIA 在将芯片适配多种用途方面的灵活性。值得强调的是，每颗芯片以及特定架构内的每个芯片版本，都可能根据 GPU 型号甚至制造商的不同，以不同的频率运行。这是因为 NVIDIA 会向市场上的多家公司提供 GPU 芯片和授权，由这些公司生产 GPU ，它们在频率等细节上可能略有差异。

在本章最后需要说明的是，本课程的目标是帮助大家掌握独立处理这些主题的知识与技能。请记住，牢固的基础是掌握任何学科的关键。

7 NVIDIA GPU 架构历史

本章将带大家了解 NVIDIA 推出的各种 GPU 架构。从下图可以看出，自 1998 年以来 NVIDIA 已推出了众多架构。我们将从 2010 年推向市场的 Fermi 架构开始探索，早期已不再使用的架构则略过不提。

下表列出了 NVIDIA 自 2010 年以来推出的所有架构。首先解释每一列的含义：第一列是 NVIDIA 引入的架构名称，第二列是它们各自的发布时间。接下来是 GPU 芯片示例列，提供了每个架构的代表性芯片。在后续讲解中，我会为每个架构突出展示两款不同的 GPU 型号，分别列在”HPC GPU”（面向高性能计算）和” 消费级 GPU”（面向普通消费者）两栏中。最



Nvidia GPU Architectures

Architecture	year	Technology (nm)	GPU Chip Example	GPU HPC Example	GPU N-HPC Example	Throughput (Tflops)	Clock speed
Fermi	2010	40	GF100	Tesla X2070	GeForce GTX 480	1.2 - 1.3	651 - 701
Kepler	2012	28	GK210	k80	----------	8.2	562
Maxwell	2014	28	GM200	M60	GeForce GTX 980 Ti	9.7 – 6.1	557 - 1000
Pascal	2016	TSMC 16 Samsung 14	GP100	P100			
Volta	2017	12	GV100	V100			
Turing	2018	12					
Ampere	2020	7	GA100	A100	RTX 30XX	19.5	Starts at 765
Hopper	2022	4	GH100	H100	----------	51 or 378	1095
Ada Lovelace	2023	4-5	AD107	L4	RTX 40XX		

图 7.1: GPU 架构历史

后两列则用于评估各架构的计算能力或性能指标。

本表仅选取了两个关键参数进行比较：第一个是吞吐量（每秒浮点运算次数），第二个是时钟频率。在大多数情况下，我会为 HPC GPU 和消费级 GPU 分别显示一个数值。

7.1 Fermi 架构

现在开始逐一讲解各架构，首先从 Fermi 架构讲起。Fermi 架构由 NVIDIA 于 2010 年推出，是 GPU 设计与技术的一次重大飞跃。

基于 Fermi 的 GPU 采用 40 纳米工艺制造。该架构的代表性芯片是 GF100，它被用于生产多款 GPU，既面向 HPC 市场，也面向消费级市场。例如，Tesla X2070 是基于该芯片的 HPC GPU，而 GeForce GTX 480 则专为普通消费者和游戏玩家设计。如果你比

较这些性能指标，可能会疑惑：为什么面向普通用户的 GTX 480 在吞吐量和频率上似乎优于 Tesla X2070？具体来看，GTX 480 的吞吐量为 1.3 TFLOPS、时钟频率为 700 MHz，而 X2070 仅为 1.2 TFLOPS 和 651 MHz。

要理解这一点，必须认识到评估 GPU 性能并非一目了然——GPU 的整体表现受多种因素影响，而非仅取决于这两个指标。仅凭 TFLOPS 或时

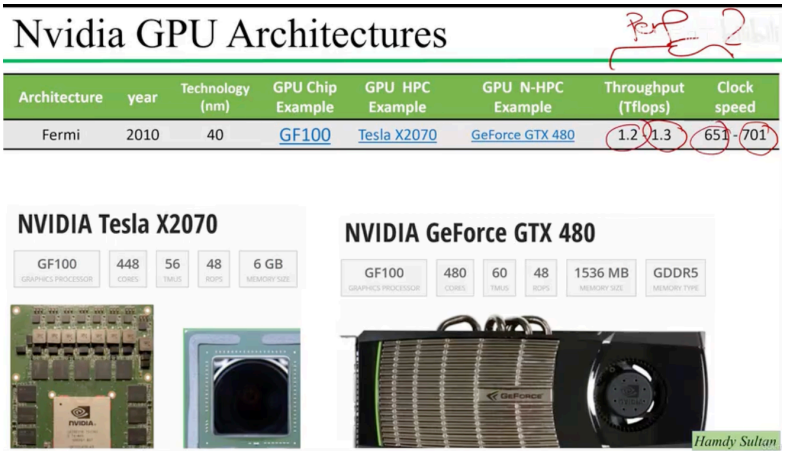


图 7.2: Fermi 架构示例

钟频率这样的单一数据，不足以全面衡量 GPU 的整体能力。此外，某一指标看似有利，在另一背景下可能恰恰相反。例如，更高的时钟频率虽然能加速任务，但也可能导致功耗增加，这对某些系统来说并不理想。

另外需要说明的是，GTX 480 在单精度 TFLOPS 上看似优于 X2070，但这仅针对单精度运算，不代表 GPU 中所有核心的整体性能。通过一个例子来说明：打开 TechPowerUp 网站对比这两款 GPU 时，你会注意到 X2070 的双精度性能约为 582.8 GFLOPS，远高于 GTX 480 的 168.1 GFLOPS——几乎相差四倍。这类高双精度任务的典型代表就是科学仿真。然而在纹理和像素填充率等依赖单精度运算的指标上，GTX 480 则优于 X2070，这也符合它面向游戏和日常应用的定位。

7.2 Kepler 架构

接下来是 Kepler 架构。在 Fermi 发布两年后，NVIDIA 推出了 Kepler。从时间线可以看出，NVIDIA 大致每一到两年就会发布一款新架构。Kepler 采用 28 纳米工艺制造，相比 Fermi 的 40 纳米实现了显著提升。

NVIDIA 的 GK210 芯片因应用于 Tesla K80 GPU 而闻名——这是一款 HPC GPU 加速卡。值得注意的是，Tesla K80 在一块板卡上集成了两颗

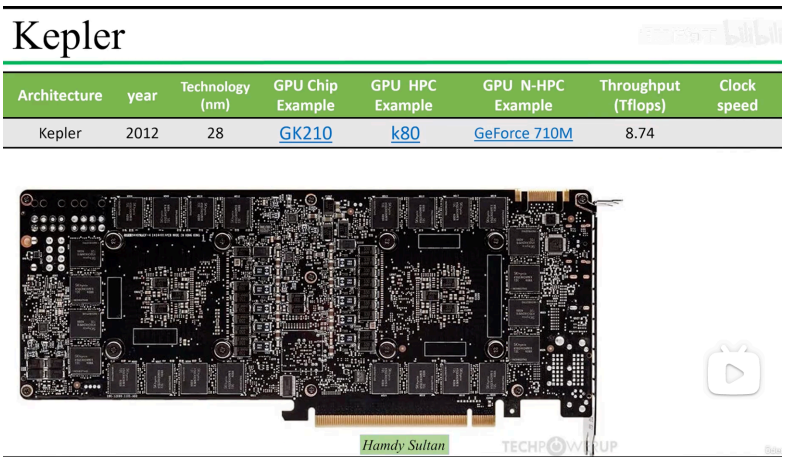


图 7.3: Kepler 架构

GK210 芯片，而非一颗。凭借双芯片设计，K80 实现了约 8.2 TFLOPS 的吞吐量，相比 Fermi 架构提升了约六倍。但需要注意的是，该数值是两颗芯片的总和，即每颗 GK210 芯片大约提供 4.1 TFLOPS。除 K80 之外，基于该芯片的其他知名 GPU 并不多，大多数面向消费市场的 Kepler GPU 均基于不同芯片。

关于 Kepler，还需提一下它的升级版本。有时 NVIDIA 会发布同一架构的增强版。例如，Kepler 最初于 2012 年推出，但在 2013 至 2015 年间出现了迭代版本。类似的迭代也发生在 Maxwell 架构上。当你在 TechPowerUp 网站上查看 GK210 芯片时，会发现其发布日期为 2014 年，与表格中的 2012 年不同——因为表格中的日期指的是该架构的最初版本。

7.3 Maxwell 架构

Maxwell 架构由 NVIDIA 于 2014 年推出，是 GPU 领域的一项重大进步，同样采用 28 纳米工艺制造。该架构的旗舰芯片之一是 GM200，被用于 Tesla M60 GPU（面向 HPC 设计）。对于游戏玩家而言，GeForce GTX 980 Ti 是另一款基于 Maxwell 架构的 GPU。在吞吐量方面，M60 达到 9.7 TFLOPS，而 GTX 980 Ti 为 6.1 TFLOPS。在时钟频率方面，M60 运行于

557 MHz，而 GTX 980 Ti 达到 1000 MHz。M60 的时钟频率较低，这直接与能耗相关——HPC GPU 更注重能效而非极限频率。

7.4 Pascal 架构

从 Pascal 架构开始，HPC GPU 卡采用了” 字母 + 100” 的命名模式。在 Pascal 架构中，GPU 名称为 P100（P 代表 Pascal），这一趋势延续到 Volta（V100）、Ampere（A100）和 Hopper（H100）。这种命名也体现在 GPU 芯片上：Pascal 有 GP100，随后的 Volta、Ampere 和 Hopper 分别有 GV100、GA100 和 GH100。Pascal 架构于 2016 年推出，采用 16 纳米和 14 纳米两种工艺制造。你可能会疑惑，为何同一架构会使用两种工艺? 原因在于当时不同制造商的技术水平不同——部分厂商已掌握更先进的 14 纳米工艺，而其他厂商仍使用 16 纳米。尽管存在工艺差异，各公司仍能基于相同的 Pascal 架构生产 GPU，仅在纳米尺寸上有所不同。

关于 Pascal 架构此处不做过多展开，后续章节将结合白皮书和数据手册详细讲解剩余架构。仅从性能提升就足以说明一切：对比 Fermi 的 1.2 TFLOPS 和 Hopper 的 378 TFLOPS，其间的飞跃一目了然（中间省略的 51 TFLOPS 将在 Hopper 架构章节中深入解释）。

8 不同架构间的核心对比参数

本章将深入探讨影响 GPU 性能的关键参数。理解这些内容将有助于我们后续阅读 GPU 白皮书和数据手册。

从前几章中，我们已知每一代新架构都在软件和硬件能力上带来了显著进步，在各类领域和应用中提供了更高的性能。一个清晰的例子就是从 Fermi 架构到 Hopper 架构的吞吐量增长，如下表所示，吞吐量增长超过了 300 倍。

如果你还记得前文关于 GPU 性能的重要提示，评估性能并非一目了然，它需要理解数十种指标和单位，而不仅仅是吞吐量或速度。那么，有

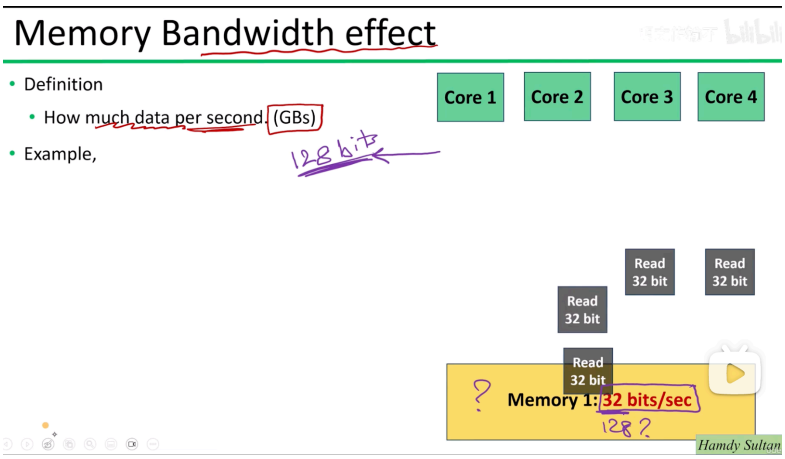


图 8.1: 带宽

哪些参数、单位和指标可用于衡量每一代新架构相较于前代的性能？例如，当 NVIDIA 在新架构中增加核心数量时，这是否足以提升架构性能，还是存在其他因素？为回答这个问题，让我们来看这些关键参数和指标——它们是理解 GPU 性能最关键的要素，但需注意，它们并非评估 GPU 性能的全部指标，只是最主要的几个。

8.1 内存带宽

本章的第一个参数是内存带宽。内存带宽指的是内存每秒能处理多少数据，以 GB/s（每秒千兆字节）来衡量。为了说明内存带宽对应用性能的影响，我们来讨论以下场景。假设我们有一块 GPU，其全局内存提供的带宽有限。再假设该 GPU 包含四个核心，而这块内存每秒只能提供 32 位数据。每个核心都需要从该内存读取 32 位数据，所需数据总量为 128 位。那么问题是：这块内存能否在一秒内处理全部四个请求？

不幸的是，答案是否定的。由于最大带宽为每秒 32 位，该内存一次只能处理一个请求。它在第一秒处理第一个请求（前 32 位），完成后再继续下一个请求，依此类推。这种串行处理意味着内存需要总共 4 秒才能响应所有读取请求。

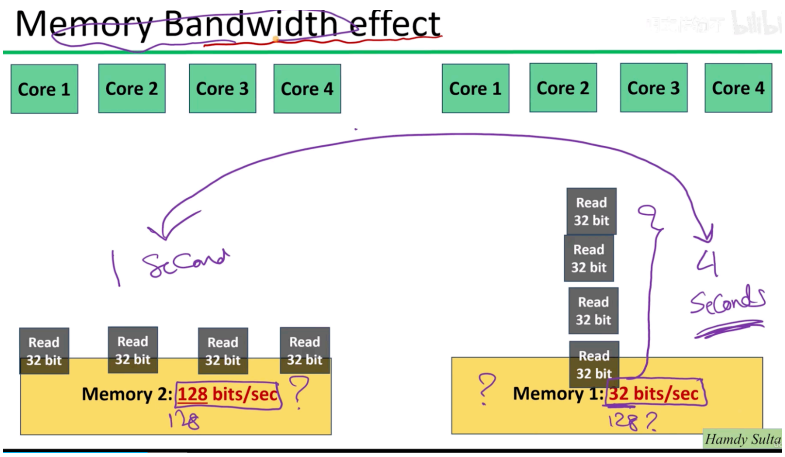


图 8.2: 带宽

现在我们将其与另一种带宽更高的内存进行比较。假设我们有一块带宽为每秒 128 位的内存，同样有来自四个核心的四个读取请求。该内存的更高带宽可以在一秒内传输 128 位，这意味着它能在一秒内响应全部四个请求。这充分说明了内存带宽在提升 GPU 性能中的关键作用——相比第一块内存，高带宽显著减少了读写操作所需的时间。

从实际角度来看，考虑视频编辑或三维渲染这类带宽密集型应用，更高的内存带宽会使这些应用或任务效率显著提升。在游戏领域，每一毫秒都至关重要，更高的内存带宽意味着更流畅的帧率和更沉浸的体验，同时也有助于减少延迟。总结一下，更高的带宽使 GPU 能一次处理更多数据，从而提升性能。

你可能会疑惑，为什么这里的带宽会与内存速度和总线宽度并列？答案是这两个因素会直接影响内存带宽。为说明这一点，让我们深入看看总线宽度对内存带宽的影响。

8.2 总线宽度

计算机架构中总线宽度的概念可以类比为道路上的车道数量。如果我们把每份数据看作一辆汽车，那么总线宽度就是这些汽车可通行的车道数

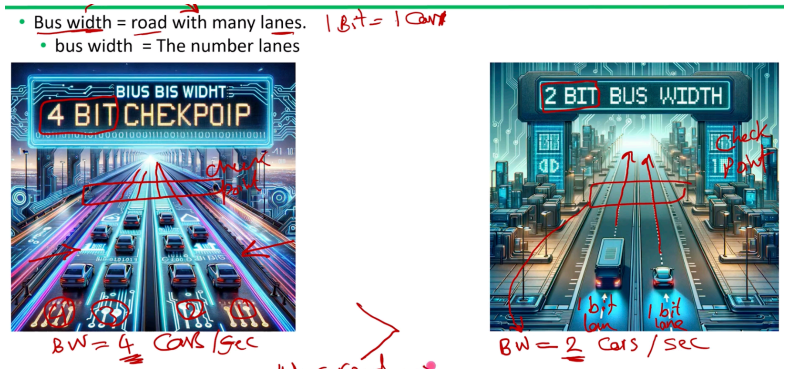


图 8.3: 带宽

量。考虑两种不同场景：第一种场景假设我们有 2 位总线宽度，相当于一条双车道的道路，因此一次能通过检查点的汽车数量是两辆。如果每条车道的限速是每秒一辆车，那么在道路尽头的检查点，一秒内总共能通过的汽车数量就是两辆——因为有两条车道，每条车道每秒一辆车。

这条路可让两辆车同时通过检查点，该检查点的带宽是每秒两辆车。第二个例子，假设我们有 4 位总线宽度，即一条四车道道路。采用相同的限速（每秒一辆车），这条路每秒能处理四辆车。第一条路（2 位总线）的带宽是每秒两辆车，第二条路（4 位总线）的带宽是每秒四辆车。总结这两个例子：更宽的总线宽度带来更高的带宽。

现在让我们回到计算机术语，应用同样的类比：将总线宽度翻倍，例如从 32 位提升到 64 位，会增加在给定时间内可传输的数据量。但这仅在内存速度相同的情况下才成立。在同样语境下，内存速度通常称为内存时钟频率，也是 GPU 性能的关键因素。更高的内存速度使 GPU 能更快地访问和处理数据，从而减少瓶颈。关键不仅在于可用内存有多少，更在于内存运行得多快，这对数据密集型任务的整体效率至关重要。

在之前的类比中，假设每条车道由一名警官管理。检查点的效率不仅取决于道路宽度（车道数量），还关键取决于每名警官检查一辆车的速度有多快。如果每名警官工作效率更高，能更快地检查每辆车，自然就能在给定时间内让更多车辆通过检查点。

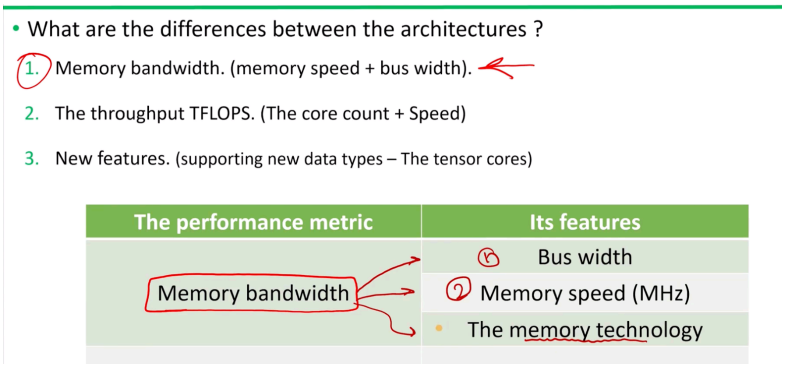


图 8.4: 带宽

这个类比直接对应 GPU 中内存的工作方式。如果内存具备更高的速度，就能在特定时间范围内处理更多数据、事务或操作，从而有效提升内存带宽。正如检查点上效率更高的警官能带来更顺畅、更高效的车流，GPU 中更高的内存速度也会带来更大的内存带宽。这种增强的带宽对 GPU 整体性能至关重要，因为它决定了数据从内存传输到计算单元（核心）的速度有多快。

我们在此考察的指标是内存带宽。现在来总结一下：该指标会受到不同因素的影响，例如内存速度、内存总线宽度以及内存技术（DDR4、DDR5、DDR6，甚至更新的 HBM 等）。现在让我们在两款属于不同类别的 GPU 上检验这一指标及其影响因素。这两款 GPU 分别是来自 Tesla 系列的 A100 和来自 GeForce 系列的 RTX 3090，如果你还记得我们之前的讨论，Tesla 系列专用于高性能计算，而 GeForce 则面向游戏和普通用户。需要再次指出的是，这两款 GPU 都基于 Ampere 架构，因此它们共享相同的架构，但面向不同的应用场景。

在高性能计算 A100 GPU 中，我们观察到其内存总线宽度异常宽——超过 5000 位，即 5000 条车道。为便于理解，我们将此数值与 RTX 3090 的总线宽度进行比较。尽管共享相同架构，RTX 3090 GPU 的总线宽度却只有约 400 位。你看到差别了吗？A100 是 5000 位，而 RTX 3090 只有 400 位，两者差距超过十倍。此外，当考察内存速度时，它们大致相当。但当我

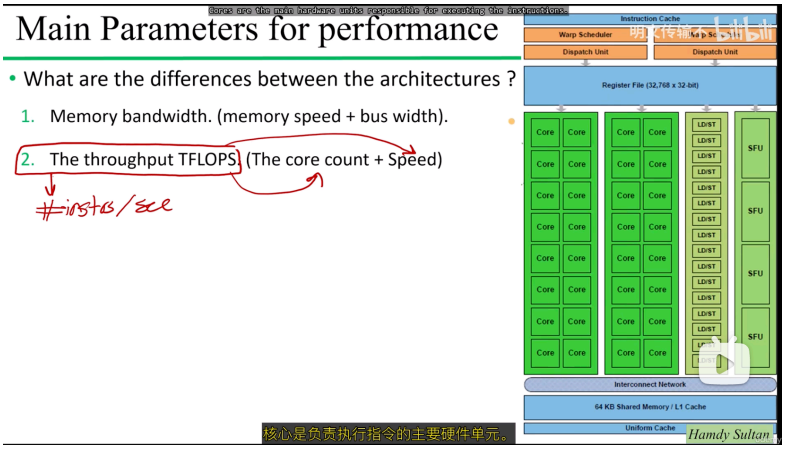


图 8.5: 带宽

们综合这些因素——内存速度、总线宽度以及内存技术——就会发现 A100 GPU 在内存带宽方面显著优于 RTX 3090。

在 A100 中，我们使用了名为 HBM2 的内存技术，而在 RTX 3090 中则使用了另一种名为 GDDR6 的技术。在介绍下一个参数之前，先思考一个问题：你认为 GPU 中包含的核心数量是否直接影响其性能？来看一个例子，假设需要比较两款 GPU，第一款有 100 个核心，第二款有 200 个核心。我们能否说第二款性能更好？

8.3 指令吞吐量

我将通过讲解第二个参数——指令吞吐量——来回答这个问题。在此上下文中，吞吐量指的是整个 GPU 每秒执行的指令总数。该指标直接受两个因素影响：核心数量和核心速度，是评估和比较不同 GPU 型号的关键度量。

为更深入理解这一概念，让我们考虑 GPU 中核心的作用。核心是负责执行指令的主要硬件单元，更多的核心允许同时执行更多指令，从而带来更高的吞吐量。为具体说明，假设有两款不同的 GPU，一款配备 100 个核心，另一款配备 200 个核心。再假设有一个包含 200 条指令的应用程序，当

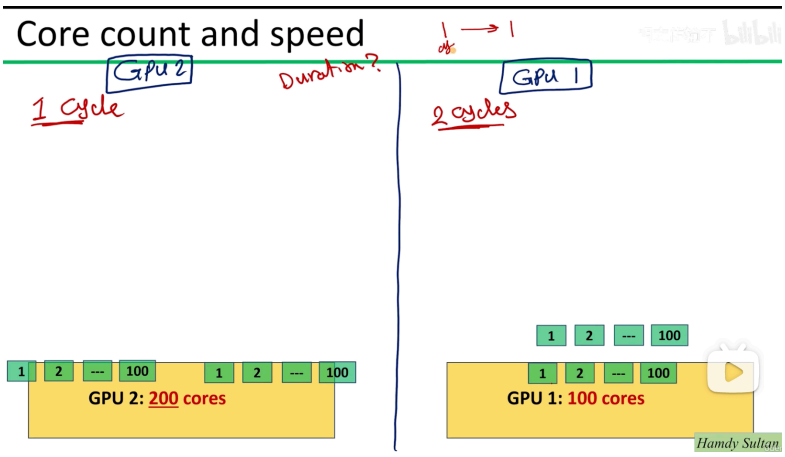


图 8.6: 带宽

该应用程序在两款 GPU 上运行时，第一款 GPU 将为每个核心分配一条指令，在第一个周期中处理 100 条指令，然后将剩下的 100 条指令再次分配给各核心。

在第二款 GPU 的场景中，200 条指令将在一个周期内执行完毕，因为我们有 200 个核心，每个核心接收一条指令。这使得 GPU 仅用一个周期而非两个就能完成整个应用程序，性能相较第一款 GPU 提升了一倍。更高的核心数量意味着更强的并行处理能力，从而实现更快、更高效的任务执行，尤其适用于可有效并行化的应用程序。这个例子突显了核心数量对 GPU 吞吐量的重大影响，但这并非绝对真理——在某些情况下，第一款 GPU（核心更少）的性能可能优于第二款 GPU。

让我们回顾这个例子：第一款 GPU 需要两个周期来执行整个应用程序，而第二款 GPU 只需一个周期。然而，如果我们不知道两款 GPU 中每个周期的持续时间，我们的比较就缺乏上下文。为说明这一点，再考虑一个假设：如果第一款 GPU 每个周期耗时 1 秒，而第二款 GPU 每个周期耗时 4 秒呢？在这种情况下，第一款 GPU 将在 2 秒内完成，而第二款 GPU 需要 4 秒。

这一场景凸显了 GPU 架构中的一个关键方面：核心数量与核心时钟频

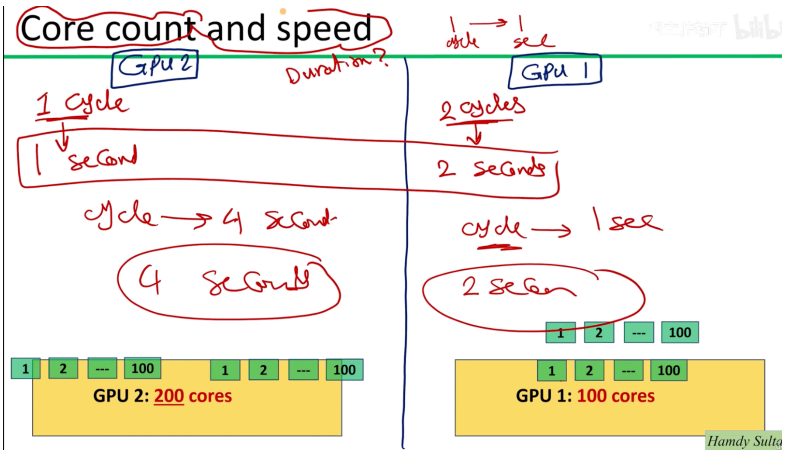


图 8.7: 带宽

率之间的平衡。时钟频率——即每个核心执行指令的速率——同样至关重要。核心数量确实影响同时处理多条指令的能力，这是 GPU 并行处理的基本原则。但更高的时钟频率意味着每个核心能更快完成分配的任务，这在需要快速处理复杂小任务的应用中尤为关键。我们可以通过吞吐量这一指标——即单位时间内可执行的指令数量（如每秒）——来衡量它们对性能的综合影响。

总结一下，核心数量加上核心速度是重要的硬件特性。在查阅白皮书或数据手册时，我们可以直接关注吞吐量指标，而不必深入考察核心数量。它们固然重要，但关注吞吐量可以绕过对核心数量和时钟频率的逐一分析。不过时钟频率始终对能耗有重要影响。

8.4 能耗权衡

让我们探讨另一个关键问题——这一点非常重要。假设我们有两款 GPU，第一款 GPU 不仅核心数量更多、核心频率更高，其运行速度也快于第二款。第一款 GPU 在核心数量和速度上都强于第二款，这是否自动使其成为更优选择？根据之前的讨论，你可能会初步认为：更多的核心、更高的速度，还需要什么呢？

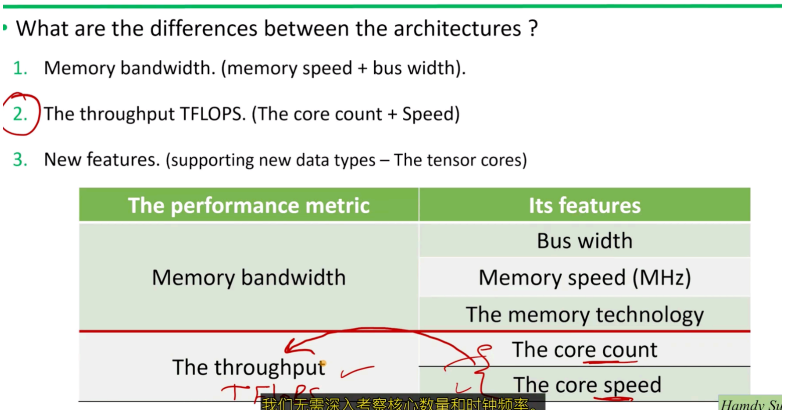


图 8.8: 带宽

然而，还有另一个关键因素需要考虑——能耗。更高的速度加上更多的核心数量会导致更高的能耗。如果一家公司优先考虑能效（这通常与节省成本相关），它可能会选择第二款 GPU，尽管其规格较低。这一场景体现了一种权衡：如果你想要更高的计算能力和速度，就必须承担更高的能耗成本，反之亦然。如果我问你哪款 GPU 更优越，你的回答应该是一个反问——在什么上下文中？你是考虑能效还是计算能力？

评估 GPU 性能时，最后一个要考虑的参数是新架构中引入的新特性或硬件单元。

8.5 新特性

有时一个新单元能显著增强 GPU 的计算能力。例如，NVIDIA 在 2017 年推出的 Volta 架构中加入了张量核心（Tensor Core）。请看这张图，它展示了从 Pascal 到 Volta 的性能飞跃——同一应用的性能大约提升了八倍。请想象一个复杂的人工智能应用，通常需要消耗八天的计算资源；而在 Volta 架构中，借助张量核心，同样的任务可能只需一天即可完成。这个例子突显了张量核心作为硬件单元在 GPU 计算领域中的革命性影响。顺便一提，本课程中有专门章节讲解张量核心及其使用方法。

本章最后，让我们来看一下 NVIDIA 所有白皮书和数据手册中都存在

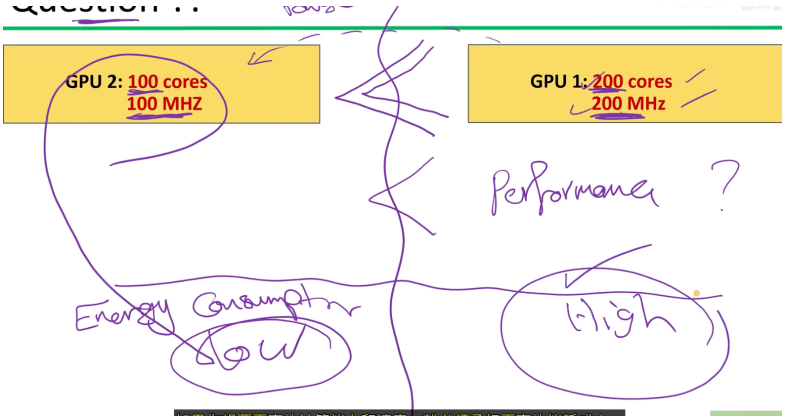


图 8.9: 带宽

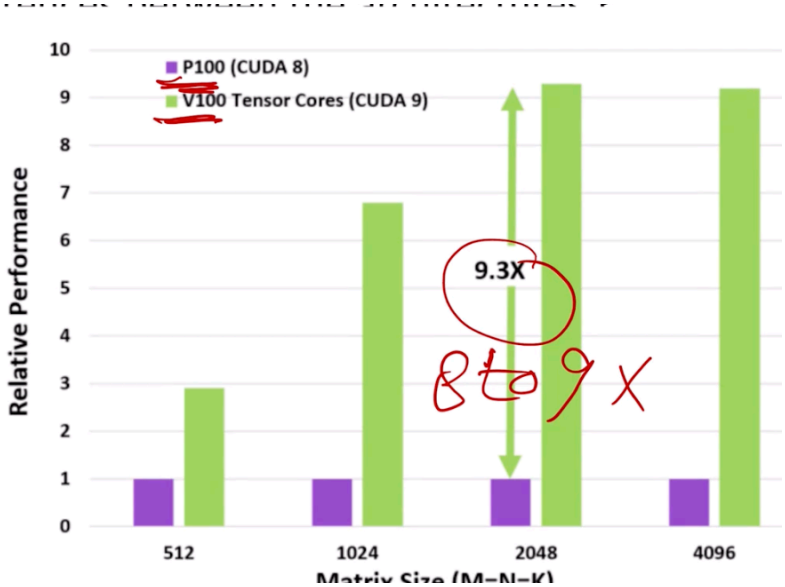
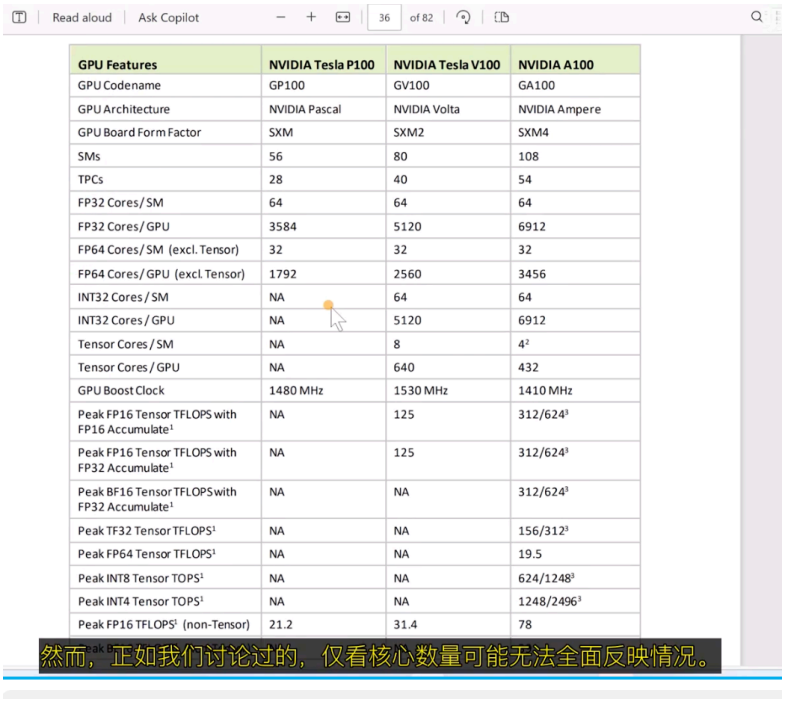


图 8.10: 带宽



GPU Features	NVIDIA Tesla P100	NVIDIA Tesla V100	NVIDIA A100
GPU Codename	GP100	GV100	GA100
GPU Architecture	NVIDIA Pascal	NVIDIA Volta	NVIDIA Ampere
GPU Board Form Factor	SXM	SXM2	SXM4
SMs	56	80	108
TPCs	28	40	54
FP32 Cores / SM	64	64	64
FP32 Cores / GPU	3584	5120	6912
FP64 Cores / SM (excl. Tensor)	32	32	32
FP64 Cores / GPU (excl. Tensor)	1792	2560	3456
INT32 Cores / SM	NA	64	64
INT32 Cores / GPU	NA	5120	6912
Tensor Cores / SM	NA	8	4[2]
Tensor Cores / GPU	NA	640	432
GPU Boost Clock	1480 MHz	1530 MHz	1410 MHz
Peak FP16 Tensor TFLOPS with FP16 Accumulate[1]	NA	125	312/624[3]
Peak FP16 Tensor TFLOPS with FP32 Accumulate[1]	NA	125	312/624[3]
Peak BF16 Tensor TFLOPS with FP32 Accumulate[1]	NA	NA	312/624[3]
Peak TF32 Tensor TFLOPS[1]	NA	NA	156/312[3]
Peak FP64 Tensor TFLOPS[1]	NA	NA	19.5
Peak INT8 Tensor TOPS[1]	NA	NA	624/1248[3]
Peak INT4 Tensor TOPS[1]	NA	NA	1248/2496[3]
Peak FP16 TFLOPS[1] (non-Tensor)	21.2	31.4	78

图 8.11: 性能数据

的一张表格。在这张表格中你会注意到列出了大量特性和指标——这些行提供了关于各种数据类型（从浮点到整数运算）的核心数量信息。然而，正如我们讨论过的，仅看核心数量可能无法全面反映情况：理解核心速度与核心数量同样重要。或者我们可以直接关注指令吞吐量来简化分析，该指标以 TFLOPS（每秒万亿次浮点运算，即 10^{12} 次浮点运算/秒）来衡量。在接下来的章节中，我们将基于本章的讨论，深入探讨最新 GPU 的规格，这些规格来自白皮书和数据手册中的架构信息。

9 半精度、单精度与双精度运算

本章将介绍 GPU 中不同的数据类型及其大小。在开始前需要说明一点：理解数据类型和大小，就像在工具箱中选择合适的工具一样，是高效

且准确完成任务的基础。在 GPU 计算领域，与 CPU 类似，我们会根据需求采用不同的方式来存储数值。先从整数开始：对于整数，我们可以使用 8 位、16 位、32 位甚至 64 位，位数越多能存储的数值就越大。例如，一个 8 位无符号整数可以存储 0 到 255 之间的值；如果是有符号变量，范围则为 -128 到 127。

9.1 整数与浮点类型

对于 64 位整数，可以存储大得多的数值，最高约 1.8×10^{19}（约 1800 亿亿）。当处理带小数点的数字时，则会使用浮点表示法：16 位为半精度，32 位为单精度，64 位为双精度。

在 C 或 CUDA 编程语言中，用 int 声明变量使用 32 位；将 int 替换为 long long，则通常使用 64 位。类似地，用 float 声明变量使用 32 位，替换为 double 则使用 64 位。

9.2 半精度、单精度与双精度的差异

半精度（FP16）是一种二进制浮点格式，在内存中占用 16 位（2 个字节），数值精度较低，但所需的计算资源更少，适用于机器学习和图形处理等对精度要求不高的应用。

单精度（FP32）在内存中占用 32 位，在速度和精度之间取得了很好的折衷，因此成为各类计算任务的首选。

双精度（FP64）则被视为精度上的强劲引擎。在科学计算和复杂仿真中，每一个小数点都可能引发蝴蝶效应，因此这类应用依赖 FP64 的高精度来确保对气候模式、分子结构等现象的仿真尽可能接近现实。不同的精度层级使 GPU 能够灵活适应各种行业和应用场景。

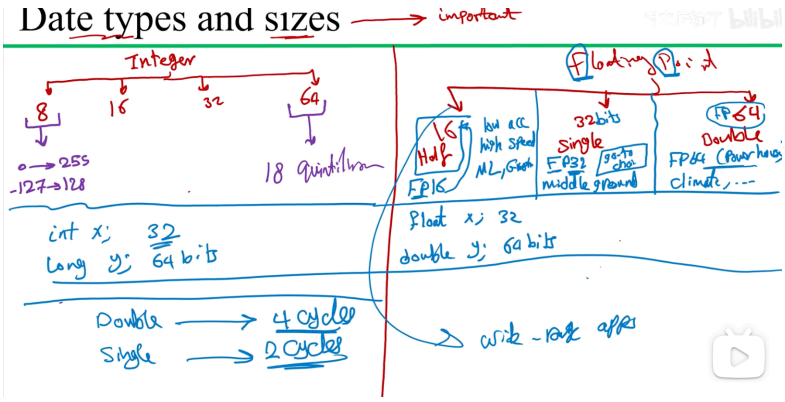


图 9.1: 数据类型

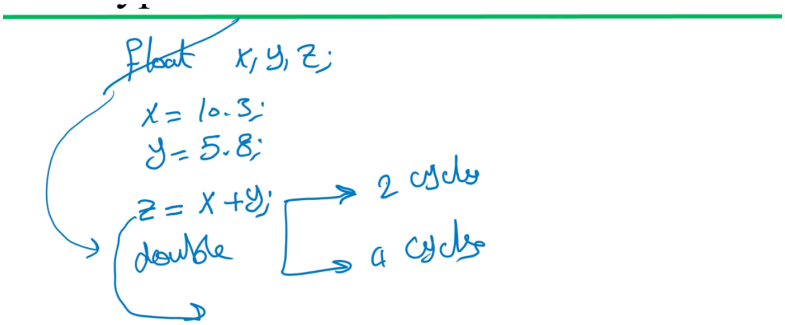


图 9.2: 不同计算类型所需时钟周期

9.3 PI 的精度对比

研究显示，双精度计算需要四个执行周期，而单精度仅需两个周期——双精度虽然更慢，但提供了更高的精度。例如，声明 float 变量 X、Y、Z，设 X = 10.3，Y = 5.8，执行 Z = X + Y，该加法操作仅需两个周期；如果将类型替换为 double，同样的加法操作则需要四个周期。这种差异源于双精度数据类型更高的精度和处理需求。

为了更清楚地说明精度的差异，以圆周率 在不同浮点格式下的表示为例：在半精度中， 仅保留约 2 位小数；在单精度中增加到约 7 位；在双精度中则进一步扩展到约 15 位。此外，对于极小的数值（如接近零的值），

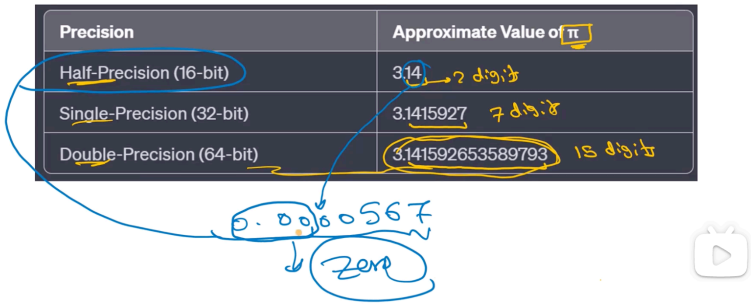


图 9.3: PI 的精度

由于半精度最多只支持 2 到 3 位有效小数，这类数值可能被近似为零而无法准确表示。

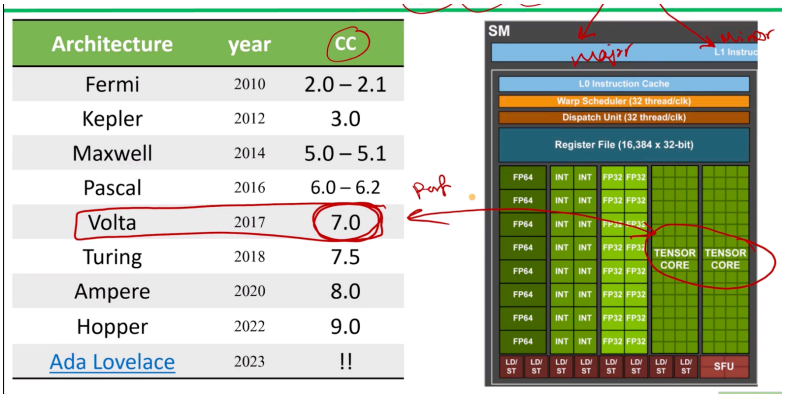
10 GPU 计算能力

本章将探讨 GPU 领域中的一个关键术语——计算能力（Compute Capability，简称 CC 编号）。它是 NVIDIA 对 GPU 架构采用的一种设计分类，表明 GPU 的通用特性和处理能力。为了更深入地理解这一点，我们先从这个问题开始：

计算能力是如何编号或赋予版本号的?

10.1 计算能力编号

计算能力以版本号的形式表示，例如 3.0、5.1 或 7.5。小数点前的第一个数字通常表示重大的架构变更，而小数点后的第二个数字代表较小的改进或扩展。每个版本对应特定一代的 GPU 及其所支持的功能。以 Volta 架构为例，其计算能力被标记为 7。Volta 因引入了张量核心（Tensor Core）而备受瞩目，大幅提升了特定类型计算的性能，尤其是在人工智能和深度学习领域。这一数字标识反映了该架构在发布时所引入的一系列前沿特性、



Architecture	year	CC
Fermi	2010	2.0 – 2.1
Kepler	2012	3.0
Maxwell	2014	5.0 – 5.1
Pascal	2016	6.0 – 6.2
Volta	2017	7.0
Turing	2018	7.5
Ampere	2020	8.0
Hopper	2022	9.0
Ada Lovelace	2023	!!

图 10.1: CC 版本

能力与改进。

转向 Ampere 架构，其计算能力版本为 8.0。这一提升表明与 Volta 相比，该架构在硬件和软件方面均实现了显著增强：包括更高效、更强大的张量核心，更大的内存带宽，更高的能效等。这些增强不仅提升了性能，还提高了 GPU 在更广泛应用场景中的效率与通用性。CC 编号这一概念适用于所有其他架构。

10.2 功能支持

接下来是本章的第二点——功能支持。不同版本的计算能力支持不同的功能，包括 GPU 能执行的数学运算类型、可处理的并行程度、内存访问能力等等。例如，当你访问 CUDA 编程文档时会发现许多关于 GPU 编程的章节，其中一节专门介绍计算能力。

我将聚焦于该节中的一个特定表格，它包含了关键信息。在表 17 中，你可以看到一个结构化的布局——不同计算能力版本列于各列，对应的功能列于各行。该表格充当对比图表，使开发者能够快速查看特定计算能力的 GPU 支持哪些功能。以计算能力 5.0 为例，从上到下逐行扫描会发现其中混杂着可用与不可用的功能：具备该计算能力的 GPU 不支持半精度运算，如表所示，在代表计算能力低于 7 的各列中，该项标记为 No。张量核

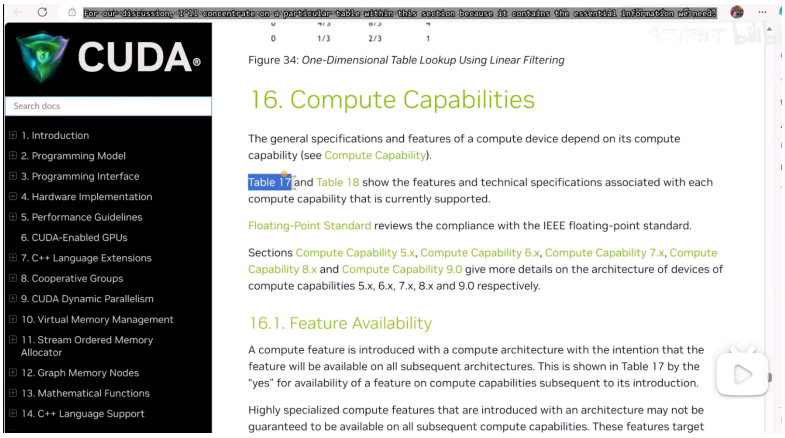


图 10.2: cuda 编程手册

心是另一类重要单元，仅从计算能力 7 及以上版本才开始提供（对应列中标记为 Yes）。表 18 也体现了相同的概念，留待读者自行查阅。

10.3 软件兼容性

更高的计算能力版本能够兼容更新的 CUDA 工具包，而新版本工具包又支持更先进的计算功能和更优的性能优化。换言之，GPU 的计算能力决定了它可运行的 CUDA 工具包的版本。为了说明这一概念，以 PTX 兼容性为例——为简化理解，可将 PTX 视作一种 GPU 汇编语言。部分 PTX 指令只能在特定计算能力版本的设备上执行，因为它们依赖于仅存在于特定计算能力 GPU 上的硬件单元或功能。

通过链接进入 CUDA C 编程指南，你会发现文档明确指出了某些 PTX 指令对计算能力的版本要求，这些操作或函数仅在计算能力高于 5 的设备上可用。例如 Warp Shuffle 函数，它能在 Warp 内的线程之间共享数据，而无需使用共享内存或全局内存。

如果还不够清楚，我们再来看另一个例子。假设你正在从事一个需要最新机器学习技术的项目，你的 GPU 的计算能力将决定你能否使用最新的 CUDA 工具包——该工具包包含最新的功能和性能优化，可能正是你的

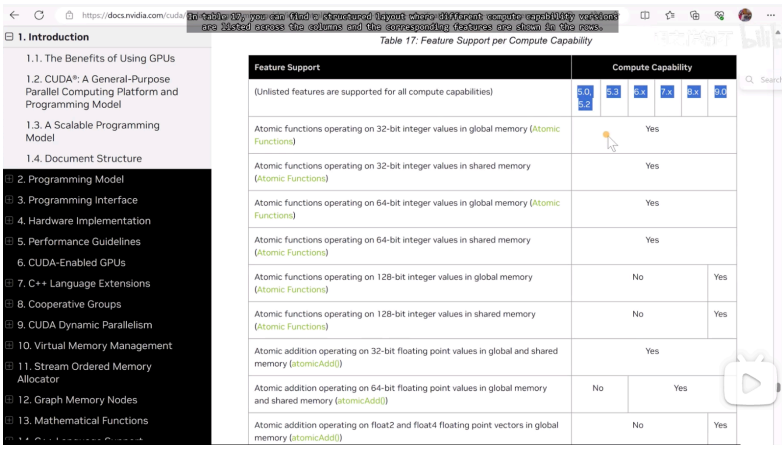


图 10.3: cuda 编程手册

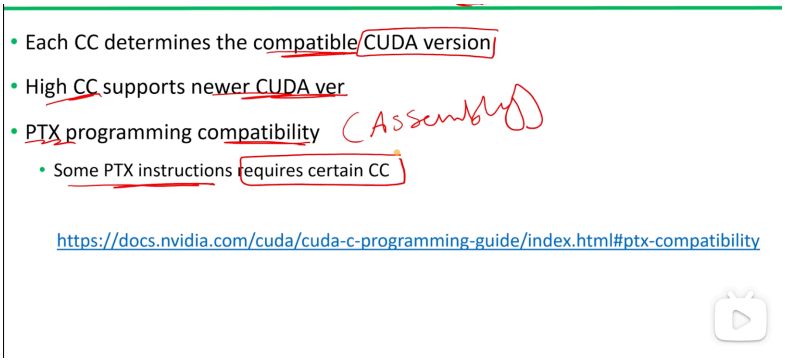


图 10.4: cuda 编程手册

Hardware Generation	Compute Capability	CTK Support
Hopper	9.x	11.8 - current
NVIDIA Ampere GPU Arch.	8.x	11.0 - current
Turing	7.5	10.0 - current
Volta	7.x	9.0 - current
Pascal	6.x	8.0 - current
Maxwell	5.x	6.5 - current

图 10.5: cuda 版本

应用所需的。

再比如说，如果你使用的是 Hopper 架构的 GPU（目前全球最新一代 GPU 之一），那么对应的计算能力为 9.x。如本表所示，这不仅仅是对前几代产品的小幅升级，意味着 Hopper GPU 已准备好处理最先进的计算任务。它对 CUDA 工具包的最低版本要求为 11.8，因此在 Hopper 架构中无法使用低于 11.8 的 CUDA 版本。类似地，Maxwell 架构的计算能力编号为 5.x，其最低 CUDA 工具包版本要求为 6.5——如果版本不兼容，你会收到错误提示。

如本表所示，计算能力与所支持的 CUDA 版本之间的关系是理解如何使硬件选择与软件需求相匹配的关键。无论你是开发者、研究人员，还是依赖 GPU 进行高强度任务的用户，了解这一关系都有助于确保项目运行顺畅、高效并避免意外问题。最后，你可以在此处展示的网页上发现专为不同应用场景设计的各类 GPU，每一款 GPU 都附带其对应的计算能力，详情见各表格。

Architecture	year	CC	GPU Chip Example	GPU HPC Example	GPU N-HPC Example	Throughput (Tflops)	Clock speed
Fermi	2010	2.0 – 2.1	GF100	Tesla X2070	GeForce GTX 480	1.2 - 1.3	651 - 701
Kepler	2012	3.0	GK210	k80	----------	8.2	562
Maxwell	2014	5.0 – 5.1	GM200	M60	GeForce GTX 980 Ti	9.7 – 6.1	557 - 1000
Pascal	2016	6.0 – 6.2	GP100	P100			
Volta	2017	7.0	GV100	V100			
Turing	2018	7.5					
Ampere	2020	8	GA100	A100	RTX 30XX		
Hopper	2022	8.6	GH100	H100	----------		
Ada Lovelace	2023	!!	AD107	L4	RTX 40XX		

图 11.1: GPU 架构

11 如何阅读 GPU 技术白皮书

白皮书本质上是 GPU 架构的深入技术规格说明书，在比较不同 GPU 时具有不可估量的价值，因为它们提供了对每个架构规格和能力的详细洞察。回到下面的表格，你会注意到针对每个架构，我们都列出了一个基于该架构的芯片示例。

11.1 搜索白皮书

要获取任何架构的白皮书，只需一次简单的互联网搜索：打开浏览器，输入芯片名称后加上” 白皮书”，芯片名称可从上述表格中获取。例如，搜索表格中的芯片名称 GF100 加上白皮书，回车后即可看到一系列搜索结果。

寻找指向官方白皮书的 PDF 链接——该链接应指向一份包含完整技术规格的 PDF 文档。同样地，如果你在研究 Ampere 架构，搜索 GA100 白皮书即可。注意不要被文章或摘要所误导，你真正需要的是原始的全篇白皮书。

英伟达有一个很有帮助的做法：在其白皮书中将新架构与旧架构进行比较。例如，A100（Ampere）的白皮书会与其前任 Volta（V100 ）和 Pascal（P100）进行性能对比。

你可能会在 A100 和 H100 的白皮书中看到展示不同单元和指标的同

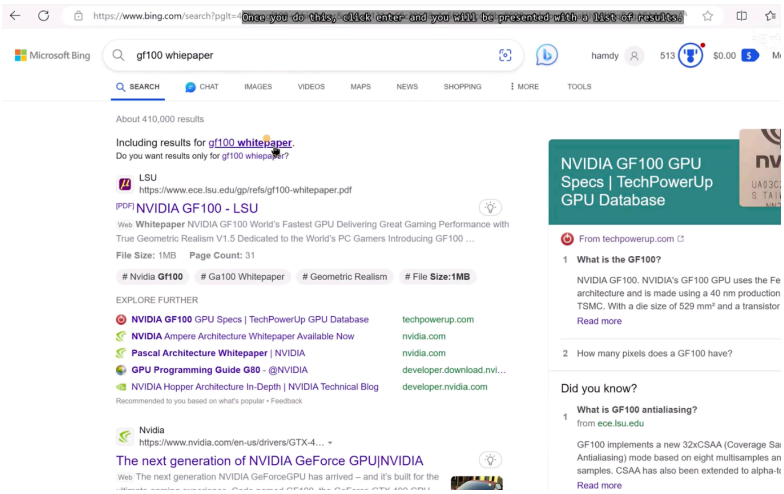


图 11.2: 搜索白皮书

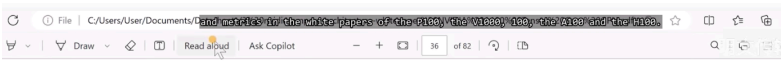


Table 4. Comparison of NVIDIA Data Center GPUs

GPU Features	NVIDIA Tesla P100	NVIDIA Tesla V100	NVIDIA A100
GPU Codename	GP100	GV100	GA100
GPU Architecture	NVIDIA Pascal	NVIDIA Volta	NVIDIA Ampere
GPU Board Form Factor	SXM	SXM2	SXM4
SMs	56	80	108
TPCs	28	40	54
FP32 Cores / SM	64	64	64
FP32 Cores / GPU	3584	5120	6912
FP64 Cores / SM (excl. Tensor)	32	32	32
FP64 Cores / GPU (excl. Tensor)	1792	2560	3456
INT32 Cores / SM	NA	64	64
INT32 Cores / GPU	NA	5120	6912
Tensor Cores / SM	NA	8	4^2

图 11.3: 架构比较

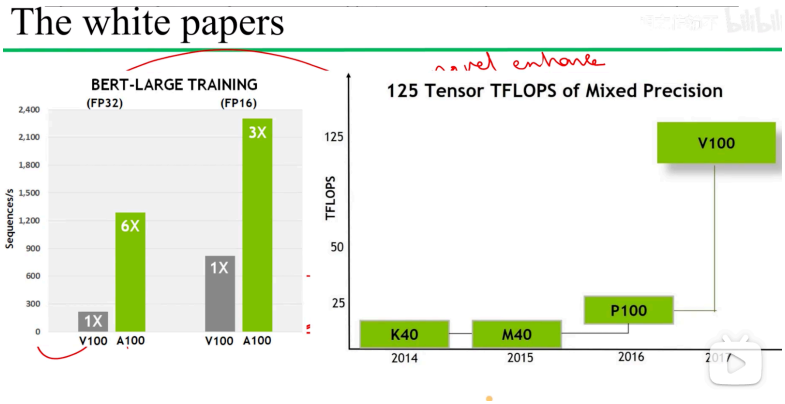


图 11.4: 架构比较

一张表格。所有白皮书中的这种一致性意味着一旦你理解了如何浏览并解读一份英伟达白皮书，也就具备了理解其他白皮书的能力。

11.2 白皮书的结构

英伟达的白皮书通常遵循标准化的结构，按顺序包含以下四个要点：第一，新技术与特性的介绍；第二，流式多处理器（SM）的内部组件和结构；第三，性能基准测试与比较；第四，技术规格。

第一点——新特性的介绍：每份白皮书都以突出展示新技术和增强功能的部分开头，内容涵盖从人工智能专用核心（如张量核心）到改进的图形着色技术，乃至软件或硬件的任何增强。

第二点——SM 架构分析：所有白皮书都会深入探讨新型 SM 的内部结构，正如你在不同白皮书的对比图中所见，每个架构都会与其前任进行比较。第四点——技术规格：每份白皮书都包含详细规格，如核心数量、内存大小、内存带宽、内存技术、功耗要求等，全面展示 GPU 的能力。

11.3 SM 内部结构

我们将重点关注第二点——即 SM 的内部结构。英伟达的白皮书深入剖析了每个 GPU 的 SM 设计，其后通常是前文讨论过的通用对比表。例如，查看 P100（Pascal 架构，GP100 芯片）的白皮书，你会看到一张展示整个 GPU 的图片，然后是 SM 的详细图表及其内部组件，如 CUDA 核心和内存层级结构。转向 V100（Volta 架构）白皮书时会发现类似的布局，然而一个显著的增强是增加了张量核心单元——Volta 架构中包含了张量核心，而 Pascal 架构中则没有。

接下来看 A100（Ampere 架构）的白皮书。同样，这份白皮书展示了整个 GPU，然后是 SM 内部结构的详细信息。对比表被移到了另一个部分，但仍在白皮书中。Ampere 架构中 SM 的内部结构与 Volta 非常相似——比较两者图片即可发现，差异并不显著。

查看 H100（Hopper 架构）白皮书时，这种一致性依然存在：先展示整个 GPU，然后是 SM 内部结构的详细信息。Hopper 与前代

架构的主要区别在于每个 SM 中可用的核心数量。例如，可以数一下 Hopper 中有多少个 32 位浮点核心，再与 A100 比较，即可看出差异。

总结而言，虽然每份白皮书都针对其特定的 GPU，但所有白皮书提供的结构和信息类型都遵循相似的模式。这种延续性和一致性不仅使专业人员更容易跟上最新技术，而且有助于清晰地理解技术进步，以及每个新架构相较于其前任的定位。

12 Volta 架构白皮书阅读

前几章中，我们已详细探讨了英伟达白皮书中常见的关键参数。在深入探讨 Volta 架构之前，回顾前几章的内容非常重要——它们奠定了理解本章主题所必需的基础知识。熟悉这些参数至关重要，因为我们将在本章中引用并验证它们。

现在就来看看 Volta 架构的规格。打开 Volta 白皮书（即 V100 白皮

图 11.5: Volta 架构

书），你可以在开头看到目录。在目录的第二页，你会找到一个名为” 关键特性” 的部分。这不仅仅是一系列改进的列表，更是 V100 相比前代架构重大增强的全面概述。

点击” 关键特性” 标题后，我们将看到定义 V100 GPU 即 Volta 架构的核心特性，从新的 SM 设计到第二代 NVLink 技术等。本章将逐一讨论所有这些增强功能。回到目录，这份白皮书的一个突出方面是专门聚焦于人工智能和深度学习。考虑到 Volta 架构首次引入了张量核心（Tensor Core），这种对 AI 和深度学习的聚焦是十分自然的。

12.1 Volta 关键特性

张量核心是专门设计用来执行矩阵运算的计算单元，这对神经网络的训练和推理阶段至关重要，也是 Volta 架构的重大突破。请注意目录中“深入硬件架构”这个关键标题。这一标题并非 Volta 白皮书独有，它在英伟达各款白皮书中反复出现。例如，Pascal 白皮书和最近的 Hopper (H100) 白皮书目录中都有相同的术语。这提醒了我们之前讨论过的重要一点：尽管

KEY FEATURES

Following are some key compute features of Tesla V100:

- **New Streaming Multiprocessor (SM) Architecture Optimized for Deep Learning**
  Volta features a major new redesign of the SM processor architecture that is at the center of the GPU. The new Volta SM is 50% more energy efficient than the previous generation Pascal design, enabling major boosts in FP32 and FP64 performance in the same power envelope. New Tensor Cores designed specifically for deep learning deliver up to 12x higher peak TFLOPS for training and 6x higher peak TFLOPS for inference. With independent parallel integer and floating-point data paths, the Volta SM is also much more efficient on workloads with a mix of computation and addressing calculations. Volta's new independent thread scheduling capability enables finer-grain synchronization and cooperation between parallel threads. Finally, a new combined L1 data cache and shared memory unit significantly improves performance while also simplifying programming.

图 12.1: Volta 架构关键特性

- **Second-Generation NVIDIA NVLink™**
  The second generation of NVIDIA's NVLink high-speed interconnect delivers higher bandwidth, more links, and improved scalability for multi-GPU and multi-GPU/CPU system configurations. Volta GV100 supports up to six NVLink links and total bandwidth of 300 GB/sec, compared to four NVLink links and 160 GB/s total bandwidth on GP100. NVLink now supports CPU mastering and cache coherence capabilities with IBM Power 9 CPU-based servers. The new NVIDIA DGX-1 with V100 AI supercomputer uses NVLink to deliver greater scalability for ultra-fast deep learning training.
- **HBM2 Memory: Faster, Higher Efficiency**
  Volta's highly tuned 16 GB HBM2 memory subsystem delivers 900 GB/sec peak memory bandwidth. The combination of both a new generation HBM2 memory from Samsung, and a new generation memory controller in Volta, provides 1.5x delivered memory bandwidth versus Pascal GP100, with up to 95% memory bandwidth utilization running many workloads.
- **Volta Multi-Process Service**
  Volta Multi-Process Service (MPS) is a new feature of the Volta GV100 architecture providing hardware acceleration of critical components of the CUDA MPS server, enabling improved performance, isolation, and better quality of service (QoS) for multiple compute applications sharing the GPU. Volta MPS also triples the maximum number of MPS clients from 16 on Pascal to 48 on Volta.

图 12.2: Volta 架构关键特性

Table 4.1: Latency of frequently used instructions on Volta and Pascal.

Architecture	Instructions	Latency (cycles)
Pascal	BFE, BFI, IADD, IADD32I, FADD, FMUL, FFMA, FMNMX, HADD2, HMUL2, HFMA2, IMNMX, ISCADD, LOP, LOP32I, LOP3, MOV, MOV32I, SEL, SHL, SHR, VADD, VABSDIFF, VMNMX, XMAD	6
	DADD, DMUL, DFMA, DMNMX	8
	FSET, DSET, DSETP, ISETP, FSETP	12
	POPC, FLO, MUFU, F2F, F2I, I2F, I2I	14
	IMUL, IMAD	∼86
Volta	IADD3, SHF, LOP3, SEL, MOV, FADD, FFMA, FMUL, ISETP, FSET, FSETP	4
	IMAD, FMNMX, DSET, DSETP	5
	HADD2, HMUL2, HFMA2	6
	DADD, DMUL, DFMA	8
	POPC	10
	FLO, BREV, MUFU	14

图 12.3: Volta 单精度和双精度核心改进

每份白皮书详述不同的架构，但英伟达白皮书采用了相似的结构方法。此外，Volta 白皮书还包含软件增强部分以及提供其他设备详细信息的附录。

现在，让我们深入浏览白皮书。文中提到 V100 GPU 拥有约 210 亿个晶体管。作为对比，H100 包含 800 亿个晶体管。这意味着从 Volta 到 Hopper，晶体管数量增加了近 4 倍。晶体管数量是可用的计算资源指标，更多的晶体管通常对应硬件单元数量的增加。

12.2 SM 增强

接下来讨论 Volta 架构的关键特性和增强。首先是 **SM 增强**：

1. **能效提升**：Volta 的新 SM 设计比前代 Pascal 架构能效高出 50%。对于管理大规模计算系统的企业来说，这意味着 V100 有可能将能源成本减半，且不牺牲计算能力。

2. **单精度和双精度核心改进**：在功耗不变的情况下，新 SM 为 FP32（单精度）和 FP64（双精度）性能提供了重大改进。研究论文中的指令延迟对比表显示，Pascal 中加法/减法指令需要 6 个周期，而 Volta 仅

需 4 个周期。后续的 Ampere 架构进一步降至 2 个周期。这说明英伟达不仅增加了核心数量，也在不断提升现有单元的执行速度。

3. **用于 AI 计算的张量核心**：这是新 SM 中的全新单元，能为神经网络训练带来高达 12 倍的吞吐量提升（TFLOPS），推理方面提升 6 倍。虽然张量核心的单条指令延迟在 4-8 个周期之间（取决于矩阵形状和数据类型），但它们是在矩阵级别而非单个元素上执行操作，从而实现了巨大的性能飞跃。

4. **增强的整数和浮点数并行性**：在 Pascal 及更早架构中，整数和浮点运算无法同时执行。但在 Volta 的新 SM 中，每种运算都有独立路径，允许整数和浮点运算并行运行。这对于混合数据类型的现代工作负载至关重要。

5. **高级线程调度与内存合并**：除了提升访问速度外，还将 L1 高速缓存和共享内存合并到了一个物理单元中。

第二个架构级增强是**第二代 NVLink 技术**（稍后详述）。

第三个特性是**全局内存增强**。Volta 的内存带宽达到 900 GB/s，而 Pascal 约为 700 GB/s。相比之下，Ampere 架构达到了约 1500 GB/s。更高的内存带宽意味着更快的数据访问和更高的数据处理效率。

此外，还有**多进程服务（MPS）**，旨在优化单个 GPU 上同时执行的多个应用程序的性能；以及**统一内存**增强，将 GPU 和 CPU 内存合并为单一虚拟地址空间。最后，常用的 GPU 加速计算库也进行了更新，以有效利用这些新硬件单元。

白皮书强调了针对 AI 和高性能计算的“极致性能”。引入张量核心后，V100 实现了 125 TFLOPS 的吞吐量，相比 Pascal 的约 21 TFLOPS 是显著提升。

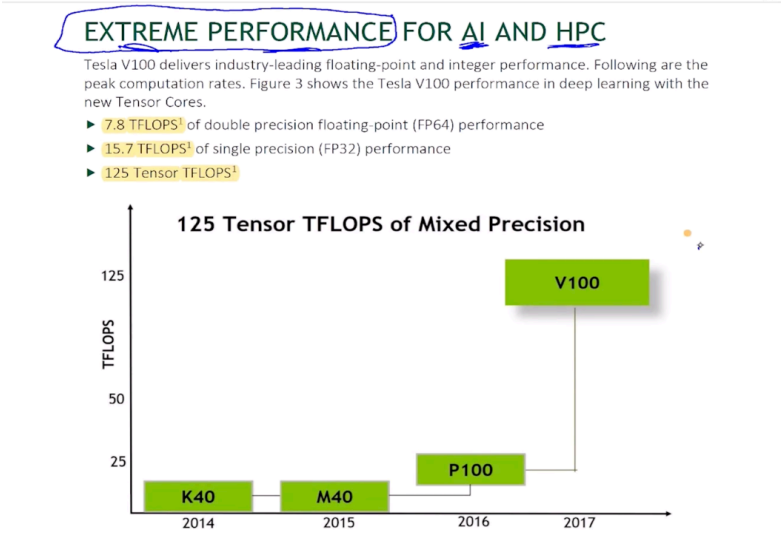


图 12.4: Volta 性能提升

12.3 深入硬件架构

接下来进入最重要的部分：**深入探讨 GV100 GPU 硬件架构**。每个 GPU 包含多个 SM，每个 SM 包含 L1/共享内存及各类计算单元。在 Volta 中，每个 SM 配备 64 个单精度核心、64 个整数核心、32 个双精度核心和 8 个张量核心。整个 GPU 拥有 84 个 SM，总计约有 2 万个计算单元。GPU 内部还包括 6 MB 的二级高速缓存（L2 Cache），通过内存控制器连接所有 SM，底部则是用于互联的 NVLink 单元。

深入单个 SM，它由**四个分区**组成。这种分区策略允许将不同的软件块分配到不同分区并发执行，最大化资源利用率。每个分区包含：

- **L0 指令高速缓存**：存储待执行指令。
- **Warp 调度器**：负责管理线程束（Warp，即 32 个线程为一组）。采用 SIMT（单指令多线程）模式，即一个 Warp 内的 32 个线程在不同核心上并行执行相同指令。
- **分发器（Dispatch Unit）**：与调度器协同，将指令分发到执行单元。

SM
L1 Instruction Cache
L0 Instruction Cache
Warp Scheduler (32 thread/clk)
Dispatch Unit (32 thread/clk)
Register File (16,384 x 32-bit)
FP64
INT
FP32
TENSOR CORE
LD/ST
SFU
128KB L1 Data Cache / Shared Memory
Tex

图 12.5: SM 内部架构

Table 1. Comparison of NVIDIA Tesla GPUs

Tesla Product	Tesla K40	Tesla M40	Tesla P100	Tesla V100
GPU	GK180 (Kepler)	GM200 (Maxwell)	GP100 (Pascal)	GV100 (Volta)
SMs	15	24	56	80
TPCs	15	24	28	40
FP32 Cores / SM	192	128	64	64
FP32 Cores / GPU	2880	3072	3584	5120
FP64 Cores / SM	64	4	32	32
FP64 Cores / GPU	960	96	1792	2560
Tensor Cores / SM	NA	NA	NA	8
Tensor Cores / GPU	NA	NA	NA	640
GPU Boost Clock	810/875 MHz	1114 MHz	1480 MHz	1530 MHz
Peak FP32 TFLOPS[1]	5	6.8	10.6	15.7
Peak FP64 TFLOPS[1]	1.7	.21	5.3	7.8
Peak Tensor TFLOPS[1]	NA	NA	NA	125
Texture Units	240	192	224	320
Memory Interface	384-bit GDDR5	384-bit GDDR5	4096-bit HBM2	4096-bit HBM2
Memory Size	Up to 12 GB	Up to 24 GB	16 GB	16 GB

图 12.6: 架构对比

- **寄存器文件**：保存临时数据。
- **执行单元**：包括单精度/整数核心、张量核心及特殊功能单元（SFU）。

四个分区共享同一个 L1 高速缓存、共享内存和纹理内存。

12.4 架构对比与 NVLink

让我们参考 A100 白皮书中的对比表来比较 Ampere、Volta 和 Pascal 架构：

- **SM 数量**：Pascal (56) → Volta (80) → Ampere (108)。
- **单精度核心**：Pascal (∼3500) → Volta (∼5000) → Ampere (∼7000)。
- **整数核心**：Pascal 无独立整数核心（与浮点共用）；Volta (∼5000)；Ampere (∼7000)。
- **张量核心**：Pascal 无；Volta (640)；Ampere (432)。注意：虽然 Ampere 张量核心数量少于 Volta，但其单核能力和效率大幅提升，整体吞吐

量反而是 Volta 的两倍以上。评估性能不能只看核心数量，还要看核心能力。

- **FP16 吞吐量（带 FP32 累加）**：Ampere 远超 Volta。这里涉及“使用单精度累加的半精度计算”概念，即输入为 FP16，但输出/累加使用 FP32 以保证精度。
- **常规单精度吞吐量**：Pascal (21 TFLOPS) → Volta (31 TFLOPS) → Ampere (78 TFLOPS)。
- **显存位宽**：Volta (∼4096-bit) vs Ampere (>5120-bit)。
- **显存容量**：Volta (16/32 GB) vs Ampere (40 GB)。
- **内存带宽**：Pascal (∼700 GB/s) → Volta (900 GB/s) → Ampere (1500 GB/s)。带宽受显存大小、速度和位宽共同影响。
- **L2 缓存**：Volta (6 MB) → Ampere (40 MB)，增长超 7 倍。
- **晶体管数**：Volta (210 亿) → Ampere (540 亿)。工艺从 Volta 的 12 nm 演进到 Ampere 的 7 nm。

编写 CUDA 程序时，还需参考”CUDA 计算能力” 表，了解最大线程数、最大块数等硬件限制。如果应用规模超出限制，需将其拆分为多个核函数（kernel）执行。

关于 **NVLink**：这是一种高速互联技术。第二代 NVLink 相比 Pascal 的第一代有两大改进：一是链路数量增加（GPU 间从 1 条增至 2 条以上）；二是链路速度提升。这使得 GPU 间通信带宽从 Pascal 的 160 GB/s 提升至 Volta 的 300 GB/s。NVLink 是 DGX 服务器的标准特性。例如 DGX Station（4 块 V100）和 DGX-1（8 块 V100）。对比 DGX-1 的 Pascal 和 Volta 版本，虽然整机功耗同为 3200 W，但 Volta 版本凭借张量核心在同等能耗下提供了数倍于 Pascal 的计算性能（能效比提升）。

本章涵盖了 Volta 架构的重点，并对比了 Pascal 和 Ampere。建议读者继续阅读白皮书剩余部分。

Table 4. Comparison of NVIDIA Data Center GPUs

GPU Features	NVIDIA Tesla P100	NVIDIA Tesla V100	NVIDIA A100
GPU Codename	GP100	GV100	GA100
GPU Architecture	NVIDIA Pascal	NVIDIA Volta	NVIDIA Ampere
GPU Board Form Factor	SXM	SXM2	SXM4
SMs	56	80	108
TPCs	28	40	54
FP32 Cores / SM	64	64	64
FP32 Cores / GPU	3584	5120	6912
FP64 Cores / SM (excl. Tensor)	32	32	32
FP64 Cores / GPU (excl. Tensor)	1792	2560	3456
INT32 Cores / SM	NA	64	64
INT32 Cores / GPU	NA	5120	6912
Tensor Cores / SM	NA	8	4[2]
Tensor Cores / GPU	NA	640	432
GPU Boost Clock	1480 MHz	1530 MHz	1410 MHz

图 12.7: 架构对比

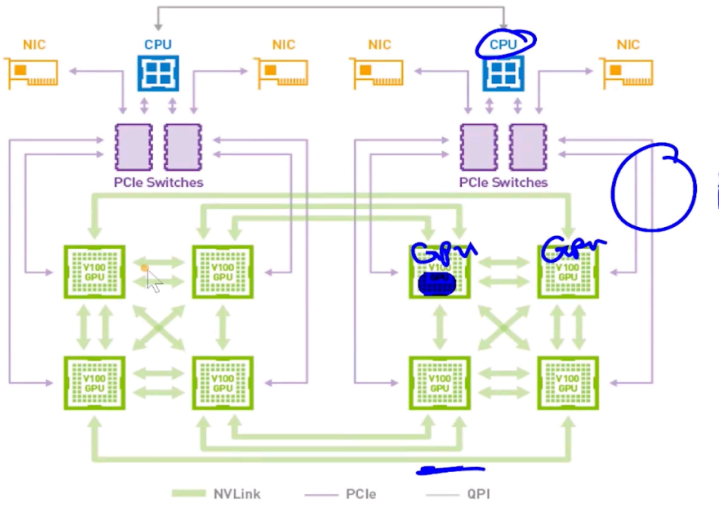


图 12.8: NVLink

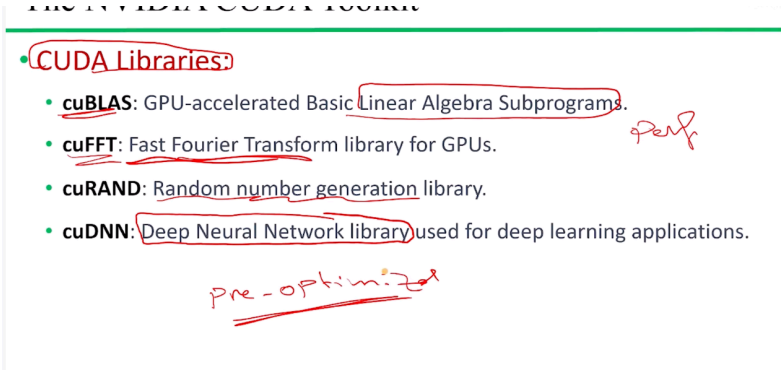


图 13.1:

13 CUDA 工具包概述

本章将深入探讨 CUDA 工具包。CUDA 工具包是 GPU 编程领域的基石，正在改变高性能计算和深度学习应用的格局。CUDA（Compute Unified Device Architecture，统一计算设备架构）是英伟达的并行计算平台。这个工具包不仅仅是一套工具的集合，更是释放英伟达 GPU 巨大潜力的入口。让我们探索其关键组件，并了解它们如何帮助开发者充分利用 GPU 计算的全部能力。

13.1 nvcc 编译器

CUDA 开发的核心是 CUDA 编译器，即 `nvcc` 编译器。它供开发人员、研究人员以及专业人士使用，以简化从简单的 CUDA 编码到复杂 GPU 应用程序的一切工作。该编译器是将高级 CUDA 代码转换为 GPU 可理解形式的关键：它可以将 CUDA 代码转换为与各种英伟达架构兼容的 PTX 代码。简而言之，`nvcc` 编译器用于将 CUDA 代码转换为可以在 GPU 上高效运行的可执行文件。

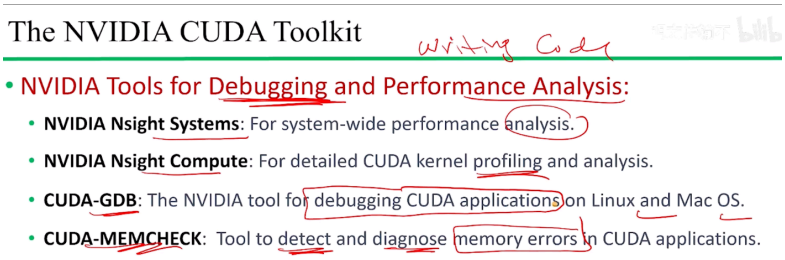


图 13.2:

13.2 优化库

CUDA 拥有一套优化库的支持，每个库都有其专门用途。例如，**cuBLAS** 和 **cuFFT** 分别在线性代数和快速傅里叶变换运算方面表现突出。这些库经过高度优化，能显著提升性能。此外还有 **curand** 库，专门用于随机数生成；以及 **cuDNN** 库，这是一个为神经网络加速设计的深度学习库。这些库是高效 GPU 编程的支柱，提供了预先优化的即用型函数，节省开发时间并增强性能。

13.3 运行时 API

CUDA 工具包还提供了运行时 API（Runtime API），用于管理设备初始化、内存分配以及核函数启动。它们对于管理应用程序与 GPU 的交互至关重要。例如，对于 CPU 和 GPU 之间的内存操作或数据传输，可以使用以下 API 函数：

- `cudaMalloc`：在 GPU 设备上分配指定大小的内存。这类似于标准 C 语言中的 `malloc`，但专用于 GPU 而非 CPU 内存分配。
- `cudaMemcpy`：用于在主机（CPU）和设备（GPU）之间复制数据。

以上是运行时 API 函数的两个典型示例。

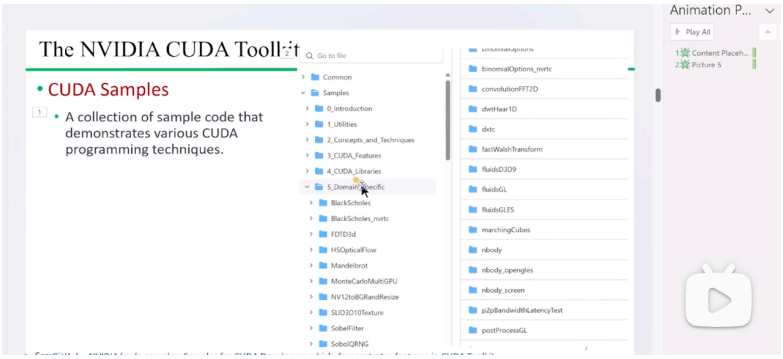


图 13.3:

13.4 调试与分析工具

对于 GPU 应用程序来说，工作不止于编写代码，还关乎确保效率和稳定性。为此，英伟达提供了以下强大的调试和分析工具：

- **Nsight Systems** 和 **Nsight Compute**：用于分析和剖析应用程序的性能。
- **cuda-gdb**：提供在 Linux 和 macOS 操作系统上调试 CUDA 应用程序的强大环境。
- **cuda-memcheck**：专门用于检测和诊断内存问题的工具。

所有这些工具共同提供了一个全面的环境，不仅用于编写代码，还用于完善和优化 GPU 加速应用程序。

13.5 示例代码

CUDA 工具包附带了一套丰富的示例代码。这些示例不仅仅是代码，更是实用的指南，阐明了各种 CUDA 编程技术——从基本概念到高级操作，涵盖了广泛的场景和应用案例。它们为高效 CUDA 编程提供了宝贵的见解和实用的学习体验，是对理论知识的补充。

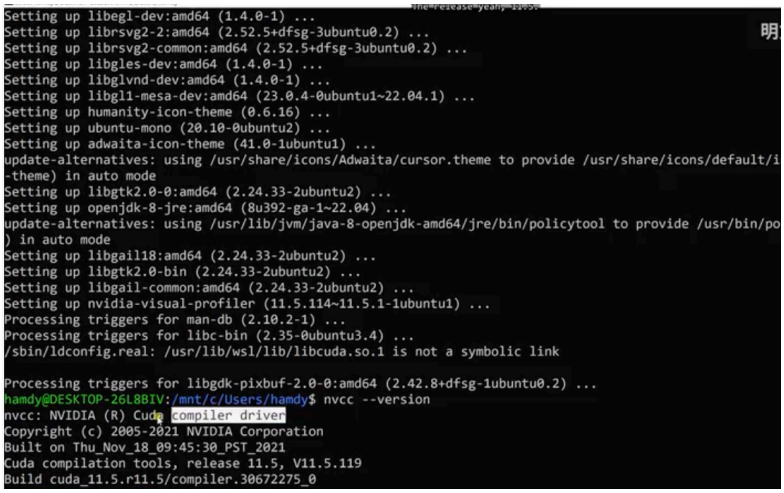


图 14.1:

总结而言，CUDA 工具包不仅仅是编译 GPU 应用程序的编译器，或是编写 CUDA 程序的编辑器。它是一个综合平台，包含了编写、编译和剖析 GPU 应用程序所需的一切。

14 在 Linux 系统上安装 CUDA 工具包

本章讲解如何在 Linux 系统上安装 CUDA 工具包。假设你正在使用前文介绍过的 WSL 工具，首先打开” 开始” 菜单，搜索” 命令提示符” 并点击打开，然后输入 `wsl` 即可在当前 Windows 系统上启动 Linux 子系统。

14.1 检查 nvcc

现在检查 `nvcc` 编译器是否已安装在 Ubuntu 系统中。启动 Ubuntu 后，终端提示 `command not found`，说明 `nvcc` 尚未安装。

14.2 安装 CUDA 工具包

需要安装 CUDA 工具包才能使用 `nvcc` 编译器。打开 CUDA 工具包下载页面，选择操作系统、架构以及发行版本。有多种发行版可选，这里选择 Ubuntu；如果你使用的是 WSL，也可以选择 WSL-Ubuntu。接着会看到三个安装选项：其中两个先下载本地安装包（标为 Local），中间那个则是通过网络直接安装（Network）。不过，最简单、最不容易出错的方式是直接使用 `apt` 安装。

再检查一次 `nvcc` 版本，虽然提示命令未找到，但第二行给出了安装建议：

```
sudo apt install nvidia-cuda-toolkit
```

复制这条指令并在终端中输入，然后输入密码。系统会询问是否继续，输入 Y 确认即可。安装过程大约需要 30-40 分钟，请耐心等待。CUDA 工具包安装完成后，你将拥有 `nvcc` 编译器以及前文介绍的所有其他功能。再次检查 `nvcc` 编译器版本，可以看到 CUDA 编译器已成功安装（版本 11.5），输出第一行显示 `nvcc: NVIDIA (R) Cuda compiler driver`，表示 CUDA 工具包已就绪，对入门来说已经足够了。

另外提醒一点：如果你按照官网的步骤操作，可能会遇到一些依赖问题或错误。因此建议一开始先使用上述 `apt` 指令，它简单且不易出错。等你熟悉之后，需要更新 CUDA 工具包时，再到 CUDA 官网选择适合你系统的最新版本进行安装。

15 NVIDIA GPU 硬件与软件层级结构及 CUDA 介绍

本章将深入探讨 NVIDIA GPU 的硬件与软件层级结构，并介绍 CUDA。

CUDA 全称为 Compute Unified Device Architecture（统一计算设备架构），是由 NVIDIA 开发的一个并行计算平台和应用程序编程接口（API）。

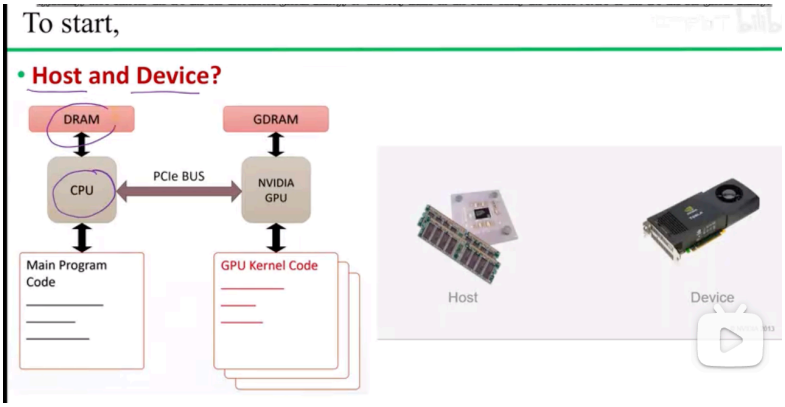


图 15.1:

CUDA 允许软件开发者使用 NVIDIA 的图形处理单元进行通用目的计算，这种方法被称为 GPGPU（General-Purpose computing on Graphics Processing Units，即图形处理单元上的通用计算）。

在开始之前，强调三个关键点。第一，CUDA 基于 C 编程语言，这是一个优势，它让熟悉 C 语言的开发者能够轻松编写可在 NVIDIA GPU 上执行的软件。第二是 CUDA 编译器 `nvcc`——它不仅仅是一个普通的编译器，而是 CUDA 工具包的核心。`nvcc` 的作用远不止将 CUDA 代码转换为可执行文件或机器码，它在开发过程中扮演着多重关键角色。随着本课程的推进，我们将逐步揭示 `nvcc` 的多功能性，以及它如何优化 CUDA 应用程序。

第三点是主机（Host）与设备（Device）这两个术语的区别，这是为了充分利用 GPU 的资源。通常，主机指的是 CPU 及其关联的全局内存（即 RAM），而设备则指 GPU 及其全局内存。这种区分不仅在术语上非常重要，在编写软件的方式上也至关重要——它影响代码结构、数据传输和内存管理。本图中左侧部分代表主机，右侧部分代表设备（即 GPU）。

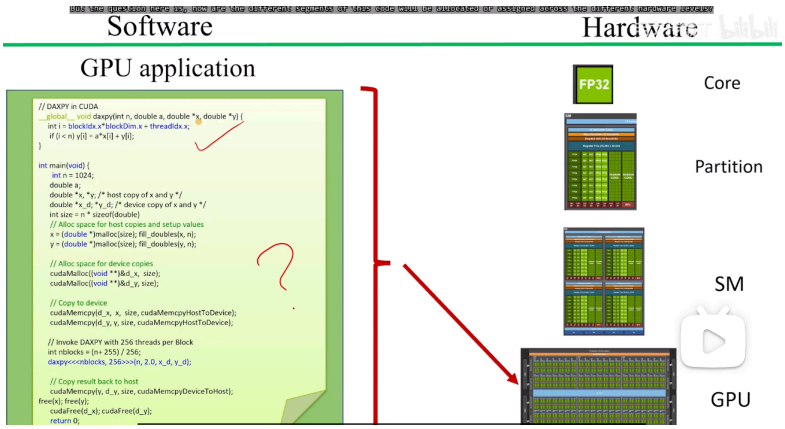


图 15.2:

15.1 硬件四级层级

回顾 GPU 硬件架构层级与其软件部分之间的关系。GPU 的结构包含四个明确的硬件组织层级，且是分层的：最顶层是 GPU 本身，它封装了所有单元；每个 SM（Streaming Multiprocessor，流式多处理器）构成该层级结构的第二级。这一设计已在 Volta 白皮书章节中详细说明。

接着，每个 SM 又被进一步划分为若干个分区（Sub-partition / Processing Block），构成硬件结构的第三级。在 Volta 和 Ampere 架构中，每个 SM 包含四个这样的分区。如图所示，来自 Volta 白皮书的图中 SM 被划分为四个分区。Ampere 白皮书中也能看到类似的结构。

在第四级（最细粒度的层级），每个分区内包含独立的计算单元，例如负责执行浮点运算的 FP32 核心。这些核心是 GPU 内部处理计算的基本单元。

综上所述，GPU 的硬件架构可视为一个四级层级结构：从整个 GPU 开始逐级向下，经过 SM，再到其内部的分区作为第三级，最终到达各个独立的计算核心。这些不同的硬件层级反过来要求对应的软件应用程序具备相似的结构。

例如，假设我们有一个 CUDA 应用程序（如图所示，用方框表示），该应用程序设计用于在整个 GPU 上运行。问题是：这段代码的不同部分将如

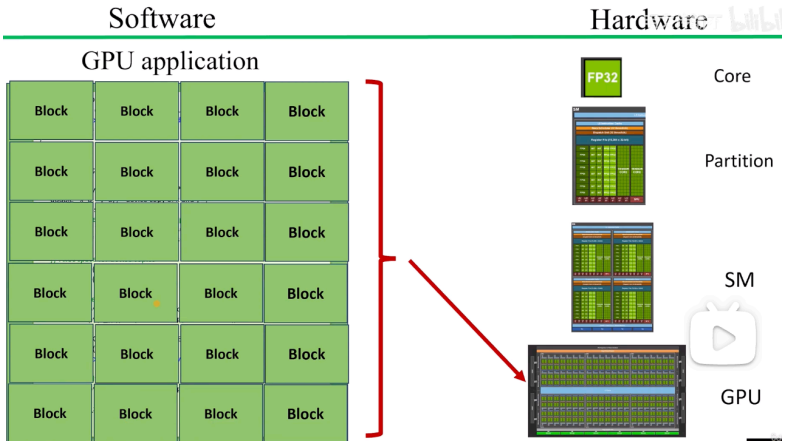


图 15.3:

何分配或指派到不同的硬件层级？这正是需要理解的内容——软件代码的不同部分如何映射到硬件的不同层级。

15.2 软件层级：块与 GigaThread Engine

编写 CUDA 代码的第一步是将 GPU 核函数（Kernel）划分成更小的软件单元，称为块（Block）或线程块（Thread Block）。对于每个核函数，首先需要配置其块数量。如果将整个方框视为一个 CUDA 应用程序，就可以将其划分为更小的块。如图所示，每个小方框代表一个块（或线程块，两个名称均正确）。这种划分对应了 GPU 的前两个硬件层级——整个应用程序设计用于在整个 GPU 上执行。

然而这引发了一个问题：软件中的第二级是指不同的块在不同的 SM 上执行。在左侧，有由软件块（线程块）表示的任务；在另一侧，有可视为”工人” 的 SM。谁负责将这些软件任务分配给工人（SM）？

要回答这个问题，需要了解另一个单元——GigaThread Engine。它是 GPU 内部最重要的单元之一，其关键功能就是高效地将这些软件块（任务/作业）分配到各个 SM 上。参考 Volta 白皮书可以看到，GigaThread Engine 位于整个 GPU 的整体结构之中，并连接到所有 SM——它负责为这

图 15.4:

些 SM 提供必要的软件块或软件任务。

将同样的原理应用到下一层级。假设图中的第一个块被分配到某个 SM 上执行，但该 SM 中有四个分区，而我们这里只有一个块。这个被称为" 块" 的单一软件单元应如何分配到 SM 内部的四个硬件分区中?

如果将整个块只分配给其中一个分区，就会导致资源利用率不足（三个分区空闲）。这一考虑引出了软件层级的第三级——Warp（线程束），以避免浪费可用的硬件资源。

15.3 Warp 与调度

每个软件块都会被细分为更小的单元，称为 Warp。例如，如果将第一个块划分为四个 Warp，那么每个 Warp 就可以分配到 SM 中的一个不同分区——四个 Warp 对应四个分区，每个分区分配一个 Warp。

考虑另一个例子：如果每个块有 32 个 Warp，而有四个分区，那么每个分区将执行八个 Warp。当一个分区分配了多个 Warp 时，就需要一个管理者——这就是为什么每个分区中都有一个 Warp Scheduler（Warp 调度器）单元。

放大该分区，可以看到内部的 Warp 调度器单元。正如前文所述，该单

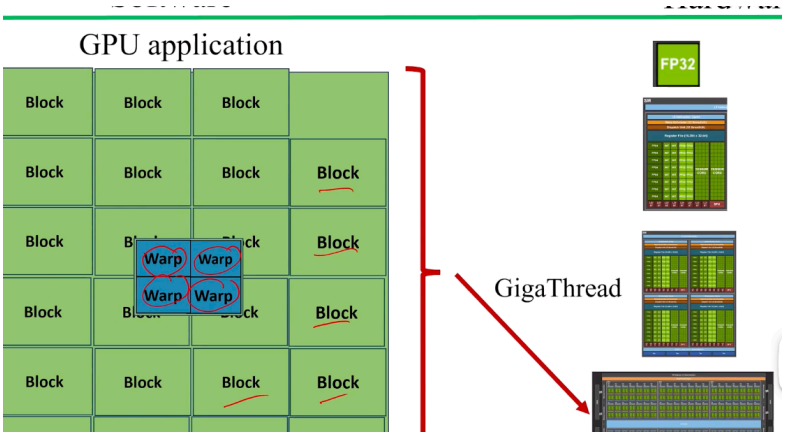


图 15.5:

元存在于每个分区中，负责在每个时钟周期中调度某个 Warp 。这是在硬件层面；而在软件层面，一个 Warp 默认通常由 32 个线程组成。

进入第四级，每个分区包含若干个核心。Warp 中的每个线程会在分区内独立的核心上执行，确保每个核心都积极参与计算。理解硬件和软件中的这四个层级，对于高效配置 CUDA 应用程序至关重要。

块的数量以及每个块中的线程数量，是启动任何 CUDA 应用程序时需要指定的两个关键参数（将在后续章节中详细探讨）。关于 Warp ，可以通过将块大小（即每个块的线程数）除以 32 来计算每个块中的 Warp 数量，因为 Warp 的大小固定为 32 个线程。

16 第一个 CUDA 项目：核函数与执行配置

本章将开始第一个项目——打印块和线程的 ID。这是一个非常简单的项目。从开始菜单启动 Visual Studio，创建一个新项目。在项目中选择 CUDA 语言，然后为你的项目命名（例如 `test_01`），点击创建。

16.1 项目创建与核函数

这是第一个项目，核函数文件默认为 `kernel.cu`。删除模板中的所有代码，只保留已准备好的几行头文件包含代码。本节不关注头文件部分，只关注核心代码。

假设读者已具备 C 编程语言的基础。本应用程序中有两个核函数，或者说一个核函数和一个普通函数。普通函数是主函数（`main`）——每个 C 程序都必须包含主函数，而 CUDA 代码基于 C，因此同样需要主函数。

核函数（Kernel）是将在 GPU 上执行的函数，而普通函数则在 CPU 上执行。区分两者的标志是 `__global__` 修饰符——带有该修饰符的即为核函数。其声明方式与普通函数类似：有返回类型、函数名、括号和大括号。当前这是一个空核函数，接下来将填充具体内容。

主函数中包含两条指令：第一条是核函数调用，第二条是 `return 0` ——因为主函数类型为 `int`，所以必须返回一个整数值。

16.2 执行配置参数

调用核函数时，需要写出核函数名称，并配置执行参数（将影响应用程序的输出）。核函数还可以接收输入参数，例如后续章节中将对两个向量进行相加，需要在核函数中传入这些向量。本节不涉及输入参数。

本程序包含一个核函数和一个主函数，主函数调用核函数以在 GPU 上执行。

目标是打印块和线程的 ID。使用 `printf` 函数即可实现：以 `\n` 开头换行，然后以 `%d` 格式输出线程 ID 和块 ID，最后传入对应的变量。

CUDA 提供了预定义的块和线程索引变量：`blockIdx` 提供块 ID，`threadIdx` 提供线程 ID。CUDA 支持多维索引，本节只使用一维（`.x`）。例如，如果程序中有十个块，`blockIdx.x` 将打印从 0 到 9 的 ID。

核函数编写完成后，点击” 生成（Build）”→” 生成解决方案（Build Solution）” 即可编译。Windows 下 Visual Studio 会自动调用 `nvcc` 编译器；Linux 用户则需要学习使用 `nvcc` 命令行编译（将在后续章节中讲解）。

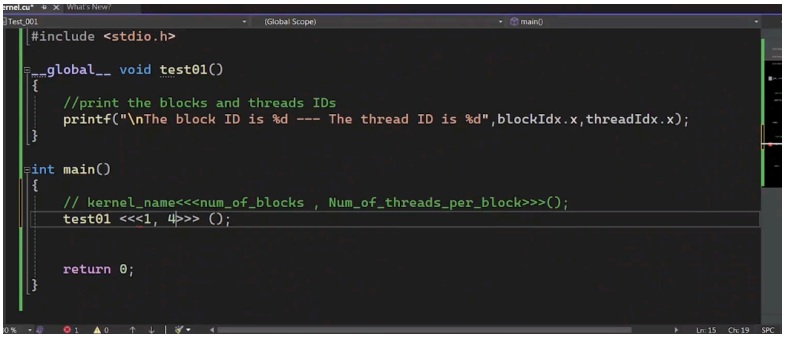


图 16.1:

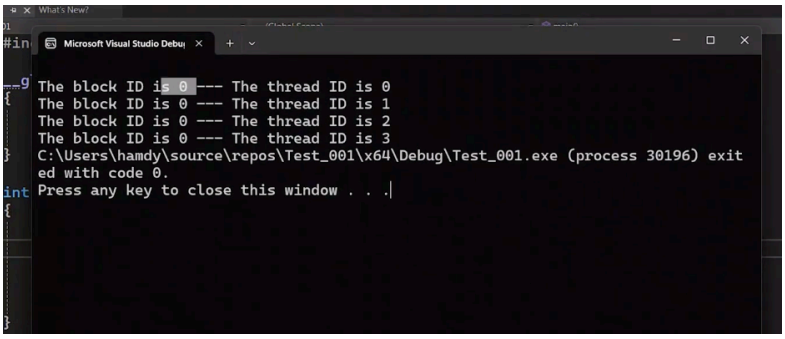


图 16.2:

核函数调用语法如下：

核函数名 <<< 块数量，每块线程数 >>>();

其中，尖括号 <<<>>> 内的第一个参数是网格中的块总数，第二个参数是每个块中的线程数（注意：这不是应用程序的线程总数）。当前配置为 <<<1,1>>>，即只有一个块、一个线程，因此输出块 ID 为 0、线程 ID 为 0。

将每块线程数增加为 4，重新构建并运行：

16.3 逐步增加线程数与块数

此时只有一个块，块 ID 为 0，四个线程 ID 从 0 到 3。将每块线程数增加到 32，重新构建并运行：

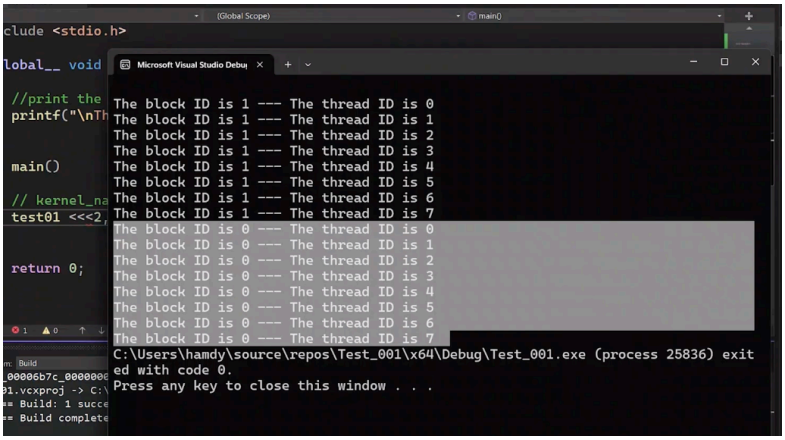


图 16.3:

将块数量改为 2，每块线程数设为 8：

输出显示两个块（ID 分别为 0 和 1），每个块有 8 个线程（从 0 到 7）。你可能好奇为什么块 ID 1 先于块 ID 0 执行——这取决于哪个 SM 先接收到它的块，顺序并不重要。

接下来结合白皮书内容尝试更多配置。线程数量通常使用 2 的幂次，例如 512。将每块线程数设为 512 并运行：输出正常，块 ID 为 0，线程 ID 从 0 到 511。注意输出顺序可能不是连续的——因为线程是并行执行的，哪个线程先完成就哪个先输出，这是正常现象。

将每块线程数翻倍到 1024，运行正常，线程 ID 从 0 到 1023。再次翻倍设为 2048，运行：

16.4 硬件限制

程序无法运行，没有任何输出。编译成功但运行失败，原因在于硬件限制。

以 Ampere 架构白皮书为例，其中的表格列出了最大线程数、最大 Warp 数、最大块数、最大寄存器数等限制。每个块的最大线程数（最大线程块大小）在 Ampere 架构中为 1024，这一限制适用于大多数架构。因此，不能

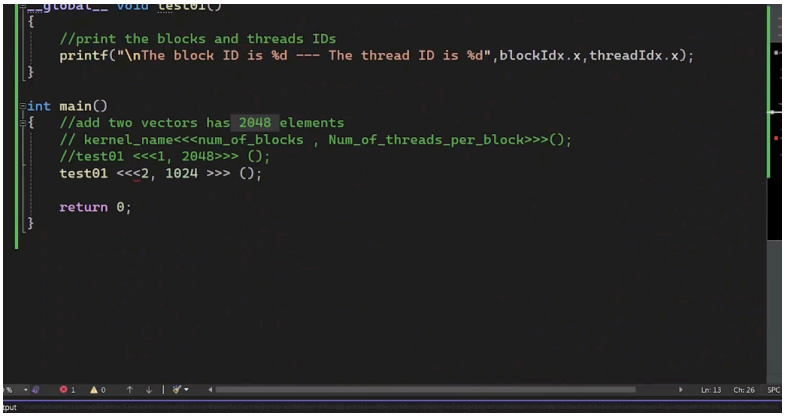


图 16.4:

将每块线程数设定为超过 1024。

这就是为什么需要同时理解软件和硬件。回到程序，假设需要对两个向量进行相加，每个向量包含 2048 个元素。如果将每块线程数设为 2048，程序将无法运行。解决方案是增加块的数量：将每块线程数设为 1024，块数设为 2（即 `<<<2, 1024>>>`），总线程数等于 $2 \times 1024 = 2048$，正好满足需求。运行后程序正常工作。

如果将块数设为 200 万，是否能运行？需要查阅白皮书中的限制。每个 SM 的最大线程块数为 32，但这是每个 SM 的限制，而非整个 GPU 的限制。整个 GPU 可运行的最大块数 = 每个 SM 最大块数 × SM 数量。例如，基于 GA100 芯片的 GPU 有 108 个 SM，因此最大并发块数为 $32 \times 108 = 3456$。

这就是软件配置与硬件限制之间的关系：每个块中的线程数与块的数量必须与硬件资源相匹配。

17 在 Linux 系统上编译和运行 CUDA 程序

本章讲解如何在 Linux 系统上编译和运行 CUDA 程序。请先阅读前一章的内容，因为本章不再重复解释相同的项目。

打开包含本章所有项目的文件夹，使用任意编辑器打开第一个项目

project_01。例如，使用 Visual Studio Code 打开后，可以看到一个 GPU 核函数，用于打印块 ID、线程 ID 和 Warp ID。我们使用两个块，每个块包含 64 个线程来启动该核函数。

17.1 使用 nvcc 编译

Visual Studio Code 本身只是编辑器，不具备编译功能。需要打开命令提示符，输入 wsl 在 Windows 上启动 Linux 子系统（安装方法详见前文章节），然后进入项目文件夹。

使用 nvcc --version 可以检查 nvcc 编译器是否已安装。编译命令如下：

```
nvcc -o project_01 project_01.cu
```

其中 -o 用于指定可执行文件的名称（建议在所有编译命令中都使用它）。编译完成后，输入 ls 即可看到生成的可执行文件。

17.2 运行程序与输出问题

运行程序时，输入 ./project_01 并按回车。但有时终端上没有任何输出——项目本应为每个线程打印一行内容。

再运行一次，这次有输出了。程序本身没有问题，但第一次运行时终端上缺少输出，这是怎么回事?

17.3 cudaDeviceSynchronize

原因在于 CPU 与 GPU 的执行是异步的。在 CPU 一侧，主函数会逐条执行指令。当遇到启动 GPU 核函数的指令时，CPU 会将核函数发送到 GPU，但不会等待 GPU 的执行结果，而是立即执行下一条指令。因此，GPU 的 printf 输出可能来不及在程序结束前显示。

解决方案是使用 cudaDeviceSynchronize()——”同步”意味着让 CPU 等待，直到 GPU 完成核函数的执行。在核函数调用之后添加这一行：

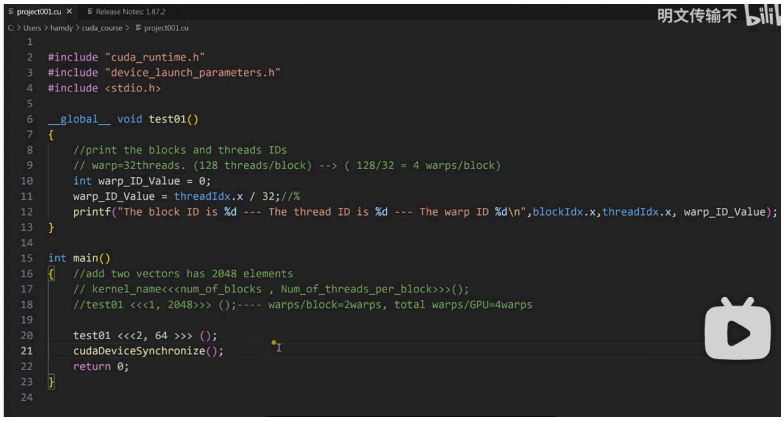


图 17.1:

```
kernel<<<blocks, threads>>>();
cudaDeviceSynchronize();
```

编译并运行后,每次都能看到完整的输出。不使用 `cudaDeviceSynchronize()` 时，输出时有时无；使用后则始终稳定输出。

这一同步机制不仅适用于 Linux，也适用于 Windows。

17.4 编译错误排查

如果故意制造编译错误（例如删除某个分号），编译器会提示错误出现在某一行。需要注意的是：分号缺失通常会反映在下一条指令上，因此应检查报错行的上一条语句。修正错误后保存并重新编译即可。

以上就是在 Linux 系统上编译和运行 CUDA 应用程序的完整流程。

18 CUDA 课程项目二：线程束 ID（Warp ID）

在前一个项目中，我们研究了块 ID（Block ID）和线程 ID（Thread ID）。本章将研究第三层级——**线程束 ID**（Warp ID）。

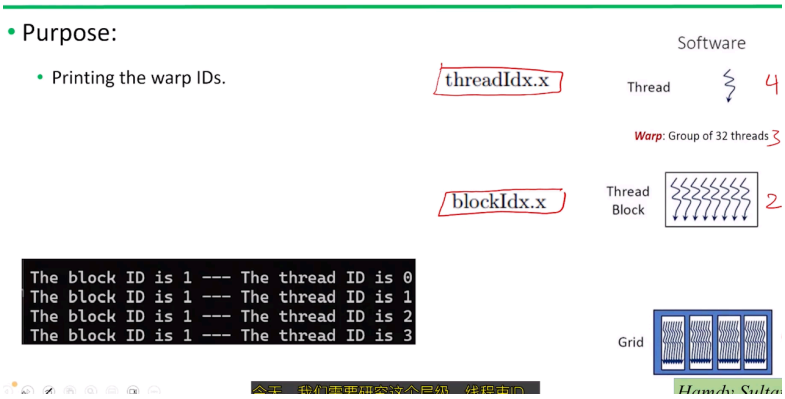


图 18.1:

18.1 Warp ID 概念

每个 GPU 应用程序包含网格（Grid），网格包含块（Block），块包含线程束（Warp），每个线程束由 32 个线程组成。这就是 CUDA 程序中的四级软件层级结构。

前文已经学习了如何打印线程 ID（X、Y、Z 三个维度）和块 ID。本章的目标是计算并打印线程束 ID。这里的关键问题是：CUDA 中**没有**预定义变量直接提供线程束 ID 值，需要我们自行计算。

18.2 计算 Warp ID

已知每个线程束由 32 个线程组成，如果每个块有 128 个线程，则每块有 $128/32 = 4$ 个线程束。

计算线程束 ID 的方法如下：声明变量 `WarpID`，将其初始化为 0，然后令其等于当前线程的线程 ID 除以 32。每个线程的线程 ID 不同，除法运算后得到该线程所属的线程束 ID。

编译并运行后，可以看到输出结果。首先关注块 ID：当只有 1 个块时，块 ID 始终为 0。线程 ID 从 0 开始（由于并行执行，输出顺序不一定连续），最大值为 127。线程束 ID 从 0 开始，共有 4 个（0、1、2、3），每个线程束包含 32 个线程——线程束 0 包含线程 ID 0 到 31，线程束 3 包含线程 ID

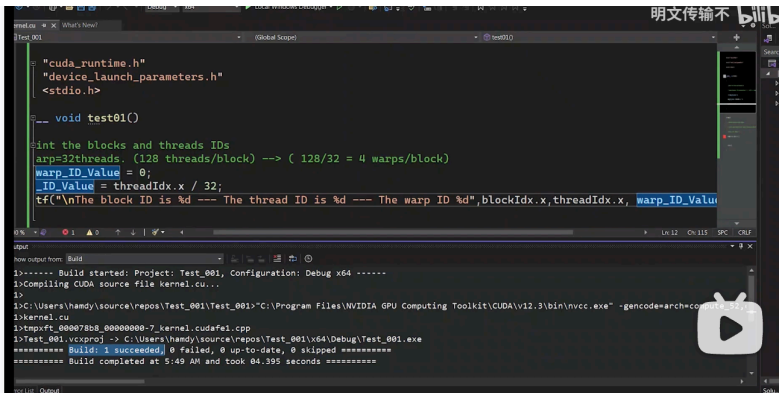


图 18.2:

96 到 127。

此时项目中有 1 个块、4 个线程束，每个线程束由 32 个线程组成。计算线程束 ID 还有其他方法（例如使用取模运算），读者可以自行探索。

18.3 多块场景下的 Warp ID

将每块线程数改为 64，则每块有 $64/32 = 2$ 个线程束。如果同时将块数改为 2，每块的线程束数仍为 2（因为未改变每块线程数），但整个 GPU 上的总线程束数变为 $2 \times 2 = 4$。

编译并运行后，输出显示两个块（块 ID 分别为 0 和 1）。关于线程束 ID，可以看到每个块都有线程束 ID 0 和线程束 ID 1。线程束 ID 0 在输出中出现了两次——因为线程束 ID 是**按块计算的**，而非针对整个 GPU。每个块都有自己独立的线程束编号，因此两个块各自拥有线程束 ID 0 和线程束 ID 1。

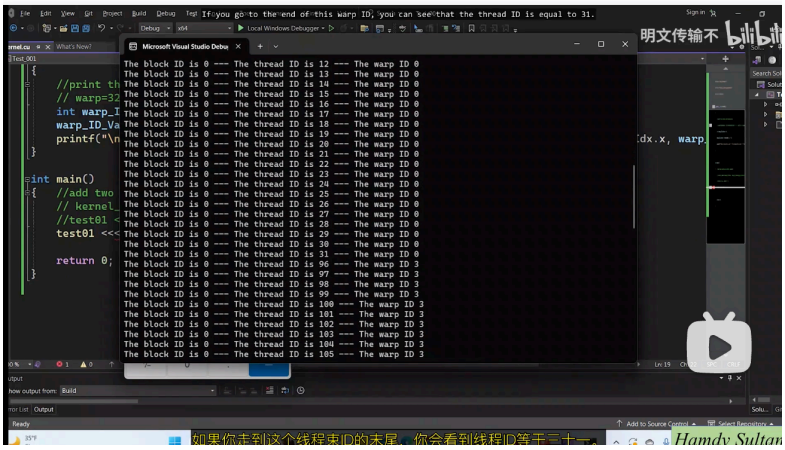


图 18.3:

19 CUDA 课程项目三：向量加法（Vector Addition）讲解

本章将解释第三个项目——向量加法（Vector Addition）。向量 A 包含 1024 个元素，索引范围从 0 到 1023。在本项目中，假设有两个向量，每个向量包含 1024 个元素。类似地，有向量 B，如图所示。当前的任务是将这两个向量相加，结果存储在向量 C 中，如图所示。这意味着向量 A 的第一个元素将与向量 B 的第一个元素相加，加法操作以逐元素的方式进行，结果存储在 C 的零位置，即 C 的第一个元素。

19.1 CPU 上的向量加法

在深入 GPU 代码之前，先展示一下这个应用在 CPU 上的代码。在每次迭代中，将向量 A 和向量 B 的对应元素相加，并将结果保存在向量 C 中。如图所示，这个过程简单地涉及编写一个 for 循环来遍历所有 1024 个元素。以上就是关于在 CPU 上运行向量加法的全部内容。需要注意的是，for 循环的迭代是顺序执行的，而非并行的。

基于之前的讨论，第一步是为 GPU 应用确定块和线程的数量。这里首

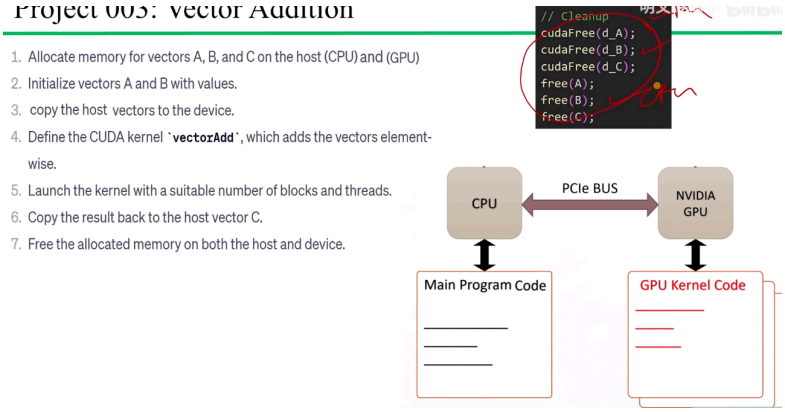


图 19.1:

先假设只使用一个块和 1024 个线程。根据这个信息，这个块的 ID 将为 0，而线程 ID 的范围将从 0 到 1023。这个选择使线程数量与每个向量中的元素数量一致。每个向量有 1024 个元素，这意味着每个线程 ID 等于向量中某个元素的索引。稍后将会用到这个信息。

现在写出第一个元素的加法操作，仅仅是第一个元素。一个小例子看起来是这样：C 的零位置等于 A 的零位置加上 B 的零位置。考虑到块和线程的 ID，在本例中只有一个块，ID 为 0，因此块 ID 没有实际用途。这个模式被复制到第二个元素、第三个元素，依此类推，覆盖这些向量中的所有元素。然而线程 ID 表明，正如前文所述，它们等于元素的索引。这个观察使我们能够有效地为每个线程分配不同的元素。线程 ID 范围从 0 到 1023，与元素的索引范围相同。例如，线程 0 可以与第一个元素对齐，第二个线程可以与第二个元素对齐，以此类推。

19.2 CUDA 核函数实现

理论部分已讲完，现在的问题是如何将其转化为 CUDA 代码。实际上答案非常简单，实现目标所需的代码很简洁，这里只需要一个操作——加法操作。这意味着一个操作将由多个线程执行。在展示的代码中，声明了一个名为 `vectorAdd` 的 GPU 核函数（kernel）。在接下来的指令中，对每个元

素执行加法操作。在核函数的第一条指令中，首先读取线程 ID，其范围从 0 到 1023。这里的每个块以及整个 GPU 的线程总数等于 1024。这里的操作将被执行多次，次数等于线程数。线程 ID 控制着将要执行加法操作的元素。这 1024 条指令将被分配给 GPU 中不同的核心，以便并行执行。这意味着这条指令将被隐式地执行 1024 次，每次都有一个不同的线程 ID。正如这部分提到的，在 GPU 上相加这两个向量的核函数只需要展示的 1 到 2 条指令，无需任何循环。与在 CPU 上运行相同函数时需要编写循环不同，线程 ID 会显式地变化以遍历不同的元素，因此不需要循环。

在打开 Visual Studio 并检查代码及其执行之前，先回顾一下任何 CUDA 应用的基本步骤。以本应用为例，任何 CUDA 应用的第一步都是为输入和输出数据分配空间。我们需要为三个向量 A、B 和 C 分配内存，并确保在主机（CPU）和设备（GPU）上都为这些向量分配了空间。例如，如这三部分所示，这些是实现第一步目的所需的指令。这里分配了一些指针，三个用于 CPU 分配，另外三个用于 GPU 分配。然后使用这些指针和 `malloc` 函数，在 CPU 的内存中为三个向量分配所需空间。类似地，使用 `cudaMalloc` 函数在 GPU 内存中为这些向量分配空间。

如图所示，下一步是初始化输入。在本例中，这意味着为向量 A 和 B 赋值，因为如果没有先为它们填充值，就无法执行 A 和 B 的加法。这是一个 CPU 步骤，意味着它将在 CPU 上执行，而非 GPU。这就是为什么使用 for 循环来执行这一步。如图所示，for 循环遍历向量 A 和 B 的所有元素，然后在每个元素中存储值。

第三步涉及将这些初始化的 A 和 B 值传输到 GPU。因为目标是在 GPU 上执行加法操作，这一步可以通过 2 个 API 或两条指令实现。有必要将输入数据移动到 GPU 的内存中。第一个是复制向量 A 的数据，第二个用于复制向量 B 的数据。一旦数据进入 GPU 内存，就可以调用向量加法核函数。如这里的指令所示，这个核函数处理 A 和 B 的元素以执行加法操作，并将结果保存在向量 C 中。正如前文讨论的那样，这条指令包含三个主要部分：核函数名称、核函数配置（即块和线程的数量），以及核函数的输入参数。

在结果安全存储在 GPU 内存中之后，下一步是将它们传输回 CPU 内存。在那里可以打印它们，或将它们发送到另一个阶段，或发送到另一个 GPU 核函数进行进一步计算等。这只是一个内存复制操作，但这次的配置是从 GPU（设备）到 CPU（主机）。如图所示。这个过程的最后一步是释放之前在 CPU 和 GPU 上分配的内存空间，这确保了高效的资源管理，并避免内存泄漏。

19.3 完整代码解析

Listing 1: 向量加法完整 CUDA 程序

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define VECTOR_SIZE 1024
#define BLOCK_SIZE 1024

__global__ void vectorAdd(int *A, int *B, int *C, int N) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;

    if (tid < N) {
        C[tid] = A[tid] + B[tid];
    }
}

int main() {
    int *h_A, *h_B, *h_C;
    int *d_A, *d_B, *d_C;
    const int bytes = VECTOR_SIZE * sizeof(int);
```

```
20
21      // 第一步: CPU 和 GPU 上分配内存
22      h_A = (int *)malloc(bytes);
23      h_B = (int *)malloc(bytes);
24      h_C = (int *)malloc(bytes);
25      cudaMalloc(&d_A, bytes);
26      cudaMalloc(&d_B, bytes);
27      cudaMalloc(&d_C, bytes);
28
29      // 第二步: CPU 端初始化向量
30      for (int i = 0; i < VECTOR_SIZE; i++) {
31          h_A[i] = i;
32          h_B[i] = VECTOR_SIZE - i;
33      }
34
35      // 第三步: 将数据从 CPU 传输到 GPU
36      cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
37      cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);
38
39      // 第四步: 启动 GPU 核函数 (1 个块 x 1024 个线程)
40      vectorAdd<<<1, BLOCK_SIZE>>>(d_A, d_B, d_C, VECTOR_SIZE);
41
42      // 第五步: 将结果从 GPU 传输回 CPU
43      cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost);
44
45      // 第六步: 打印验证结果
46      for (int i = 0; i < VECTOR_SIZE; i++) {
47          printf("%d + %d = %d\n", h_A[i], h_B[i], h_C[i]);
48      }
```

```
49
50      // 第七步: 释放 CPU 和 GPU 内存
51      free(h_A); free(h_B); free(h_C);
52      cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
53
54      return 0;
55  }
```

如图所示，代码以每个向量的大小开始，大小等于 1024 个元素。每当需要改变应用中向量的大小时，只需更改这个值。这就是为什么需要在应用开始时预定义大小。然后是 GPU 核函数，我们知道这是一个 GPU 核函数而非 CPU 函数，因为有 `__global__` 修饰符，这意味着这些指令将在 GPU 端执行，而非 CPU 端。核函数名称是 `vectorAdd`，它接受四个值：向量 A、向量 B 作为输入，向量 C 作为输出（存储结果的地方）。还有一个变量 N，实际上等于大小（1024）。这里暂不详细解释 N 变量，将在后续章节中说明。

然后是两条指令，实际上可以将它们合并为一条指令，但这里保留两条以便为后续更复杂的应用做准备。这里是读取线程 ID，有 1024 个线程，这个变量将包含线程 ID 的值，不同的线程 ID 将用于遍历这些向量中的所有元素。线程 ID 将从 0 到 1023。这就是 GPU 核函数，它非常简单。

然后是主函数的开始。第一步是分配一些指针以供使用，下一步是在 CPU 端和 GPU 端分配空间。例如这三个向量，将用于在 CPU 内存上为向量 A、向量 B 和向量 C 分配空间。使用 `malloc` 函数来分配这些空间。需要注意的是，这是每个向量的大小，预定义的大小并不正好是 1024，因为乘以了整数值的大小（`sizeof(int)`），这就是使用 `malloc` 函数的方式。总之需要具备一定的 C 语言基础才能理解这里的一切。

然后使用另外三个指针在 GPU 内存上分配空间。这三个指令是为在 GPU 内存上分配三个向量所需的空间，分别是第一个向量、第二个向量和第三个向量，它们都使用相同的大小。

下一步是初始化向量 A 和向量 B。不需要初始化向量 C，因为这是将

要存储结果的地方。这里使用一个 for 循环来遍历向量 A 和向量 B 内的所有元素，存储一些值。这个 for 循环的结束条件是 1024。例如在 A 中存储 I 的值，I 从 0 到 1024。第一个元素将等于 0，第二个元素将等于 1，最后一个元素将等于 1023，以此类推。如果 I 等于 0，B 的值将等于 1023。B 的值等于 1024 减去 I 值。这非常简单，不需要关注值本身，可以用任何想要的值来初始化 A 和 B。

一旦初始化了 A 和向量 B 的值，就需要将这些值从 CPU 传输到 GPU。因为这里的 A 和 B 在 CPU 端，不在 GPU 端。A 和 B 是 CPU 的，`d_A`、`d_B` 是 GPU 端的。需要做的就是使用 `cudaMemcpy` 函数，意思是 CUDA 的内存复制操作。将从主机复制到设备，主机指的是 CPU，设备指的是 GPU，这就是这条指令的用法。将从变量 A 复制到变量 `d_A`，从 CPU 到 GPU。要复制的数据量等于这个大小。向量 B 也是如此，将移动数据，大小等于这个值，从 CPU 到 GPU，从变量 B 到 GPU 上的变量 `d_B`。不需要复制向量 C，因为向量 C 将存储结果。

一旦通过这两个指令或 API 将数据放入 GPU 内存，就需要执行向量加法核函数 `vectorAdd`。在这里启动，调用 `vectorAdd` 核函数，它将执行这里的这两条指令。使用的配置是一个块，每个块 1024 个线程。并且核函数将使用向量 A、向量 B、向量 C 以及所有这些向量的大小作为输入。一旦调用这个函数，执行将从 CPU 端转移到 GPU 端。它会执行所有线程，这意味着对所有元素执行加法操作，并将结果存储在 C 中。

一旦在 C 中有了结果，这一步之后，就需要将数据从 GPU 内存复制回 CPU 内存。如所示，将结果复制回主机，只是为了将这部分与这部分进行比较。`cudaMemcpyDeviceToHost`，这里有 `HostToDevice`。要传输或移动的数据量等于这个大小，并且从 GPU 向量移动到 CPU 向量。需要注意这里的顺序，这里从 GPU 到 CPU。

一旦在向量 C 上有了数据，而向量 C 在 CPU 端，就可以打印结果 C 值。这里使用 for 循环遍历所有元素并打印 A、B 和 C 值。如所示，A 加 B 等于 C。一旦打印完结果，就需要释放两端所有分配的空间，CPU 端和 GPU 端。如所示，这里使用 `cudaFree` 释放 GPU 的内存，CPU 端也是一

样，但只使用一个叫 `free` 的函数。它们非常相似，这里的 `free`，这里的 `cudaFree`。但因为这将释放 GPU 上的空间，这里使用了 CUDA 术语。

如所示，这些是主要步骤。需要做的就是来到这里并构建，以确保没有任何错误。看这里，这个部分从下面移到了右边。如所示，应用编译正确，构建成功，没有任何失败。现在需要做的就是运行这个应用。如所示，正在打印值，因为所有操作都已经执行完毕了。这些值来自这个 for 循环，这意味着之前所有的步骤都已正确执行。是的，这些就是结果。为了让它更简单，就在这里用比如 32 个值，然后编译并运行。让我们放大看看这些值。这是第一个值，A 的第一个元素加上 B 的第一个元素，这就是结果。如果还记得这里的初始化过程，这个部分用相同的值从 0 到 32 或 31 初始化了 A 向量，而 B 向量一开始以 32 结束。这些是 A 的值，从零开始到 31 结束。这就是初始化过程，可以选择任何其他值，看看它们是否正确。结果给出了正确的值。例如，23 加 9 等于 32。

现在需要做的就是改变块和线程的数量。这里暂不展开，将在后续章节中详细讲解。

20 CUDA 编程教程：向量加法扩展与 GPU 利用率优化

本章将继续讨论上一个项目。在上一个项目中，处理了两个向量，每个向量包含 1024 个元素，因此选择了一个块和 1024 个线程。本章计划将向量大小扩展到 2048。

20.1 问题提出与硬件限制

问题来了：是否可能用一个块来运行 2048 个线程？要回答这个问题，需要查阅白皮书。在 Ampere 架构的情况下，白皮书给出了明确的答案：这是不可能的。每个块的最大线程数是 1024。

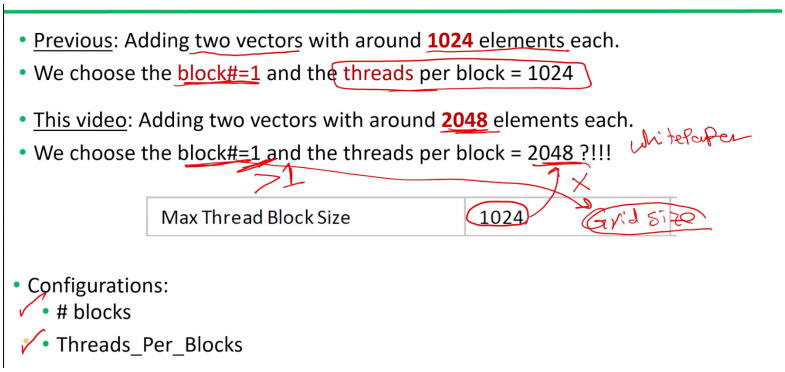


图 20.1:

20.2 解决方案：调整网格配置

为了解决这个问题，需要调整网格大小，即增加项目中的块数。与上一个项目使用一个块不同，这里需要使用多个块。可以灵活地决定块的数量以及每个块应包含的线程数，但必须遵循的关键规则是：确保应用程序中的总线程数与向量中的最大元素数相匹配。这意味着如果块数增加（这里的块数指的是网格大小），每个块的线程数必须相应减少，以确保总线程数保持在 2048。

这种情况确实会引起需要关注的性能问题，但将从一个简单的方法开始，只使用两个块，每个块配备 1024 个线程。性能影响将在后续阶段深入探讨。

20.3 索引映射原理

如图所示，有一个 2048 个元素的向量。方法是将这个向量分成两部分，每个部分由各自的块来执行。有一个 ID 为 0 的块用于第一部分，一个 ID 为 1 的块用于第二部分。第一部分有 1024 个线程，每个线程专门用于在单个向量元素上执行加法操作，这些线程的 ID 范围从 0 到 1023。这个配置在第二部分中镜像，即第二部分中的线程 ID 范围也是从 0 到 1023。

现在必须将两个部分中的块 ID 和线程 ID 与向量索引对齐，以确保每

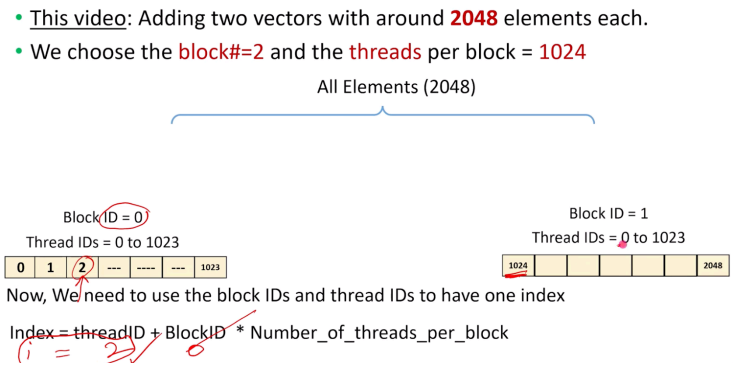


图 20.2:

个向量索引都与一个专用线程匹配来执行加法操作。对于第一部分，这很简单，因为线程 ID 与向量索引相同。然而在第二部分中，向量的索引从 1024 开始，一直到 2047，而线程 ID 仍然从 0 开始，在 1023 结束。

为了解决这个问题，特别是在第二部分，应用以下等式：向量中任何元素的索引计算为**线程 ID 加上块 ID 乘以每个块的总线程数**。举个例子来验证这个等式对两个部分都有效。

以向量中的第三个元素为例，块 ID 为 0、线程 ID 为 2。将这些值代入等式，计算结果为索引 2，这是正确的。另一方面，对于后半部分的第一个元素，其线程 ID 为 0，块 ID 为 1，得到索引 1024，这也是准确的。这个等式对两个部分都适用。

需要强调的是，这个等式至关重要，并将继续是 CUDA 编程工作的重要组成部分。

20.4 代码实现

请注意，这个等式在编码方面与上一章中的项目非常相似。主要任务涉及直接在核函数中应用索引等式。在这里，将替换这条指令，写成：

```
1 index = threadIdx.x + blockIdx.x * blockDim.x;
```

这里的 blockDim.x 指的是 X 方向上每个块的最大线程数。别忘了在这里将向量大小从 1024 改为 2048。同时需要修改应用程序配置，在这条指令中，需要将这个值从 1 改为 2，以反映对两个块的需求。

在满足其他要求后，现在是编译和运行项目的时候了，并确保每个块中有 1024 个线程。如图所示，项目运行良好，这些就是结果：A 的第一个元素加 B 的第一个元素将存储在 C 向量的第一个位置。

```
1
2  // This program computes the sum of two vectors of length N
3  // By: Nick from CoffeeBeforeArch
4
5  #include <cuda_runtime.h>
6  #include <device_launch_parameters.h>
7  #include <cstdlib>
8  #include <stdio.h>
9  #include <cassert>
10 #include <iostream>
11
12 // CUDA kernel for vector addition
13 __global__ void vectorAdd(int* a, int* b, int* c, int n) {
14        // Calculate global thread ID (tid)
15        int tid = (blockIdx.x * blockDim.x) + threadIdx.x;
16        // Vector boundary guard
17        if (tid < n) {
18               // Each thread adds a single element
19               c[tid] = a[tid] + b[tid];
20        }
21 }
22
```

```
23 // Initialize vector of size n to int between 0-99
24 void vector_init(int* a, int n) {
25         for (int i = 0; i < n; i++) {
26                 a[i] = rand() % 100;
27         }
28 }
29
30 // Check vector add result
31 void check_answer(int* a, int* b, int*c, int n) {
32         for (int i = 0; i < n; i++) {
33                 assert(c[i] == a[i] + b[i]);
34         }
35 }
36
37 int main() {
38         // Vector size of 2^16 (65536 elements)
39         int n = 1 << 16;
40         // Host vector pointers
41         int *h_a, *h_b, *h_c;
42         // Device vector pointers
43         int *d_a, *d_b, *d_c;
44         // Allocation size for all vectors
45         size_t bytes = sizeof(int) * n;
46
47         // Allocate host memory
48         h_a = (int*)malloc(bytes);
49         h_b = (int*)malloc(bytes);
50         h_c = (int*)malloc(bytes);
51
```

```
52          // Allocate device memory
53          cudaMalloc(&d_a, bytes);
54          cudaMalloc(&d_b, bytes);
55          cudaMalloc(&d_c, bytes);
56
57          // Initialize vectors a and b with random values between
                0 and 99
58          vector_init(h_a, n);
59          vector_init(h_b, n);
60
61          // Copy data from
62          cudaMemcpy(d_a, h_a, bytes, cudaMemcpyHostToDevice);
63          cudaMemcpy(d_b, h_b, bytes, cudaMemcpyHostToDevice);
64
65          // Threadblock size
66          int NUM_THREADS = 256;
67
68          // Grid size
69          int NUM_BLOCKS = (n + NUM_THREADS - 1) / NUM_THREADS;
70
71          // Launch kernel on default stream w/o shmem
72          vectorAdd<<<NUM_BLOCKS, NUM_THREADS>>>(d_a, d_b, d_c, n)
               ;
73
74          // Copy sum vector from device to host
75          cudaMemcpy(h_c, d_c, bytes, cudaMemcpyDeviceToHost);
76
77          // Check result for errors
78          check_answer(h_a, h_b, h_c, n);
```

```
79
80      // Free memory on host
81      free(h_a);
82      free(h_b);
83      free(h_c);
84
85      // Free memory on device
86      cudaFree(d_a);
87      cudaFree(d_b);
88      cudaFree(d_c);
89
90          printf("COMPLETED SUCCESFULLY\n");
91
92          return 0;
93  }
```

20.5 GPU 利用率分析

有两个问题需要探讨，同样的规则适用于 2048 个元素。

20.5.1 问题一：SM 的使用数量

第一个问题：基于选择的配置，有多少个 SM（流式多处理器）将被用来执行这个项目？提出这个问题是为了鼓励时刻留意设置对所分配的硬件资源的影响。这将如何影响分配的硬件资源？例如，在这个应用程序中，选择了两个块，每个块 1024 个线程，这将如何影响应用程序的性能？

在回答这个问题时，有必要参考之前提供的图片。如图所示，并且如前所述，一个线程在 GPU 核心上运行（这里的核心是 GPU 中最小的硬件单元，这是软件和硬件中的第二层级）；块在整个 SM 上运行；网格在整个

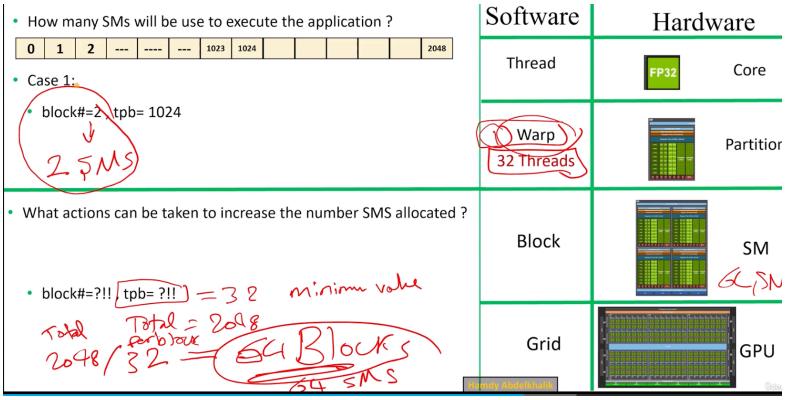


图 20.3:

GPU 上执行。线程束（Warp）在 SM 分区上执行，因为每个 SM 包含多个分区。

据此，在应用程序中，有两个块，这表明只有 2 个 SM 将参与执行这个应用程序。这个衡量标准被称为 **GPU 利用率**。假设拥有 Ampere 架构的 A100 GPU，这个 GPU 拥有 108 个 SM，那么 GPU 利用率将低于总可用 SM 的 2%。因为只使用了 2 个 SM，而剩下的 106 个 SM 处于空闲状态，没有工作。GPU 利用率是 GPU 性能的一个关键指标。在这个例子中，GPU 利用率极低。

20.5.2 问题二：提高 GPU 利用率的措施

现在看第二个问题：可以采取哪些措施来增加为运行此应用程序分配的 SM 数量？这里希望在不增加向量大小的情况下回答这个问题。这个问题也意味着如何提高 GPU 利用率。

这非常简单：如果能使用更多的 SM，那么就在使用更多的资源，这意味着提高了 GPU 利用率。应该增加块的数量，并且如果可能，应该增加每个块的线程数。在这种特定情况下，同时增加两者不是一个选项，因为总线程数需要与向量中的元素数量匹配。拥有比向量中元素总数更多的线程会引发一个问题：这些额外的线程将执行什么操作？

为了保持总线程数为 2048，策略是增加块数，同时减少每个块的线程数。应该使用的最小每块线程数等于 32，这相当于一个线程束（Warp）的大小，也就是在任何块中可以使用的最小线程数。需要将 2048 个线程除以 32，得到在此应用程序中可以使用的总块数：$2048 \div 32 = 64$ 个块。

这个配置意味着将使用该 GPU（即 A100 GPU）的 64 个 SM。与第一种情况中只有两个块和两个 SM 相比，这里有 64 个块将在 64 个 SM 上运行。

20.6 执行时间测量

可以使用两种不同的方法来计算执行时间。为了验证工作并了解这将如何影响应用程序性能，需要测量两种不同场景下的执行时间。一种方法是利用 CUDA 的内置 API，另一种可以使用分析器（Profiler）。本章将采用 CUDA API 方法。这里的分析器指的是 NVIDIA 的诊断工具，如 Nsight Systems 或 Nsight Compute。

现在打开 Visual Studio 看看代码。如图所示，添加了几行专门用于测量执行时间的代码。这个过程的第一条指令是声明两个变量 `start` 和 `stop`。之后应用一个名为 `cudaEventCreate` 的函数。这两个变量是由 `cudaEvent_t` 类型声明的，这个函数准备这些变量以供使用，类似于初始化。然后使用 `cudaEventRecord` 函数来开始记录时间，在这里将初始时间或开始时间存储在 `start` 变量中。

在这条指令中，正在 GPU 上执行核函数。正如之前讨论的那样，一旦执行完成，再次记录时间并将值存储在 `stop` 变量中。下一步是使用 `cudaEventElapsedTime` 函数，从 `stop` 值中减去 `start` 值。这个计算提供了以毫秒（ms）为单位的执行时间。

现在编译并运行项目。如图所示，这就是执行时间，请注意这个值。然而再运行一次，读取这个值并记在脑子里。现在再次运行——这印证了对这种计时方法的保留意见。正如观察到的，执行时间可能因每次运行而异。这种方法缺乏精确性，使用分析器是更好的选择。

20.7 性能对比与总结

总之，现在回到代码并调整块数和每块线程数。可以在这里选择 64 个块，每个块有 32 个线程，这是第二种场景。再次运行应用程序后，如图所示，可以轻易观察到当前的执行时间比第一种情况减少了。

现在应该清楚了。回到演示幻灯片，更深入地探讨这个话题。

在第一种场景中，有两个块，在总共 108 个可用 SM 中的两个上并行运行。这里需要记住，这些块中的每一个都配置了 1024 个线程，而 1024 个线程是在一个块中可以使用的最大线程数。这意味着这里的块尺寸非常大。

在第二种场景中，有 64 个块在 64 个 SM 上并行运行。与第一种场景相比，这里的块明显更小，每个块包含 32 个线程，而第一种场景中是 1024 个线程。

这里有一个问题：看看第一种情况或第二种情况下可用的 SM，告诉我在任何这些 SM 上将要执行的最大块数是多少。在第一种情况下，第一个 SM 上面将要执行的最大块数是多少？第二个 SM 呢？无论如何，可以说每个 SM 的最大块数等于 1。

再看第二种情况回答同样的问题。有 64 个块，每个 SM 将只执行一个块，这些块将分布在 64 个 SM 上。再次强调，每个 SM 的最大块数等于 1 个块。别忘了这两种情况下的块将并行执行，而不是顺序执行。这意味着执行时间将根据块的大小来计算。如果第二种情况下的块更小，第一种情况下的块更大，那么第二种情况将比第一种情况执行得更快。

希望现在明白了应用程序配置如何影响其性能。本章到此结束。

21 大规模向量加法：内存限制与分块处理策略

21.1 引言：从中小规模到超大规模向量

在之前的项目中，创建了 CUDA 程序来实现两个向量的加法，但那些向量的规模较小。本章将涵盖相同的应用程序——向量加法，但专注于尺寸大得多的向量。这个任务不会像之前的项目那样直截了当，将更加复杂。

很可能会遇到几个挑战，本章将在遇到每一个挑战或问题时指出它，并提供相应的解决方案。

在上一个项目中,向量的大小是 2048 个元素,分配了两个线程块(block),每个块包含 1024 个线程。本章将把向量大小增加到超过 4 亿个元素。

当前的向量大小设定为 $1024 \times 432 \times 1024$，确实超过了 4 亿个元素。

转到启动核函数（Kernel）的指令。在这条指令中，使用每块 1024 个线程。为了处理所有这些元素，必须调整应用程序中的主要配置，即块数（Grid Size）和每块线程数（Block Size）。这代表了 GPU 的最大限制。对于网格大小（即块的数量），使用 1024×432 个块。这段代码中的其他所有内容都与之前的项目类似，不编辑其他部分。这两个配置的乘积应等于每个向量中的元素总数。

现在编译并运行。由于在处理很大的向量尺寸，这个应用程序的执行将花费一些时间。如所示，一切正常，结果显示如图。可以在这里选择任意一行，测试 $A + B$ 是否等于 C 的值。无论是编译还是运行应用程序，都没有出现编译错误或运行时错误。

21.2 挑战一：超出主机内存限制

进一步增加向量尺寸时，问题出现了。到向量大小定义处复查一下，将 432 替换为 1024。现在每个向量都有超过 10 亿个元素。编译后发现没有任何编译错误。运行应用程序，如图所示，代码未能正确运行。代码中用于验证结果的 for 循环输出没有再显示在终端上。此外，可以注意到，如果错误码为零，则表示没有错误；如果是非零数字，则表示遇到了运行时错误。该应用程序中没有编译错误。

需要思考一下这个问题：为什么在这个应用程序中遇到了错误？可能的原因是什么?

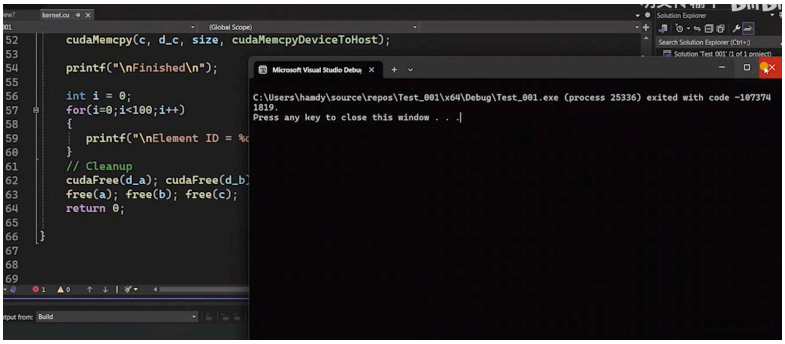


图 21.1:

21.3 调试策略：使用 printf 定位问题

下面提供解决方案。有很多调试策略可以找到确切的问题所在。在这种情况下，最有效的调试技巧之一是使用 `printf` 函数。在应用程序的不同位置放置许多 `printf` 指令，然后运行应用程序，利用终端上的输出来定位有问题的代码行。

例如，在开头写第一个 `printf("hello 00\n");`，然后将这条指令复制到不同的位置，并依次标记为 01、02 等。为什么需要在不同的位置放置 `printf` 指令？因为需要识别错误发生的具体位置。这与语法错误不同——如果移除一个分号，编译器会明确告知问题所在。但在当前情况下，遇到的是运行时错误而非编译错误。

编译成功后，看到终端打印了”hello 00” 和”hello 01”，但”hello 02” 没有打印。这意味着错误发生在”hello 01” 之后、”hello 02” 之前。结合上下文，发现 GPU 内存分配（`cudaMalloc`）本身没有报错，问题出在 CPU 端的主机内存分配上。

21.4 根本原因分析

为了理解这一点，查看 RAM（主机内存）中的可用空间。假设计算机总共有 16 GB RAM，当前已使用约 10 GB，剩余约 6 GB 可用空间。回到代码，当运行应用程序时，流程是为向量 A 分配空间并初始化，然后为 B

分配并初始化，最后为 C 分配空间。

当向量维度设置为 $1024 \times 1024 \times 1024$ 时，单个向量的元素数量为 $1,073,741,824$。由于每个整数元素占 4 字节，单个向量的大小为：

$$\frac{1024 \times 1024 \times 1024 \times 4\text{Bytes}}{1024^3} = 4 \text{ GB}$$

要存储三个向量（A、B、C），总共需要 12 GB 主机内存，而计算机上没有这么多可用空间。这就是导致运行时失败的根本原因。虽然 GPU 显存可能足够，但 `malloc` 在主机端分配内存时失败了。

21.5 解决方案：数据分块（Chunking）处理

解决此问题的方案是将每个向量分成小块（chunks）。具体使用多少块取决于可用内存。假设有 4 至 5 GB 可用空间，每个数据块的大小应小于此限制。

例如，将维度中的 1024 替换为 128。这意味着将把每个向量分成 8 块，因为 $1024/128 = 8$。使用这个” 块大小”（chunk size）来分割向量。处理流程如下：取 A 的第一块和 B 的第一块相加，将结果存入 C 的第一块；完成后释放这些临时缓冲区占用的内存；然后转到向量的第二部分，依此类推，直到处理完所有部分。

21.6 分块代码实现逻辑

以下是修改后的核心逻辑说明：

1. **参数定义**：定义总大小（total size）、数据块大小（chunk size）以及块数量（num chunks = total size / chunk size）。线程块大小仍保持为每块 1024 个线程。

2. **核函数**：GPU 核函数和初始化函数无需修改，仅需传入数据块指针而非完整向量指针。

3. **内存分配**：不再为整个向量分配空间，而是仅为当前数据块分配空间。使用 `malloc(chunk_size * sizeof(int))` 为 `chunk_A`、`chunk_B`、`chunk_C` 分配主机内存。GPU 端同样只分配数据块大小的显存。

4. **迭代处理**：使用 for 循环遍历所有数据块。在每次迭代中：

   - 计算当前偏移量（offset = i * chunk_size）；
   - 初始化或加载当前块的 A 和 B 数据；
   - 使用 `cudaMemcpy` 将数据块拷贝到 GPU；
   - 执行核函数进行向量加法；
   - 使用 `cudaMemcpy` 将结果从 GPU 拷回主机的 `chunk_C`；
   - 验证或输出当前块的结果；
   - 释放当前块的主机和设备内存，为下一次迭代做准备。

21.7 验证与结果

编译并运行修改后的代码。编译成功。运行时，终端打印了每个数据块的偏移量：0、128、256……直到第 8 个块。这证实了循环正确执行了 8 次迭代。

观察内存使用情况，仅使用了约 1 GB 左右的额外空间，因为只为两个数据块（A 和 B 的当前块）分配了空间，而不是三个完整的向量。当程序完成并释放内存后，可用空间恢复到了初始水平。

最终结果是正确的，例如 $13 + 77 = 90$ 等验证均通过。这就是本章讨论的第一个挑战及其完整的解决方案：通过数据分块技术突破主机内存限制，实现超大规模向量运算。

22 CUDA 运行时 API 详解与设备属性查询

本章将解释 CUDA 运行时 API（Runtime API）。CUDA 运行时 API 为 CUDA 提供了一个高级接口，使开发者能够高效地利用 NVIDIA GPU

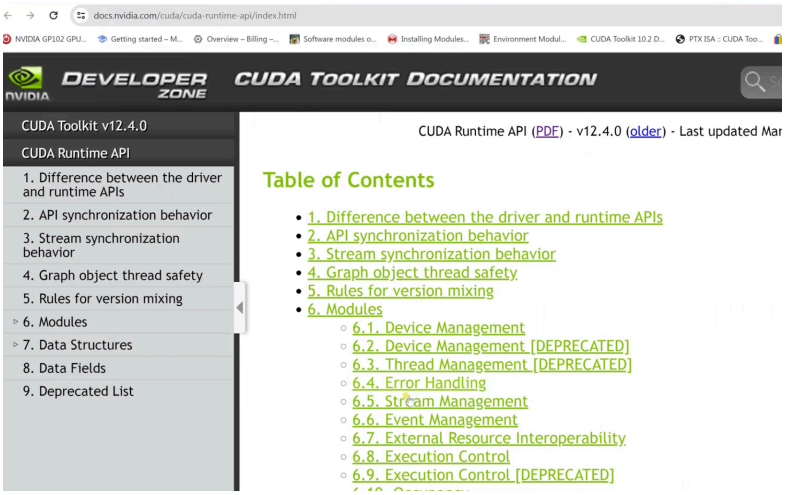


图 22.1:

的能力来完成广泛的计算任务。这些 API 简化了管理 GPU 设备、内存分配以及执行并行核函数的过程。

运行时 API 可供程序员用来开发优化或运行 GPU 加速的应用程序。这种方法是一个省时的替代方案，无需阅读 GPU 白皮书中详细的文档。例如，无需深入研究白皮书来查找设备限制和约束。正如在前一章中解释的那样，例如每个块的最大线程数，这些 API 提供了函数，可以直接从 GPU 查询该值。

查看专门用于运行时 API 函数的网页，如所示，有数十个函数，涵盖设备管理、内存处理以及异步计算。如果想从运行时 API 中找到任何函数，可以参考该网页。

22.1 核心函数：cudaGetDeviceProperties

API 函数最常见的例子之一是 `cudaGetDeviceProperties` 函数。该函数旨在查询支持 CUDA 的 GPU（或一般来说 NVIDIA GPU）的属性和能力。这个函数为开发者提供了重要的信息，例如 GPU 名称、总全局内存、每个块的最大线程数、计算能力以及其他指标。这些丰富的信息和指标使

开发者能够优化他们的 CUDA 应用程序。现在让我们将这个函数应用到我们的 GPU 上来查询它的属性。

22.2 项目创建与代码实现

下面这段代码的主要目的是查询 GPU 的一些属性，不需要进行任何计算，因此在主函数之前没有 GPU 核函数。

22.3 代码解析

```
#include <stdio.h>

int main() {
    int nDevices;

    // 获取系统中可用GPU设备的数量
    cudaGetDeviceCount(&nDevices);

    // 遍历所有设备
    for (int i = 0; i < nDevices; i++) {
        cudaDeviceProp prop;

        // 获取设备ID为i的属性
        cudaGetDeviceProperties(&prop, i);

        printf("Device Number: %d\n", i);
        printf(" Device name: %s\n", prop.name);
        printf(" Memory Clock Rate (KHz): %d\n", prop.
           memoryClockRate);
```

```
19          printf(" Memory Bus Width (bits): %d\n", prop.
               memoryBusWidth);
20
21          // 峰值内存带宽计算示例 (GB/s)
22          // 注意: 此处读取的是理论峰值而非实际运行值
23          double peakBandwidth = 2.0 * prop.memoryClockRate *
24                                 (prop.memoryBusWidth / 8) / 1.0e6;
25          printf(" Peak Memory Bandwidth (GB/s): %f\n",
               peakBandwidth);
26
27          printf(" Total Global Memory (Bytes): %zu\n", prop.
               totalGlobalMem);
28          printf(" Compute Capability: %d.%d\n", prop.major, prop.
               minor);
29          printf(" SM Count: %d\n", prop.multiProcessorCount);
30
31          // 网格与块的限制
32          printf(" Max threads per block: %d\n", prop.
               maxThreadsPerBlock);
33          printf(" Max block dims: (%d, %d, %d)\n",
34                 prop.maxThreadsDim[0],
35                 prop.maxThreadsDim[1],
36                 prop.maxThreadsDim[2]);
37          printf(" Max grid dims: (%d, %d, %d)\n",
38                 prop.maxGridSize[0],
39                 prop.maxGridSize[1],
40                 prop.maxGridSize[2]);
41      }
42
```

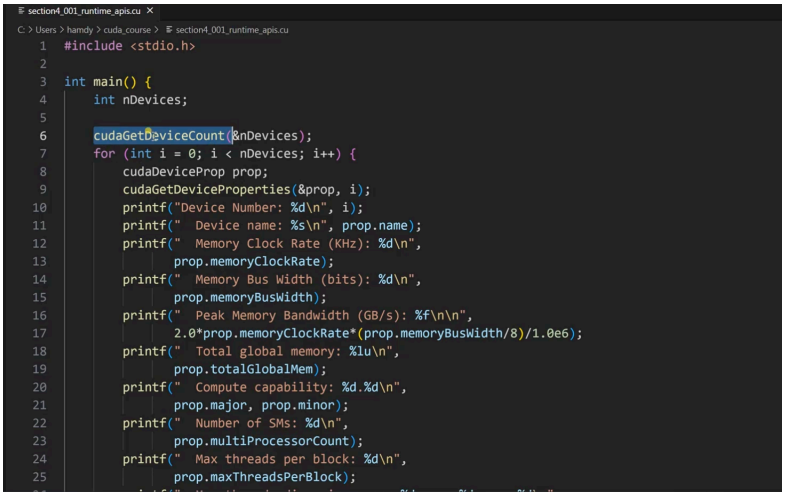


图 22.2:

```
43      return 0;
44  }
```

从代码开始分析：

1. 我们包含 `stdio.h` 库，以便能够使用 `printf`。
2. 声明 `main` 函数，并为整型变量在内存中分配空间。
3. 在第 6 行我们开始使用运行时 API 函数 `cudaGetDeviceCount`。函数的名称指的是它的任务：获取系统中设备的数量。如果系统中有 3 个 NVIDIA GPU，那么数字 3 将存储在 `nDevices` 中。
4. 使用 `for` 循环遍历计算机上所有的设备。我们从 `i=0` 开始，这指的是 GPU 编号 0。
5. 创建一个类型为 `cudaDeviceProp` 的结构体 `prop`。结构体意味着 `prop` 将包含不同的值，而不仅仅是一个值。

6. 使用 `cudaGetDeviceProperties` 函数读取设备 `i` 的属性，并将结果存储在 `prop` 结构体中。

7. 通过点操作符（`prop.name`, `prop.memoryClockRate` 等）访问并打印各项指标。

22.4 编译与运行验证

使用 NVCC 编译器编译代码：

```
nvcc -o runtime_api section_4_01_runtime_api.cu
```

运行可执行文件后，输出示例如下（以 RTX 3090 为例）：

- **设备编号**: 0
- **设备名称**: GeForce RTX 3090
- **内存总线宽度**: 384 bits（与 TechPowerUp 数据库一致）
- **总全局内存**: 约 24 GB（需将字节数除以 1024^3 转换）
- **计算能力**: 8.6
- **SM 数量**: 82
- **每块最大线程数**: 1024
- **最大块维度**: X=1024, Y=1024, Z=64

思考题：已知每个 Warp 由 32 个线程组成，根据每块最大线程数 1024，每个块的最大 Warp 数量是多少?

$$\text{Max Warps per Block} = \frac{1024}{32} = 32$$

请注意区分” 块”（软件概念）和”SM”（硬件概念）。块在 SM 上执行，这个值可用于计算另一个性能指标——占用率（Occupancy）。

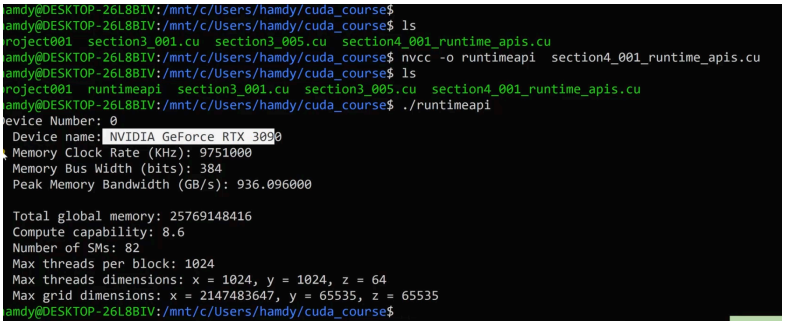


图 22.3:

22.5 关键要点总结

1. **参数传递机制**: 运行时 API 函数的运行方式与标准函数不同。它们不通过典型的返回机制返回结果，而是将输出分配给函数调用中提供的指针参数。例如 `cudaGetDeviceCount(&nDevices)` 通过参数存储检测结果。

2. **错误检查**: 运行时 API 函数返回它们的执行状态（`cudaError_t`），该状态可以被捕获以供后续检查。如果成功则返回 `cudaSuccess`，否则返回错误代码，可用于分析故障原因。

3. **效率优势**: 使用运行时 API 节省了大量时间。替代方案是在白皮书中查阅大量内容才能获取每一个参数值。虽然白皮书中有表格包含许多值，但并非全部，且 API 能动态反映当前设备的真实状态。

需要掌握如何使用运行时 API，因为在后续章节中，检查或分析性能时需要这些值。

23 NVIDIA 系统管理接口（nvidia-smi）详解

本章将解释 NVIDIA 系统管理接口(System Management Interface)，即 `nvidia-smi`。`nvidia-smi` 是 NVIDIA 提供的一个命令行工具，为 NVIDIA

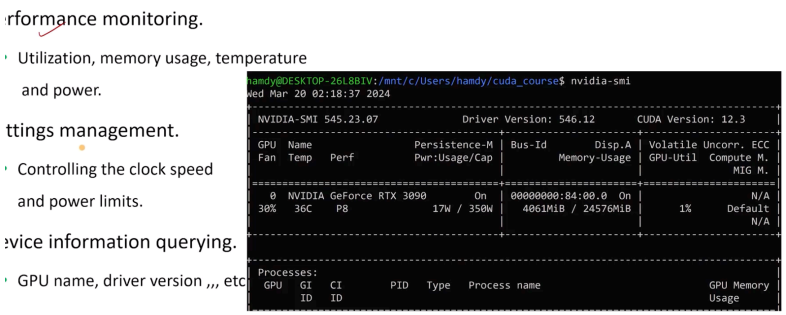


图 23.1:

GPU 提供监控和管理功能，可实时检查 NVIDIA GPU 状态。本章将探讨 `nvidia-smi` 的一些关键特性及使用示例。

23.1 nvidia-smi 的三大关键领域

`nvidia-smi` 可用于三个关键领域：性能监控、设置管理和设备信息查询。

1. **性能监控**: 可用于实时监控 GPU 性能，包括 GPU 资源利用率、内存使用情况、温度以及功耗。这些数据对于高效优化 GPU 资源至关重要。

2. **设置管理**: 提供了调整或控制 GPU 内存和核心时钟速度的灵活性。不仅可以读取性能指标，还可以控制影响性能的参数。此外，可以为 GPU 功耗设定限制。

3. **设备信息查询**: 提供关于 GPU 型号、驱动程序版本、CUDA 版本以及当前时钟速度的全面详细信息。这确保开发者和管理员拥有必要的数据来做出关于软件兼容性和性能调优的明智决策。

23.2 基础命令与输出解读

从最简单的命令开始：`nvidia-smi`。通过执行这个简单命令，我们可以获得性能指标的全面概览。输出包含两个主要表格。

23.3 第一个表格：性能概览

第一行显示两个重要指标：

- **驱动程序版本**: 例如 546.12。确认驱动版本有助于排查库兼容性问题。
- **CUDA 版本**: 例如 12.3。可通过 `nvcc --version` 验证编译器版本是否匹配。

第二行是指标名称，第三行是对应值。该表格分为多列：

- **GPU 信息**: GPU 名称（如 GeForce RTX 3090）、GPU ID（如 0）。
- **功耗与温度**: 当前功耗（如 31W）/ 最大功耗限制（如 350W）；当前温度（如 53°C）。
- **内存使用**: 已用显存 / 总显存（如 4000 MiB / 24 GiB）。
- **GPU 利用率**: 即使未运行自定义 CUDA 程序，若后台有录屏等应用调用 GPU 编码器，利用率也可能显示为 30%-40%。注意：`nvidia-smi` 的进程列表通常只显示 CUDA 计算进程，不一定显示视频编码/解码进程。

23.4 第二个表格：进程列表

此表格显示当前在 GPU 上运行的计算进程。例如运行矩阵乘法（Matrix Multiplication）应用时：

- GPU 利用率从空闲时的 ∼19% 升至 100%。

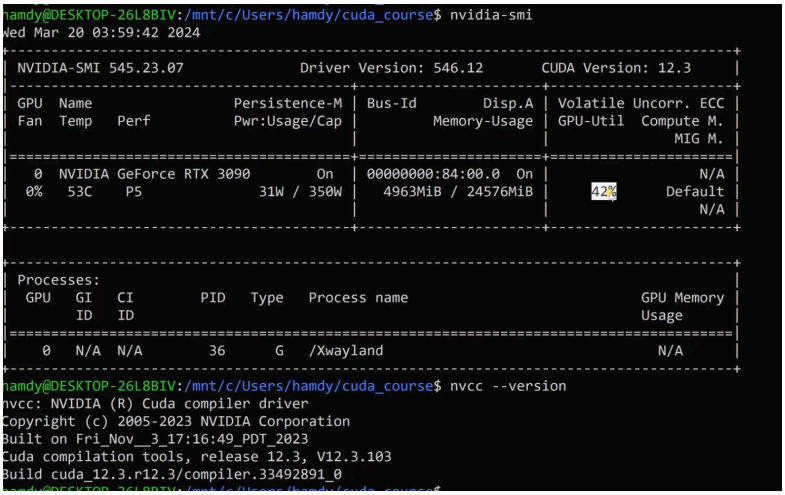


图 23.2:

- 功耗从 ~29W 升至 ~318W（接近 350W 上限）。
- 温度从 36°C 升至 63°C。
- 显存占用显著增加（如增至 ~8 GiB）。

23.5 持续监控与数据导出

23.6 循环查询模式

使用 `-l` (loop) 参数实现持续监控：

```
# 每5秒刷新一次
nvidia-smi -l 5

# 每2秒刷新一次
nvidia-smi -l 2
```

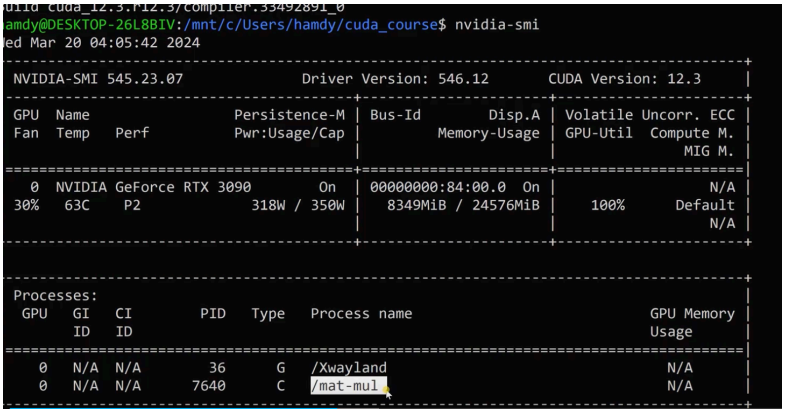


图 23.3:

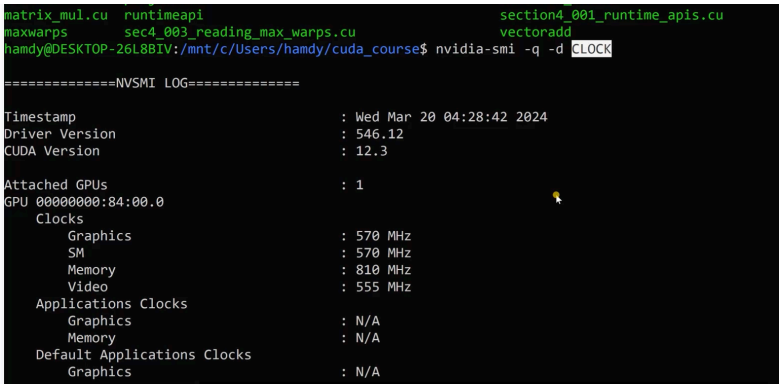


图 23.4:

按 Ctrl+C 退出。通过对比刷新间隔内的数值变化，可以观察应用程序执行期间的实时性能波动。

23.7 定向查询与 CSV 导出

使用 --query-gpu 和 --format 参数提取特定指标：

```
# 查询指定指标并以CSV格式输出
nvidia-smi --query-gpu=name,driver_version,temperature.gpu \
```

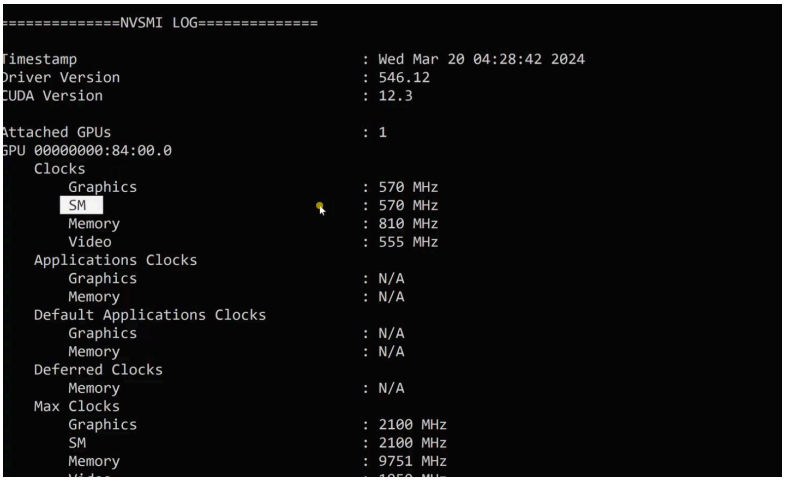


图 23.5:

```
          --format=csv

# 将结果保存到文件（Linux重定向）
nvidia-smi --query-gpu=name,driver_version,temperature.gpu \
          --format=csv > gpu_stats.csv
```

输出的 CSV 文件可直接用 Excel 打开进行分析。

23.8 设置管理：功耗与时钟控制

23.9 功耗限制

使用 `-pl` (power limit) 设置功耗上限：

```
# 将GPU 0的功耗限制设为200W
sudo nvidia-smi -i 0 -pl 200
```

注意: 并非所有 GPU 都支持修改功耗限制，需提前确认硬件支持情况。

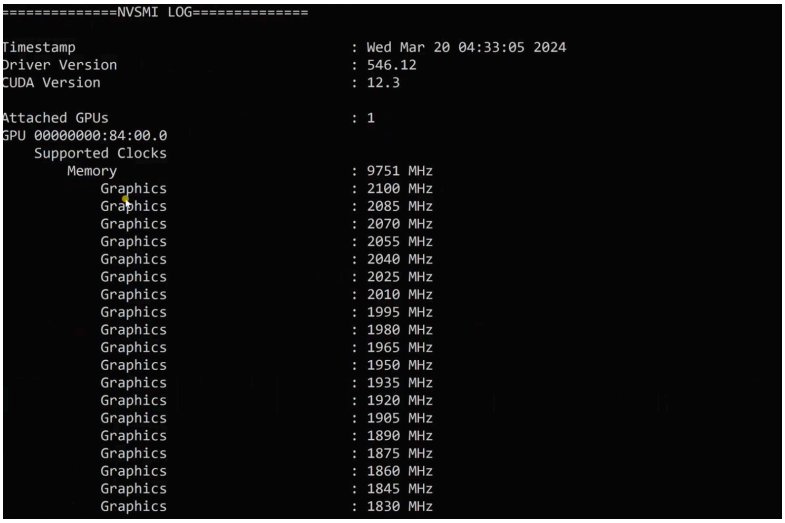


图 23.6:

23.10 持久模式（Persistence Mode）

持久模式保持 GPU 驱动常驻内存，避免频繁加载/卸载开销。修改时钟等设置前可能需要先启用：

```
# 启用持久模式
sudo nvidia-smi -i 0 -pm 1

# 禁用持久模式
sudo nvidia-smi -i 0 -pm 0
```

23.11 时钟速度查询与控制

23.12 查询时钟

使用 `-q` (query) 查看时钟详情：

```
nvidia-smi -q -d CLOCK
```

输出区分“当前时钟”与“最大时钟”。空闲时 GPU 会自动降频节能（如 SM 时钟从 2100 MHz 降至 570 MHz）；负载增加时自动升频（如升至 1695 MHz）。

```
colin@colin-BATTLE-AX-B860M-P-WIFI:~$ nvidia-smi -q -d CLOCK

==============NVSMI LOG==============

Timestamp                                           : Sat Sep 26
    11:19:58 2026
Driver Version                                      : 595.84
CUDA Version                                        : 13.2

Attached GPUs                                       : 1
GPU 00000000:02:00.0
    Clocks
        Graphics                                    : 180 MHz
        SM                                          : 180 MHz
        Memory                                      : 405 MHz
        Video                                       : 600 MHz
    Applications Clocks
        Graphics                                    : Requested
            functionality has been deprecated
        Memory                                      : Requested
            functionality has been deprecated
    Default Applications Clocks
        Graphics                                    : Requested
            functionality has been deprecated
        Memory                                      : Requested
```

```
            functionality has been deprecated
    Deferred Clocks
        Memory                                    : N/A
    Max Clocks
        Graphics                                  : 3090 MHz
        SM                                        : 3090 MHz
        Memory                                    : 14001 MHz
        Video                                     : 3090 MHz
    Max Customer Boost Clocks
        Graphics                                  : N/A
    SM Clock Samples
        Duration                                  : N/A
        Number of Samples                         : N/A
        Max                                       : N/A
        Min                                       : N/A
        Avg                                       : N/A
    Memory Clock Samples
        Duration                                  : N/A
        Number of Samples                         : N/A
        Max                                       : N/A
        Min                                       : N/A
        Avg                                       : N/A
    Clock Policy
        Auto Boost                                : N/A
        Auto Boost Default                        : N/A
```

23.13 查询支持的时钟频率

设置固定时钟前，必须查询 GPU 支持的离散频率点：

```
nvidia-smi -q -d SUPPORTED_CLOCKS
```

只能选择列表中存在的值（如内存 9501 MHz、5001 MHz；SM 12085 MHz、12070 MHz 等），不支持任意值。

23.14 设置应用时钟

使用 `-ac` (application clocks) 锁定 SM 和内存时钟：

```
# 格式：-ac <mem_clock>,<sm_clock>
sudo nvidia-smi -i 0 -ac 9501,12085
```

23.15 重置为默认值

使用 `-r` 或 `-rac` / `-pmc` 重置设置：

```
# 重置应用时钟为默认
sudo nvidia-smi -i 0 -rac

# 重置功耗限制为默认
sudo nvidia-smi -i 0 -pmc
```

23.16 帮助与总结

查看所有可用选项：

```
nvidia-smi -h
```

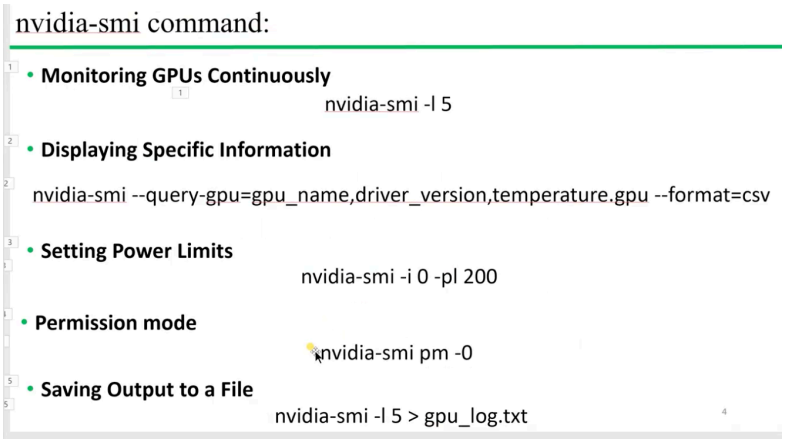


图 23.7:

```
1  colin@colin-BATTLE-AX-B860M-P-WIFI:~$ nvidia-smi -h
2  NVIDIA System Management Interface -- v595.84
3
4  NVSMI provides monitoring information for Tesla and select
       Quadro devices.
5  The data is presented in either a plain text or an XML format,
       via stdout or a file.
6  NVSMI also provides several management operations for changing
       the device state.
7
8  Note that the functionality of NVSMI is exposed through the
       NVML C-based
9  library. See the NVIDIA developer website for more information
       about NVML.
10 Python wrappers to NVML are also available. The output of NVSMI
       is
11 not guaranteed to be backwards compatible; NVML and the
```

```
    bindings are backwards
12  compatible.
13
14  http://developer.nvidia.com/nvidia-management-library-nvml/
15  http://pypi.python.org/pypi/nvidia-ml-py/
16  Supported products:
17  - Full Support
18      - All Tesla products, starting with the Kepler architecture
19      - All Quadro products, starting with the Kepler
            architecture
20      - All GRID products, starting with the Kepler architecture
21      - GeForce Titan products, starting with the Kepler
            architecture
22  - Limited Support
23      - All Geforce products, starting with the Kepler
            architecture
24  nvidia-smi [OPTION1 [ARG1]] [OPTION2 [ARG2]] ...
25
26      -h,  --help             Print usage information and exit.
27
28           --version          Print version information and exit.
29
30    LIST OPTIONS:
31
32      -L,  --list-gpus        Display a list of GPUs connected to
             the system.
33
34      -B,  --list-excluded-gpus Display a list of excluded GPUs
            in the system.
```

```
35
36    SUMMARY OPTIONS:
37
38      <no arguments>          Show a summary of GPUs connected to
            the system.
39
40      -col, --columns         Show a summary of GPUs connected to
            the system in multi-column format.
41
42      [plus any of]
43
44      -i,  --id=              Target a specific GPU.
45      -f,  --filename=        Log to a specified file, rather
           than to stdout.
46      -l,  --loop=            Probe until Ctrl+C at specified
           second interval.
47
48    QUERY OPTIONS:
49
50      -q,  --query            Display GPU or Unit info.
51
52      [plus any of]
53
54      -u,  --unit             Show unit, rather than GPU,
           attributes.
55      -i,  --id=              Target a specific GPU or Unit.
56      -f,  --filename=        Log to a specified file, rather
           than to stdout.
57      -x,  --xml-format       Produce XML output.
```

```
58          --dtd                When showing xml output, embed DTD.
59      -d, --display=           Display only selected information:
            MEMORY,
60                                   UTILIZATION, ECC, TEMPERATURE,
                                        POWER, CLOCK,
61                                   COMPUTE, PIDS, PERFORMANCE,
                                        SUPPORTED_CLOCKS,
62                                   PAGE_RETIREMENT, ACCOUNTING,
                                        ENCODER_STATS,
63                                   SUPPORTED_GPU_TARGET_TEMP,
                                        VOLTAGE, FBC_STATS
64                                   ROW_REMAPPER,
                                        GSP_FIRMWARE_VERSION
65                                   POWER_SMOOTHING, POWER_PROFILES
66                               Flags can be combined with comma e.
                                    g. ECC,POWER.
67                               Sampling data with max/min/avg is
                                    also returned
68                               for POWER, UTILIZATION and CLOCK
                                    display types.
69                               Doesn't work with -u or -x flags.
70      -l, --loop=              Probe until Ctrl+C at specified
            second interval.
71
72      -lms, --loop-ms=         Probe until Ctrl+C at specified
            millisecond interval.
73
74  SELECTIVE QUERY OPTIONS:
75
```

```
76      Allows the caller to pass an explicit list of properties to
            query.
77
78      [one of]
79
80      --query-gpu             Information about GPU.
81                              Call --help-query-gpu for more info
                                    .
82      --query-supported-clocks List of supported clocks.
83                              Call --help-query-supported-clocks
                                    for more info.
84      --query-compute-apps    List of currently active compute
            processes.
85                              Call --help-query-compute-apps for
                                    more info.
86      --query-accounted-apps  List of accounted compute processes
            .
87                              Call --help-query-accounted-apps
                                    for more info.
88                              This query is not supported on vGPU
                                    host.
89      --query-retired-pages   List of device memory pages that
            have been retired.
90                              Call --help-query-retired-pages for
                                    more info.
91      --query-remapped-rows   Information about remapped rows.
92                              Call --help-query-remapped-rows for
                                    more info.
93
```

```
94      [mandatory]
95
96      --format=                Comma separated list of format
           options:
97                                 csv - comma separated values (
                                      MANDATORY)
98                                 noheader - skip the first line
                                      with column headers
99                                 nounits - don't print units for
                                      numerical
100                                       values
101
102     [plus any of]
103
104     -i,  --id=               Target a specific GPU or Unit.
105     -f,  --filename=         Log to a specified file, rather
           than to stdout.
106     -l,  --loop=             Probe until Ctrl+C at specified
           second interval.
107     -lms, --loop-ms=         Probe until Ctrl+C at specified
           millisecond interval.
108
109   DEVICE MODIFICATION OPTIONS:
110
111     [any one of]
112
113     -pm, --persistence-mode= Set persistence mode: 0/DISABLED,
           1/ENABLED
114     -e,  --ecc-config=       Toggle ECC support: 0/DISABLED, 1/
```

```
        ENABLED
115     -p,  --reset-ecc-errors= Reset ECC error counts: 0/VOLATILE
        , 1/AGGREGATE
116     -c,  --compute-mode=     Set MODE for compute applications:
117                              0/DEFAULT, 1/EXCLUSIVE_THREAD (
                                    DEPRECATED),
118                              2/PROHIBITED, 3/EXCLUSIVE_PROCESS
119          --gom=              Set GPU Operation Mode:
120                                  0/ALL_ON, 1/COMPUTE, 2/LOW_DP
121     -r   --gpu-reset         Trigger reset of the GPU.
122                              Can be used to reset the GPU HW
                                    state in situations
123                              that would otherwise require a
                                    machine reboot.
124                              Typically useful if a double bit
                                    ECC error has
125                              occurred.
126                              Reset operations are not guaranteed
                                     to work in
127                              all cases and should be used with
                                    caution.
128                              Reset triggered without extra
                                    arguments, will be a.
129                              default Function Level Reset (FLR).
                                     To issue a
130                              Bus Reset, use -r bus.
131     -vm  --virt-mode=        Switch GPU Virtualization Mode:
132                              Sets GPU virtualization mode to 3/
                                    VGPU or 4/VSGA
```

```
                            Virtualization mode of a GPU can
                               only be set when
                            it is running on a hypervisor.
    -lgc --lock-gpu-clocks= Specifies <minGpuClock,maxGpuClock>
        clocks as a
                               pair (e.g. 1500,1500) that
                                  defines the range
                               of desired locked GPU clock
                                  speed in MHz.
                               Setting this will supersede
                                  application clocks
                               and take effect regardless if
                                  an app is running.
                               Input can also be a singular
                                  desired clock value
                               (e.g. <GpuClockValue>).
                                  Optionally, --mode can be
                               specified to indicate a special
                                   mode.
    -m   --mode=            Specifies the mode for --lock-gpu-
       clocks.
                               Valid modes: 0, 1
    -rgc --reset-gpu-clocks
                            Resets the GPU clocks to the
                               default values.
    -lmc --lock-memory-clocks= Specifies <minMemClock,
       maxMemClock> clocks as a
                               pair (e.g. 5100,5100) that
                                  defines the range
```

```
149                                 of desired locked Memory clock
                                       speed in MHz.
150                                 Input can also be a singular
                                       desired clock value
151                                 (e.g. <MemClockValue>).
152     -rmc --reset-memory-clocks
153                               Resets the Memory clocks to the
                                     default values.
154     -lmcd --lock-memory-clocks-deferred=
155                                 Specifies memClock clock to
                                       lock. This limit is
156                                 applied the next time GPU is
                                       initialized.
157                                 This is guaranteed by unloading
                                        and reloading the kernel
                                       module.
158                                 Requires root.
159     -rmcd --reset-memory-clocks-deferred
160                               Resets the deferred Memory clocks
                                     applied.
161     -ac  --applications-clocks= Specifies <memory,graphics>
           clocks as a
162                                 pair (e.g. 2000,800) that
                                       defines GPU's
163                                 speed in MHz while running
                                       applications on a GPU.
164     -rac --reset-applications-clocks
165                               Resets the applications clocks to
                                     the default values.
```

```
166     -pl  --power-limit=     Specifies maximum power management
           limit in watts.
167                             Takes an optional argument --scope.
168     -sc  --scope=           Specifies the device type for --
           scope: 0/GPU, 1/TOTAL_MODULE (Grace Hopper Only)
169     -cc  --cuda-clocks=     Overrides or restores default CUDA
           clocks.
170                             In override mode, GPU clocks higher
                                 frequencies when running CUDA
                                applications.
171                             Only on supported devices starting
                                from the Volta series.
172                             Requires administrator privileges.
173                             0/RESTORE_DEFAULT, 1/OVERRIDE
174     -am  --accounting-mode= Enable or disable Accounting Mode:
           0/DISABLED, 1/ENABLED
175     -caa --clear-accounted-apps
176                             Clears all the accounted PIDs in
                                the buffer.
177           --auto-boost-default= Set the default auto boost
                 policy to 0/DISABLED
178                             or 1/ENABLED, enforcing the change
                                only after the
179                             last boost client has exited.
180           --auto-boost-permission=
181                             Allow non-admin/root control over
                                auto boost mode:
182                             0/UNRESTRICTED, 1/RESTRICTED
183     -mig --multi-instance-gpu= Enable or disable Multi Instance
```

```
        GPU: 0/DISABLED, 1/ENABLED
                              Requires root.
  -gtt --gpu-target-temp= Set GPU Target Temperature for a
     GPU in degree celsius.
                              Requires administrator privileges
  -den, --dram-encryption= Toggle DRAM Encryption support: 0/
     DISABLED, 1/ENABLED
                              Requires root.
        --set-hostname=    Set the hostname associated with
           the GPU.
                              Requires root.
        --get-hostname     Get the hostname associated with
           the GPU.

 [plus optional]

  -i,  --id=                Target a specific GPU.
  -eow, --error-on-warning Return a non-zero error for
     warnings.

UNIT MODIFICATION OPTIONS:

  -t,  --toggle-led=        Set Unit LED state: 0/GREEN, 1/
     AMBER

 [plus optional]

  -i,  --id=                Target a specific Unit.
```

```
206   SHOW DTD OPTIONS:
207
208         --dtd               Print device DTD and exit.
209
210      [plus optional]
211
212      -f,  --filename=        Log to a specified file, rather
            than to stdout.
213      -u,  --unit             Show unit, rather than device, DTD.
214
215      --debug=                Log encrypted debug information to
            a specified file.
216
217   Device Monitoring:
218      dmon                    Displays device stats in scrolling
            format.
219                              "nvidia-smi dmon -h" for more
                                    information.
220
221      daemon                  Runs in background and monitor
            devices as a daemon process.
222                              This is an experimental feature.
                                    Not supported on Windows
                                    baremetal
223                              "nvidia-smi daemon -h" for more
                                    information.
224
225      replay                  Used to replay/extract the
            persistent stats generated by daemon.
```

```
                            This is an experimental feature.
                            "nvidia-smi replay -h" for more
                               information.

  Process Monitoring:
    pmon                    Displays process stats in scrolling
         format.
                            "nvidia-smi pmon -h" for more
                               information.

  TOPOLOGY:
    topo                    Displays device/system topology. "
       nvidia-smi topo -h" for more information.

  DRAIN STATES:
    drain                   Displays/modifies GPU drain states
       for power idling. "nvidia-smi drain -h" for more
       information.

  NVLINK:
    nvlink                  Displays device nvlink information.
         "nvidia-smi nvlink -h" for more information.

  C2C:
    c2c                     Displays device C2C information. "
       nvidia-smi c2c -h" for more information.

  CLOCKS:
    clocks                  Control and query clock information
```

```
      . "nvidia-smi clocks -h" for more information.
247
248   ENCODER SESSIONS:
249     encodersessions         Displays device encoder sessions
        information. "nvidia-smi encodersessions -h" for more
        information.
250
251   FBC SESSIONS:
252     fbcsessions             Displays device FBC sessions
        information. "nvidia-smi fbcsessions -h" for more
        information.
253
254   GRID vGPU:
255     vgpu                    Displays vGPU information. "nvidia-
        smi vgpu -h" for more information.
256
257   MIG:
258     mig                     Provides controls for MIG
        management. "nvidia-smi mig -h" for more information.
259
260   COMPUTE POLICY:
261     compute-policy          Control and query compute policies.
         "nvidia-smi compute-policy -h" for more information.
262
263   BOOST SLIDER:
264     boost-slider            Control and query boost sliders. "
        nvidia-smi boost-slider -h" for more information.
265
266   POWER HINT:
```

```
267     power-hint                Estimates GPU power usage. "nvidia-
            smi power-hint -h" for more information.
268
269   BASE CLOCKS:
270     base-clocks               Query GPU base clocks. "nvidia-smi
            base-clocks -h" for more information.
271
272   CONFIDENTIAL COMPUTE:
273     conf-compute              Control and query confidential
            compute. "nvidia-smi conf-compute -h" for more
            information.
274
275   GPU PERFORMANCE MONITORING:
276     gpm                       Control and query GPU performance
            monitoring unit. "nvidia-smi gpm -h" for more
            information.
277
278   PCI:
279     pci                       Display device PCI information. "
            nvidia-smi pci -h" for more information.
280
281   Power Smoothing:
282     power-smoothing           Control and query power smoothing
            information. "nvidia-smi power-smoothing -h" for more
            information.
283
284   Power Profiles:
285     power-profiles            Control and query workload power
            profiles information. "nvidia-smi power-profiles -h" for
```

```
        more information.
286
287   RUSD:
288     rusd                        Set RUSD (read only shared data
          buffer) settings. "nvidia-smi rusd -h" for more
          information.
289
290   PRM:
291     prm                         Query GPU PRM registers. "nvidia-
          smi prm -h" for more information.
292
293   SOC:
294     soc                         Query system on chip metrics. "
          nvidia-smi soc -h" for more information.
295
296   Please see the nvidia-smi(1) manual page for more detailed
        information.
```

nvidia-smi 是一个非常重要的命令。在后续章节中,将使用 nvidia-smi 来检查各种性能指标，验证优化效果，并诊断 GPU 运行状态。

24 GPU 占用率与延迟隐藏技术详解

24.1 引言：占用率的概念区分

本章将深入探讨占用率（Occupancy）和利用率这两个概念，即比较不同架构时所需的关键参数。本章开头有必要重温课程中一个重要章节的内容——讨论了一系列关键参数，如内存带宽和指令吞吐量。在本章中，将引入占用率作为另一个评估性能的关键术语。

这个术语的真正问题在于，大多数涉及该主题的教程倾向于只解释理论占用率，而不是实际达到的占用率。GPU 的占用率是衡量 GPU 中资源利用率的一个指标。在 GPU 性能领域，占用率被分为两种不同的类型：理论占用率代表理想化场景，达到的占用率表示 GPU 资源的实际使用情况。

理论占用率的计算方法是将核函数中使用的 Warp 数除以每个 SM（流式多处理器）所能容纳的最大 Warp 数。这里涉及两个关键值：分子根据应用程序中使用的 Warp 数而变化；分母对应 GPU 的 Warp 数限制，这个限制可以从 GPU 白皮书中获取，或者使用前文介绍的运行时 API 获取。

```
1
2  colin@colin-BATTLE-AX-B860M-P-WIFI:~/cuda_ebook$ sudo /usr/
       local/cuda-13/bin/ncu ./vec_add
3  ==PROF== Connected to process 3036 (/home/colin/cuda_ebook/
       vec_add)
4  ==PROF== Profiling "vectorAdd" - 0: 0%....50%....100% - 9
       passes
5  COMPLETED SUCCESSFULLY
6  ==PROF== Disconnected from process 3036
7  [3036] vec_add@127.0.0.1
8    vectorAdd(const int *, const int *, int *, int) (64, 1, 1)x
         (1024, 1, 1), Context 1, Stream 7, Device 0, CC 12.0
9      Section: GPU Speed Of Light Throughput
10     ----------------------- ----------- ------------
11     Metric Name          Metric Unit Metric Value
12     ----------------------- ----------- ------------
13     DRAM Frequency              Ghz       13.45
14     SM Frequency                Ghz        2.40
15     Elapsed Cycles            cycle      5,931
16     Memory Throughput             %       24.84
```

```
17      DRAM Throughput                %       24.84
18      Duration                      us        2.46
19      L1/TEX Cache Throughput        %       10.44
20      L2 Cache Throughput            %       13.04
21      SM Active Cycles           cycle    2,484.90
22      Compute (SM) Throughput        %        6.93
23      ----------------------- ----------- ------------
24
25      OPT  This kernel grid is too small to fill the available
           resources on this device, resulting in only 0.91 full
26           waves across all SMs. Look at Launch Statistics for
                more details.
27
28      Section: Launch Statistics
29      -------------------------------- ---------------
           ---------------
30      Metric Name                         Metric Unit  Metric Value
31      -------------------------------- ---------------
           ---------------
32      Block Size                                              1,024
33      Cluster Scheduling Policy                         PolicySpread
34      Cluster Size                                               0
35      Function Cache Configuration                   CachePreferNone
36      Grid Size                                                 64
37      Preferred Cluster Size                                     0
38      Registers Per Thread           register/thread            16
39      Shared Memory Configuration Size      Kbyte          8.19
40      Driver Shared Memory Per Block   Kbyte/block          1.02
41      Dynamic Shared Memory Per Block   byte/block             0
```

```
    Static Shared Memory Per Block   byte/block            0
    # SMs                                  SM             70
    Stack Size                                          1,024
    Threads                               thread        65,536
    # TPCs                                               35
    Enabled TPC IDs                                     all
    Uses Green Context                                    0
    Waves Per SM                                       0.91
    -------------------------------- ---------------
       ---------------

    OPT  Est. Speedup: 8.571%
         The grid for this launch is configured to execute only
             64 blocks, which is less than the 70
            multiprocessors
         used. This can underutilize some multiprocessors. If
            you do not intend to execute this kernel
            concurrently
         with other workloads, consider reducing the block size
             to have at least one block per multiprocessor or
         increase the size of the grid to fully utilize the
            available hardware resources. See the Hardware
            Model
         (https://docs.nvidia.com/nsight-compute/ProfilingGuide
            /index.html#metrics-hw-model) description for more
         details on launch configurations.

    Section: Occupancy
    ------------------------------ ----------- ------------
```

```
    Metric Name                 Metric Unit Metric Value
    ------------------------------ ----------- ------------
    Max Active Clusters            cluster          0
    Max Cluster Size                 block          8
    Overall GPU Occupancy                %          0
    Cluster Occupancy                    %          0
    Block Limit Barriers             block         24
    Block Limit SM                   block         24
    Block Limit Registers            block          4
    Block Limit Shared Mem           block          8
    Block Limit Warps                block          1
    Theoretical Active Warps per SM   warp         32
    Theoretical Occupancy                %       66.67
    Achieved Occupancy                   %       54.80
    Achieved Active Warps Per SM      warp       26.30
    ------------------------------ ----------- ------------

    OPT  Est. Local Speedup: 17.81%
         The difference between calculated theoretical (66.7%)
            and measured achieved occupancy (54.8%) can be the
         result of warp scheduling overheads or workload
            imbalances during the kernel execution. Load
            imbalances can
         occur between warps within a block as well as across
            blocks of the same kernel. See the CUDA Best
            Practices
         Guide (https://docs.nvidia.com/cuda/cuda-c-best-
            practices-guide/index.html#occupancy) for more
            details on
```

```
84          optimizing occupancy.
85
86      OPT  Est. Local Speedup: 33.33%
87          The 8.00 theoretical warps per scheduler this kernel
               can issue according to its occupancy are below the
88          hardware maximum of 12. This kernel's theoretical
               occupancy (66.7%) is limited by the number of warps
                within
89          each block.
90
91      Section: GPU and Memory Workload Distribution
92      ------------------------- ----------- ------------
93      Metric Name             Metric Unit Metric Value
94      ------------------------- ----------- ------------
95      Average DRAM Active Cycles   cycle      8,236
96      Total DRAM Elapsed Cycles    cycle    265,216
97      Average L1 Active Cycles     cycle   2,484.90
98      Total L1 Elapsed Cycles      cycle    414,270
99      Average L2 Active Cycles     cycle   2,386.17
100     Total L2 Elapsed Cycles      cycle    126,432
101     Average SM Active Cycles     cycle   2,484.90
102     Total SM Elapsed Cycles      cycle    414,270
103     Average SMSP Active Cycles   cycle   2,411.47
104     Total SMSP Elapsed Cycles    cycle  1,657,080
105     ------------------------- ----------- ------------
106
107     OPT  Est. Speedup: 7.462%
108         One or more SMs have a much lower number of active
               cycles than the average number of active cycles.
```

```
          Maximum
109       instance value is 17.77% above the average, while the
          minimum instance value is 100.00% below the average
          .
110
111   OPT Est. Speedup: 7.972%
112       One or more SMSPs have a much lower number of active
          cycles than the average number of active cycles.
          Maximum
113       instance value is 19.56% above the average, while the
          minimum instance value is 100.00% below the average
          .
114
115   OPT Est. Speedup: 7.462%
116       One or more L1 Slices have a much lower number of
          active cycles than the average number of active
          cycles.
117       Maximum instance value is 17.77% above the average,
          while the minimum instance value is 100.00% below
          the
118       average.
119
120   OPT Est. Speedup: 7.754%
121       One or more L2 Slices have a much higher number of
          active cycles than the average number of active
          cycles.
122       Maximum instance value is 17.12% above the average,
          while the minimum instance value is 8.81% below the
123       average.
```

24.2 获取硬件限制：运行时 API 示例

运行一个简单的代码，它包含运行时 API 函数，用于获取 GPU 每个 SM 的最大 Warp 数。首先使用 `nvidia-smi` 命令来识别 GPU 的名称，以获取方程的分母。如输出所示，GPU 是 GeForce 系列的一员，具体型号是 RTX 3090。

既然已经确定了正在处理的是哪个 GPU，现在把注意力转向代码，类似于前文的方法。从 `cudaGetDeviceCount` 函数开始，它提供系统中可用的支持 CUDA 的 GPU 总数。如果有两块 NVIDIA GPU 和一块 AMD GPU，AMD GPU 将不会包含在此变量中，即该变量只会包含 2 而不是 3。

然后使用 `cudaDeviceProp` 结构体来创建对象，用于存储所选 GPU 的所有属性。如果只有一个 GPU，使用索引 0 即可，无需使用 for 循环遍历所有 GPU。这里的 `cudaGetDeviceProperties` 函数读取 GPU 的所有属性，从 GPU ID=0 读取的属性将被存储在这个对象中。

这里打印的是每个 SM 的最大 Warp 数。如果想打印此对象中的任何度量指标，需要使用点运算符。例如 `prop.maxThreadsPerMultiProcessor` 获取每个 SM 的最大线程数，再除以 32 即可得到最大 Warp 数，因为每个 Warp 包含 32 个线程。

还有另一种读取这些度量指标的方法,涉及使用 `cudaDeviceGetAttribute` 函数。从其名称可知，此函数可用于从任何设备读取单个属性。例如在这种情况下，需要读取每个 SM 的最大 Warp 数，只需写入属性的名称，属性名称可以在该函数的文档中查到。

在文档中搜索该函数，它接受三个参数：第一个是值（存储结果），第二个是度量指标或属性的名称，最后一个是设备 ID。如果计算机中有多个 GPU，需要确定是从 GPU ID=0 还是 ID=1 或 ID=2 读取。如所示，有几十个可以读取的属性，例如每个块的最大寄存器数、多处理器数量、ECC 错误纠正、内存时钟频率等。前文已通过 `nvidia-smi` 命令检查过内存时钟

频率，这里是读取它的另一种方法。

这里需要的度量指标是 cudaDevAttrMaxThreadsPerMultiProcessor。这个函数需要两个输入：设备 ID 和属性名称，然后属性值将被存储在定义为整数的变量中。同时可以将该值除以 32 以获得每个 SM 的最大 Warp 数。

进入 CUDA 课程文件夹，列出所有可用文件。复制文件名，然后使用 nvcc 编译器，将可执行文件命名为 read_metrics，放上 .cu 文件，点击回车。执行后，如所示，RTX 3090 GPU 中每个 SM 的最大 Warp 数是 48（1536/32）。使用第二个函数结果也一样，两种方法都可以使用。

```
1
2  #include <cuda_runtime.h>
3  #include <stdio.h>
4
5  int main() {
6      int device;
7      cudaGetDevice(&device); // Get current CUDA device
8      cudaDeviceProp prop;
9
10     cudaGetDeviceProperties(&prop, device);
11     printf("Max_threads_per_SM 0: %d \n", prop.
           maxThreadsPerMultiProcessor);
12     printf("Max_warps_per_SM 0: %d \n\n\n", (prop.
           maxThreadsPerMultiProcessor)/32);
13
14     int maxThreadsPerMP = 0;
15     cudaDeviceGetAttribute(&maxThreadsPerMP,
           cudaDevAttrMaxThreadsPerMultiProcessor, device);
16     printf("Max_threads_per_SM 1: %d \n", maxThreadsPerMP);
```

```
17      printf("Max_warps_per_SM 1: %d \n", maxThreadsPerMP/32);
18
19      return 0;
20  }
```

NVIDIA GeForce RTX 5070 Ti 上运行结果

```
1   Max_threads_per_SM 0: 1536
2   Max_warps_per_SM 0: 48
3
4
5   Max_threads_per_SM 1: 1536
6   Max_warps_per_SM 1: 48
```

24.3 理论占用率计算示例

现在已知每个 SM 的最大 Warp 数是 48。看一些例子来理解理论占用率。

在第一个场景中，考虑一个设计为使用每个 SM 48 个 Warp 的 CUDA 应用程序。计算比率得出理论占用率为 100%，因为应用程序中使用的 48 个 Warp 与每个 SM 的 Warp 上限 48 相匹配。

在第二个场景中，如果应用程序配置为使用每个 SM 12 个 Warp，那么 12/48 的比率表明理论占用率降低到了 25%。然而在后续两个章节中，将探讨更复杂的场景。因为占用率与另一个称为” 每个 SM 分配的活动块” 的术语有关，该术语将在后续章节中探讨，这里只是简化的例子。

24.4 向量加法案例研究

深入评估与向量加法应用相关的理论占用率。本章的主要目的是专注于占用率这个术语，因此不会解释 Nsight Compute 工具中的不同选项，仅

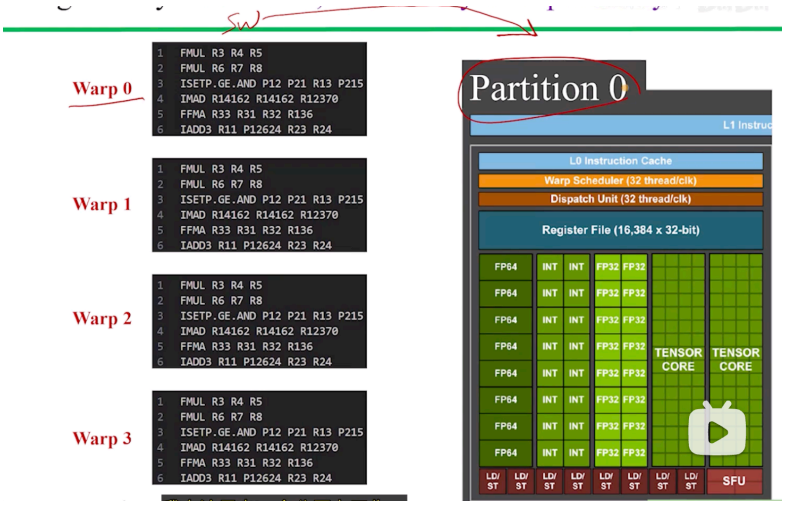


图 24.1:

使用最简单的命令。

列出文件夹中所有的 CUDA 文件，复制向量加法文件并进行编译：`nvcc -o vector_add vector_add.cu`。如所示，它正确执行了，执行时间大约为 6.5 毫秒。

需要分析这个应用，可以使用 `ncu` 命令（即 Nsight Compute）。如所示，分析过程已启动。从头开始看：核函数名称是 `vectorAdd`，这些是核函数的输入参数和维度。可以从 CUDA 文件本身读取这些值：每个块的线程数是 32，块的总数是通过方程计算得出的。

往下滚动到占用率部分。有 SM 块限制、寄存器块限制、共享内存块限制、Warp 块限制等参数，这些将在后续章节中解释。然后是理论上的每个 SM 活动 Warp 数和理论占用率。如所示，理论占用率是 33.33%，意味着只使用了 GPU 总可用资源的三分之一。即使往下查看达到的占用率，有更低的值，这意味着在实际执行中，只使用了理论占用率的一半。

这就是为什么选择每个块的线程数对于应用程序的占用率非常重要。举例来说，把这个值增加到 64，看看改变它将如何影响占用率。重新编译并重新分析。理论占用率翻倍了，达到的占用率也从 15% 增加到了 51%。

理论占用率现在是 66.67%。

为什么改变每个块的线程数会增加理论和达到的占用率？关注理论占用率：看这些限制值，取最小值，现在是 16。这意味着将在每个 SM 上同时执行 16 个块。块大小是指每个块的线程数，取这两个值相乘：$64 \times 16 = 1024$ 个线程。除以 32 得到 32 个 Warp。已知最大值是 48，所以 $32/48 = 66.67\%$。

以上展示了如何获得理论占用率。同时也解释了为什么这个值影响了理论占用率：因为它将每个块的 Warp 数从 1 个增加到了 2 个（64 线程 = 2 个 Warp）。

再将这个值增加到 96。不能写 100，因为 100 意味着 4 个 Warp（即使选择 100 个线程，也将执行 128 个，因为这取决于有多少个 Warp）。重新编译并重新分析。可以计算那种情况下的理论占用率：每个块 96 线程 = 3 个 Warp，最小值仍然是 16 个并发块，$3 \times 16 = 48$ 个 Warp，这与每个 SM 的最大 Warp 数相同，意味着理论占用率是 100%。

如果查看结果，理论占用率确实是 100%，然而达到的占用率大约为 73%。这就引出了接下来要解释的内容：为什么这两个值之间有差异。

24.5 理论占用率总结

总之，理论占用率指的是在理想情况下核函数可以使用的 GPU 计算资源的最大百分比。这里讨论的是理想情况，它被称为理论的而不是实际的，因为它假设了最优条件：既有足够的独立任务或足够的线程来完全占用 GPU 的资源，而不受内存/计算指令间依赖或同步约束的瓶颈影响。

24.6 达到的占用率与延迟隐藏

现在把重点转移到达到的占用率。首先回顾 GPU 的结构：一个 GPU 由多个 SM 组成，每个 SM 包含多个分区。在 Ampere 和 Volta 架构中，每个 SM 包含四个分区。这是一张仅一个分区的图片，在本章剩余内容中会用到它。

在这个层级中，应用程序在整个 GPU 上运行，块被指定在 SM 上运

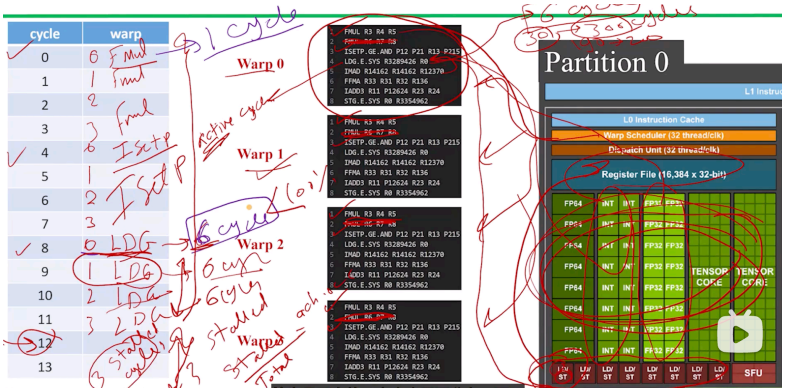


图 24.2:

行，而 Warp 在分区上执行。Warp 是本章的焦点，需要来自软件的 Warp 和来自硬件的分区。调度单元负责获取就绪 Warp 的就绪指令，并将它们发送到适当的计算单元。GPU 内的每个分区包含一个调度器，它负责在每个周期分配一个 Warp 来执行。

24.7 简化示例设定

假设有一个应用程序，其中每个 SM 只取一个块，并且块大小是 512 个线程。这意味着块大小等于 16 个 Warp（512/32）。同时假设使用一个不同的 GPU，其限制是每个 SM 16 个 Warp。16/16 得出的理论占用率为 100%。

然而对于达到的占用率，存在 Warp 未就绪的情况。例如当一个 Warp 正在等待来自内存的值时，这可能需要 30 到 300 个周期不等，具体取决于访问的缓存级别。在此期间，其他 Warp 可能被调度执行。但如果所有 Warp 都遇到显著的内存请求或指令依赖，调度器可能会遇到没有 Warp 准备好执行的周期，这就引入了所谓的停顿周期（Stall Cycle）。

24.8 场景一：无内存请求无指令依赖

有两个场景来澄清这一点。在第一个场景中，假设没有内存请求或指令间的依赖。这里涉及两个术语：占用率和延迟隐藏，将在后面解释。

从这张图片开始，包含了某个代码应用的部分汇编指令。可以使用名为 `nvbit` 的工具获得这个指令跟踪。在 NVIDIA GPU 的情况下，汇编指令被称为 SASS 指令。可以搜索该工具以了解更多信息，同时也搜索 SASS 指令这个术语。

这张图片仅代表一个 Warp，但假设有四个 Warp。在硬件部分，假设将在一个分区上执行这四个 Warp（不是一个 SM，不是整个 GPU）。请问：每个分区利用了多个 Warp，在这种情况下有四个 Warp，能计算每个 SM 利用的 Warp 数吗？答案是肯定的。如果一个 SM 有四个分区，每个分区执行 4 个 Warp，那么每个 SM 的最大 Warp 总数将是 $4 \times 4 = 16$。

能计算理论占用率吗？理论占用率等于应用程序中使用的 Warp 数除以总数再乘以 100%。在这种情况下，最大 Warp 数是 16，应用程序中使用的 Warp 总数也是 16，所以理论占用率是 100%。

下面解释 Warp 调度器的功能。Warp 调度器开始与所有 Warp 对话，询问它们谁准备好执行其指令之一。假设 Warp 0 会说：”是的，我准备好执行这里的第一个指令，即 FFMA（F 指的是浮点乘法）。”调度器会说是的，有就绪 Warp，它准备好执行 FFMA。调度器将获取这个指令并将其发送到所有这些核心。可能会疑惑为什么取一个指令并发送给所有这些核心，只需要一个核心？答案实际上不止一个指令，有 32 个 FFMA 指令。

为什么有 32 个指令？需要记住每个 Warp 由 32 个线程组成。这些线程执行相同的指令但处理不同的数据，这意味着这个指令后面应该跟着另一个 FFMA 指令用于第二个线程但使用不同的寄存器。对于所有 32 个线程都是如此，这就是为什么这个指令将被获取并发送到这些核心执行。在周期 0 中，拥有就绪 Warp 准备好执行 FFMA 指令。

然后在下一个周期，调度器会再次询问谁准备好执行指令。假设它们按顺序执行，Warp 1 应答并就绪，可以执行第一个 FFMA。调度器将类似于第一个 Warp 那样获取这个指令并发送到核心。对于 Warp 2 和 Warp 3

同样适用。然后这个周期将再次重复，Warp 0 将在周期 4 回复 FFMA 指令。

然后在周期 8，Warp 调度器将再次询问所有 Warp 谁就绪以及什么指令就绪。Warp 0 会说：” 我就绪了，有一个整数指令 ISETP。”ISETP 是一个用于比较的整数指令，它比较两个值并选择其中一个。比方说如果其中一个大于另一个，它会取第一个并存储在这里；如果不是更大的那个，它会取第二个并存储在这里。在这个例子中它是一个整数指令，调度器会获取这个指令并将其发送到整数流水线。对于所有其他 Warp 也同样适用。

同样的周期再次重复，可以说达到的占用率等于 100%。为什么？因为每个周期都有一个 Warp 准备好执行它的指令之一。如果出于任何原因，在这些周期中的任何一个发现 Warp 都没有准备好执行，这将被称作一个停顿周期。没有 Warp 就绪意味着停顿周期，在这种情况下达到的占用率将小于 100%。当指令间没有依赖时，达到的占用率是 100%，等于理论占用率。

24.9 延迟隐藏的概念

但场景一中没有内存请求，这是一个非常简单的例子。下面解释延迟隐藏这个术语。

延迟隐藏是 GPU 相较于 CPU 的主要特性之一。在 CPU 的情况下，没有这么多计算资源，可能有 4 个、8 个或 16 个核心，这就是为什么在 CPU 的情况下没有 Warp。CPU 使用一些技术，例如乱序执行（Out-of-Order Execution）。

举例来说。假设有 `ADD R1, R2, R3`，然后 `MUL R4, R1, R5`，再假设有 `SUB R10, R11, R12`。如果关注前两条指令，可以看到这里有依赖关系：MUL 指令依赖 ADD 指令的结果 R1。

当 ADD 指令进入 CPU 执行时，下个周期不会发送 MUL 指令，因为有依赖关系。MUL 必须等待 ADD 完成然后才开始执行，在这种情况下有一些停顿周期。CPU 使用乱序执行技术：SUB 指令没有依赖，CPU 改变指令顺序使其准备好执行，当 ADD 在执行时 SUB 可以发送给 CPU 准备

在下个周期执行。

这可以应用于 CPU，但在 GPU 中不需要那样。为什么? 在 GPU 的情况下，大多数时候不止有一个 Warp，有很多 Warp，它们可能包含一些就绪的指令。如果这个 Warp 没有就绪（比如有依赖关系），不需要对指令进行重排序，因为可以和另一个 Warp 对话，通常可以找到一个就绪的指令。这就是所谓的通过提供大量 Warp、大量线程、大量指令在所有计算单元上执行来隐藏延迟。

24.10 场景二：内存请求与指令依赖

在第二个场景中，假设只有一个内存请求并且还有一个指令依赖。看看从这个地址读取值并存储到这个寄存器。假设 LDG 指令（全局内存加载）将等待来自内存的值。

在 LDG 指令和后续指令之间存在依赖关系，有一个内存请求和一个依赖。还假设有四个 Warp。假设只有一个 FFMA，在第一个周期调度器询问所有 Warp 谁就绪，第一个 Warp 用 FFMA 回复，调度器获取指令并发送。Warp 0 就绪执行 FFMA。下一个周期 Warp 1 就绪又是 FFMA，Warp 2、Warp 3 同样，前四个周期执行 FFMA，与场景一类似。

假设加载指令将消耗 6 个周期（通常实际值在 30 到 300 个周期之间，取决于访问的缓存级别：L1 缓存约 30 周期，L2 缓存约 190-200 周期，全局内存约 300 周期）。为了简单起见假设 6 个周期。

那么在周期 4 执行完 FFMA 之后，调度器再次与所有 Warp 对话。Warp 0 说准备好执行整数指令 ISETP，调度器将其带到整数单元。Warp 1、2、3 同样适用。在这里完成了 ISSUED。

调度器再次问谁就绪，Warp 0 说准备好了但现在在执行加载指令。在周期 8，加载指令将被调度器移到加载/存储单元（Load/Store Unit），而不是计算单元。加载/存储单元负责内存不同级别（L1、L2、全局内存）之间的连接。这个指令消耗 6 个周期，意味着这个 Warp 将不可用除非 6 个周期结束。

然后 Warp 调度器问第二个 Warp 准备好了吗？它说准备好执行加载指令，但这也要花费 6 个周期。Warp 2 和 3 同样加载 6 个周期。

现在在周期 12，最重要的一点是调度器将与 Warp 0 对话：”准备好了吗？”Warp 0 会说：”不，还没准备好。因为想执行 ISETP 指令，但这个指令依赖于从内存来的结果，现在没准备好因为加载指令还没完成。”为什么没准备好？这里只过了 3 个周期（从周期 8 到周期 12 是 4 个周期，但加载需要 6 个周期），还必须再等 2 个周期才会准备好。

调度器将询问下一个 Warp，Warp 1 也会说不可用因为加载指令需要 6 个周期还没完成。这就是为什么这里至少会有 2 到 3 个停顿周期。停顿意味着所有这些 Warp 中没有一个准备好执行任何指令，因为内存请求。

在这种情况下可能会看到一些活动周期（Active Cycle），因为在这个周期中计算单元之一是活动的，正在执行 FFMA 或 ISETP 甚至是加载。但这里有停顿的情况。将活动周期除以总周期（而不是停顿周期）将得到达到的占用率。

举一个带数字的例子。假设总共有 100 个周期，有 85 个活动周期，那么将有 15 个停顿周期。达到的占用率等于 $85/100 = 85\%$。

现在已知什么是达到的占用率以及它与理论占用率的区别。为什么理论占用率是 100% 而达到的占用率是 85%？因为理论占用率只取决于 Warp 的数量，这很容易计算，所以它被称为理论的。但达到的占用率取决于每个 Warp 指令在执行时的动态实时行为。

24.11 Nsight Compute 实测验证

再次运行 `ncu ./vector_add`。达到的占用率是 72.8%，而理论占用率是 100%。

达到的占用率不能通过方程计算，因为它取决于运行时执行。总的达到的占用率是由所有分区和所有 SM 上的运行时计算得出的，而不仅仅是一个分区。这里的计算仅由一个分区完成，这就是为什么这里的计算会复制到 4 个分区中的每一个，然后取平均值以获得每个 SM 的达到的占用率。

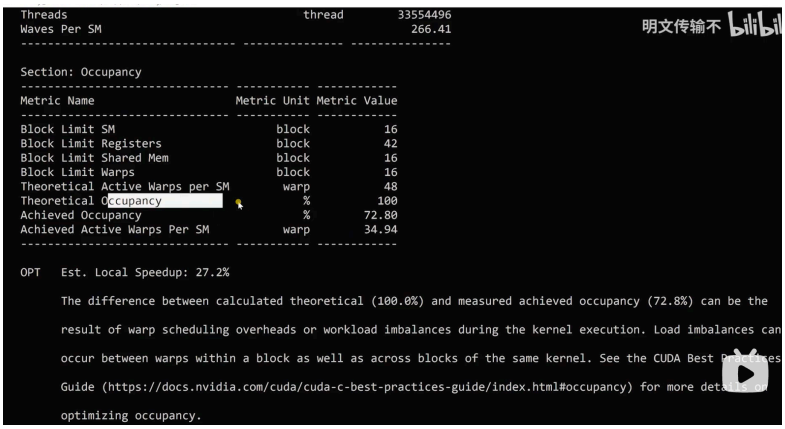


图 24.3:

然后总的达到的占用率值通过对所有 SM 的值取平均来确定：将所有 SM 的达到的占用率加起来除以 SM 的数量。一旦有了每个 SM 的达到的占用率，这个方法将应用于所有 SM，意味着将拥有等于 GPU 中可用 SM 数量的达到的占用率值。这就是计算整个 GPU 而非一个分区的达到的占用率的方法。

24.12 为什么 GPU 占用率很重要?

1. 令人惊讶的是，高占用率并不总是等同于高性能，但它确实表明 GPU 资源中有很大一部分正在被利用。当看到占用率是 100% 时，意味着许多资源将被利用。

2. 识别和理解占用率可以帮助查明性能问题和优化机会。回顾第一个关于修改每个块线程数的例子：将每个块的线程数从 32 增加到 64 再到 96 的影响，以及对理论占用率的差异。这是其中一个例子。理解占用率将帮助查明性能问题。另一方面，低占用率表明存在瓶颈阻碍了 GPU 被充分利用。

3. 如需进一步信息，建议访问 NVIDIA 官方文档页面。它也解释了在决定这些占用率时起作用的因素，配备了足够的细节以帮助理解理论和

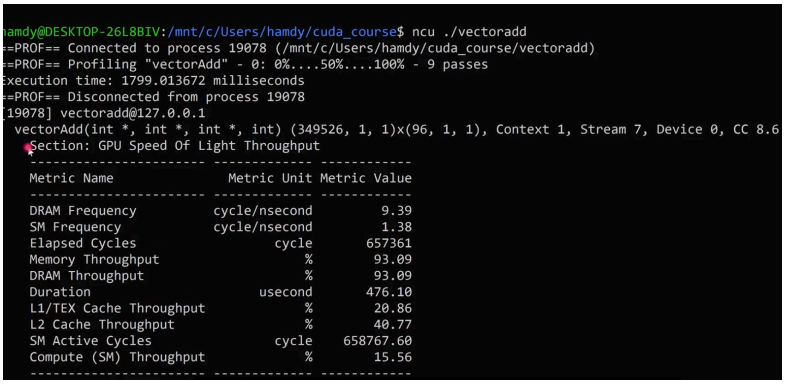


图 25.1:

达到的占用率的细微差别。

25 每个 SM 分配的活跃线程块详解 (阅)

25.1 引言：核心概念与硬件限制表

本章聚焦于 GPU 性能分析中的一个基本概念，即每个 SM 分配的活跃块（Active Blocks per SM）。这个术语不仅与占用率同等重要，而且直接影响占用率或 GPU 利用率。

要完全理解这个术语，必须将注意力转向这张表格。展示的表格来自 A100 GPU 白皮书，显示了 Ampere 架构的硬件限制或约束。本章不会涵盖表格中的所有行，只重点介绍四个关键行。这意味着在任何给定时间，每个 SM 上不能超过 32 个块同时运行或并发运行。

25.2 四个关键硬件限制

25.3 每个 SM 的最大线程块数

本章要检查的第一行是每个 SM 的最大线程块数，在这种情况下其值为 32 个块。

25.4 每个 SM 的最大 Warp 数

要检查的第二行呈现了每个 SM 的最大 Warp 数，即单个 SM 上可以同时执行的最大 Warp 数。对于 A100 GPU，这个限制是每个 SM 64 个 Warp。这意味着无论每个 SM 分配的块数是最大值 32 个块还是更少，这些块中所有 Warp 的总和不能超过每个 SM 的最大 Warp 数，即 64。

这是这里的主要关键点：**Warp 数限制控制着每个 SM 可以分配多少个活跃块**。最大值是每个 SM 32 个块，但应用程序可能使用少于 32 个块，这取决于 Warp 数限制。需要牢记这一要点。

如果每个块包含 4 个 Warp，那么 32 个块的 Warp 总数将是 128 个 Warp。为了理解这一点，考虑一个例子：应用程序每个 SM 分配 32 个块，这等于表格中提到的每个 SM 最大块数。但这个值 128 个 Warp 超过了表格中规定的 Warp 数限制，编译器将尝试解决这个问题。

根据表格该值不应超过 64 个 Warp，编译器会介入遵循以下逻辑：编译器理解每个 SM 最多可以利用 32 个块，然而如果每个块由 4 个 Warp 组成，那么总的 Warp 数将是 128。这超过了可接受的 Warp 数限制（而不是块限制）。编译器将决定将活跃块的数量从 32 减少到 16。经过这个调整，总的 Warp 数将变成 $16 \times 4 = 64$ 个 Warp。这个值落在可接受的范围内并且没有超过 Warp 数限制。

25.5 块限制 Warp 数（Block Limit Warp）

如何控制每个 SM 分配的活跃块？这个例子揭示了一个关键概念，称为” 块限制 Warp 数”（Block Limit Warp）。正如从 Nsight Compute 工具分析中获取的图片所示，很明显” 块限制 Warp 数” 这个术语出现在占用率部分。这强调了前文关于占用率与每个 SM 分配的活跃块之间关系的观点。这个例子表明，尽管最大块限制是每个 SM 32 个块，但最终每个 SM 分配的块数受 Warp 数限制的约束。

25.6 全局块分配与并发块的区别

需要提一个关键点：可以在整个 GPU 上运行的最大块数非常大，可能达到数百万。可以通过之前讨论过的 CUDA 运行时 API 来检查这个限制。这个巨大的数字需要分布在数十个 SM 上。在 A100 GPU 中有 108 个 SM，这意味着有可能为每个 SM 分配超过 1000 个块。然而这里的关键问题是：这 1000 个块中有多少个可以在一个 SM 上**并发运行**？

提到每个 SM 32 个块的限制时，并不意味着只能分配 32 个块；相反意味着可以分配超过 32 个，但只有 32 个或更少的块会同时运行或并发运行。

25.7 寄存器限制与寄存器溢出

同样的原则适用于每个 SM 的最大寄存器数。A100 GPU 中的最大值超过 65000。

例如，考虑一个应用程序利用一个块，每个 SM 块大小为 1024 个线程。每个 SM 只有一个块，然而表格设定的限制仅为 65536。如果每个线程需要 100 个寄存器，那么每个 SM 所需的总寄存器数将是 $1024 \times 100 = 102400$ 个寄存器，超过了允许的最大限制。

在这种情况下，不能通过减少每个 SM 的块数来解决这个问题，因为应用程序每个 SM 只利用一个块。可以减少每个块的线程数来解决这个问题，但如果不这样做，所需的寄存器将超过允许的最大限制。

寄存器溢出（Register Spilling）会导致性能下降，这是因为本地内存比普通寄存器慢。这将导致使用本地内存来存储这些寄存器的值。检查应用程序是否过度依赖寄存器溢出是至关重要的，应该寻求改进应用程序的方法来缓解这个问题。

25.8 寄存器限制计算示例

在这个例子中，需要计算每个 SM 分配的活跃块数。设块大小为 512 个线程（可配置参数），每个线程需要 16 个寄存器（取决于应用程序代码，不在控制范围内）。

每个块所需的总寄存器数：$512 \times 16 = 8192$ 个寄存器。

在寄存器限制为每个 SM 65536 个寄存器的情况下，可以并发运行的最大块数：

$$\left\lfloor \frac{65536}{8192} \right\rfloor = 8 \text{ 个块}$$

这意味着尽管规定的块限制是每个 SM 32 个块，但在寄存器限制下将只选择 8 个块来并发运行。

25.9 四重限制取最小值原则

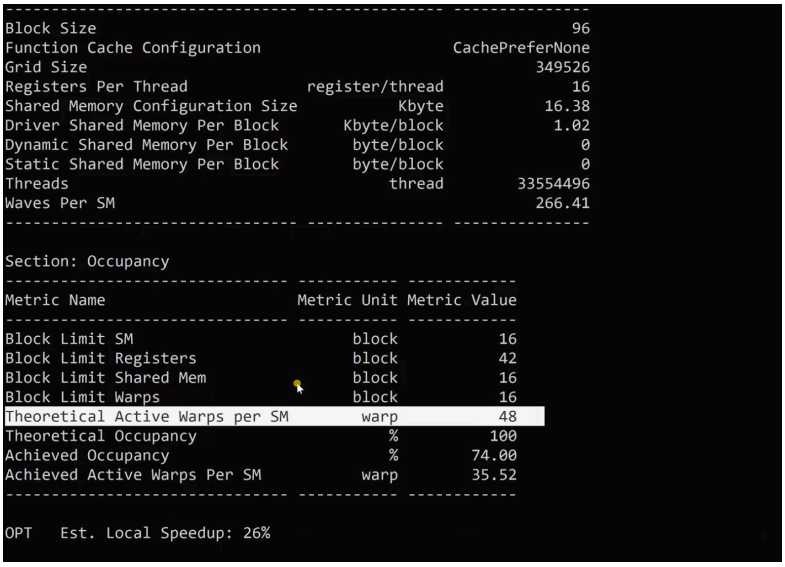
同样的原则适用于可用的 Warp 数（Warp 限制）、可用的寄存器数（寄存器限制），以及表格最后一行的共享内存限制。虽然本章不会深入探讨共享内存约束的细节，但将在后面更深入地探讨这个主题。

下面解释最后一个例子，以说明 Warp 限制、寄存器限制和共享内存限制对每个 SM 分配的活跃块的影响。" 分配的活跃块" 指的是这些块中有多少个可以**并发运行**，可以为每个 SM 分配数百或数千个块，但并发数受限。

假设以下条件：

- **块限制 Warp 数**: 4 个块（如果使用超过 4 个块，将超过 Warp 数限制）
- **块寄存器限制**: 8 个块
- **块共享内存限制**: 16 个块
- **硬件最大块限制**: 32 个块

在这四个值中，NVCC 编译器将选择其中**最小的值**。尽管块限制是 32，寄存器限制是 8，共享内存限制是 16，但 Warp 限制约束会超越它们，规定最多只能并发运行 4 个块。寄存器说可以运行 8 个，但 Warp 限制说不能运行超过 4 个。最终值将取最小的那个值。



```
------------------------------- -------------- ---------------
Block Size                                                  96
Function Cache Configuration                     CachePreferNone
Grid Size                                               349526
Registers Per Thread             register/thread            16
Shared Memory Configuration Size           Kbyte         16.38
Driver Shared Memory Per Block       Kbyte/block          1.02
Dynamic Shared Memory Per Block       byte/block             0
Static Shared Memory Per Block        byte/block             0
Threads                                  thread        33554496
Waves Per SM                                             266.41
------------------------------- -------------- ---------------

Section: Occupancy
------------------------------- ----------- ------------
Metric Name                     Metric Unit Metric Value
------------------------------- ----------- ------------
Block Limit SM                        block           16
Block Limit Registers                 block           42
Block Limit Shared Mem                block           16
Block Limit Warps                     block           16
Theoretical Active Warps per SM        warp           48
Theoretical Occupancy                     %          100
Achieved Occupancy                        %        74.00
Achieved Active Warps Per SM           warp        35.52
------------------------------- ----------- ------------

OPT   Est. Local Speedup: 26%
```

图 25.2:

现在应该已经明白为什么在计算理论占用率时要取最小值。这就是为什么需要理解” 每个 SM 分配的活跃块” 这个术语，以便能够理解为什么在分析应用程序时要在这四个值中选择最小值。

25.10 Nsight Compute 实测验证：RTX 3090

再次分析向量加法应用程序。输入 `ncu ./vector_add`（即 Nsight Compute）。如所示，有三个部分：GPU Speed of Light、Launch Statistics 和 Occupancy。需要重点关注的是占用率部分。

在这个部分内，前四个度量指标就是本章研究过的：

- **块限制 Warp 数**: 16 个块
- **块共享内存限制**: 对应值
- **块寄存器限制**: 42 个块（本章影响较小）
- **硬件最大块限制**: 32 个块

这四个值中的最小值是 16。可能想知道为什么是 16 而不是 32？因为这里是在 RTX 3090 GPU 上解释，而不是 A100 GPU。通过 `nvidia-smi` 来检查一下，或者使用运行时 API 检查每个 SM 的最大分配活跃块数。

在这种情况下，每个 SM 分配的活跃块数是 16 个块，这就是用于理论占用率计算的值，不能并发运行超过 16 个块。

25.11 理论占用率 100% 的验证

例如这里的理论占用率是 100%。为什么是 100%？已知块大小是 96。进行前文已经做过的相同计算：

$$\text{每块 Warp 数} = \frac{96}{32} = 3 \text{ 个 Warp}$$

RTX 3090 中每个 SM 的最大 Warp 数是 48（前文占用率章节中解释过这个值）。如果有 3 个 Warp 乘以从四个值得到的最小分配活跃块数 16：

$$3 \times 16 = 48 \text{ 个 Warp}$$

而在应用程序中利用了 48 个 Warp。如果每个 SM 的最大活跃 Warp 数是 48，那么：

$$\text{理论占用率} = \frac{48}{48} \times 100\% = 100\%$$

这就是为什么理论占用率是 100%。这只是理解这些参数的影响及其含义的一个例子，因为如果用最大值除以应用程序中利用的值，百分比将是 100%。

26 掌握 CUDA GPU 并行编程（硬件与软件）Nsight Compute 入门

本章本来应该开始讲解如何使用 Nsight Compute 工具来分析 CUDA 应用。然而开始着手做这件事时，发现了一个可能遇到的问题，尤其是使用 Windows 操作系统的话，下面解释这个问题及其解决方案。

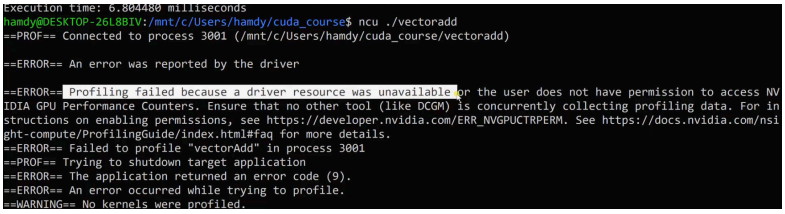


图 26.1:

26.1 问题分析：ncu 工具报错

首先这是 CUDA 课程文件夹，使用 `ls` 命令来列出所有文件和文件夹。如所示，有一些 CUDA 文件，还有一些可执行文件，接下来执行这个应用——向量加法项目的可执行文件。前文已经解释过很多次了，如图所示，它正确执行没有任何问题，执行时间等于 6.8 毫秒。然而当开始分析这个应用时：

如果想使用 Nsight Compute 进行分析，需要输入 `ncu`，它指的是 Nsight Compute。点击回车看看输出。如所示，有一些错误，这是第一个。此外，还有另外三个错误。

为什么 Nsight Compute 工具不工作呢？它安装在系统里了吗？答案是肯定的。当安装 CUDA 工具包时，安装了许多库和工具，其中之一就是 Nsight Compute，但这不是遇到的错误。除非在这里遇到错误说 `ncu` 命令未找到，否则不需要安装 Nsight Compute。

26.2 GPU 性能计数器权限设置

现在检查一下这五个错误中的每一个。第一个提到驱动程序有问题，还记得这个吗？如何检查驱动程序呢？前文在解释 `nvidia-smi` 工具时提到过驱动程序。输入这个命令，看看驱动程序是否已安装。这个表格不会在这里出现。如果驱动程序有问题，这个表格将不会显示。

会遇到一个问题或错误，类似这里的情况。如所示，这里提到了驱动程序版本，甚至还有 CUDA 版本，以及 NVIDIA GeForce RTX 3090 GPU 被

检测到了，驱动程序版本被检测到了，CUDA 版本被检测到了，这不是问题所在。如果这不是问题，为什么错误提到有驱动程序问题呢？这可以从第二个错误中理解。如果阅读第二个错误，就会找到答案。它说分析失败，因为一个驱动程序资源不可用，” 或者” 解答了问题，” 或” 意味着这可能是个问题，也可能不是。

那么现在检查的问题不是第一个。检查问题是否在这个部分。它说用户没有访问 GPU 性能计数器的权限。需要搜索关于 GPU 性能计数器限制的信息，以获得更多信息。下面给出解决方案：在 Windows 系统上安装 GPU 驱动程序时，它只将 GPU 性能计数器的访问权限授予管理员，而当前不是以管理员身份运行，是以普通用户身份运行。

可能会问为什么不用 sudo？因为使用 sudo 时，如所示，可能会遇到另一个问题——`ncu` 命令未找到。但确定已经安装了 Nsight Compute 工具，接下来尝试搜索 NVIDIA 面板。

如所示，右键点击并选择 NVIDIA 控制面板选项，需要几秒钟来打开一个窗口。从这个窗口，需要进入菜单并选择桌面。然后从桌面启用开发者设置，启用这个选项，将在这里显示性能计数器。请看这一部分启用。

如所示，有了开发者设置管理 GPU 性能计数器其中之一，也是这里唯一可用的。点击这里，如所示，有两个选项。第一个是仅限管理员用户访问 GPU 性能计数器，这是推荐的。但现在想允许所有用户访问 GPU 性能计数器。勾选这个选项并应用，显示器可能会闪烁。一旦选择并应用这个选项后，可以继续操作。

26.3 重启与验证

现在已经允许所有用户访问 GPU 性能计数器。回到这里，尝试使用命令。可以检查到错误中有一些变化。如所示，这些错误与之前的错误不同了。为了验证这一点，检查第一个错误，这里提到驱动程序，但现在这里不是关于驱动程序的。

接下来需要重启 Ubuntu 系统。输入 `sudo reboot`，然后点击回车。如

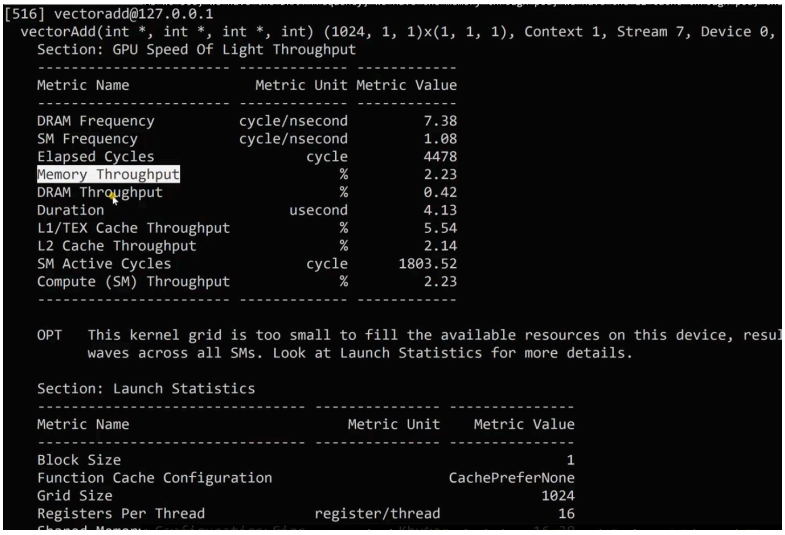


图 26.2:

果在使用 WSL（即在 Windows 上安装 Ubuntu 系统或 Linux 系统的工具），需要耐心一点。需要等待大约一分钟，然后再启动 Ubuntu 系统。等待超过一分钟后，通过输入 `wsl` 再次启动 Ubuntu 系统。等待几秒钟，然后进入文件夹。

输入 `cd cuda_course`，然后再次运行相同的命令。如所示，得到了一些结果。有 DRAM 频率、内存吞吐量、L2 缓存吞吐量、活动周期，这些是 `ncu` 命令提供的度量指标。这里有一些配置，如块大小、网格大小，每个线程的寄存器数。还有一个名为占用率的部分，与前文讨论的占用率相关，这里有一些限制。最后有这个度量指标，即达到的占用率。本章不会解释这些度量指标，因为本章只是为了解决在 Windows 系统上运行 `ncu` 或 Nsight Compute 的问题。

27 NVIDIA 全系性能分析工具（Nsight Systems/Compute/nvprof)

本章将深入探讨 NVIDIA 为分析和剖析 CUDA 应用程序提供的基本工具。在深入介绍这七个工具之前，需要注意的是 `nvprof` 分析工具在许多教程中被广泛涵盖，但现已被弃用。建议不要使用它，这就是它不会包含在本课程中的原因。

27.1 CUDA-MEMCHECK 与 Visual Profiler

如图所示，有七个由 NVIDIA 提供的用于分析 CUDA 应用程序的工具。从本章中的工具开始，例如 CUDA-MEMCHECK 工具。该工具有助于识别和诊断 CUDA 应用程序中的内存错误，包括越界和未对齐的内存访问错误，类似于可用于调试 CUDA 程序的 cuda-gdb 工具。CUDA-MEMCHECK 工具对于确保 CUDA 应用程序的正确性至关重要，而这反过来又会影响其性能。

本章的第二个工具是 NVIDIA Visual Profiler，简称 NVVP，以及针对 CUDA、OpenCL、Direct3D 和 OpenGL 应用程序的硬件计数器。NVIDIA Visual Profiler 是一个跨平台的性能分析工具，为开发者提供详细的时序信息。从其名称中的"Visual" 一词可以知道其主要特点。该工具提供应用程序时间线和达到的占用率的图形化视图，有助于识别 GPU 核函数和内存传输中的瓶颈。

27.2 Nsight 套件三大工具

本章不解释所有这些工具，只专注于 Nsight 套件中三个最重要的工具：Nsight Compute、Nsight Systems 和 Nsight Graphics。这张图解释了这三个工具之间的关系。如图所示，该图从 Nsight Systems 开始，它相对于其他两个工具是外层的，这表明该工具更适合将应用程序作为一个整体来检查，而不是专注于单个核函数。这个工具即 Nsight Systems 被推荐给开发

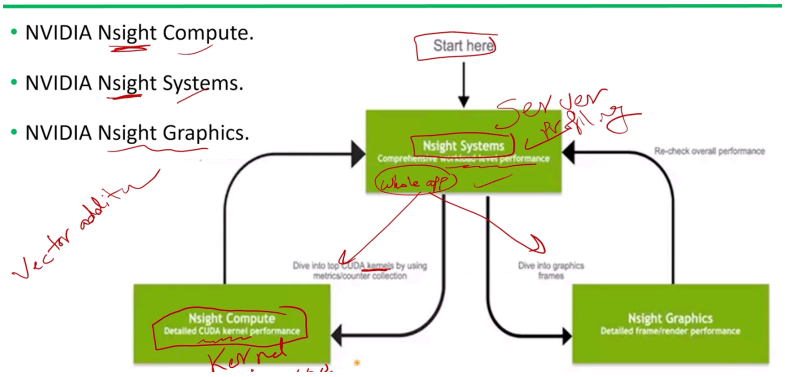


图 27.1:

者，用于对其应用程序性能进行全系统范围的分析。当考虑到一个应用程序可能包含多个核函数，而不仅仅是一个时，这种区别就变得明显了。该工具可以可视化 CPU 到 GPU 的交互使用情况，以及 CUDA Runtime API 的性能，还能分析多节点工作负载。这就是为什么可以说 Nsight Systems 是一个服务器端分析工具，而不是客户端分析工具。这个工具就是前文分析向量加法应用程序时演示过的同一个工具。

然后为了对应用程序或工作负载中的每个核函数进行更细致的检查，Nsight Compute 是首选工具。它提供了大量针对每个核函数的详细见解和度量指标，包括全局内存、缓存、SM。甚至深入到 SM 内每个独立单元的度量指标，可用的度量指标超过 10 万个。

Nsight Graphics 提供有关图形帧和渲染性能的详细度量指标，专为图形应用程序分析和剖析的需求而定制。最后，当涉及到以图形为中心的应用程序时，Nsight Graphics 是该领域的专业工具。

在结束本章之前，留一个重要的提示。本课程中的分析重点将完全集中在 Nsight Compute 工具的使用上，可能还会涉及 Nsight Systems。

28 错误检查 API 详解

在深入探讨使用 Nsight Compute 工具分析 CUDA 应用之前，需要强调 CUDA 函数中错误检查的重要性。用一个例子来说明这一点。在进行性能分析之前，确保 CUDA 函数运行流畅且准确至关重要。在前文中提到过一个问题，即应用程序编译时没有错误，然而该应用程序却无法正确执行。经过调查，定位到了一个故障的 `malloc` 函数，原因是用于分配所需存储或所需空间的 RAM 不足。这类错误在编译期间无法检测到，在运行时也可能不会表现为可读的错误。这凸显了为所有 CUDA 函数使用错误检查的必要性。

28.1 内存不足问题分析

这大致可以分为两类：同步错误检查和异步错误检查。CUDA 提供了几种机制来检查其操作或函数的成功与失败。从第一类开始：同步错误检查。在 CUDA 中，像内存分配这样的操作是同步的，这意味着这些操作会阻塞主机代码的执行，直到设备完成操作。大多数 CUDA Runtime API 函数都会返回一个错误代码，这使得错误处理更加直接，因为错误会在操作完成后立即报告。可以通过检查这个错误代码来确定是否发生了错误。

现在打开向量加法代码来做一些错误检查测试。这是在添加任何错误检查指令之前，是之前章节的向量加法应用程序。首先需要测试这个应用程序是否工作正常。打开终端，列出第四节文件夹中的所有文件。如所示，有三个文件，而本章将重点关注第 2 个 `02_enhanced_vector.cu`。需要编译和运行，检查编译和执行步骤中是否有任何错误。从编辑器中可以看到，打开的是同一个文件。正如之前讨论的，正在使用 NVCC 编译器。`nvcc -o` 这是可执行文件的名称。以 CUDA 文件结束命令，编译时没有出现任何错误。现在运行应用程序，执行也没有任何问题。

回到这里，把这个值，也就是 32 替换成一个更大的值，1024×25。增加每个向量中元素总数的原因是为了增加每个向量所需的总存储量，从而在给这个应用程序分配所需空间时制造一个问题。一旦出现问题，就需要

通过错误检查来确切地知道问题是什么。在那之前先简化一下，把 20 这个乘数去掉。在编译和运行应用程序之前，用任务管理器检查一下电脑资源。右键点击，然后选择任务管理器。选择性能选项卡，它显示了电脑中的所有资源。从 CPU 开始到 GPU 结束，需要关注的是内存和 GPU。

原因是关注内存，因为这里有三条指令要在 RAM 中分配空间，而 RAM 在这里由内存选项卡表示。同时使用 `cudaMalloc` 函数在 GPU 中分配空间，这就是关注 GPU 的原因。

可以注意到 GPU 现在使用了大约 48% 的资源，内存使用了 2.5 GB。也可以从图表检查这个值。具体来说关注的是专用 GPU 内存，专用 GPU 内存指的是 GPU 内部真实或实际的内存。这是因为录制程序在运行，最后一个选项卡说明这个录制程序使用了硬件加速，硬件加速指的就是 GPU 本身。需要从这里这些步骤查看 GPU 和内存。

需要计算每个向量所需的总存储量。这里就是结果，这个数字代表了每个向量中的元素总数。复制这个数字，然后写下结果代表了每个向量中的元素数量。有三个向量，那么现在需要做的是乘以 4。为什么需要乘以 4 呢？因为应该熟悉 C 语言，在 C 语言中整数值使用四字节的存储空间，因为这些元素中的每一个都包含一个整数值。需要乘以 4 来得到每个向量所需的总字节数。但现在想以 GB 为单位计算，而不是字节。然后再除一次转换成 MB，除以 1024 转换成 KB。再进行一次转换变成 GB。如所示，每个向量将使用 4 GB 存储空间。因为有三个向量，所以将使用 12 GB。现在需要再次检查资源，GPU 现在使用了 2.5 GB，当运行这个正常的应用程序时，这个值应该从 2.5 增加到大约 14 或 15 GB。

来编译。编译时出现了一些警告，需要处理警告，因为它可能在运行时、在执行时造成问题。这不是编译时的问题。现在可以执行，编译已经完成了，但在执行时应用程序可能工作也可能无法正常工作。先关注第一个问题。它指出问题从这里这部分开始。如前所述，第十七行有个问题。读一下警告本身，它说整数转换导致截断。回到应用程序，每当看到截断部分，就应该知道这个值太大了，无法存储在分配的空间里。第十七行有一个整数值，叫做 `size`，这个变量应该存储乘以 4 字节之后的结果。取 `size`，它

代表这个数字，然后乘以四字节，结果将存储在这个变量中。这里的结果太大了，无法只存储在四字节中，需要超过四字节。这就是为什么需要写 `long`，而不是 `int`。然后编译，如所示，现在没有问题或警告了，然后运行。当代码应用程序正在执行时，可以检查专用 GPU 内存，它正从 2.5 增长到 15，大约 14.7 左右，GPU 内存会出现一个峰值。

28.2 同步错误检查实现

现在没有问题，想把应用程序所需的空间增加到一个很大的值。可能会想错误检查在哪里? 大到无法装入专用 GPU 内存，甚至共享 GPU 内存。因为如果使用例如 30 GB，那么将用掉全部 24 GB 的专用内存和 6 GB 的共享内存。想增加更多，把这个值乘以 20。如果把这个值乘以 20，那么每个向量需要 80 GB。需要三个向量乘以 80 GB，这个值将大到无法放入 CPU 的 RAM 内存，甚至 GPU 内存。现在编译看看一些其他的警告。如所示，有更多警告。从第十七行开始还是同一行。因为这里的乘法不能以这种方式进行，所有这些值 1024 都是整数。但当乘以另一个 20 时，这个值将无法装入一个整数。也许 `long` 可以，`long` 足够了，但它不足以进行乘法运算本身。需要做的是复制这部分，并定义一个空间足够的变量。写 `long`，但因为 `long` 可能还不够，所以再写一个 `long`，以确保分配超过八字节。然后写等于，加上 `LL`。因为如果去掉这个 `LL`，那么就是在整数乘整数乘整数，结果应该是 `long`。这里的 `L` 指的是需要对 `long` 值进行乘法，而不是整数。但如果乘两个值，在最终值存储到这个 `size` 之前，它不足以存储在整数值中。这就是为什么如果看警告，会在其他指令上也有警告。比如第 28 行，看 for 循环，不能在这里乘这些值，因为它们现在被定义为整数。当写 `LL` 时，意味着是在乘 `long` 值，而不是整数值。然后编译并运行。

如所示，遇到了段错误。在这种情况下，知道问题所在，这个应用程序需要的空间超过了 CPU 的 RAM 和 GPU 专用内存的可用空间。但如果有一个应用程序在输出中给出了段错误，但不知道所需空间，这就是为什么需要错误检查。需要为这三个函数，也就是 `cudaMalloc` 内存分配函数

添加一些错误检查。复制指令，为这三个函数放在这里。顺便说一下，需要对 cudaMemcpy、cudaEventRecord、核函数启动等做同样的事情，这里只是把这些作为例子。这只是错误对象的声明，这个对象将用于存储每个 CUDA 函数的错误。这就是为什么这个变量或对象的类型是 cudaError_t，这是 CUDA 编程中的另一种结构。需要再次写上 error =。如前所述，每个 CUDA 函数都会提供一个错误。这个错误随后可以被检查，以确切地知道执行这个函数的问题是什么。取这个函数的输出，并将其存储在错误对象中，然后测试这个错误是否不是 cudaSuccess。如果不是成功，那么就有问题。需要打印这个错误。如所示，使用了 fprintf，但这里的错误值不是字符串值，它更像是 ID 号之类的东西。使用另一个 CUDA API 函数叫做 cudaGetErrorString，它接收错误 ID 并返回对应于该 ID 的字符串。这就是为什么对所有 CUDA 函数重复相同操作，并且应该对此应用程序中的其他 CUDA 函数做同样的事。

现在保存、编译和运行。同样的问题，没有从添加到应用程序的错误检查指令中得到任何指示。原因是如果在为 GPU 内存分配之前，先为主机内存分配了空间——malloc 和 cudaMalloc——输出中的段错误出现在 malloc 这里，而不是 cudaMalloc。而且没有为 malloc 做错误检查。需要把这些指令向下移动。现在在为 CPU 或主机的 RAM 分配空间之前，先在 GPU 中分配空间。在这种情况下，如果 cudaMalloc 无法在 GPU 中分配所需空间，应该能看到错误检查的输出。

编译并运行，在运行时看看 GPU 资源会发生什么。如所示，可以检查专用值。因为需要 80 GB，编译器现在正尝试分配全部的 24 GB 专用 GPU 内存，然后尝试增加共享 GPU 内存。编译器正尝试分配第一个向量的空间。但这两者都不够，所以会看到一些峰值，它尝试然后失败，如此反复。最终编译器会说” 无法为第一个向量分配空间”，它会为第二个向量 B 和第三个向量 C 做同样的事。从这部分可以看到，” 未能分配设备内存不足”，这是这条指令的结果。分配内存失败原因就是内存不足。一旦完成了 GPU 的分配，无论是成功还是失败，将使用这些指令为主机端分配空间。没有足够的空间来分配所需的存储，这些指令也应该会失败，这就是为什么也有

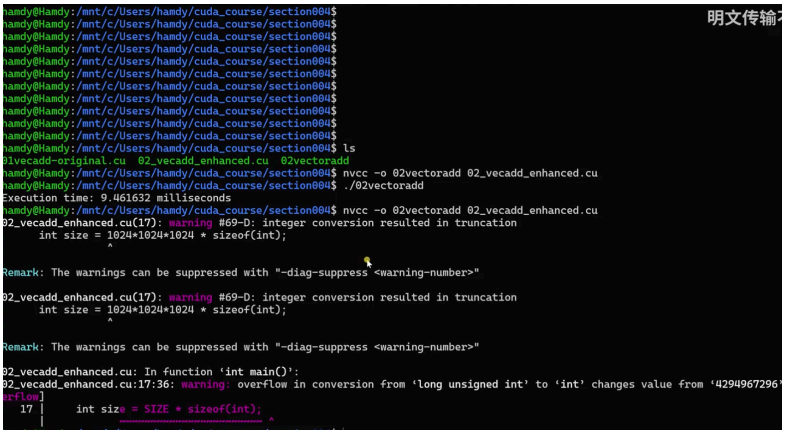


图 28.1:

一个段错误。但在尝试先在 GPU 中分配之后，这就是它的重要性。这就是错误检查在 CUDA 应用程序中的使用方式，它给出一些指示，确切地指出问题在哪里。特别是如果有一个由他人设计的应用程序，而正试图开发或增强它，错误检查就是这样工作的。

回顾本章开头提到的内容，错误检查有两类：同步和异步错误检查。对于同步错误检查，GPU 任务、GPU 函数或 GPU 操作会阻塞主机执行任何其他指令，直到 GPU 任务完成其执行。它将在这里被阻塞，直到 `cudaMalloc` 完成其执行。例如，如果编译器执行到这里的第 27 行，那么它将停在这里。在执行中编译器将尝试为向量 A 分配空间，但它不会在完成向量 A 的分配之前尝试为向量 B 分配。完成向量 A 的分配，然后进入向量 B 的下一步，无论失败还是成功都没问题。一旦完成向量 B 进入第三步，以此类推。正如所示，现在正尝试为向量 B 分配。完成了向量 A 的步骤，它失败了，但没问题。这就是同步操作的含义。这就是为同步操作使用错误检查的方式：一个错误对象使用这条指令，然后读取同步函数的输出，然后检查该错误是成功还是失败。如果操作失败，那么需要打印错误。

28.3 异步错误检查

然而异步类型不同，因为异步类型在这里有 CUDA 启动函数代表，不会阻塞主机端或 CPU 执行下一条指令。这就是为什么如果不在这里放一个 cudaDeviceSynchronize 函数，执行 cudaMemcpy，可能会在执行完成之前复制结果。因为编译器不会停在这里，编译器将启动 GPU 核函数，然后进入下一条指令，这与在 cudaMalloc 函数中发生的情况不同。这就是为什么有不同的方法来检查异步操作的错误。这样做的方式就是使用一个叫做 cudaGetLastError 的函数。需要写一些像那样的东西，复制并把它放在这里。如所示，并不直接来到 GPU 任务本身写 error = CUDA 函数或 CUDA 启动，不像之前做的那样。为什么？因为编译器不会停在这里，直到这个函数完成并返回一个错误存储在这里，它不会停在这里。这就是为什么不需要存储核函数执行的输出。相反，使用 cudaGetLastError 这个 CUDA Runtime API 函数。需要获取最后一个错误，它将存储在内存中的某个地方。而这个错误应该是 GPU 执行的最后一个函数的错误，在这种情况下就是向量加法函数。有两种不同的方法来处理这两类错误。同步错误，使用上面的方法；异步错误，使用 cudaGetLastError 方法。

28.4 错误检查宏封装

这里提一下，这三条指令在代码中重复了很多次。如所示，这里有这三条指令，这里也是同样的三条指令，这里也有。如果对所有 CUDA 函数使用错误检查，应该会看到它们在 cudaMemcpy 这里这两条函数以及 cudaEventRecord、用于输出的 cudaMemcpy 等等。这三条指令将在应用程序中重复多次，而没有必要。可以复制这部分，展示如何做到这一点。需要把这三条指令放在一个宏或一个函数里，然后在需要的时候调用它。如所示，有几行中有两个部分。第一部分是定义一个名为 CUDA_CHECK_ERROR 的宏。这个宏调用了一个名为 cudaAssert 的函数。cudaAssert 是一个函数，如所示，它接收一个来自 CUDA Runtime API 函数的错误作为输入，接收执行一个 CUDA 函数后返回的错误，然后检查错误是成功还是失败。如果成功，

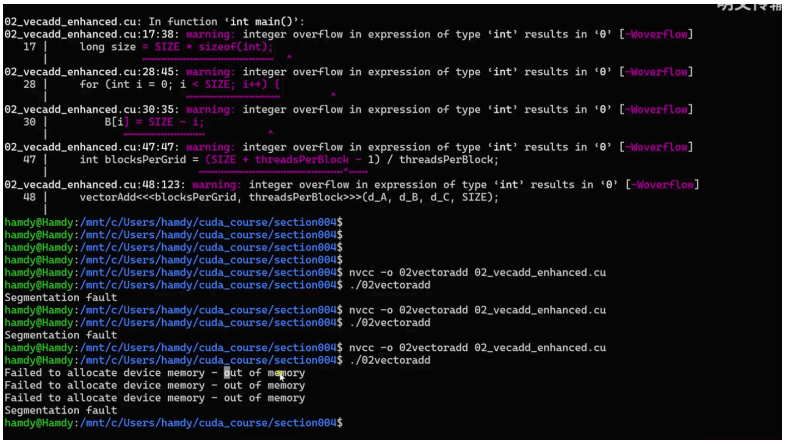


图 28.2:

那么什么都不做。如果不成功，如所示，需要打印错误。但如果看这里的第一个关键字，会发现它叫做 `inline`。`inline` 指的是将这些指令内联到另一个代码中。这里不是一个分支，不是另一个函数，只是组织一下。需要复制 `CUDA_CHECK_ERROR`，并把它放在需要的地方。每当看到 `CUDA_CHECK_ERROR` 时，就是在调用 `cudaAssert`。而当调用 `cudaAssert` 时，并不是在执行另一个函数，只是复制这些指令，并把它们放在看到 `CUDA_CHECK_ERROR` 的任何地方。

编译并运行。不打算对所有函数都这样做，只是做一个示例。现在会看到区别，编译器正在尝试为第一个向量分配空间。看输出。如所示，看到一个叫做"GPUAssert: Out of memory" 的输出，这发生在第 37 行。这就是在输出中看到的 GPUAssert。有 `%s`，它对应错误本身，这就是为什么这里有错误本身，也就是内存不足分配。然后有文件名，也就是 `02_enhanced_vector.cu`。然后有行号，也就是 37。去看看 37 行，正确。但是为什么在这种情况下，没有看到三次向量 B 和向量 C 的失败？只看到向量 A 失败。因为如果看这里的 `cudaAssert`，会看到使用了 `exit`。这是因为如果无法为向量 A 分配空间，在没有足够空间的情况下，为什么要继续执行过程呢？需要退出，不需要执行那些不能正常工作的东西。在后续项目中，将使用这种错误检查方法，但不会再解释。这就是为什么需要按顺序观看所有章节。因为在每

个章节中，都建立在之前解释的内容之上。

可以为异步操作做同样的事。复制这部分，来到这里。这是用于同步 CUDA 操作的，这是用于异步 CUDA 操作的。可以注意到区别，这里的 `cudaGetLastError` 函数在同步类别中没有。需要复制宏和 `cudaKernelAssert` 的定义。复制宏名，然后到这里用于核函数启动，但不需要写类似于 `error = 某物`的东西，因为编译器不会停在这里。在完成之后，需要写一些类似那样的东西。在编译之前，把这个值减小到比如 32，以确保它在没有任何问题的情况下运行。现在执行完成没有任何问题。

以上就是关于错误检查的全部内容。通过本章的内容，应该理解了错误检查的重要性，以及如何将其用于不同的代码函数。在后续项目中，将使用错误检查而不再进行解释。

29 命令行模式下的 Nsight Compute 性能分析

前言

本章将解释如何使用 **Nsight Compute** 工具来分析任何 CUDA 应用程序，该工具由 NVIDIA 提供。不过在开始之前，先说明一点：任何公司的性能分析工具都应提供两种分析方法。

1. **命令行分析 (CLI Profiling)**：如所示，可以使用一些命令来收集特定指标以评估 CUDA 应用程序的性能。
2. **图形用户界面分析 (GUI Profiling)**：本章将从第一种方法即命令行分析开始。在后续章节中，将详细讲解 GUI 分析。

这两种方法都非常重要。有时可能只需要收集几个指标来评估特定硬件单元的性能，在这种情况下，最佳选择可能就是命令行分析。但如果想进行深入的性能评估，最好使用 GUI 分析，因为它可以提供诸如性能屋顶线分析 (Roofline Analysis)、图表以及每个硬件单元的利用率等信息。

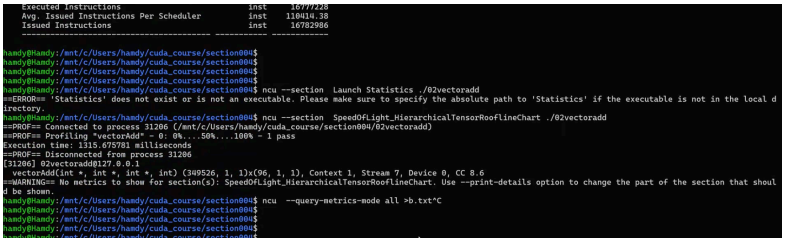


图 29.1:

29.1 文档与基础命令

照例打开文档先大致了解一下命令行分析。搜索”Nsight Compute CLI” 并打开第一个链接。该网页包含了关于 Nsight Compute 命令行分析需要知道的一切。

基础命令如下：

```
ncu --set full -o profile ./my_cuda_app
```

如果正在使用 MPI，可以针对特定进程或者针对所有进程进行分析。此外，还可以指定输出文件、分段文件夹，或过滤想要分析的核函数 (Kernel)。

29.2 分段 (Sections)

Nsight Compute 将分析数据组织为不同的” 分段”。当运行最简单的 `ncu` 命令时，默认会显示四个分段：

1. **GPU Performance Ceiling (GPU 性能上限)**：显示 GPU 中不同单元的吞吐量信息。
2. **Launch Statistics (启动统计)**：包含 DRAM 吞吐量、L1 和 L2 缓存吞吐量、SM 利用率等。
3. **Occupancy (占用率)**：包含块大小、网格大小、每个线程的寄存器数量、每个 SM 使用的共享内存大小等。

4. **GPU and Memory Workload Distribution (GPU 与内存工作负载分布)**：显示理论占用率与实际占用率（例如本例中实际占用率为 81%），以及 DRAM、L1、L2 缓存等的活跃周期数。

Nsight Compute 实际上包含约 23 个分段（从 ID 4 到 27）。在命令行中，需要使用分段的标识符 (ID) 而非显示名称。例如，若只想查看"Launch Statistics"，可使用：

```
ncu --section LaunchStats ./my_cuda_app
```

其他分段还包括 NVLink 通信统计、调度器统计 (Scheduler Stats)、线程束状态统计 (Warp State Stats) 等。

分析建议与警告

Nsight Compute 有时会提供建议或警告。例如提示" 可选指标无法找到"，这可能是因为当前 GPU 架构不支持该指标。另外，工具可能会给出停顿原因分析，例如：" 平均而言，此核函数的每个线程束因等待记分牌依赖 (Scoreboard Dependency) 而停顿了 111 个周期"。理解这些信息需要结合总周期数来判断瓶颈严重程度。

29.3 指标 (Metrics)

Nsight Compute 包含海量指标（约 10 万个）。为了高效使用，需掌握其命名规则。

29.4 指标命名结构

指标名称通常遵循以下格式：

< 硬件单元 >__< 指标描述 >.< 聚合方式 >

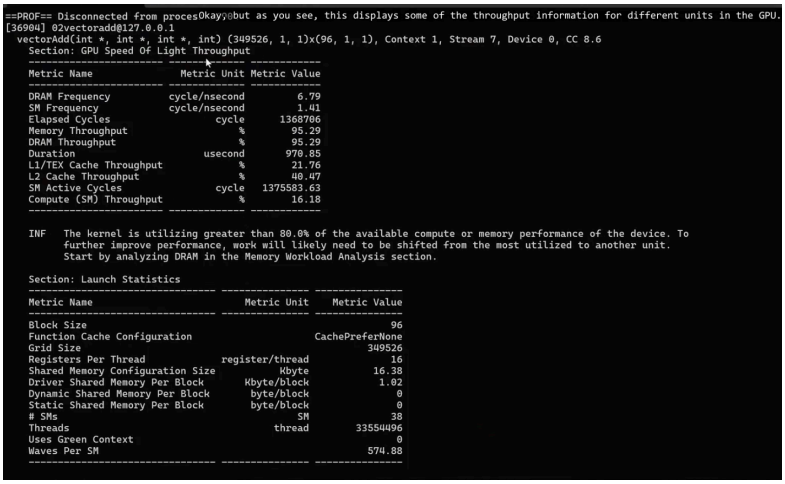


图 29.2:

29.5 硬件单元前缀

双下划线之前的部分定义了被分析的硬件单元:

- `dram`: 设备内存 (DRAM)
- `l1tex`: L1/纹理缓存
- `lts`: L2 缓存 (Level Two Slice)
- `sm`: SM (Streaming Multiprocessor)
- `gpu`: 整个 GPU

29.6 聚合方式后缀

点号 (.) 之后的标识符表示数据的聚合方式:

- `.min`: 所有 SM 中的最小值
- `.max`: 所有 SM 中的最大值

- `.avg`: 所有 SM 的平均值
- `.sum`: 整个 GPU 的总和

重要技巧： `.sum` 是基于最大值计算的，而非平均值。即：Sum = Max × SM_Count。如果在命令行中省略点号及后缀(例如只写 `smsp_inst_executed`)，工具将自动收集该指标的所有聚合类型（avg, max, min, sum)。

29.7 常用指标示例

- **L1 缓存命中率**: `l1tex_hit_rate`
- **DRAM 访问字节数**: `dram_bytes.sum`
- **扇区数量**: NVIDIA GPU 中每个扇区 (Sector) 为 32 字节。相关指标如 `dram_sectors.read.sum`。
- **指令执行数**: `smsp_inst_executed.avg`（SM 内执行的平均指令数)。
- **管道利用率**: 可搜索 `pipe_lsu`（加载存储单元)、`pipe_tensor`（张量核心)、`fp16` / `fp64` 等。

29.8 实战应用与导出

29.9 收集特定指标

可以使用 `--metrics` 参数收集指定指标：

```
1 ncu --metrics l1tex__hit_rate,smsp__inst_executed ./02
    _vector_add
```

如果 L1 命中率为 0%，说明所有内存操作都访问了全局内存，这是严重的性能瓶颈，需要优化数据局部性。

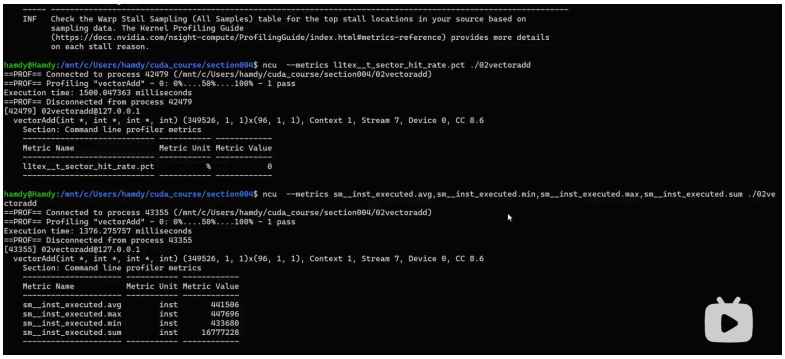


图 29.3:

29.10 使用正则表达式过滤

若要分析特定硬件单元的所有指标，可使用正则表达式：

```
ncu --metrics ".*shared.*" ./02_vector_add
```

这将筛选出所有包含”shared” 的指标（如共享内存相关指标）。

29.11 导出结果

可以将分析结果导出为 CSV 文件以便在 Excel 中查看：

```
ncu --metrics smsp__inst_executed --csv --log-file result.csv
    ./app
```

导出的表格包含指标名称、单位、以及各聚合方式对应的数值。

结语

本章学习了如何使用 Nsight Compute 命令行工具进行基础性能评估，包括分段的选择、指标的命名规则解析以及结果导出。在后续章节中，将应

用这些知识，通过特定指标深入分析代码，例如探究块数和线程数对核函数执行时间及周期数的具体影响。

30 图形化 Nsight Compute 使用

在前文中，讨论了 Nsight Compute 工具的命令行分析。其中提到有两种方法来分析任何 CUDA 应用程序。第一种是使用命令行分析，前文已经讨论过了，其中讨论的一些分析内容是本节所需要的。第二种方法是使用 Nsight Compute 工具的图形用户界面，这就是本章的主要内容。将探索 Nsight Compute 的图形界面，即 GUI。照例，这是文档链接。需要说明的是，有两份文档，不是一份。一份是关于 Nsight Compute 工具图形界面的；另一份是关于命令行分析的。

30.1 安装与项目创建

当打开一个新项目时，需要编辑一些配置，以便能够对 CUDA 应用程序进行性能分析。完成初始步骤后，需要查看输出。但在那之前，需要确保系统上已经安装了这个工具，无论使用的是 Linux 还是 Windows 操作系统。如果系统上没有安装 Nsight Compute 工具，安装起来非常简单。

当安装 CUDA 时，CUDA 工具包包含了所有内部工具，例如 Nsight Compute 或 Nsight Systems 等。搜索"Nsight Compute tool for Windows"，打开第一个链接，点击这里。它可能会要求在 NVIDIA 网站注册，可以使用任何电子邮箱进行注册。注册后，会看到这个网页，可以从这个链接下载 Nsight Compute。安装过程非常简单。然后搜索 Nsight Compute，打开工具。

这里有两个版本，一个是 24.11，第二个是 24.3.2。将打开这个版本。

它是这样启动的，有一些选项，可以打开一个文件、创建一个新项目，或者加载一个已有的项目。这些是 Nsight Compute 图形界面最近的一些增强功能。

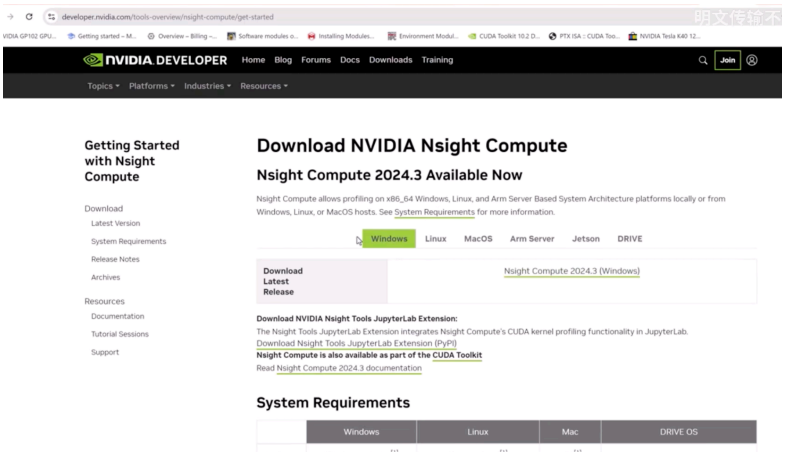


图 30.1:

需要做的只是从文件菜单打开一个新项目。但在那之前，需要知道 CUDA 应用程序的确切位置。这是 CUDA 课程的文件夹，这是第四节的文件夹。如所示，这里有一些 CUDA 应用程序，还有可执行文件。但在那之前先编译这个 CUDA 应用程序，输入 `ls`。

有一些 `.cu` 文件，还有之前章节生成的可执行文件。但今天需要做的是生成带 `.exe` 扩展名的可执行文件。使用 NVCC 编译器，写例如 `-o`，这是为了保存编译过程的输出。随便起个名字，例如就叫 `aa.exe`。

这是可执行文件，这是 CUDA 文件。这个命令会生成这个可执行文件。如所示，这里还没有这个可执行文件。按回车键，然后输入 `ls`。如所示，这里有了可执行文件。然后回到这里，点击新项目，给它起个名字，点击确定，这里就是配置项。只解释主要的几个。例如这个选项是必须的，这是应用程序可执行文件，已经用这个命令生成了，从这里选择。

复制路径，选择 `aa.exe` 文件打开。如所示，感叹号被移除了。然后可能需要选择工作目录，例如选择第四节文件夹。

从这里开始有一些配置，比如说是否想启用 CPU 调用栈？是否想启用对 CPU 和 GPU 通信的性能分析过程？如何处理缓存？需要在时钟控制中刷新它们全部。刷新的意思是清除先前存在于缓存中的所有内容。

需要使用基础频率还是无基础? 这将指 GPU 的基础时钟频率。对于图分析需要使用节点 1。如果想了解更多，可以从这里查阅文档。

如果现在点击立即启动，它是不会工作的。因为需要开始执行这个应用程序，才能对其进行分析。将添加附加（Attach），可以看到进程名，这里没有任何进程，可以通过这个命令来做到。需要来到这里并开始执行，以便从 CPU 到 GPU 创建一个新进程。

这个命令叫做 `ncu`。将使用的模式是 `--launch-mode attach`，然后是 `./aa.exe`。

那个模式意味着需要让 GPU 开始执行，但它不应该结束。因为需要分析，需要一个正在运行的进程。当按回车键时，例如 `./aa`，如所示它执行了。但当使用这个命令时，它只是启动了一个进程，消耗了大约 8.2 毫秒。可以通过点击这里的附加来完成。现在可以将这个进程附加到 Nsight Compute UI。

如所示，`aa.exe` 可以显示在这里，这是进程 ID。先按 Ctrl+C 取消，然后来到这里。

点击附加，再点击这个。需要将参数改为对应的值，因为这是今天的应用程序。点击这个，然后点击附加。现在可以附加了，或者可以来到这里点击启动，如所示一切运行正常。现在需要看看左侧有什么。但它现在是空的，因为还没有开始性能分析。

30.2 附加进程与指标选择

在下面的标签页中，可以看到一个叫" 指标选择"、"API 统计" 的标签页。只是附加到了进程，但还没有开始分析本身。如何开始分析呢? 在指标选择中，可以选择任何想要分析的项目，即需要分析哪些指标，需要做哪些性能分析。

从这里开始，选择所有与 Speed of Light 相关的内容，例如 Speed of Light 吞吐量、Speed of Light 屋顶线图（Roofline）、Speed of Light 分层屋顶线图等。如果点击这里，可以看到其他一些选项。同时不选择 PM 采

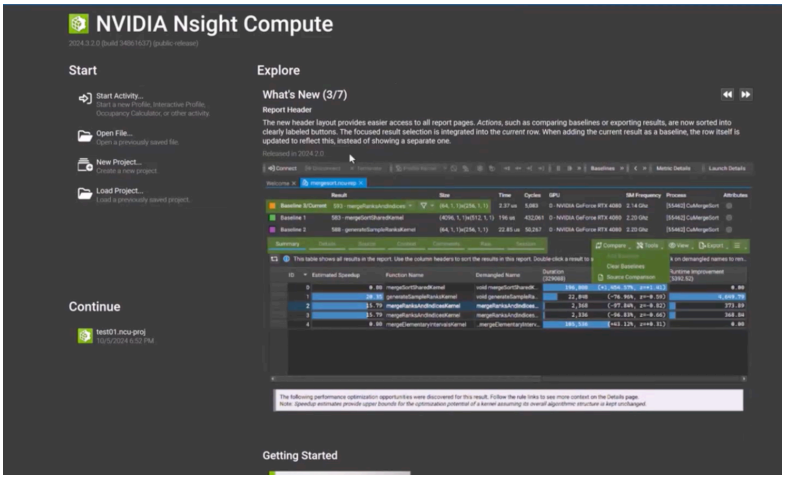


图 30.2:

样，但选择计算工作负载（Compute Workload）。如果只想要这些选项中的一个，可以选择它。然后可以在这里选择所有已选的选项，因为它们都非常重要，后面将解释每一个。然后也可以选择指令统计、启动统计、占用率（Occupancy）、GPU 和内存工作负载分布等。

如果还记得之前的讨论，会知道在这里选择的项目在命令行分析中是以部分（Section）的形式呈现的。例如还记得前文中的表格吗？这些就是 Nsight Compute 中包含的部分。

例如看计算工作负载分析，如果来到这里，可以看到计算工作负载分析，然后看一下，例如 Sub-warp 统计。然后再回到这里，会看到 Sub-warp 统计。一旦为分析过程选择了所需的部分和指标，就可以来到这里。同样适用于调度器统计、内存工作负载分析等。然后可以开始点击这里” 运行至下一个核函数”。

VectorAdd，这是下一个核函数。选择核函数，然后点击分析核函数，这就是分析过程的开始。一旦点击分析核函数，它将开始执行应用程序，并在执行过程中收集性能指标。

30.3 分析结果概览

如所示，分析过程已经完成，可以查看结果了。这里没有结果，因为有很多标签页需要选择。例如有一个摘要（Summary），从这个摘要中可以看到一些信息：已实现的占用率。它解释了理论占用率和已实现占用率。例如理论占用率是 100%，前文已经讨论过这个了。在命令行分析中的占用率，理论占用率是 100%，已实现的占用率大约是 80% 或 81%。

使用的是相同的指标，但显示指标的方式不同。这里通过一些信息图表来显示，而那里是通过表格来显示的。如果来到这里，可以看到同样的理论 100%，已实现 81%。也可以看到这些内容并阅读，但将从另一个标签页阅读。

那个标签页有更多细节，例如详情（Details）标签页，如所示，这里会显示包含的所有部分。例如 GPU Speed of Light 吞吐量，这是第一个；然后有计算工作负载分析；然后是内存工作负载分析。这里有一些技巧可能会给出更多信息，因为可以看到一些表格。

在指标选择中包含的所有内容，都会在详情标签页的这个部分看到其输出。组织得很好。例如可以看到吞吐量，有 16% 的吞吐量。同样可以再次回到命令行分析，看到这个值也显示在那里。这里只是在两种方法之间建立联系，以便选择最适合的方法。

先看这里，内存吞吐量是 95%，L1 和 L2 缓存的吞吐量分别是 21% 和 40%。在这里可以看到经过的周期数，这个核函数执行的持续时间等。此外如果选择这些选项或指标中的任何一个，可以在这里看到细节。例如，如果在这里选择持续时间，它会显示单位，单位是纳秒，显示值。这里的值是 970，也就是 978，但这里是纳秒，那里是微秒。如果想在命令行分析中做同样的事情，可以使用这个指标。

如所示，`gpu__time_duration`。还记得那个分析吗？所有想从图形界面获取、以便于命令行分析一起使用的内容，都可以从这里获取。然后在这个右侧部分的末尾，可以看到 Average、Sum、Minimum。这些在前文中也讨论过。

现在可能会问，为什么最小值、最大值、总和所有这些都有相同的值？

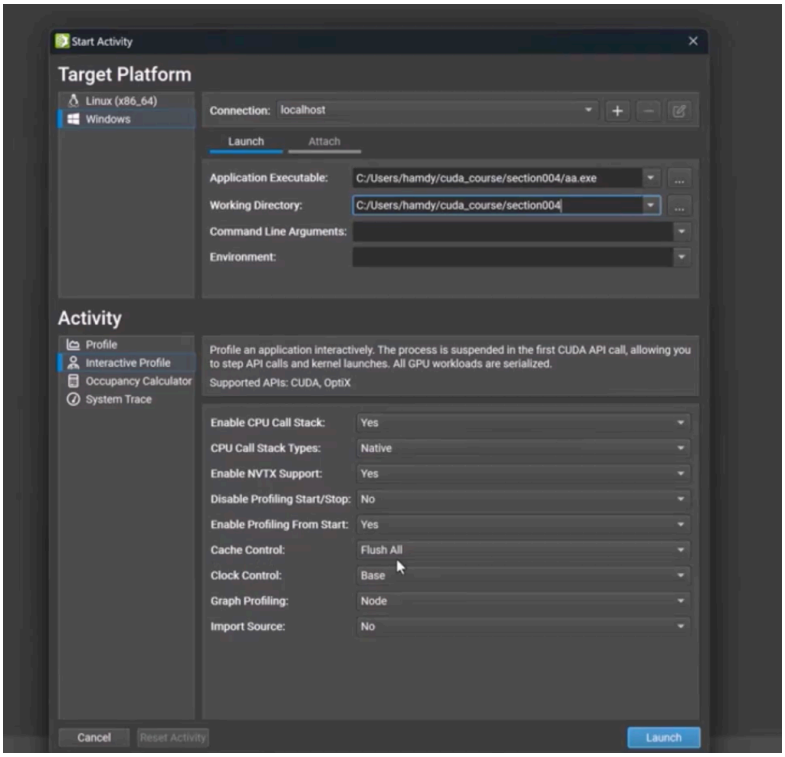


图 30.3:

因为现在讨论的是持续时间，而对于整个 GPU 的时间，它们应该是相同的。但对于其他情况则不同，例如周期数，每个 SM 使用的字节数就不同。但讨论的是总时间，总时间它们应该相同。

这是第一部分，GPU Speed of Light 吞吐量。然后这里有计算工作负载分析。如所示，有每个周期的指令数（IPC），每个活动周期的指令数，需要搜索它们之间的区别，这是经过的（Elapsed），这是活动的（Active）。这些将在后续章节中讨论。

但这不是本节的重点，因为本节的主要重点是熟悉 Nsight Compute 工具，而不是优化应用程序，也不是理解每个术语。重点是理解 Nsight Compute 工具。在其他章节中将使用 Nsight Compute 工具进行优化。

30.4 内存瓶颈分析

接着还有内存工作负载，这里有 SM 繁忙时间的百分比。这里有 L1 的命中率，有 0% 的命中率。在命令行分析章节中测量过这个，33% 的命中率，这也非常糟糕。对于 L2 命中率有 33%，这意味着在内存操作上消耗了太多时间或太多周期。因为必须访问 L1，却找不到需要的数据。然后访问 L2 能找到一些，但其他的数据不在那里。需要从全局内存进行大量读取甚至写入操作，这需要很长时间，需要很多周期，数百个周期。这里只是逐步说明这些数字的含义。

然后有内存繁忙百分比、最大带宽，实现了大约 95% 的内存带宽。有 95%，现在可能会想这很好，是个好兆头。但这并不好。因为如果实现了全局内存最大带宽的 95%，这意味着访问全局内存太频繁了，需要减少这种情况。

理解这些指标的含义非常重要。再看看 L2 缓存中包含了一个叫做压缩的功能，这与 Ampere 架构也有关系。因为这些功能中的一些是从 Ampere 架构开始引入的，而不是之前的架构，比如 Volta、Turing、Pascal 等。然后还有调度器统计，前文已经讨论过调度器统计了。

在讨论 Sub-warp 统计之前，先停在这里。可能会问，这些表格看起来和这里的表格非常相似，简直一模一样。在这里做什么？命令行分析和图形用户界面之间有什么区别？它们显示的是相同的表格吗？但事实并非完全如此。稍后将跳转一下，展示图形用户界面中包含的最佳功能。

这里有 Sub-warp 统计，如所示，有一个非常重要的指标：每个已发布指令的 Sub-warp 周期数。需要阅读所有这些信息。可以说大约需要 119 或 120 个周期来处理每个已发布的指令，这非常重要。需要阅读所有内容，因为这些说明中的一些可以看作是对你的建议或者是一些要点，能帮助优化应用程序以实现高性能。

例如读一下这里的说明：”平均而言，这个核函数的每个 Sub-warp 大约花费 111 个周期处于停滞（Stall）状态。”这非常糟糕。停滞意味着这个 Sub-warp 在约 111 个周期内什么都没做。想象一下，有 111 个周期，每个 Sub-warp 就只是停在那里不退出，也不执行任何指令。如果继续看会找到

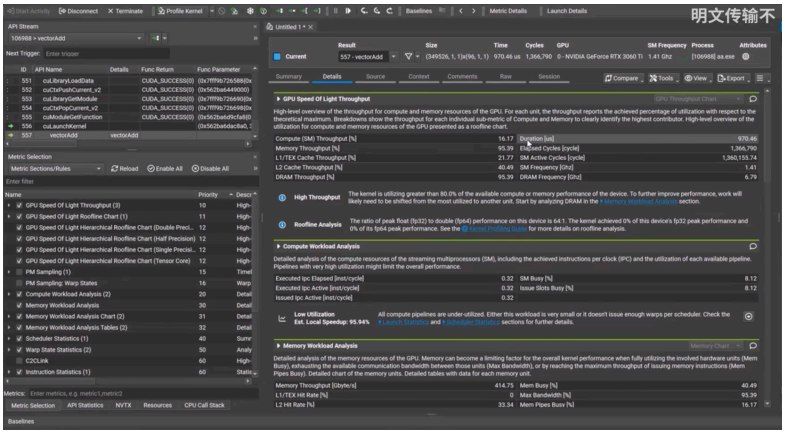


图 30.4:

导致这个问题的原因。需要改进这一点，以确保在每个周期内尽可能让每个 Sub-warp 执行一些操作，执行一条指令。

这背后的原因是什么？说过有 111 个周期的停滞。”等待长计分板依赖（Long Scoreboard Dependency）”，可以阅读来了解，这里的依赖意味着某个指令正在等待前一个结果或从内存的读取操作。它说明依赖是由于从 L1/纹理缓存读取数据，或者可能是来自其他内存层级。例如本地内存、全局内存、表面内存或纹理操作单元。可以从这里知道遇到了内存瓶颈问题。

30.5 源代码与汇编分析

回到这里，还记得最大带宽吗？当说如果正在实现一个高带宽时，高内存带宽在某些情况下可能是好的，但在其他情况下就非常糟糕。这就是这些指标与在 Sub-warp 统计中看到的停滞是如何关联的。特别是当 L1 和 L2 命中率很低时。然后可以看到每个 Sub-warp 有 32 个线程，每个 Sub-warp 有 30 个未被屏蔽（Unmasked）的线程。

继续看查找产生数据的指令。但在那之前来到这里，点击视图标签页，然后点击这里的展开所有部分。它说需要找到导致停滞的指令，下面展示这一点。如所示，这就是图形用户界面和命令行分析之间的主要区别。这是

添加到 Nsight Compute 图形界面中的最佳功能。看到这个部分了吗？如所示，拥有相同的部分。除了图表之外，这里没有添加任何新内容。

看到计算工作负载了吗？看到内存工作负载了吗？可以很容易地阅读它，但添加了一些图表，以便用一种好的方式将数据或分析输出可视化。

例如，这里有两个重要的吞吐量或利用率计算：SM 吞吐量和内存吞吐量。说过内存吞吐量是 95%。但计算吞吐量是多少呢？16%，这意味着这个应用程序是内存瓶颈型（Memory Bound）的，为什么？因为它只使用了 16% 的算术逻辑单元（ALU）。只利用了 16%，但从内存方面利用了 95%。

这意味着应用程序大部分时间在进行内存操作，而不是算术运算。不需要这样，需要在提高 L1 和 L2 命中率的同时，稍微降低这个数值。因为如果提高了 L1 和 L2 的命中率，那将减少内存带宽的使用。同时增加算术运算，以充分利用可用的硬件核心。这就是如何解释内存瓶颈或者说应用程序的瓶颈——它是计算瓶颈型（Compute Bound）还是内存瓶颈型。

这是第一个图表，非常重要。也可以阅读这两条说明，因为可能会给出一些启发。也可以点击这些链接，一些是在线的。如果点击这里，这将带你到内存工作负载分析部分。

看看计算工作负载分析。如所示，这只是对不同类型算术操作及其利用率的可视化呈现。例如这里有 FMA 单元或 FMA 指令。FMA 指的是融合乘加运算（Fused Multiply-Add），而 ALU 则包含除双精度外的所有其他算术操作。这将包括例如单精度、半精度、整数精度等。这里使用了大约 3.55% 的可用资源用于融合乘加运算。

而这里使用了大约 4% 左右，但这指的是活动周期中的百分比，意思是如果整个 GPU 有 100 万个活动周期，大约使用了其中的 4% 来执行 ALU 运算。另外 4% 或 3.55% 用来执行乘加运算。

后面将给出一个关于这个的实际例子。这里有其他单元，例如加载/存储单元（LSU）。不确定 IDU 指的是什么，但双精度在这里。无论如何，这是相对于执行的峰值指令数而言的。意思是从白皮书中知道，在这款 GPU 这个架构上，不同类型的算术操作有一个可执行数量的峰值。假设例如加载/存储单元的峰值是 1000 TOPS。这里只使用了峰值数的 16%。可以看到

这里执行了太多的加载和存储单元指令。

加载和存储单元执行什么操作？内存操作、全局内存、L1/L2 内存操作或者共享内存操作。如果把它与 ALU 和 FMA 相比，这并不好。可以看到，这个单元的待执行指令数量很大。而这是因为如果内存操作相比 ALU 来说很大，这将导致内存瓶颈。

看看内存工作负载分析，这非常重要。因为它会给出一个概念，并提供更多关于有多少请求、在不同内存层级之间移动了多少数据等细节。这里没有使用任何单精度或双精度，这就是为什么它们都是零。

这里有主要的内存单元，例如有共享内存加载、全局内存存储、共享内存。这是稍后会讨论的单元，先别关注它。另一个单元叫做表面内存、纹理内存、本地内存和全局内存。这是第一级，这里有它们的第一级。还记得命令行分析吗？`l1tex__...`，这是 L1 缓存。还记得这些指标以及它们是怎么写的吗？如所示，它们显示命中率是 0%。

从这个图形界面可以读取许多性能指标，而无需费力去阅读指标名称、硬件单元名。需要提醒一下，看到这个了吗？有十万个指标，还记得吗？

这里一切都组织得很好。这比试图搜索需要知道的指标在哪里、描述是什么等要好得多。如所示，L1 缓存的命中率是 0%。对于共享内存没有请求。因为在这个应用程序中，如果没有使用共享内存，只是在不使用任何共享内存的情况下将两个向量相加。这就是为什么这里的请求数量是 0。

然后是第二级 L2 缓存，这里是零或者说是 33%。如所示，在它们之间有传输的数据量，可以计算所有这些。然后从 L1 到 L2，这意味着写入结果，因为正在读取向量 A 例如从 L2 到 L1，移动了大约 268 兆字节和向量 B，然后计算单元将完成执行，写入的数据量大约是此值的一半。因为读取两个向量，但只写入一个向量即 C 向量。然后 L2 缓存中的命中率是 33%。

接下来给你一个任务，需要搜索如何提高 L2 缓存的命中率。只是思考一下，看看是否能做些什么。然后这是设备内存，也就是最后一级。有全局内存、L2 缓存、L1 缓存以及计算单元本身之间的请求数量。这来自 ALU，来自 SM。而内存单元会将这些操作发送到 L1 级。大约有 315 万条指令从 SM 发出，到达加载和存储单元或全局内存操作单元。

然后 L1 级如果没有命中，会将这些请求发送到 L2。L2 如果命中，它会将结果或数据发送回这里。如果没有命中，它将向设备内存请求这个数据。这里展示的就是这个情况。这就是内存图表图形界面的样子。大约有 100 万次请求从 SM 发往 L1，大约有 210 万次对这些请求的回复。

如所示，不太确定具体如何解读。这里说例如如果看到这个颜色，这意味着正在达到峰值。例如看这里，如果记得设备内存的吞吐量是 95%，可以看这里，用了这个颜色。因为有 95%，但对于 L2 有 33%。

这就是为什么它在这个区域是这个颜色。对于 L1 有 0%，它是深色的。如所示，这是一个非常重要的图表。然后这里有调度器统计。再次建议阅读这里的所有内容，GPU 每个调度器最大 Warp 数，如果记得应该是 16。

在这个应用程序中实现了 12 个，有一个不错的值。如果有 16 个，而实现了 12 个，这很好。这是每个调度器的理论值，数值也是 12。每个调度器的活动 Warp 数大约是 9.69，这是理论的，这是实际达到的。然后每个调度器的合格（Eligible）Warp 数和每个调度器已发出的 Warp 数。总之现在不关注所有这些，这里相同的数据用图表可视化了。

如所示长时间计分板停滞（Long Scoreboard Stall），大约是 111 个周期停止等待。可以查看这个 Warp 采样。搜索一下长时间计分板停滞。但不管怎样，这里有短时间计分板停滞（Short Scoreboard Stall）。

这是指如果命中了某个内存层级，但这是指如果计算所有层级的平均值。然后有指令统计。这与不同的指令有关，停滞、耗尽（Throttle）、选定存储屏障。需要查找更多关于这些类型停滞的信息。例如整数乘加指令，意思是所有这些指令都在这个应用程序中执行了。并且获取每一条汇编指令，试图告诉你每条指令在应用程序中执行了多少次。

例如看乘加指令，可以看到它执行了超过 400 万次。`S2R` 这是用来读取时钟之类的，大约是二百多万次。有一个从内存加载值的加载指令，如所示，它也执行了超过 200 万次。移动指令超过 200 万次，退出指令等。还有将两个值相加的 `IADD` 指令，执行了一百多万次。

如果想查看这个应用程序中的所有指令，可以转到源代码（Source），这是这个应用程序核函数的汇编语言。这是汇编语言，这是 CUDA 的代码。

这是这个核函数的汇编语言。还记得这里的内容吗?

需要加载 A 值，然后加载 B 值，将它们相加并将结果存储在 C 中。这里读取线程 ID、块 ID 并进行一些计算，比如加法和乘法，并将值存储在 i 中。这里使用 i 值从内存加载 B、从内存加载 A、将两个值相加，并将结果存储在 C 中。这就是将在汇编语言输出中看到的内容。

例如看看这个，还记得这个指令吗? 这只是读取一个寄存器，一个预定义的寄存器，如所示这个值线程 ID，这个值 CTID，前文中说过 CTA 指的是块 ID。一旦读取了线程 ID 和块 ID，如果还记得这里，需要进行加法和乘法。这就是为什么这里有 `IMAD`。`IMAD` 指令将在这里执行，这里还有另一个 `IMAD`。一旦通过 `IMAD` 计算完所有这些值，需要将它们存储在这个值即 R6 值中，这就是 i 变量。然后有 `EXIT`，因为 if 语句的条件没有满足，那么需要退出。

如果 i 不小于 N 需要退出核函数，然后需要执行 if 语句。这就是在汇编语言中做 if 语句的方式。但如果不是，如果 i 小于 N 需要执行这个，这就是为什么这里有 `EXIT` 指令。如果达到了语句的条件，意思是需要执行这里的这个命令，那么将执行 if 语句内部的内容。

如所示，不会看所有的指令。但这里是前几条指令，前四条指令只是获取 i 值。需要在这里做的是有一个加载 A 的指令，一个加载 B 的指令。这是从内存加载，这也是从内存加载，然后需要乘加。因为一旦加载了 B 和 A 需要将两个值相加。这就是为什么需要 `IMAD` 或 `ADD` 指令。

30.6 指令依赖与停滞分析

现在不解释为什么这里两者都有。R4、R3。但想象一下可以把 `IMAD` 当做 `ADD` 使用，然后还有 `ADD`，这是加法过程。读取 B 值并存储在 R3，因为读取 A 值并存储在 R4，然后如所示，在这里将 R4 和 R3 相加，并将值存储在 R9。但不管怎样，现在不想关注这个命令。但能看到这里的感叹号吗? 如所示，如果把鼠标移到这里，会看到这个信息。这一行指令要对 84% 的停滞负责，所有 Sub-warp 停滞的 84%。

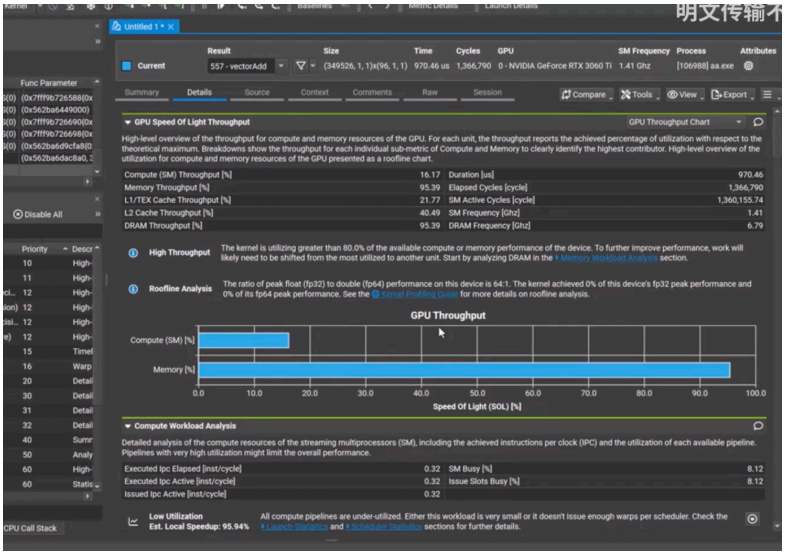


图 30.5:

再看一下这一行指令停滞的 99.7%，这一行指令导致了停滞。可能会问为什么是这一行？这只是加法，为什么加法过程会导致大量停滞？

如所示，有一个依赖，如果不从内存读取两个值，就无法对它们进行加法。这个加法将等待直到从内存获取 R4 和 R3。这就是为什么这里有依赖，这个依赖可以在这里显示出来。确认一下，有这个信息。

是的，如果引用这个指令，这些就是对依赖的解释。这个依赖于这个和这个。每当在这里看到这个，这意味着这个依赖于前面的那个。看看这个指令依赖什么。如果到这里看看这个三角形符号，从汇编代码看，它应该依赖于 R3 和 R4。这是三角形符号。它具体在哪里？是的，这里有一个依赖。

看看这个，看到 R4 了吗？如所示，依赖于 R4。但也应该看到，这里有什么依赖于 R3。

集中在这里，R4 在这里和这里都存在，这个指令需要等待来自这里的 R4，然后这个指令需要等待来自这里的 R4，情况就是这样。还有看看这个，每当看到右侧的这个部分，就会看到依赖关系。例如再次集中注意力，这里有一个向右的箭头，如果向上看会发现它依赖于 R4。这里同样这个指令有

另一个向右的三角形。R3 也有一个向右的三角形，它依赖于这里的这个指令。如果点击这里，会发现它也依赖于 R3。如所示 R3，这就是这里展示的依赖关系。

回到这里查看已执行的指令，这是每条指令执行时间的百分比。但可以看看这里的 Warp 停滞采样，可以看到这个指令导致了大约 93% 的 Sub-warp 停滞。因为它等待前面所有指令的结果，而前面所有指令又依赖于更前面的指令，这是最后一条指令，它依赖于前面的所有指令。

同样对于存储指令，也是因为一旦完成了加法过程或加法操作，需要将值存储到内存中的 C 里。这就是为什么这里有一条存储指令，`STG` 存储到全局内存。需要从加法过程或加法操作中取出 R9，并将其存储在 C 值中。如所示，同样这条指令导致了 93% 的停滞。这解释了这里的一切。在 Sub-warp 统计中，看看有什么。

是的，从这些选项中可以确定其他一些选项。例如有地址空间，可以从这里开始查看已执行的指令，但可以专注于地址空间。地址空间意思是在这里访问的是全局内存，但可以左右移动这个来查看所有内容，这也适用于全局内存。如果看这条指令，正在从全局内存加载，这些信息非常重要。

这是加载不是存储，但这里是存储。而扇区（Sector）是一个扇区，这里的扇区大小是 32 字节。因为如果还记得前文，一个扇区等于 32 字节。很多线程不仅仅是相加两个元素，每个是四字节加四字节，正是相加两个元素。但是是针对不同的线程，所以每个线程将需要四字节，这就是为什么每次指令执行加载 32 字节。

是的，这是寄存器的活跃性。还有其他内容吗？不确定活跃寄存器的含义，但可以稍后讨论。如果有其他核函数，可以从这里选择核函数。这里是 SASS，但没有生成 PTX，所以关注的是 SASS 指令，现在先跳过它。上下文，这是稍后需要讨论的内容。

环境（Environment），在这里可能会看到一些关于操作系统的信息。比如 Windows 11 名称、创建时间，拥有的 CPU 主机架构等。如果想查看这些信息，这只是其中一部分。如果 Nsight Compute 图形界面提供了关于性能分析的注释，这里应该包含一些注释，但不需要看这个。

回到详情，看看是否还有其他需要查看的东西。这里有一些说明，如果增加每个线程的寄存器数量会发生什么？实现了大约超过 40%，好像是 50% 左右。在纵轴上有 Sub-warp 占用率。这里这个 48%，这是因为每个线程使用了 16 个寄存器。但它说如果改变应用程序代码来增加每个线程的寄存器数量，将会降低 Sub-warp 占用率，正如这个图所示。这意味着可以从这个图中了解到，可以将每个线程的寄存器数量从 16 增加到 40，而不会导致 Sub-warp 占用率的下降。

这就是需要从这里学习的。可以将每个线程的寄存器数量从 16 增加到 40，而不会导致任何下降或损害 Sub-warp 占用率，这对性能来说是非常重要的。如果将每个线程的寄存器数量增加到超过 40，那么将会导致性能问题。同样适用于块大小——应用程序性能。它说在这里，在这个点上块大小是每个块 96 个线程，使用每个块 96 个线程，可以从这里的启动统计中看到这一点。它说，如果将这个数字从 96 增加到 128，Sub-warp 占用率将不会有任何影响。

这里只讨论 Sub-warp 占用率，但如果到这里将其增加到例如 800，将大大降低占用率，从 48% 降到大约 24%，相当于占用率下降了 50% 左右。但如果例如将其增加到 224，将稍微降低占用率。如果降低了占用率，将会影响应用程序的整体性能。如果增加了使用的共享内存，将会导致占用率的下降。同样适用于每个块的共享内存，但这里是因为在这个应用程序中没有使用共享内存。

不打算再次解释所有这些，但需要看看这部分。这是总数，不是每个 SM。在解释 Warp 停滞采样，并说有 47000 次停滞。它说可以点击这里来了解是哪条指令导致了所有这些停滞。如果还记得从源代码标签页，知道这条指令导致了大部分的停滞。但可以点击这里，它会带我们到同一条指令。

可以从这里的百分比值看到停滞的第二大原因。不知道是否有更多，但可以来到这里点击导致停滞的第二条指令，它会带我们到那个分支。再次点击这里，它选择了这里的这个，因为这个导致了 2% 的停滞。无论如何，这是针对执行最多的指令。如果点击这里，它会带你去那条执行了一百多

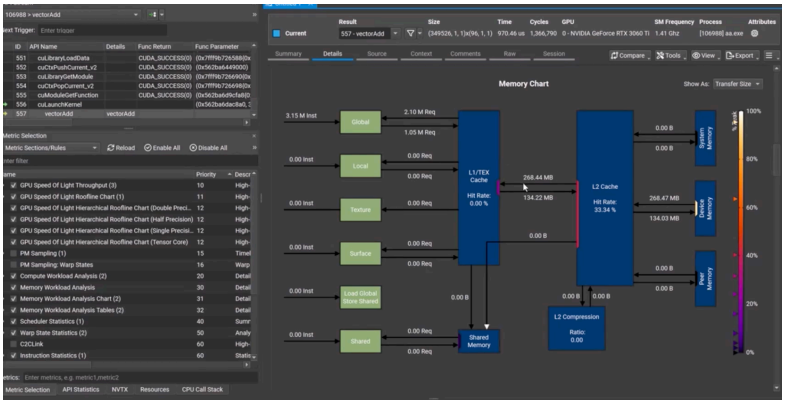


图 30.6:

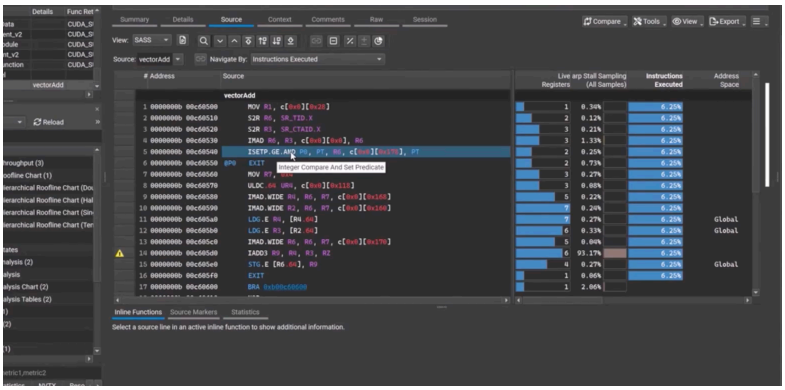


图 30.7:

万次的指令。点击这里，这条执行了一百多万次，可以对其他的做同样操作，但不全部演示。

30.7 占用率计算器与优化

这就是 Nsight Compute 用户界面的工作方式。关于它就是这样了，只是确认没有其他内容了。是的，这里指标详情这个标签页，当点击这里并点击这里，这会显示任何所选项目的详情。当选择某个项目时，它会把详情显示在这里，然后也可以显示占用率计算器（Occupancy Calculator）。前文讨

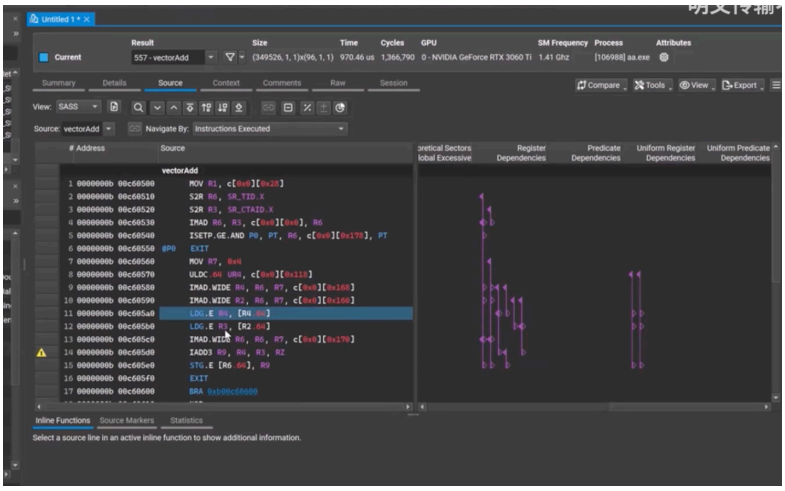

Untitled 1 *
Result
Size
Time
Cycles
GPU
SM Frequency
Process
Attributes
Current
557 - vectorAdd
970.46 us
1,366,790
0 - NVIDIA GeForce RTX 3060 Ti
1.41 Ghz
[106988] aa.exe
Summary
Details
Source
Context
Comments
Raw
Session
Compare
Tools
View
Export
View: SASS
Source: vectorAdd
Navigate By: Instructions Executed
Address
Source
Register Dependencies
Predicate Dependencies
Uniform Register Dependencies
Uniform Predicate Dependencies
vectorAdd
1 0000000b 00c60500 MOV R1, c[0x0][0x28]
2 0000000b 00c60510 S2R R6, SR_TID.X
3 0000000b 00c60520 S2R R3, SR_CTAID.X
4 0000000b 00c60530 IMAD R6, R3, c[0x0][0x0], R6
5 0000000b 00c60540 ISETP.GE.AND P0, PT, R6, c[0x0][0x178], PT
6 0000000b 00c60550 @P0 EXIT
7 0000000b 00c60560 MOV R7, 0x4
8 0000000b 00c60570 ULDC.64 UR4, c[0x0][0x118]
9 0000000b 00c60580 IMAD.WIDE R8, R6, R7, c[0x0][0x160]
10 0000000b 00c60590 IMAD.WIDE R2, R6, R7, c[0x0][0x168]
11 0000000b 00c605a0 LDG.E R4, [R8.64]
12 0000000b 00c605b0 LDG.E R3, [R2.64]
13 0000000b 00c605c0 IMAD.WIDE R6, R6, R7, c[0x0][0x170]
14 0000000b 00c605d0 IADD3 R9, R4, R3, RZ
15 0000000b 00c605e0 STG.E [R6.64], R9
16 0000000b 00c605f0 EXIT
17 0000000b 00c60600 BRA 0xb00c60600
Inline Functions
Source Markers
Statistics
Select a source line in an active inline function to show additional information.

图 30.8:

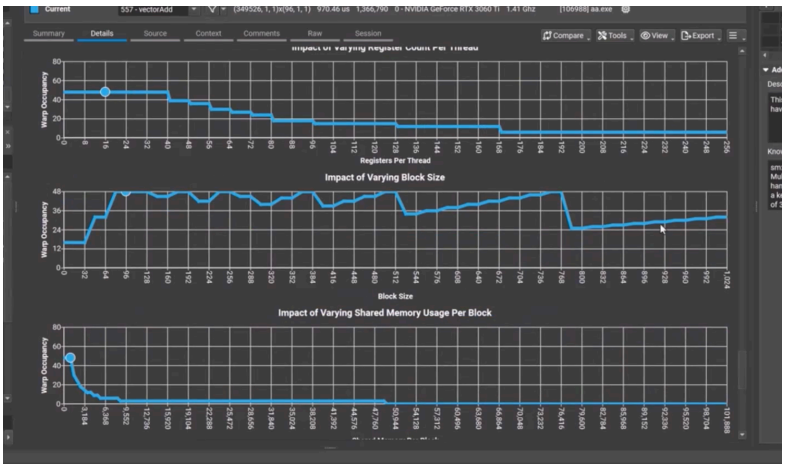

Warp Occupancy
Registers Per Thread
Impact of Varying Block Size
Block Size
Impact of Varying Shared Memory Usage Per Block

图 30.9:

论过这个计算器以及如何计算占用率。可以在这里修改这些数字，看看它们对占用率的影响。

同样前文在另一个标签页看到的这些图表在这里也有解释，但在这里可以操作它们。可以在这里修改并观察对数值的影响。不确定现在是否有效。是的，它起作用了。比如说 1024，比如说如果点击这里的 256，看到了吗？看到变化了吗？

这也是一个很好的功能，可以看到这些配置对性能的影响。可以折叠各个部分以隐藏图表，只看数字。还有什么功能？然后可以把性能输出在这里，导出或保存为 PDF、图像或其他格式。

可以设定一个基线（Baseline）。如果有一个核函数并想将其与其他核函数进行比较，可以在这里进行比较。可以将一个核函数设定为基线。还能在这里查看什么？从这里开始有一个叫做 API 统计的功能，可以查看或者说隐藏这里的任何部分。从这里这将展示 CUDA 的运行时 API。还记得吗？前文有关于运行时 API 的章节，所以需要看那个章节来理解这里讨论的内容。

`cudaMemcpy` 操作，这被称为 CUDA 运行时 API。`cudaMalloc` 在 GPU 的全局内存中分配空间，这也被称为 CUDA 运行时 API。同样适用于在应用程序中使用的所有其他函数，但这非常重要，因为它解释了这些 API 在这个应用程序中各消耗了多少时间。例如仅仅将数据从 CPU 复制到 GPU 或反之，这就消耗了大约 40 毫秒。在 GPU 中分配空间，这大约需要 7 毫秒。同样这与核函数执行无关。这里只是将数据从 CPU 复制到 GPU，而这里再进行分配。还记得这里的指针吗？这显示了曾经执行需要的时间。大约是 7 毫秒，这些 API 需要多少时间？可以看看这些 API 各需要多少时间来执行。

不需要看所有这些东西，只需要关注指标选择和 API 统计。据所知，还可以从这里选择许多其他选项，例如这个显示或隐藏这里的内容。可能需要做的事情之一是，例如添加另一个选择项。这个分析如果想要分析核函数，比如说修改了核函数中的任何内容并想再次分析它，只需点击这里或这里。如果添加了另一个选择项，需要从这里再次分析核函数。

然后从工具菜单有项目资源管理器、输出消息。看看输出消息，没有任何输出消息，也可以看看这里的选项，可以查看诸如字体等选项，这是针对用户界面本身的，可以选择一些配置，轻松地操作这些设置。然后从窗口菜单，可以显示或隐藏这里的一些窗口。

这就是关于 Nsight Compute 工具的图形用户界面的全部内容。但这并不是使用 Nsight Compute 工具的结束。因为当开始讲解性能章节和优化时，将基于从 Nsight Compute 获得的结果进行优化。

31 性能分析方法论

本章将对上一节最后一个应用程序进行一些性能分析。需要做的只是改变块大小（Block Size），看看它如何影响不同的性能指标。从 CUDA 代码开始，如所示，这里有一个将两个向量相加并将结果存储在第三个向量中的核函数，该向量称为 C 向量。然后在主函数中，遵循一般 CUDA 应用程序的步骤。如所示，定义了一些指针，然后在 GPU 中为这些向量或指针分配空间，然后在 CPU 端做同样的事情，并初始化向量 A 和 B，然后将数据从 CPU 中的 A 和 B 复制到 GPU 中的 A 和 B，以便能够在 GPU 中执行加法操作。然后执行核函数来执行加法操作，最后使用 `cudaMemcpy` API 返回结果。这就是在这个应用程序中需要做的和需要了解的全部内容。

这里需要控制或操作的是每个网格的块大小变量。实际上有两个变量，不是一个。因为如果还记得，对于任何 CUDA 核函数，有两个配置。第一个是每个块的线程数，第二个是应用程序中的块数。如果搜索这个，可以往上翻，看到正在从终端读取这个值。当执行这个应用程序时，需要在执行时添加块大小，并且这个值将被输入到这里的这条指令中。相比于上一节讨论的内容，这只是在应用程序中所做的唯一更改。

回到这里，这个文件叫 `test_01`。这里有一个名为 Section 05 的新章节，输入 `ls` 命令，这是文件。需要编译这个文件，写 `nvcc -o`，然后需要为可执行输出起个名字。可以说例如 `test_01`，可以就这样写，或者也可以写 `.exe`，两者都可以。然后需要 CUDA 应用程序的 `.cu` 文件，接着按回

车键。这里有可执行文件，可以直接复制这个并按回车键。

需要做的就是来到这里写一个空格，然后假设块大小是 256 个线程。如所示，它没有运行，因为先去掉这个空格，还是不行。因为它在这里要求块大小。然后按回车，应用程序成功执行，没有任何问题。终端上不会显示任何内容，因为没有打印任何东西。总之这就是需要做的。可以在这里放一个 `printf` 或类似的东西，以确保执行成功完成。

31.1 Python 自动化分析脚本

需要改变块大小，并用不同的块大小来分析这个应用程序，以观察这个变化的影响。这里有一个 Python 文件，它可以完成所有这些分析。将读取一些性能指标，然后有 L1 命中率、L2 命中率、占用率（Occupancy）。第一个是执行时间，第二个是活动周期数（Active Cycles）。因为通常可以用秒或周期数来衡量时间。这是用来读取占用率的指标。这里测量一个叫做分支效率（Branch Efficiency）的东西，将在关于分支发散（Branch Divergence）的章节中讨论它。今天不需要关注那个，但只是想提前开始了解。这个叫做分支效率。

最后有 DRAM 吞吐量。DRAM 吞吐量是通过内存每秒访问的千兆字节数（GB/s）来衡量的。需要知道这个应用程序的执行时间是多少秒，并使用该值来计算 DRAM 吞吐量。时间会影响 DRAM 吞吐量，因为吞吐量是通过每秒千兆字节数来衡量的。工具会帮做这件事，但只是在解释所有东西，这些就是指标。把它们写了两次，一次在这里，另一次在这里，但每个都有其用途，这个将和 NCU 一起使用。如果还记得，可以来到这里，像这样写 `ncu`，然后两个短横线 `--metrics`，就像在上一节讨论的那样。

然后可以在这里列出所有这些指标，如所示，得到了所有值。但在这里只是使用 Python 文件来为不同的块大小做这个，而不仅仅是一个。复制所有这些，把它们夹在这里，然后点击回车。不想看着终端显示很多表格，然后把这些值复制到 Excel 表格中再进行比较。需要 Python 脚本来完成所有这些。这就是为什么这里准备了 Python 脚本。可以用这个，但需要它们作

为一个数组。如所示，使用这里的 header 变量来为从这个 Python 脚本输出的 CSV 文件创建一个表头。然后这里打开输出文件，以便在运行 NCU 时存储这些指标名称及其值。

然后对于每个块大小，从 32 开始，因为最小的块大小是一个 Warp。从 32 个线程开始，一直到 1024。一个 Warp 由 32 个线程组成，并且取最大值 1024，因为根据白皮书，不能使用超过这个数字的块大小。

如果还记得在硬件章节，是第二节或第一节。说过根据白皮书，不能使用超过 1024 个线程的块大小。然后这里创建命令，这里是 1024。因为范围在这里，所以只需要在 1024 上加 1，以确保在 for 循环中包含 1024 这个值。

这是命令，如果看这里，它和这里的命令完全一样，将使用这个变量，而不是写出所有指标。但没有尝试这些指标，因为一开始已经在这里写了，如所示。如所示，在这里使用这个变量，然后这是可执行文件。这里添加块大小，块大小如所示，范围从 32 到 1024，步长为 32。即是 32、64、96、128，以此类推，每一步增加一个 Warp 或增加 32 个线程。

然后这里这是一条 Python 指令，用于运行这个命令。这完全等同于执行这里的这个即按回车执行这个命令。这就是在 Python 编程中执行终端命令的方式。这只是一个 `print`，然后这只是一个 `print`，用于显示当前正在为这个块大小运行的信息。这意味着这里的输出或这里的表格将被存储在 result 变量中。然后当执行这个 NCU 命令时，获取输出并将其存储在 result 变量中。现在需要做的就是从这个输出中读取指标名称及其值，并将它们全部存储在一个文件中。

需要分割这个 result 变量。把它们全部存储在这里的输出文件中，在这个 CSV 文件中分成不同的行。然后遍历拥有的所有行，逐行读取。对于每一行将查看或检查指标名称。一旦有了一个指标名称，就尝试读取它的值。

一旦有了值，需要将这个值存储到输出文件中。这里有很多细节，这些细节只是为了去除所有不需要的东西。因为如果看一下输出，例如不需要所有这些。只需要指标名称和它的值。如果这样做，会发现它比这个表格更奇怪。

顺便说一下，如果重复运行，因为在这里使用两个短横线 `--csv` 来确保它们以 CSV 文件格式显示，而不是表格。看到它是什么样子了吗? 需要去掉这个逗号，需要去掉这个双引号等。所有这些只是为了去除不需要的任何东西，或者只是从输出中移除它。

从这个文件中需要的就是这两样东西：指标名称和它的值。然后需要的是获取指标值，并将其存储在输出文件中。一旦在输出文件中有了所有内容，只是提一下具体存储在哪里。

是的，打开输出文件，从这里开始写表头，需要看到写入表头。看到表头了吗? 因为表头应包含块大小以及每个指标的名称。是的，这里尝试获取指标值，然后将其写入输出文件。然后应该看到另一个写入指令来写入值，然后这里需要绘制每个指标随块大小变化的关系图，意思是将块大小从 32 改到 1024，与例如周期数的关系等。需要绘制所有这些，这就是在这里做的。

需要在图表中显示块大小与占用率的关系，与时间的关系，与执行时间的关系，块大小增加。

31.2 分析结果图表

运行，这就是全部的内容。在纵轴上绘制所有指标，在 X 轴上应该是块大小。想象一下从 32 到 1024 步长为 32，这大概有 15 次左右。

将添加这个 Python 脚本，以便不仅可以将它用于这个应用程序。因为只是给出一个如何分析许多配置的想法，只需一次运行。因为想象一下在这里运行了多少次。

来到这里使用这个命令 15 次，然后手动读取这些值并把它们放进文件，这是非常困难的，将会花费太多时间。而且如果想用不同的配置再次重复它也会非常累人。这就是 Python 文件的样子，以及这里需要做的所有事情。

需要做的就是来到这里输入 `python3`，文件名是 `get_insight.py`，然后按回车。

它将为所有步骤或所有迭代重复运行。现在正在为块大小 32 运行。可以在这里看到这个 `print` 通过这条指令体现出来。这就是从这个脚本中需要的全部。一旦它完成了所有迭代，脚本将给出一些图表。除此之外，想再看另一个文件，需要查看图表，看看块大小的变化如何影响不同的性能指标。现在完成了，输入 `ls`，但不想让你困惑，所以等等看从这个输出中能得到什么。看看这些图表里的内容。

如所示，有一些图表，所以想到这里，在第一个图表里有 GPU 时间、执行时间和周期数。看看第一步或第一次迭代，其中块大小是 32 个线程或一个 Warp。如所示，对于两者——时间和活动周期——与其他所有迭代相比，都有更大的值。当每个块使用一个 Warp 时，得到了更长的执行时间。如在此所示，其他所有迭代中，这里只是稍微增加了一点，但大多数几乎相等。需要拿这个值和其他值进行比较，和其中任何一个值比较。这意味着需要进行一些性能分析，以理解为什么这里的执行时间比其他迭代长。

看看这如何影响其他指标。这里有 L1 缓存和 L2 缓存。对于 L1 看到对于所有迭代它都是零。因为改变块大小并不会改变访问全局内存的方式，将访问相同的扇区（Sector）或相同的字节。这就是为什么 L1 缓存是恒定的且等于 0，但在这个应用程序中没有那样做。如果改变例如步长（Stride），即每两个元素之间的步长，这可能会改变。

好的，这里 L2 大约等于 33 点几，它略有变化或变化很小，但几乎相同。在这个图表里没有看到任何变化。再次只是想说明，这里有块大小，这里是指标值，L1 命中率和 L2 命中率。因为如果看核函数时间，可以看到第一个值很大，而所有其他值都很小。

看最后一个，如所示，内存带宽与核函数时间成反比关系。对于内存带宽和 Warp 活动度或 Warp 占用率，可以在这里看到相反的情况。可以看到第一个值很小，但所有其他值都很高。那么开始做吧，这就是为什么需要关注第一个值，并拿例如第二个值对两者进行一些性能分析。来到这里说，例如需要输入 `ncu`，需要的是 `./test.exe`，然后输入 32。

需要看看当块大小是 32 个线程时，不同的部分（Section）是什么样的。网格大小（Grid Size）指的是这个应用程序中所有的块。这些是部分，需要

看的是块大小，如所示是 32，然后是网格大小。这是一个很大的数字，因为有数百万个元素需要相加。如果再看这里的应用程序，可以看到这些向量中每一个的大小是这个值，这是一个非常大的值。以及块大小，如所说是 32 个线程。如果有这个数量的元素，这是一个很大的数字，而只有 32 个线程。如果用这个除以这个就得到块数，这就是得到块数的方法。

总之对于每一个网格的块数，通常使用这个公式：总大小加每个块的线程数减一，除以每个块的线程数。而这个应用程序结合给每个块的线程数的值，两者共同给出了在这个应用程序中需要的线程总数。这给了每个网格正确的块数。因为记住，想确保应用程序中的线程总数等于要相加的元素总数。

总之，这里需要做的是取这个值和这个值，并把它们放在这里。如所示，每当在应用程序中使用块大小 32 时，总块数将是这里的这个值。那么在应用程序中有多少个活动块（Active Blocks）呢?

如果还记得在上一节解释了" 已分配活动块"（Allocated Active Blocks）这个术语，并且说过这真的非常重要。在继续本章之前，需要先看那个章节。总之具体在哪里可以找到这个值? 有 16、128、48，可以来到这里进入占用率部分，然后取这四个值中最小的那个。已分配活动块的数量将是 16。这就是为什么这里加了 16。很好，使用 CUDA API，可以获取 GPU 上可用的 SM 数量。在 GPU 上，得到的数量是 38 个 SM。

31.3 波次与执行时间分析

现在想计算波次（Wave）数量，这里需要了解更多关于波次定义的信息。说过已分配活动块的数量通常指的是可以在一个 SM 中同时运行的块的数量。意思是在那种情况下，可以同时运行 16 个块在每个 SM 上。但是记住块的总数不是 16、32 甚至 64，而是超过 100 万。从那个巨大的数字中，可以在每个 SM 上同时运行 16 个，那么剩下的块怎么办? 这就是所说的波次。

例如除以这个值，将这个值也就是总块数除以 16。说过可以同时运行

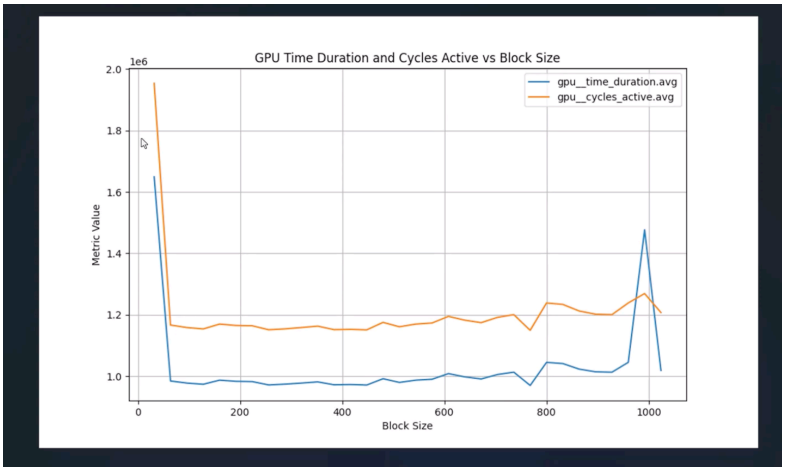

GPU Time Duration and Cycles Active vs Block Size
1e6
2.0
1.8
1.6
1.4
1.2
1.0
Metric Value
0
200
400
600
800
1000
Block Size
gpu__time_duration.avg
gpu__cycles_active.avg

图 31.1:

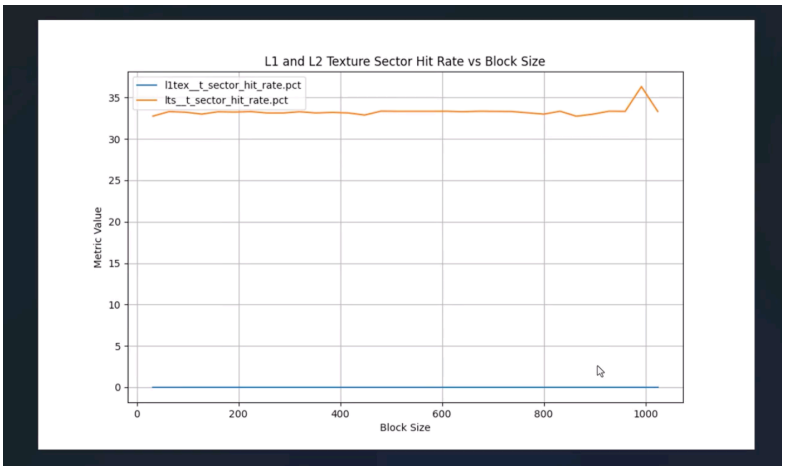

L1 and L2 Texture Sector Hit Rate vs Block Size
35
30
25
20
15
10
5
0
Metric Value
0
200
400
600
800
1000
Block Size
l1tex__t_sector_hit_rate.pct
lts__t_sector_hit_rate.pct

图 31.2:

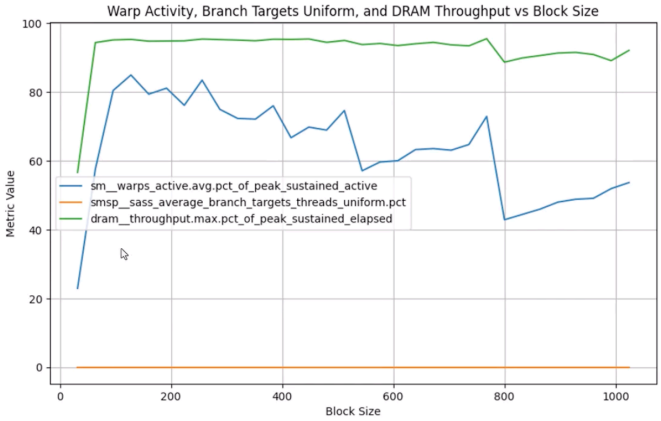


图 31.3:

16 个。然后除以 SM 的数量，因为讨论的是一个 SM。这就是在应用程序中，每个 SM 将要运行的波次数量。

再重复一遍，当这个应用程序启动时，它从这个数字中取出 16 个块交给第一个 SM，意思是对于这里的每个时间点，每个波次，将在每个 SM 上运行 16 个块。同样的情况也适用于所有其他 SM。一旦这些 SM 中的任何一个完成了它的 16 个块，就意味着它完成了第一个波次。那么它会请求另一个波次，它会请求另外 16 个块。

情况就是这样，看一下这里，这是 SM，这是第一个波次。如所示，有 16 个块 ID，从 0 到 15。一旦完成了所有这些块，SM 就可以获取另一个波次，可以获取另外 16 个块。但是再次当执行第一个波次时，不能获取这些块中的任何一个。即使所有这些块都因依赖而停顿，因内存操作而停顿或因任何其他原因停顿，同时只能有 16 个块在每个 SM 上。一旦完成那个波次，可以再获取 16 个块，就是这样。

这个术语非常重要，将解释它是如何重要的。总之这是使用块大小 32 的情况。如果使用块大小 64，这些是数字，又运行了 NCU 来获取总块数，这是总块数。为什么这里的总块数是那个数字的一半？这很简单，如果增加每个块的线程数，就必须减少总块数。因为说过总块数结合每个块的大小，

每个块的大小应该等于应用程序中元素的总数。

这就是它的工作原理。现在需要知道有多少个波次。再次只是想展示第一种情况的波次数量，看这里，看到这个数字了吗？每个 SM 的波次数量，可以从这里检查它，这完全是同一个数字。

回到这里除以这个值，这是总块数。应用程序中的所有块，所有这些块都应该在 38 个 SM 上执行。需要知道每个 SM 将执行多少个块，这就是需要做的。复制这个值并除以 38。那样的话，每个 SM 应该执行超过 13000 个块。

但不能同时执行所有这些块，可以在不同的波次中调度所有这些块。每个波次由 16 个块组成，需要用 16 除这个数以得到每个 SM 的波次数量。如所示是 862，如果在这里运行 NCU，会发现完全相同的数字。如所示是 862。

只是想在这里提一下波次数量，因为需要很好地理解它。总之回到第二张幻灯片，因为它非常重要。说过这种情况的执行时间比第二种情况的执行时间长。还记得在这里吗？不在这里，是在这个图表里。回头看看，还记得执行时间或活动周期数吗？对于块大小 32，它比块大小 64 的大，这没关系。

那么需要理解为什么情况是这样的。

31.4 块大小对性能的影响

想象有一个 GPU，两者都在执行同一个应用程序，但使用不同的块大小。想象将只分析 38 个 SM 中的一个 SM。在这种情况下有一个 SM，在这种情况下也有这个 SM，但区别在于——并且在两种情况下都有 16 个已分配活动块。确认一下。是的，如果看这里的占用率部分，可以看到这四个值的最小值是 16。在两种情况下已分配活动块的数量都是 16。这两条线代表一个 Warp，这就是为什么这里有 16 个从 0 到 15，这里也是从 0 到 15。

这里有两个 Warp，为什么？因为这个块由 64 个线程组成，块大小是 64，而一个 Warp 等于 32 个线程，这就是为什么这个块比这个大。如果用

32 除这个，那么每个块就有两个 Warp。

想象有一个内存操作（红色的那个）以及一个依赖于该内存操作的加法操作。在这种情况下，每个块将执行内存操作，并会等待内存操作完成。而内存操作通常比 GPU 中的算术操作花费更长时间。如果在 L1 缓存命中它可能花费 30 个周期，或者如果在 L2 缓存命中大约 200 个周期，或者如果在全局内存命中 300 或 400 个周期。但这里有依赖关系，一旦因这个依赖而停顿，SM 将不会执行任何操作。

所有 Warp 或所有块都会停滞。但这里有不同的 Warp。如果这里的第一个 Warp 因任何原因停顿，SM 可以获取第二个 Warp 并执行它。可以在不同的 Warp 之间进行调度，以确保最小化 SM 内发生的停顿次数。这就是为什么当块大小是 64 时，总执行时间比块大小是 32 时要短。不过将在后续章节中进行更多的性能分析。

32 非 2 的幂次方向量加法实现

在之前所有的应用程序中，使用的每个向量的元素数量都是 32 的倍数。例如在第一个应用程序中，为 A、B 和 C 这三个向量每个都使用了 1024 个元素。这个值就是 32 的倍数。如果用它除以 32，结果也是 32。在第二个应用程序或者说第二个例子中，每个向量使用了 2048 个元素，这也是 32 的倍数。这样做的原因是任何 CUDA 应用程序的最小规模都是一个 Warp。

32.1 Warp 与线程数的约束

任何一个 Warp 都由 32 个线程组成。这意味着可以用于任何 CUDA 应用程序的最小线程数是 32 个线程，不能使用例如 16 个线程，必须使用 32 个线程或 32 的倍数。因为必须选择 Warp 的数量，而每个 Warp 由 32 个线程组成。线程总数必须是 32 的倍数。

回答一个可能想到的问题。如果每个向量的向量大小是 16 个元素，这

是一个很小的规模。完全可以在 CPU 端相加这些小的规模，而不是在 GPU 端。因为把数据从 CPU 端复制移动到 GPU 端等操作会有开销。

但假设一下，假设向量大小只等于 16 个元素。需要将向量 A 中的这 16 个元素与向量 B 中对应的元素相加，并将结果存储在向量 C 的对应位置。在这种情况下，可以为这个应用程序使用 32 个线程的大小，因为这是最小值。但可以将 16 个元素分配给 16 个线程。而另外 16 个线程不执行任何操作，它们只是处于空闲状态，但在这种情况下，利用率或线程利用率是 50%。

有 32 个线程，但只使用了其中的 16 个。而这就是不选择 32 的倍数作为元素数量会带来的性能影响之一。现在明白为什么选择每个向量 1024 个元素，而不是例如 1000 个了。这只是本章的引言。必须使用 32 个线程，每个应用程序不能使用少于 32 个线程。因为最小值是一个 Warp，而一个 Warp 等于 32 个线程，每个应用程序必须是 32 或 32 的倍数。这更容易确定 Warp 数量或块大小，因为块大小等于 Warp 数量乘以 32，而每个 Warp 由 32 个线程组成。

当选择块大小时，可以计算网格大小。因为网格大小乘以块大小（以线程计）必须等于元素数量，这不必严格相等，但应该等于向量中元素数量的倍数。因为在后续章节中，将为每个线程分配多个操作。例如这里如果每个向量有 1024 个元素，那么必须选择网格大小和块大小，以提供总共 1024 个线程。在那种情况下，每个元素将被分配给一个不同的线程。

但可以调整应用程序、代码，为每个线程分配两个元素。在那种情况下，每个线程可以相加两个元素，而不仅仅是一个元素。只需要调整代码中的一两行就能做到。并且这将导致减少应用程序中的总块数，而这可能会提高应用程序的占用率（Occupancy）。因为如果每个线程将要执行两个元素，而不是每个元素一个线程，那么将需要总共 512 个线程。如果总共有 512 个线程，而不是 1024 个，那么就可以减少总块数。如果减少了总块数，这意味着在某些情况下可能对性能产生正面或负面的影响。

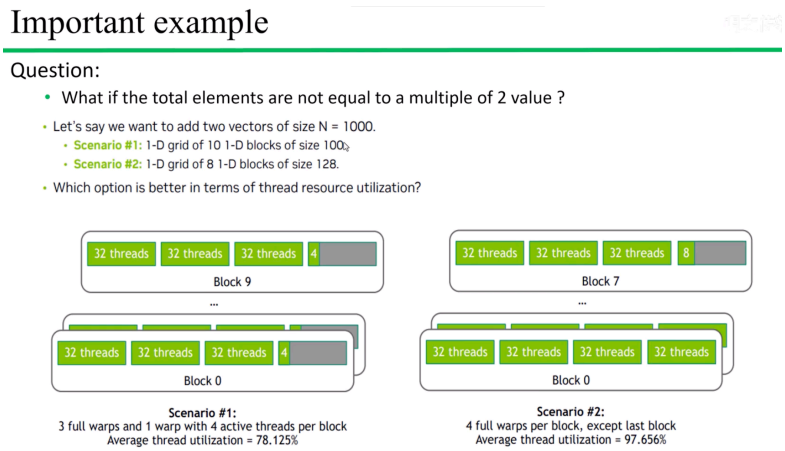


图 32.1:

32.2 块大小与线程利用率

那么现在假设有一个不同的例子。假设例如每个向量的元素数量等于 1000，而不是 1024。这里假设了两种情况，如所示，第一种情况是选择 10 个块，其中每个块包含 100 个元素。不是说 100 个线程，因为这是不可能的。每个块将由 96 个线程（等于三个 Warp）或 128 个线程（等于四个 Warp）组成。但在这里，在第一种情况下，只是说每个块将处理 100 个元素。不是说 100 个线程，有 10 个块。其中，每个块将处理 100 个项目，100 个元素。在第二种情况下，选择 8 个块，其中每个块将处理 128 个元素。

这是第一种情况，这是第二种情况。如所示，如果选择每个块 100 个元素，那么需要超过 100 个线程。这里能选择三个 Warp 吗？三个 Warp 将等于 96 个线程，这行不通。必须选择下一个 32 的倍数，下一个 32 的倍数是 128。

正如在这里看到的，例如在块内，必须选择四个 Warp。所有这些 Warp 的线程总数将是 128 个线程。只需要大约 100 个，这是 32、64、96。将只使用最后一个 Warp 中的 4 个线程，这里使用的线程总数是每个块 100 个线程。而有多少？有 28 个线程处于空闲状态。这适用于所有 10 个块。

如所示，每个块使用了完整的三个 Warp 以及最后一个 Warp 的 4 个

线程。这里可以计算线程利用率，线程利用率将是 78%。因为如果用使用的线程数 100 除以 128，这将给出 78%。即 28 个空闲线程除以线程总数，这是使用的部分，这是从所有可用线程中利用的部分。

在第二种情况下，减少了块数。为什么减少了块数? 因为每个块将处理 128 个元素，而只有 1000 个，所以不需要使用 10 个块。8 个块就足够了。将使用所有四个 Warp，如在此所示。

总之在第二种情况下，如在这里看到的，例如对于块内，将使用 128 个线程。同样的情况适用于第二个块，也适用于第三个块。以此类推，直到最后一个块。

在最后一个块中将使用四个 Warp 中的三个和最后一个 Warp 的部分线程。只有这些。能告诉所有这些块中有多少个线程处于空闲状态吗? 它们大约是 32 减 8，这大约是 24 个线程处于空闲状态。但在这里每个块大约有 28 个线程处于空闲状态（指第一种情况），不是所有块的总和。几乎是 100% 利用率（指第二种情况的前 7 个块）。这就是为什么这里的平均线程利用率是 96.6%。

现在看到选择块数量和每个块的线程数的影响了吗? 这非常重要。在这个部分（第二种情况）使用了更多线程，因为这在线程资源利用率方面会更好。而这里（第一种情况）有更多的线程处于空闲状态，同时还减少了块的数量（第二种情况），这也不错。因为这里每个块的大小和那里每个块的大小是恒定的。这里使用了四个 Warp，那里也使用了四个 Warp。但区别是，这里利用了所有线程或所有 Warp，而那里只利用了其中一部分，而且有 10 个块，而这里有 8 个块。

32.3 波次与执行效率

假设 GPU——这只是个假设，因为这是个很小的数字——假设 GPU 包含大约 8 个 SM。只有 8 个 SM。

在第二种情况下，将每个 SM 分配一个块，并且可以在一个波次（Wave）内完成执行。这里的波次执行前 8 个块。但在那里（第一种情况）必须进

行两个波次，而不是一个。因为需要执行 10 个块，每个块将被分配给一个 SM，因为只有 8 个 SM。有两个波次，这将使执行时间加倍。然后在第二个波次中，其中 2 个 SM 将执行剩余的 2 个块。

这只是一个例子，展示当每个向量的元素数量不是 32 的倍数时该怎么办——每个块的线程数，每个网格的块数等。希望能自己应用这个。因为已经多次解释了，唯一需要做的就是控制元素数量。这就是需要做的全部：相同的代码，相同的核函数，一切都相同。唯一需要操纵的就是这三个数字。

33 基于二维线程块与线程的矩阵加法

本章将开始第二阶段的讲解。在所有先前的应用中，专注于使用一维线程和线程块。将此应用在一个简单的例子中，该例子是将两个向量相加，并将结果存储在第三个向量中，并且在每个网格中使用一维线程块。

现在需要更深入一层，即进入本课程的下一阶段，在每个线程块中使用二维线程。由于需要在一个实际例子中应用这一点，在此添加了这个标题：二维矩阵加法。本章的主要目的是将两个矩阵相加，并将结果存储在第三个矩阵中。也就是说不再使用向量。在这里复习一下。

在之前的讨论中，在所有 CUDA 应用中使用线程块大小和网格大小。线程块大小在 CUDA 代码中被称为 `blockDim.x`。这一点很重要，因为需要在 CUDA 应用的方程式中使用这个标识符。并且它也可以标识为每个线程块的线程数。

记住，线程块大小等于 X 维度的 `blockDim`。因为之前只使用了一个维度，对于 Ampere 架构，在这种情况下可以使用的最大值是每个线程块 1024 个线程。这也就等于每个线程块的线程数。对于 Ampere 架构可以使用的最大值是 1024。然后还有网格大小，网格大小被称为 `gridDim.x`。需要这个是因为有时需要在 CUDA 代码中使用它，这里最大值非常大，但这里只是举个例子，所以这不是最大值。例如可以使用每个网格 1024 个线程块，那样的话就可以计算整个应用的线程总数，而不是每个线程块的线程数。只需将 1024 乘以 1024 就能得到整个应用的总线程数。这是假设使用

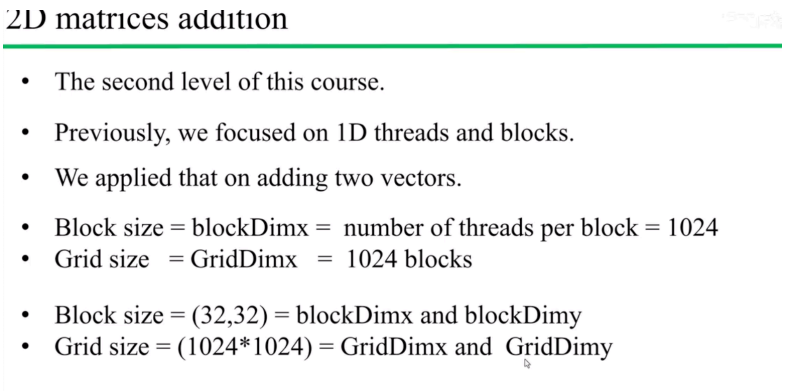


图 33.1:

一维的每个线程块线程数或网格大小时的情况。现在需要迈向下一步。

33.1 二维线程块配置

需要使用二维的大小。需要做的仅仅是像这样设置一个 X 值，这里的 32 和一个 Y 值，这就是需要做的全部。那样的话将有两个标识符，第一个用于确定 X 维度的大小，第二个用于确定 Y 维度的大小。因为这里两者都等于 32，所以 X 维度的 `blockDim` 等于 32，Y 方向的 `blockDim` 是 32。

但需要在这里提一下，这两个值的乘积绝不能超过这个值。因为说过最大线程块大小，无论是一维还是二维都不应超过这个值，绝不能超过这个值，即 1024。如果在应用中尝试使用，例如 1024 和 2，如果将它们相乘会得到类似 2048 的结果，那么应用将无法运行。即使编译是正确的，也需要留意这一点。

但只要不使用大于这个值的数字，可以自由调整这两个值，并且需要这样做，因为这里的每个选项都可能影响应用的性能。例如可以说 64 和 16，可以使用 1 和 512，可以使用 2 和 512，或者在 X 维度用 512，Y 维度用 2。需要测试这一点。

然后对于网格大小也可以进行选择。可以在 X 维度使用一定数量的线程块，在 Y 维度使用另一个数量，并且与线程块大小类似。这里有两个标

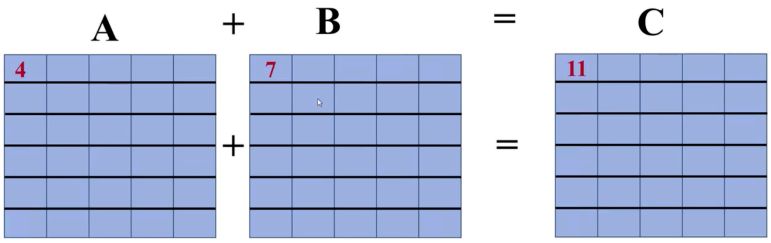


图 33.2:

识符来确定 X 维度和 Y 维度的大小。

例如在这个例子中，在这个实例中，在 X 维度有每个网格 1024 个线程块，在 Y 维度有每个网格 1024 个线程块。如果想计算这个应用中的线程块总数，可以将这两个值相乘。如果将它们相乘，会得到超过 100 万。每个网格的线程块数就是这两个值。如果想计算这个应用中的总线程数，需要将这两个数字相乘，其结果再乘以那个乘法的结果。将是 32 乘以 32 乘以 1024 乘以 1024。这就是如果使用这里的配置，每个应用的总线程数。

以上只是对将在应用中所做更改的一个介绍。但想从矩阵加法的定义开始讲起——简单的矩阵加法。因为知道每个矩阵都是二维的，需要从头开始理解。因为每当解释二维线程和线程块的索引时，这是最困难的部分。有矩阵本身的两个维度。

33.2 行主序内存存储

这并不容易，因为这里涉及不同的概念，有内存索引或每个元素将存储的内存地址。还有线程 ID 和线程块 ID，但这里有 X 维度的线程 ID 和 Y 维度的线程块 ID。需要理解所有这些概念是如何映射到一起的。从这里开始，有一个名为 A 的矩阵，需要将其与另一个名为 B 的矩阵相加。结果将存储在另一个名为 C 的矩阵中，并将结果存储在 C 的第一个元素中。需要按元素依次相加，意思是取第一个值，把它加到这里的第一个值，A 的

第一个值加到 B 的第一个值上。这就是在这个例子中做矩阵加法的方式。

这是需要很好理解的图表，因为知道这里有许多元素。但需要知道这些元素将如何存储在内存中。从这里开始，仅以矩阵 A 为例，只关注其中一个矩阵，同样的道理也适用于另外两个矩阵，这是第一个矩阵。如所示，假设在前两行有一些值。它们从较小值递增到较大值，故意这样做，以确保是按顺序读取它们的。这只是一个假设。然后想知道这些值将如何存储在内存中。因为内存不是二维的，有一个可以在代码中通过变量定义的矩阵，需要知道这些值将如何存储在内存中。

内存是一维的。这个二维矩阵如何存储在一维内存中呢？它看起来是这样的，这里聚焦于第一行，展示第一行将如何存储在内存中。需要关注的是这些值，如所示，第一个值会在这里，下一个值在这里。它们不是以一行，而是像存储在一列中那样。但这没关系，需要知道的是第一行的所有值将在内存中连续存储。连续意味着每个值都紧挨着前一个值，正如在这里看到的 4、7、9、0、2、5。

第二行，第二行会以不同的方式存储，还是相同的方式？它将以相同的方式存储，这就是要做的。第二行将会是这样，不是拥有两行，而是将所有值连续存储。就像在这里看到的，这是假设使用的是行主序（Row-major）存储。行主序意味着每一行将在内存中连续地存储其值。只是解释这两个术语，但将只关注行主序。但如果是列主序（Column-major），那么将取第一列的值，它们将在内存中连续存储。矩阵 A 中的二维值将如这里所示那样在内存中连续存储。

解释得比较慢，因为几分钟后事情会变得更复杂一点。总之请注意这里所说的，因为这非常重要。然后这里展示了内存中每个位置的地址。展示这个是因为将在代码中需要它。这里有 10 个位置，是的，从 0 到 9，10 个位置。这里假设这些内存位置的地址将从零开始，第一个位置是 0，第二个位置是 1，以此类推。在地址之后，这里有 X 维度的值，也有 Y 维度的值。在这一边，每个元素将由一个 X 值和一个 Y 值定义。但在内存中，这个值或同一个元素将仅由一个代表地址的值定义。现在需要知道的是定义这些元素中任何一个的两个值 X 和 Y 如何被转换或转移到这仅有的一个值。

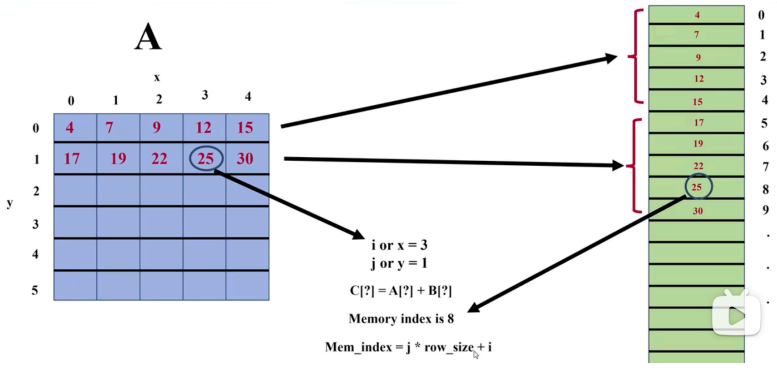


图 33.3:

33.3 二维索引映射

这就是要解释的，可以说 i 或 x，这没关系。假设在谈论这个值 25，这里的 25 的 X 值等于 3。比如说 i 等于 3，这里的 j 等于 1。所以有 i 等于 3，j 等于 1。但在 CUDA 代码中不能写 3 或 1。因为 3 或 1 并不能定义这个元素在内存中的位置。不能使用 3 或 1。需要做的是，如果看 25 这个值，可以看到它有一个索引 8。相反需要地址，或者更具体更准确地说，需要这个元素在内存中的索引。

内存索引是 8，但是 i 和 j 是 3 和 1，它是这样的，如何将 3 和 1 映射到值 8 呢？这是使用的方程。说 j 乘以行大小加上 i。把值代入这里，j 等于 1，二维矩阵的行大小是 5。因为有从 0 到 4，每行有五个元素组成，将用 1 乘以 5 再加上 3，这等于 8。这就是需要的，这就是将二维矩阵元素映射到内存中一个索引值的方法。

这容易吗？这是第一步。但这里还没有讨论线程 ID 和线程块 ID。别忘了有二维线程和二维线程块。如何将任何元素的地址或内存索引映射到任何元素的 X 和 Y 值呢？将矩阵中的元素映射到线程 ID 和线程块 ID，这里的问题在于这三者之间的映射关系。

这是标题：如何从线程 ID 和线程块 ID 映射到内存索引。先总结一下之前在章节和例子中讨论的内容。如所示，假设只有一个向量，就像之前讨论的那样。这是一个一维向量，一维应用。在应用中，假设使用了四个线程

块，这就是为什么所有的 X 线程块 ID 和线程 ID 都只存在于 X 维度。

假设这里的向量大小是这么多，可以说是 32 个元素。可以数一下所有这些值，看看这个向量里有多少个元素。第一个元素的索引是 0，最后一个元素的索引是 31。如果需要这些元素的索引，就是这样。这些就是索引。应用中有四个线程块，如果想说 A[i] 或 A[i]+B[i] 等。

每个线程块将处理八个元素，这里的线程块 idx 是 0，这里的线程块 idx 是 1。然后每个线程块由八个线程组成。如在这里看到的，每个线程块由八个线程组成。选择每个线程块八个线程并不是一个好的做法。应该使用的最小值是每个线程块 32 个线程，这等于一个 Warp 数。但这里这只是一个简单的例子。

无论如何，还记得这个方程吗？曾在代码中使用这个方程来获取这些元素中任何一个的索引。它是这样的：这是线程 ID。例如如果看这里的这个值 3，这个值的线程 idx，在这个线程块中，线程 ID 从 0 到 7。线程 ID 是 3 加上线程块 ID。顺便说一句，这个方程和这个方程非常相似，它们非常相似。但不管怎样，聚焦在这里。

线程块 idx，这是线程块 ID 1，将其乘以大小。X 维度的 `blockDim` 指的是线程块在 X 维度的大小。知道每个线程块由八个元素组成，将乘以 8，那么这里的值结果将会是 11。0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11。现在使用线程 ID 和线程块 ID 计算出了这个元素在这里的 ix 或索引值，这就是代码看起来的样子。如果还记得没有在这里添加任何新代码，这只是对所有先前讨论的复习。然后可以使用这里的 ix 来遍历从 0 到 31 的所有元素，这里的 ix 等于 `threadIdx.x` 加上 `blockDim.x` 乘以 `blockIdx.x`，就是这样。

现在需要前进到更复杂的部分，即使用二维数组而不是一个向量。为了更容易理解，想象一下将使用多个向量，而不是一个向量。这里的第一个向量，然后这里的另一个向量以及这里的另一个向量。第一个向量如所示，第二个向量以及第三个向量。

如果有一个向量，只使用这个方程来识别该向量内的任何元素。但如果有多个向量，可以使用另一个方程，这是针对 X 维度的。类似的还有一个针对 Y 维度的。如所示，ix 将完全是这里使用的那个，但这仅适用于第

一个向量。

如果有多个向量，可以使用 iy 来定义正在使用哪个向量。这是用来确定向量内的任意元素，而 iy 是用来确定正在使用哪个向量。这就是可以从向量过渡到矩阵，或者说从一维过渡到二维的方法。这是用来确定一行中的哪个元素，而是用来确定具体在处理哪一行。这里如所示，使用 ix 来确定向量内或具体到某行内的任意元素。因为如果是二维矩阵，那么需要行和列，这完全是一个与这个方程类似的方程。所以这是一个类似的方程，用于确定二维数组中的哪个元素。

ix 是从这里计算的，iy 是从这里计算的。这两个值定义了矩阵中这里的任何元素。例如这里的这个元素由从此处计算的 ix 值和从此处计算的 iy 值定义。知道这仍然很复杂，因为现在只讨论了线程和线程块 ID。但是内存索引或元素索引呢？这是元素索引，最终可以使用的元素索引。就像这里将在这里使用它。代替 X 值，这里是从线程 ID、线程块 ID 映射到元素索引。但想问的是矩阵的大小具体体现在哪里？和线程块 ID、元素索引。因为涉及三个部分：线程。

最后一部分是矩阵本身的大小。需要理解三个部分，而这只是一个实际例子，用来理解和想象这些部分是如何映射到一起的。在这个例子中，矩阵大小是从 0 行到 5 行。这个矩阵有六行，这是矩阵 A，在讨论这个矩阵的大小，有六行和 8 列，因为列从 0 到 7。是八列，记住这个。八列和六行。

33.4 二维矩阵配置实例

在这个例子中有多少个线程块？线程块的数量可以在应用中配置。例如这里有矩阵大小 X 和 Y 值，有六行和 8 列。

这里假设有六个线程块，并且它们是二维大小的。对于二维线程块和二维线程索引，有二维网格大小。网格大小指的是每个网格的线程块数，讨论的是线程块的数量，在 X 维度有三个线程块。这个应该在 Y 维度，不是 X 维度。不管怎样，所以有 1、2、3。有三行线程块，而在垂直维度，总之有两个。这是 1，这是 2。

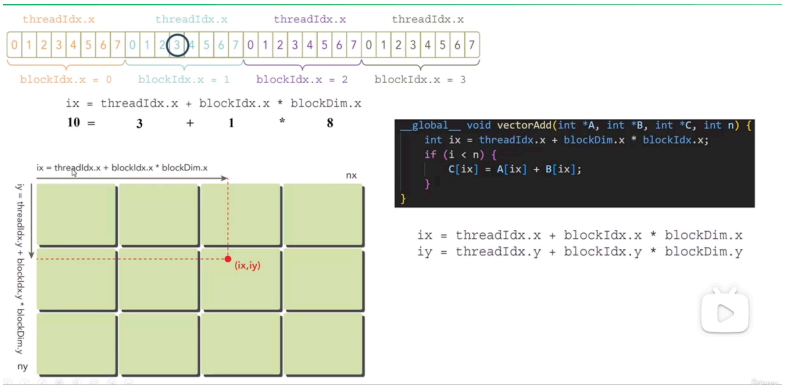


图 33.4:

这个例子中的线程块总数是六个。这就是它们如何映射到这个矩阵中的不同元素。如所示，线程块 (0,0) 将处理这里的这些元素，这八个元素。现在不需要看下一步就可以明白，每个线程块将由八个线程组成，因为它处理八个元素。这适用于所有线程块。

第二个线程块，即线程块 (1,0)——X 维度的 1 和 Y 维度的 0，它也处理八个元素，这就是二维线程块如何映射到这个矩阵中的所有元素。然后有二维线程块大小。说总线程数将是 2 乘以 4 等于八个线程。每个线程块的线程数是 2 乘以 4，因为这里每个线程块有两行和 4 列。

可以说这里的 `blockDim.x` 是 4，`blockDim.y` 是 2。

`blockDim` 指的是线程块大小。还记得 `blockDim` 是什么意思吗？如所示，在 X 维度有 4，Y 维度的线程块大小是 2。只写 `dim.x` 而不写 `blockDim.x` 是错误的。因为还有网格大小的 dim，必须在这里明确，有 `gridDim` 和 `blockDim`，像这样写是错误的。但不管怎样，`gridDim` 在 X 维度和 Y 维度将是 3 和 2，然后是内存索引。

`gridDim` 和 `blockDim` 将分别在 X 和 Y 维度是 3 和 2，这里是矩阵大小，这是二维线程块和线程，这是内存地址。不喜欢说内存地址，但只是为了更容易理解，在每个元素内添加了内存索引。但可以说内存和这些索引。例如第一个元素的索引将是 0，第二个元素是 1，第一行的索引将从 0 到 7。这就是它们在内存中的排列顺序。

如果看第一个元素它是 0，最后一个元素它是 47，可以理解这个矩阵有 48 个元素。然后继续索引 8、9、10，以此类推，直到到下一行 16、17 等。元素总数是 48 个，它们从 0 到 47 索引。但没有每个元素的具体值。想知道如何在这三个部分之间进行映射。不需要关注值本身，以这个例子为例，这个值第 44 个元素，将遮住其他线程块，以免用不同的东西造成困惑。

将以这一个为例，这是在处理的线程块，这些是正在处理的元素，这是 ix 值。记住需要计算 ix、iy，然后是索引，已经知道索引是 44，但在这里是为了验证原理。是为了证明是的，它是 44，并且这个值可以使用这里的这些方程计算出来，这是 ix 值。

想问的是，能从这个线程块中给出 `threadIdx.x` 吗？知道每个线程块由八个线程组成。这些线程有两个维度，一个在 X 一个在 Y。能给出这里 `threadIdx.x` 的值吗？只讨论 idx，不是两者。能给出这个元素的 `threadIdx.x` 吗？请仔细想一想。

在这里直接给出这个值，如所示它是 0，为什么？因为从这里开始，这是 0、1、2、3，这是针对 X 维度。对于 Y 维度这一行将是 0，这一行将是 1。如果看 X 维度这是 0、1 和 2，但这是针对两个维度的，不是针对一个维度。

现在需要考虑 `blockIdx.x`，记住这是线程块 ID，只需要 idx 即 X 维度的 ID。如果看这个值，会发现它是 1，因为这是 X 维度上的值。可以从这里得到这个值。然后需要知道 X 维度的 `blockDim`，即 X 维度的线程块大小 dim。X 维度的 `blockDim` 是 4。这里的值 ix 值将等于 4 乘以 1 加 0，所以等于 4。

现在有了 ix 值，需要计算 iy 值。是的，是 1。能告诉同一个元素的线程 ID 吗？但是在 Y 维度不是 X 维度。是 0 和 1，这个值是 1。能告诉 Y 维度的线程块 ID 吗？不是 X 维度。然后能告诉 Y 维度的 `blockDim`，即 Y 维度的线程块大小吗？线程块 ID 像是 2。

1、2。从这个方程得到的总值将是 5，这是 ix 和 iy。现在需要计算索引本身，即将在代码中使用的索引。A 的索引加上 B 的索引等于这个值将存储在 C 的索引中。那么在这里展示索引值，这里的 iy 是 5，所以这是 iy

乘以 nx。能告诉 nx 吗?

这完全来自这里矩阵大小。因为这是 nx 即矩阵 A 在 X 维度的大小，如果是这样，可以得到值的列的数量。因为每行——这里每行如果还记得每行有八个元素或八个值，每行的大小是 8，这就是 nx 的值。现在明白了吗? 线程 ID 被转换为 idx 值，而这里使用的是矩阵大小，这就是这里三个部分如何映射在一起。总之可以说这个矩阵在 X 维度的大小是 8，这正是一开始就在这里看到的那个值。5 乘以 8 加上 ix 值即 4，总值将是 44。

希望这能更容易理解和想象二维线程是如何映射到正在处理的矩阵的任何元素的。这只是所有这些内容的一个总结。需要这些方程的结果，结合以列数表示的矩阵大小来得到内存索引。就像在这里看到的。需要线程 ID 和线程块 ID 来获得 ix 和 iy。线程 ID 和线程块 ID 来计算 ix 和 iy，这只是两句话来总结所有这些。

这是 CUDA 编程最优秀的书籍之一。这些例子或这个映射，说实话是从这本书中引用的，而这正是希望看的，它会带你一步一步学习。希望看这本书并继续学习。其他例子想是在第三章，第三章或第二章记不清了，但就是从这里引用的。这将与将看到的命中和未命中数量有关，但现在不打算解释这个。

如果在处理二维线程和线程块，可以定义 `blockDim`。就像在这里看到的 `blockDim`，这只是一个变量名。然后可以使用这里的方程来计算每个维度中剩余的线程块数量。但可以将其定义为 32 乘以 32。如讨论的 X 维度每个线程块 32 个线程，Y 维度每个线程块 32 个线程。因为想说明，每个矩阵都有特定数量的元素。例如假设有一个矩阵，大小是 X 维度 4096 个元素乘以 Y 维度 4096 个元素，那么元素总数将是这两个值相乘的结果。

元素总数必须等于将在应用中使用的总线程数。这里有两个配置：每个线程块的线程数和线程块数量。如果像在这里做的那样，手动确定每个维度的线程数，比如 32 和 32，那么必须使用一个方程来计算线程块的数量。不能假设这里是 32 乘以 32，而那里例如是 1024 乘以 1024。如果只使用一个维度，可以使用这里的这个方程，这就足够了。不，这不是它的工作原理，因为需要确保所有维度中所有值的乘积将给出一个线程数，等于相

加的矩阵中的元素数量。如果使用两个维度，可以在这里使用这个逗号来为下一个维度或 Y 维度设置值。如果运行这个应用，并且如果在这里使用相同的矩阵大小，当尝试使用例如 NCU 分析时，会在输出中看到这个。将展示这个例子，将运行它，但只是在展示代码前解释这个概念，因为代码不是困难的部分，理解代码背后是什么，这个代码背后的原理是什么，才是需要知道的最重要的事情。当尝试分析这个具有这些配置的应用时。

以及这个矩阵大小。会在输出中看到这个，会看到 `blockDim` 是这里的这个指令给出的 32 和 32。并且从这个方程会看到，在 X 维度有 128 个线程块，在 Y 维度有 128 个线程块。想问为什么是 128? 为什么不是例如 256?

为了解释这个，想说明必须看一下这里的这个图表。说过从这个输出中有 128 个线程块。正如从 0、1、2 直到 127 所看到的，这里的每一行代表线程块的数量。列也是如此，从 0、1、2 开始直到 127。然后想问，能告诉 X 维度有多少线程吗? 这个图表代表线程块的数量，并且对于每个线程块，如在这里看到的，每个线程块的大小是 32 乘以 32。有 128 个线程块，每个线程块在 X 维度有 32 个线程。这很简单，只需将这个值乘以这个值，就能得到 X 维度的这个值；如果将这个值乘以这个值，就能得到这个值 Y 维度。这就是要说明的，需要确保 X 维度和 Y 维度的总线程数应该等于矩阵大小。

不打算在本章中讨论这个。将在下一章中讨论它们。希望看一下代码，是这样的，在这里要做的是复制这个，然后转到新例子。

33.5 一维与二维代码对比

但在看今天讨论的代码之前，想带你回顾一下一维线程和线程块。这里称它为 `vectorAdd` 并找出差异。需要在这里看一下核函数，GPU 核函数它由 `__global__ matrixAdd` 开始。如果看这个有 ix 和 iy，可以给它们起任何名字，可以叫 row、col 或者希望的 ix、iy。但这里只有一个，因为这是向量，向量不是矩阵。这里说 C[ix] 等于 A[ix] 加上 B[ix]。这里必须使用行值和列值重新计算索引，这是唯一的区别，没有其他区别。

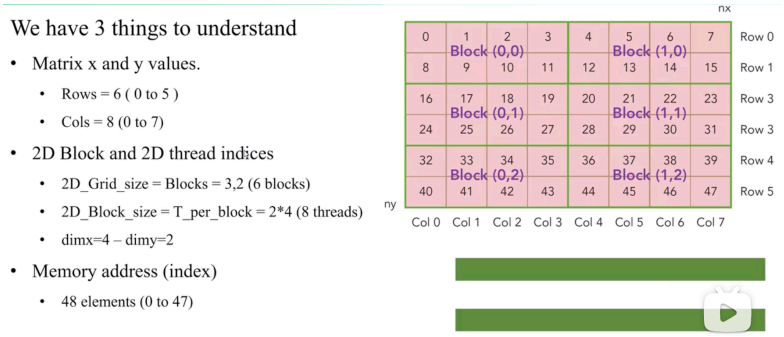


图 33.5:

好的，然后在代码内部，如所示，通过将 nSize 和 mSize 乘以 float 的大小来确定每个矩阵的大小。因为在处理单精度值，这里的大小是通过两者相乘来定义的，而不是像这里那样大小只乘以一个维度。N 和 M 在这里定义，N 是 4096，M 是 4096。看这个只有一个维度的大小，因为这是向量，而这是矩阵。一旦定义了每个矩阵的总大小，将在 CPU 端为它们分配空间。

然后初始化 A 和 B 并在 GPU 端在 GPU 内存中分配一个相似的空间。然后将 A 和 B 的数据从主机复制到设备，或者说从 CPU 复制到 GPU。如在这里看到的，从主机到设备，接着将定义 X 和 Y 维度的线程块大小，以及 X 和 Y 维度的网格大小。如在这里看到的，只是想看看 X 维度的这一部分，会发现它是相等的，它非常相似，和这里的这个方程这完全是同一个方程。但这里只使用一个，没有逗号，因为这是向量不是矩阵，所以只需要一个维度。

顺便说一下，在这个例子中，也可以使用一维的线程块大小和网格大小来进行矩阵加法。这就是为什么建议看这本书，因为它有这个例子，无法在这里解释所有例子，但需要给一些任务。它将会工作。与其使用两个维度来进行两个矩阵加法，尝试使用一个维度来相加这两个矩阵。并且可以研究这两种情况下的性能。总之这里这个只是想跳过，因为它涉及 L1 缓存，所以今天不想讨论它。不需要添加它。可以移除它。

然后在这里启动核函数来执行。发送 A、B、C 以及 X 和 Y 维度的大小。一旦这里的全局执行完成，它将在这里打印 A、B 的前两个元素及其值。

以及 C 以检查结果是否成功计算出来，这就是整个代码。需要做的是来到这里，这个文件叫做 `two_dimensions_01.cu`。在这里要做的是来到这里使用 NVCC 编译这个文件。将把输出或可执行文件保存为这个名字，然后点击回车，这里没有错误，需要做的就是点击 `./`，然后是 `.exe`，然后点击回车，如在这里看到的这等于 16 点多。

第二个值同理，因为在这里打印这两个值来验证工作的正确性，这是输出 8.4 加 7.8，这就是它的样子。但同样也可以来到这里。需要看一下这一部分，这是命令，并且写例如 `ncu` 来查看性能分析的输出。如看到的。还记得幻灯片上的这个吗？这里这个来自幻灯片。

计算出的两个维度的线程块数量是 128 和 128 个线程块。如所示，线程块大小的配置是 X 维度 32，Y 维度 32。然后可以查看 SM 的利用率，可以查看不同内存级别的吞吐量，如 DRAM、L1、L2 等。会发现线程块大小是 1024。同样也可以查看启动统计信息，线程块大小是 1024，为什么？如果将这些值相乘，32 乘以 32，这将是 1024。因为每个线程块的总线程数将是 32 乘以 32，然后网格大小是 16384。

如果将这个值乘以这个值，这像是 1600 万。这是总线程数。因为有线程块大小和网格大小，请记住这一点，将来这里计算 4096 乘以相同的数字。这将是 1600 万，与总线程数相同，元素总数。因为这是矩阵大小，矩阵大小中的所有元素，它们的总数应该等于应用中的总线程数。

33.6 性能分析与优化

占用率有 50% 的占用率，受线程块限制的线程数和这里的线程块。然后每个 SM 的波次（Wave），已经在前面的章节中解释了波次是什么意思，所以需要再次复习。每个 SM 同时只能运行一个线程块，这非常糟糕。原因是在每个线程块中使用了太多的线程数。使用了每个线程块的最大线程数，

这等于 32 个 Warp。但是如果将这个值从 32 乘以 32 改为更小的值，会看到这个值应该会增加。这里的每个线程块由 32 个 Warp 组成，这就是为什么不能使用超过一个线程块。因为每个线程块都很大，每个线程块有 32 个 Warp。试试这个，例如来到这里。写成 16 乘以 16，然后编译并分析性能。

受线程块限制的线程数数量，这里是 6。如所示看这个，这意味着可以取这里的最小值。可以同时运行每个 SM 六个线程块，将这个值与之前的值比较。不知道这是否会影响总执行时间，但可以检查一下，比较一下执行时间。这里的占用率是 74%。

因为有多个线程块，每个都有多个 Warp，这是 74%。但在第一种情况下，可以看到只有 50%，这些是性能指标，而这是分析这些指标的方法。这里把执行时间存储在这个文件中，在前面的一个章节中解释过这个。

这将在这里给出一些性能指标，在这里需要做的是写 `sh`，然后点击回车，这里的执行时间大约是 478 微秒。例如周期数、执行时间，只需要看这个以及 L1 和 L2 的命中率，L1 命中率大约是 1%。几乎是 0%，L2 也几乎是 0%，但这里另一个 L2 命中率等于 33%。将在下一章中解释它们，这是本课程中最喜欢的章节之一。

下一个。总之需要关注这两个值：时间和活动周期。然后回到这里的代码，将这些值恢复为 32 乘以 32。保存、编译。必须编译。然后再次运行相同的脚本。

看看这里。是的，如所示，这种情况下的执行时间与这种情况相比较长，想现在这很容易理解了。这里的占用率是 73% 或 74%，这里的占用率是 49% 或 50%。然后认为不同级别缓存的命中率是相同的，33%。

这就是本章的全部内容，希望这容易理解。这是进入本课程下一阶段或第二阶段的开始。

34 L1 缓存命中率为零的原因分析

摘要

本章继续探讨矩阵加法应用程序的性能分析，重点聚焦于 L1 和 L2 缓存命中率。通过回顾 NVIDIA GPU（以 Ampere 架构为例）的内存层次结构，深入解析为何在矩阵加法中 L1 缓存命中率为零，并揭示 NVIDIA Nsight Compute 工具在统计 L2 缓存命中率时将所有写入操作视为" 命中" 的特殊机制。最后通过向量加法案例验证了这一结论，强调了在性能评估中区分读取命中率与总命中率的重要性。

34.1 引言：矩阵加法回顾

本章将继续讨论之前的应用程序案例，即两个矩阵相加并将结果存储在第三个矩阵中。具体而言，将矩阵 A 的所有元素与矩阵 B 的对应元素相加，结果存入矩阵 C 的对应位置（例如 $4 + 7 = 11$）。在上一章中，已经讨论了矩阵相加的原理和概念。

本章的主要目的是进行性能分析，重点关注 L1 和 L2 缓存的命中率。

34.2 NVIDIA GPU 内存层次结构回顾

为了理解缓存行为，需要回顾 NVIDIA GPU 的硬件实现。以下讲解基于 NVIDIA 白皮书中的内存层次结构图（以 Volta 或 Ampere 架构为例）。

34.3 全局内存与 HBM

从最底层的全局内存（设备内存）开始，现代高端 GPU 通常采用 HBM（High Bandwidth Memory，高带宽内存）。相较于 DDR3/4/5 等传统内存，HBM 以其极高的带宽著称。以 A100 GPU 为例，其显存总线宽度超过 5000 位（远超 CPU 端常见的 64 位），带宽可达每秒 1.5 至 1.9 TB。这是一个极其庞大的数据传输能力。

34.4 L2 缓存

L2 缓存被划分为多个分区（Slice），在所有 SM 之间共享。HBM 通过内存控制器连接到 L2 缓存。无论请求来自哪个 SM，数据都可以通过 L2 缓存进行发送或接收。对于 HBM2e 等规格，L2 缓存同样包含多个物理分区，数据可根据 SM 的访问请求在分区间移动。

34.5 L1 数据缓存与共享内存

深入到单个 SM 内部，L1 数据缓存与共享内存（Shared Memory）共用同一个物理存储单元。在 Ampere 架构中，该单元的最大容量为 192 KB。开发者可以配置应用程序，将此容量按需分配给共享内存和 L1 数据缓存（例如：100 KB 用于共享内存，92 KB 用于 L1 缓存）。

需要注意的是：

- L1 缓存仅在单个 SM 内部共享，不在不同 SM 之间共享。
- 共享内存同理，仅局限于 SM 内部。

34.6 数据访问流程与 L1 命中率分析

34.7 数据加载路径

当应用程序运行时，数据通常初始存储于 HBM 中。当 SM 内的计算核心（ALU、Tensor Core 等）需要全局内存数据时，流程如下：

1. 核心向加载/存储单元（Load/Store Unit）发送请求。
2. 加载/存储单元首先查询 L1 缓存。若命中，数据直接返回核心；若未命中，请求转发至 L2 缓存。
3. 若 L2 缓存命中，数据返回 SM；若仍未命中，则向 HBM 发起请求。

34.8 为何矩阵加法的 L1 命中率为零?

在矩阵加法 $C = A + B$ 中，每个元素仅被访问一次：读取 A 的元素，读取 B 的元素，计算后写入 C。由于没有数据复用（Reuse），同一个元素不会被再次读取。因此，理论上该应用的 L1 缓存命中率应为 0。这与在 Nsight Compute 中观察到的结果一致。

相比之下，在矩阵乘法中，矩阵 A 的一行会与矩阵 B 的多列反复相乘，存在显著的数据复用，因此会产生大于零的 L1/L2 命中率。

注意：块大小（Block Size）的配置可能会影响实际的缓存行为，但基本原理不变。

34.9 L1 缓存技术细节与合并访问

关于 L1 缓存的关键参数：

- **最大容量**： 192 KB（可配置）。
- **缓存行大小**： 128 Bytes。

34.10 线程束（Warp）与内存合并

每个线程束由 32 个线程组成。当一个线程束执行加载指令时，32 个线程在同一周期内发出请求。加载/存储单元会根据地址连续性将这些请求合并（Coalesce）。

示例：假设 32 个线程各加载一个 4 字节的浮点数，且内存地址连续。总数据量为 $32 \times 4 = 128$ 字节，恰好等于一个缓存行的大小。此时，加载/存储单元会将 32 个独立请求合并为单次 L1 缓存行请求，极大提升效率。

34.11 L2 缓存命中率的”秘密”：写入操作被视为命中

34.12 观测到的异常现象

在矩阵加法测试中，虽然 L1 命中率为 0，但 L2 总命中率却显示约为 33%。这与理论预期不符。

34.13 原因解析

这是 NVIDIA Nsight Compute 工具的一个特殊统计机制：**L2 缓存中的所有写入（Store）操作均被计为”命中”，即使实际并未命中缓存**。

在矩阵加法中，每个元素对应三次操作：

- 加载 A（Read）：未命中
- 加载 B（Read）：未命中
- 存储 C（Write）：**被工具计为命中**

因此，命中率计算公式为：

$$\text{L2 Hit Rate} = \frac{\text{Write Ops}}{\text{Read Ops} + \text{Write Ops}} = \frac{100}{100 + 100 + 100} \approx 33.3\%$$

34.14 验证实验

通过向量加法程序验证了这一结论：

```
使用Nsight Compute运行分析
ncu --metrics l1tex__hit_rate,lts__hit_rate ./vector_add
```

结果显示：

- L1 命中率：0%
- L2 总命中率：33.33%

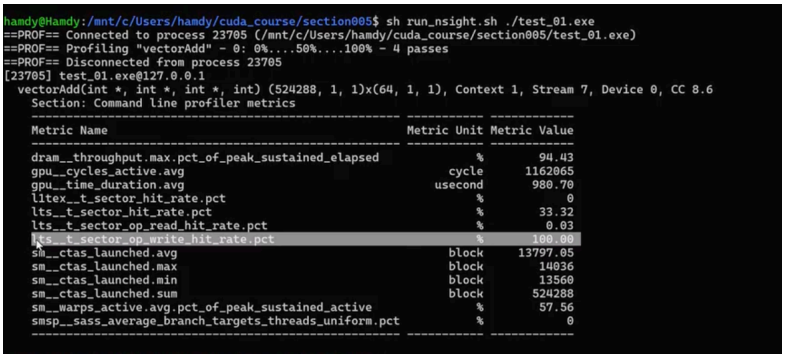


图 34.1:

- L2 读取命中率（lts__read__hit__rate）：$\approx 0\%$
- L2 写入命中率（lts__write__hit__rate）：100%

34.15 关键结论与建议

1. 在矩阵加法等无数据复用的场景中，L1 缓存命中率确实为 0。
2. NVIDIA Nsight Compute 将 L2 写入操作强制计为命中，导致 L2 总命中率虚高。
3. **性能评估建议：**在评估 L2 缓存性能时，应仅关注**读取命中率**（Read Hit Rate），而非总命中率，以获得公平准确的评估。此规则不适用于 L1 缓存。

结语

本章到此结束。矩阵加法中缓存命中率的成因及工具统计特性，如有疑问，可发送邮件或在 NVIDIA 开发者论坛中搜索相关讨论，那里包含大量宝贵信息。

35 CUDA GPU 并行编程：共享内存技术详解

35.1 引言

在 NVIDIA GPU 的内存层次结构中，共享内存（Shared Memory）是提升并行程序性能的关键组件。本章基于相关技术讲座内容，对共享内存的硬件架构、软件编程接口、存储体（Bank）组织形式以及存储体冲突（Bank Conflict）等核心概念进行系统性的修正与总结。

35.2 GPU 内存层次结构概述

NVIDIA GPU 内部包含多种内存类型，其访问延迟与可见性各不相同：

- **全局内存（Global Memory）**：对所有线程及 CPU 可见，容量大但延迟高（> 300 个时钟周期）。
- **二级缓存（L2 Cache）**：在所有 SM 之间共享，延迟约为 200 个时钟周期。
- **一级缓存（L1 Cache）**：位于 SM 内部，仅对当前 SM 内的线程可见。它是硬件管理的缓存，程序员无法直接控制其分配与替换策略，延迟约为 33 个时钟周期。
- **寄存器（Registers）**：按线程私有分配，生命周期与线程一致，是速度最快的存储单元。
- **共享内存（Shared Memory）**：位于 SM 内部，属于软件可管理的缓存，延迟极低（约 20–25 个时钟周期）。

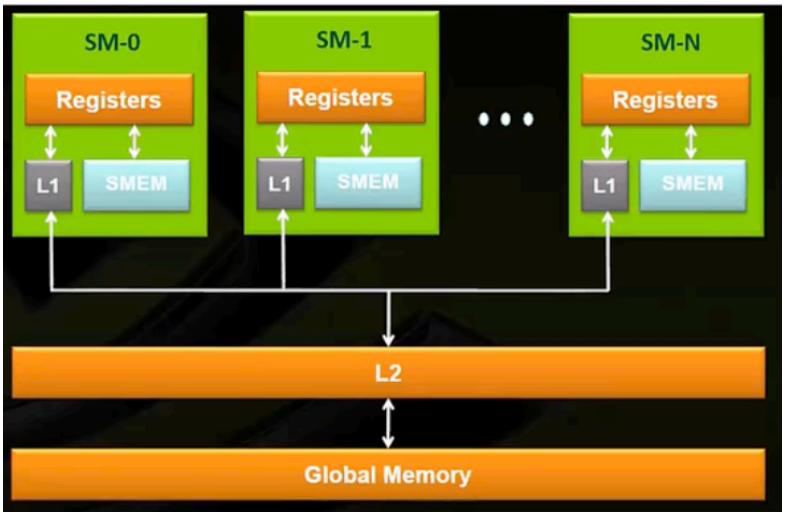


图 35.1:

35.3 共享内存的架构特性

35.4 硬件与软件的融合

在现代 NVIDIA 架构（如 Turing、Ampere）中，L1 缓存与共享内存通常位于同一个物理存储单元中。该物理单元的总容量是固定的，通过编译器或配置 API 在 L1 缓存与共享内存之间进行动态划分。

35.5 Ampere 架构配置示例

以 Ampere 架构（如 A100 GPU）为例，单个 SM 的 L1/共享内存物理单元总大小为 192KB。

- 若配置共享内存为 0 KB，则 L1 缓存为 192 KB。
- 若配置共享内存为 8 KB，则 L1 缓存为 184 KB。
- 共享内存的最大可配置容量通常为 164 KB，因为必须保留部分空间作为 L1 缓存。

通常情况下，编译器会根据代码中的 `__shared__` 变量声明自动完成最优配置，开发者无需手动干预。

35.6 可编程性与数据重用

与 L1 缓存不同，共享内存是可编程的。开发者可以通过 `__shared__` 关键字显式声明变量，精确控制数据的存储位置、步长及存储体映射。这种特性使其成为实现线程间通信、数据重用及减少全局内存访问（Memory Bottleneck）的最佳选择。

35.7 共享内存的物理组织

共享内存被划分为多个**缓存行（Cache Line）**和**存储体（Bank）**：

- **缓存行**：每行大小为 128 字节。
- **存储体**：每个缓存行包含 32 个存储体。
- **存储体大小**：128 Bytes/32 = 4 Bytes。

访问规则：每个时钟周期，每个存储体（列）仅能处理一次读/写操作。若同一周期内有多个线程访问同一存储体的不同地址，将发生存储体冲突。

35.8 存储体冲突（Bank Conflict）详解

35.9 冲突定义与计算

存储体冲突发生在同一个 Warp（32 个线程）内的多个线程同时访问**同一存储体中的不同地址**时。

$$\text{冲突次数} = \text{单存储体最大并发访问次数} - 1 \tag{1}$$

例如，若某存储体在同一周期被 4 个线程访问，则产生 $4 - 1 = 3$ 次冲突，该次访问需要 4 个时钟周期才能完成。

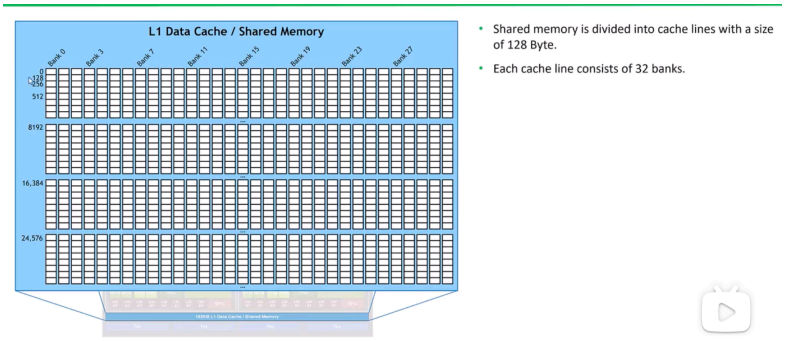


图 35.2:

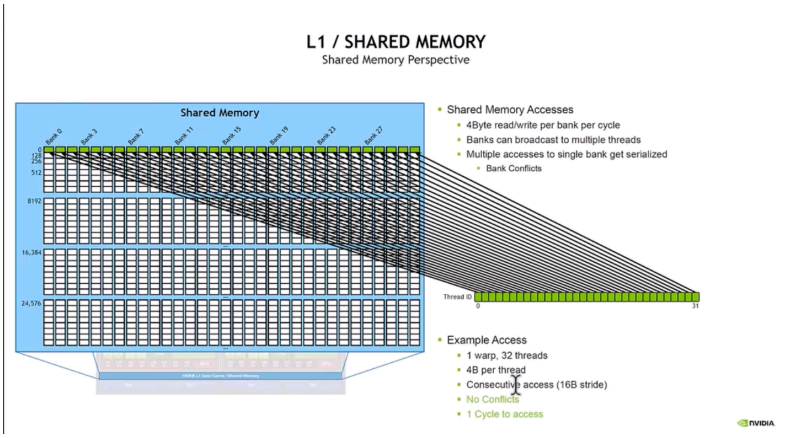


图 35.3:

35.10 典型访问模式分析

1. **无冲突访问（步长为 1）**：
   当 32 个线程以 4 字节步长连续访问时，每个线程恰好映射到不同的存储体（Bank 0 到 Bank 31）。此时无冲突，1 个周期即可完成。

2. **广播机制（Broadcast）**：
   若多个线程访问**完全相同的地址**（同一存储体、同一偏移量），硬件会触发广播机制，将数据同时发送给所有请求线程，**不产生存储体冲突**。

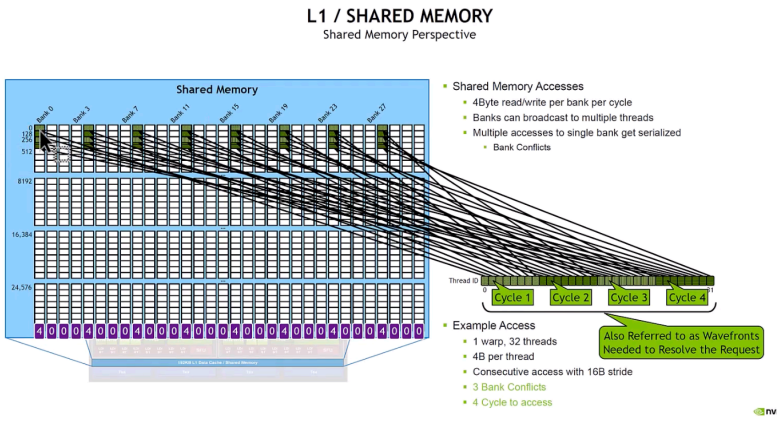


图 35.4:

3. **有冲突访问（步长为 16 字节）**：
   若步长为 16 字节（即跨越 4 个存储体），线程 0 访问 Bank 0，线程 1 访问 Bank 4，线程 2 访问 Bank 8。此时，Bank 0 会被线程 0、8、16、24 同时访问。最大并发数为 4，因此产生 3 次冲突，耗时 4 个周期。

35.11 双精度数据类型

对于 8 字节的双精度（Double）数据，每个线程需访问 2 个连续的存储体。32 个线程共需访问 64 个存储体位置（跨越 2 个缓存行）。由于每个存储体每周期仅能服务一次，理论上至少需要 2 个周期。在实际硬件实现中，NVIDIA 可能通过特殊的交织访问机制优化此类场景，导致 Nsight Compute 等工具报告的冲突数可能与纯理论计算存在差异。

35.12 总结

共享内存是 CUDA 编程中优化内存带宽的核心手段。理解其底层物理结构（32 Banks，4 Bytes/Bank）及冲突机制，对于编写高性能核函数至关重要。开发者应：

1. 尽量采用步长为 1 的连续访问模式。
2. 利用广播机制处理只读常量数据。
3. 避免不必要的跨步访问（Strided Access）。
4. 信任编译器的自动配置，但在极端性能需求下可手动调整 L1/共享内存比例。

36 Visual Studio 调试技巧

36.1 Nsight Visual Studio Edition 入门与安装

36.1.1 课程前言与工具回顾

本章将讲解本课程的第一个调试工具。在正式开始之前，先回顾一下：在本课程的每个阶段，都会介绍一款用于调试、性能分析或性能评估的工具。

例如，在第一讲中，讲解了 **NVIDIA Nsight Compute** 工具，包括其命令行分析和图形用户界面（GUI）的使用方法。这是一个非常有用的工具，可以帮助计算执行时间、SM 利用率、占用率、内存带宽，以及一级缓存和二级缓存的命中率。它甚至能提供可视化图表，帮助寻找应用程序中的性能瓶颈。

36.1.2 本章目标：向量归约与调试需求

本章将进入第二讲。在第一讲（入门级）中，讲解的是一个简单的向量加法应用程序；而本章将要讲解的是 **向量归约（Vector Reduction）** 应用程序。

针对这种更复杂的逻辑，需要的不仅仅是性能分析工具，更需要一个**调试工具**。需要通过调试工具在特定指令旁添加断点来暂停执行，从而观

察程序到达该指令时的状态：内存状态如何？变量的值是多少？这就是本章的主要目的。

36.1.3 NVIDIA Nsight Visual Studio Edition 简介

本章将使用的工具是 **NVIDIA Nsight Visual Studio Edition**。之所以称为“集成（Integrated）”，是因为它必须与 Visual Studio 应用程序集成使用。这与 NVIDIA Nsight Compute、Nsight Systems 或 Nsight Graphics 不同，后三者通常是独立运行的工具。

36.1.4 安装方法

关于如何安装该工具，可以查看官方安装文档或使用手册。安装主要有两种方式：

1. 直接从 Visual Studio 内部安装。
2. 通过 Visual Studio Marketplace 安装。

接下来将演示第一种方法（相关链接也会添加到本教程中）。

36.1.5 安装步骤

1. 打开 Visual Studio。
2. 在顶部菜单栏中，点击 **扩展 (Extensions)** 菜单，选择 **管理扩展 (Manage Extensions)**。
3. 在搜索框中输入 “Insight”。
4. 找到 **NVIDIA Nsight Visual Studio Edition (64-bit)**（如果你使用的是 32 位系统，也可以安装 32 位版本）。
5. 点击 **安装 (Install)** 按钮。

6. 安装完成后，需要 **关闭 Visual Studio 并重新打开**，以完成安装步骤。关闭时可能会弹出一些浏览器窗口或提示框，按照提示操作即可。

36.1.6 工具界面与功能概览

重新打开 Visual Studio 后，再次点击 **扩展 (Extensions)** 菜单，可以看到 **Nsight** 子菜单。这里列出了在 NVIDIA Nsight 调试工具中可以找到的选项。

36.1.7 集成工具入口

菜单中的前三个选项与操作系统上安装的其他 Nsight 工具相关。也就是说，如果操作系统上未安装 NVIDIA Nsight Compute、Nsight Graphics 或 Nsight Systems，这里可能不会显示它们。例如，在系统中搜索 “Nsight” 后，可以看到已经安装了 Nsight Compute、Nsight Systems 甚至 Nsight Graphics。这意味着可以非常方便地直接从 Visual Studio 内部打开这些工具来分析应用程序。

优势：不需要专门去开始菜单打开 Nsight Compute，然后再尝试将进程附加（Attach）到工具上才能启动性能分析。现在这些繁琐的步骤都不需要了。只需打开 CUDA 应用程序，转到 **扩展** → **Nsight** → **Compute** → **Profile** 即可直接开始分析。同样的方法也适用于 Nsight Graphics 工具。

36.1.8 核心调试选项

除了上述集成入口，菜单中还有其他用于调试的选项，这些代表了此扩展的主要目标。可以看到如下选项：

- **Start CUDA Debugging (Next-Gen)**：启动 CUDA 调试。
- **Break on Launch**：在启动时中断。
- **Freeze**：冻结线程等选项。

- **Options**：在选项中可以更改一些设置。

相关内容摘要。文档链接（NVIDIA Nsight Visual Studio Edition）也会添加到本章资源中。

36.2 调试实战：变量观察与内存分析

36.2.1 调试器的核心功能与界面概览

如上所述，需要安装这个独立的工具，它将为 Visual Studio 增添强大的调试功能。通过该调试器，可以直观地查看以下关键信息：

- **内存状态**：查看显存或内存中的数据变化。
- **线程信息**：查看当前的线程数量及状态。
- **GPU 寄存器**：直接观察 GPU 寄存器中的数值。
- **汇编代码**：查看 CUDA 代码对应的底层汇编指令。
- **内置变量**：查看块 ID（Block ID）、线程 ID（Thread ID）以及应用程序中的其他自定义变量。

这就是所说的“调试”。可以在代码的特定行设置断点，程序的执行会在此处暂停。这将深入了解当程序运行到这条指令时，汇编代码层面以及寄存器中每个变量究竟发生了什么。

36.2.2 准备工作与文档资源

在开始实战之前，请确保已经安装了上述提到的两个工具（Nsight Visual Studio Edition 及其依赖组件）。相关下载链接会添加到本章的资源列表中。

此外，官方文档中包含了关于如何使用这个调试器的详细信息。文档中解释了一些需要编辑的选项和配置，稍后将逐一解释这些内容。还会在文档中找到 Visual Studio 中 CUDA 相关的特定选项说明。

36.2.3 变量窗口与汇编级调试

重点关注调试界面中的 **变量窗口 (Variables Window)**。

- **检查变量**：可以检查自定义变量，也可以检查预定义变量（如线程 ID 和块 ID）。
- **检查内存**：可以观察特定变量（例如变量 A）在内存中的状态和数值变化。
- **代码映射**：可以看到 CUDA 代码中的每一行是如何转换成对应的汇编代码的。例如，如果应用程序中有一个变量 A，可以实时检查它在当前执行点的值。

稍后通过一个具体的例子来展示这一过程。

36.2.4 实战演示：向量加法调试

回到应用程序。这里有一个非常简单的 **向量加法 (Vector Addition)** 应用程序作为示例。虽然这个例子很简单，但调试方法同样适用于后续更复杂的程序（如向量归约）。

36.2.5 验证程序正确性

首先，需要运行应用程序以确保它正常工作。

- 观察输出结果：可以看到 $0+2=2$，$1+3=4$，$2+4=6$，以此类推。
- 确认应用程序运行良好，没有任何问题。

36.2.6 设置断点与启动调试

确认程序无误后，开始进行调试操作：

1. **添加断点**：在代码行号左侧点击，或者通过菜单栏选择 **新建断点 (New Breakpoint)** 来添加断点。

2. **启动调试**：点击工具栏上的调试按钮，或者通过菜单栏选择 **Nsight → Start CUDA Debugging (Next-Gen)** 来启动调试会话。

一旦启动，程序将在断点处暂停，就可以开始检查上述提到的各种状态了。

36.2.7 Nsight Compute 调试界面概述

在 Nsight Compute 中，需要区分 CPU 端与 GPU 端的调试视图。虽然软件提供了通用的调试菜单，但针对 CUDA 开发，应专注于 GPU 专用窗口。

36.2.8 线程视图与任务选择

进入调试模式后，界面会显示多个窗口。请注意：

- **GPU 线程视图**：专门用于显示 GPU 端的线程执行状态，这是关注的核心。
- **CPU 线程视图**：显示主机端线程，通常不在 GPU 核函数调试范围内。

在视图选项中，可以选择“Tasks”或“Threads”。对于核函数级调试，主要使用线程视图来追踪应用程序中的主要执行流。若当前无活动任务，需确保核函数已正确触发断点。

36.2.9 启动配置参数解读

在调试窗口的“Launch Details”面板中，可以查看当前核函数的启动参数：

- **Grid ID / Block IDX**：当前执行的网格索引与块索引（均从 0 开始）。

- **Thread IDX**：当前线程在块内的索引（从 0 开始）。
- **Grid Dimensions**：网格维度，即每个网格包含的块数量。
- **Block Dimensions**：块维度，即每个块包含的线程数。例如，本例中每块 96 个线程，等价于 3 个线程束（Warp，$3 \times 32 = 96$）。

36.2.10 单步执行与变量观察

使用“Step Over”按钮可逐行执行核函数代码。在执行过程中：

1. 高亮行随执行进度移动，指示当前指令位置。
2. 局部变量窗口实时更新。例如，当执行到 $C[i] = A[i] + B[i]$ 时，可观察到索引 i 的值以及 A、B、C 对应元素的数值。
3. 若变量值未立即更新，可能需要额外执行一步以完成赋值操作。

注意：在调试初期，可能会观察到变量值似乎“卡”在初始状态（如始终显示 0 和 2）。这通常是因为调试器仅聚焦于当前活动的线程束，而其他线程的计算结果尚未在当前视图中刷新。此时应结合内存视图进行验证。

36.2.11 访问内存窗口

通过 `Debug > Windows > Memory` 打开内存视图。该窗口以十六进制格式展示设备内存内容，每行包含多个地址及其对应的值。

36.2.12 定位变量与地址计算

要查看特定数组的内存布局：

1. 在内存窗口的搜索框中输入变量名（如 `A`、`B` 或 `C`）并回车，视图将自动跳转至该数组的起始地址。
2. 观察地址增量：若首地址为 `0x...00`，下一行为 `0x...30`（十六进制），则差值为 `0x30`（十进制 48），表明每行显示 48 字节的数据。

36.2.13 整数类型的内存布局

数组 A、B、C 均为 `int` 类型，理解其存储方式至关重要：

- **字节占用**：每个 `int` 占 4 字节（32 位）。
- **十六进制表示**：每个十六进制数字占 4 位，两个十六进制数字构成 1 字节。因此，一个整数由 8 个十六进制数字表示。
- **小端序存储**：在小端架构下，低位字节存储在低地址。例如，整数值 2 在内存中显示为 `02 00 00 00`；当数值增大至超过 255（`0xFF`）时，高位字节才会被填充。

36.2.14 数据验证示例

根据初始化代码 $A[i] = i$ 和 $B[i] = i + 2$：

- **数组 A**：起始值为 `00 00 00 00`（0），随后为 `01 00 00 00`（1）、`02 00 00 00`（2），依此类推。
- **数组 B**：起始值为 `02 00 00 00`（2），随后为 `03 00 00 00`（3）、`04 00 00 00`（4）。
- **数组 C**：预期结果为 $A[i] + B[i]$。前几个元素应为 `02 00 00 00`（2）、`04 00 00 00`（4）、`06 00 00 00`（6）。

36.2.15 调试暂停与部分计算结果

在调试过程中，可能会发现数组 C 仅有前几个元素有正确值，其余均为零。**这是正常现象**，原因如下：

- 调试器在断点处暂停时，仅完成了部分线程束的执行。
- 未执行的线程尚未写入结果，其对应的内存区域仍保持初始值（零）。

- 继续执行“Step Over”或运行下一个线程束后，新的计算结果（红色高亮显示）将逐步填充到内存中。

通过检查特定地址（如 `0x...420`），可以验证后续线程的计算结果是否正确写入。这种逐步填充的过程直观地展示了 GPU 并行计算的执行时序。

36.3 SASS 汇编分析与 Nsight 工具集成

36.3.1 访问反汇编视图

在 Nsight Compute 调试界面中，通过 `Windows > Disassembly` 打开反汇编窗口。需注意区分 CPU 端与 GPU 端汇编：

- CPU 端汇编通常以标准 x86/ARM 指令呈现，非本章重点。
- GPU 端汇编（SASS）以双下划线前缀标识（如 `__global__`），是分析 CUDA 核函数性能的关键。

36.3.2 高级代码与 SASS 指令映射

CUDA C++ 代码与 SASS 指令并非一一对应，编译器会进行多种优化：

1. **参数加载：**核函数启动初期，多条 `MOV` 指令用于将核函数参数（指针 A、B、C 及标量 N）从常量内存或寄存器加载到工作寄存器。
2. **线程索引计算：**对应代码中的 `int i = blockIdx.x * blockDim.x + threadIdx.x`，SASS 中体现为读取 `TID`（线程 ID）、`CTAID`（块 ID）及块维度，后接乘法与加法指令。
3. **边界检查：** `if (i < n)` 被转换为比较指令（如 `SET.LT`）与条件分支指令。若条件不满足，程序跳转至核函数末尾标签（如 `DDD0`），跳过计算体。

4. **计算与存储**：单行 $C[i] = A[i] + B[i]$ 可能展开为多条指令，包括全局内存加载（`LDG`）、浮点加法（`FADD`）及全局内存存储（`STG`）。编译器可能因循环展开或指令调度插入额外操作。

36.3.3 汇编分析的价值

查看 SASS 对性能调优至关重要：

- **识别障碍**：检测是否存在不必要的同步屏障（`BAR`）或内存依赖导致的流水线停顿。
- **验证优化**：确认编译器是否按预期进行了向量化、循环展开或寄存器分配。
- **延迟归因**：当高级代码无法解释性能瓶颈时，SASS 可揭示指令级延迟来源。

注意：NVIDIA 未公开 SASS 完整文档，分析需结合经验与社区资源。本例中未观察到明显障碍指令，表明核函数流水线较为顺畅。

36.3.4 Visual Studio 集成 Nsight 工具

此前性能分析需在 Linux 终端或 WSL 中手动执行：先运行 CUDA 程序使其暂停，再通过 `ncu --attach` 附加分析器，最后启动 GUI 加载结果。此流程繁琐且易出错。

36.3.5 VS 扩展一键启动

Visual Studio 通过 Nsight 扩展简化了全流程：

1. 确保已安装 CUDA Toolkit（自带 Nsight 工具）或通过官网单独安装。
2. 在 VS 中打开 CUDA 项目（如 `kernel.cu`）。
3. 菜单栏选择 `Extensions > Nsight`，可见 **Nsight Compute**、**Nsight Graphics**、**Nsight Systems** 三个选项。

36.3.6 Nsight Compute 集成分析

1. 选择 `Nsight Compute > Profile`，VS 自动填充可执行文件路径与工作目录。
2. 指定输出文件路径（如 `Section8/A.ncu-rep`），避免报告丢失。
3. 点击 **Launch** 启动分析。若程序含 `scanf` 等阻塞调用，需在控制台输入以继续执行。
4. 分析完成后，结果自动在 VS 内嵌窗口展示，包含：
   - **Speed of Light / Roofline**：理论性能上限与实际达成率。
   - **Compute Workload Analysis**：SM 吞吐量、ALU/FP64/Tensor Core 利用率。
   - **Memory Workload Analysis**：L1/L2 缓存命中率、显存带宽利用率。
   - **Occupancy**：每周期活跃线程束数。

关键提示：重点关注核函数 **持续时间（Duration）**，此为后续向量规约优化中对比不同实现性能的基准指标。

36.3.7 交互式分析 vs 文件分析

- **常规分析（Profile）**：生成 `.ncu-rep` 文件，支持离线复查与团队共享。
- **交互式分析（Interactive）**：实时查看指标，不保存文件，适用于快速迭代调试。

36.3.8 Nsight Systems 时间线分析

通过 `Extensions > Nsight Systems > Start` 启动。可选择收集 GPU 指标、CUDA Trace、上下文切换、CPU 采样等。本例聚焦 GPU 行为。

36.3.9 时间线核心解读

Nsight Systems 提供毫秒级精度的系统级时间线，关键轨道包括：

- **GPC Clock Frequency**：GPC（Graphics Processing Cluster）时钟频率随负载动态调整。空闲时降频节能，计算时升频至峰值。频率波动直接影响性能一致性。
- **SM Thread Occupancy**：反映 SM 活跃线程束比例。低占用率时段可能对应数据传输、核函数启动开销或 CPU 瓶颈。
- **Tensor Core Activity**：本例向量加法未使用张量核心，故该轨道为零值。若启用混合精度计算，此处将显示活动强度。
- **Memory Transfers**：Host ↔ Device 数据拷贝与核函数执行的时间重叠度，是异步优化效果的直观验证。

36.3.10 交互操作

- **缩放**：按住 `Ctrl` + 鼠标滚轮可局部放大时间线，观察微秒级事件。
- **悬停信息**：鼠标悬停任一区间，弹出详细元数据（如占用率百分比、指令类型分布）。
- **瓶颈定位**：通过时间线间隙识别 CPU-GPU 同步点、内存传输阻塞或核函数启动延迟。

本章建立了从 SASS 指令级到系统时间线级的全栈分析能力。Visual Studio 集成大幅降低了工具使用门槛，使性能分析融入日常开发流程。下一章将基于本章采集的基线数据，开展向量规约的多版本优化对比。后续课程将以 Visual Studio 为主要开发环境，VS Code 仅作辅助。

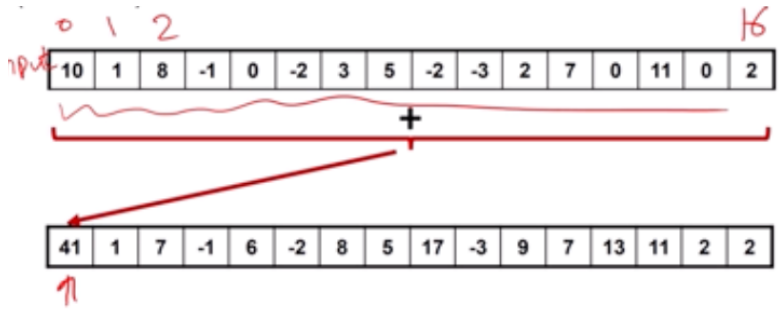


图 37.1:

37 仅使用全局内存的向量归约（基准版本）

本节开始讲解”向量归约”（Vector Reduction）应用程序。需要说明的是，本节仅涉及该主题的第一部分，后续章节还将介绍针对该应用的多种优化技术。因此，本节内容不仅对于理解向量归约至关重要，也是掌握诸多优化技术的基础。建议大家认真学习本节内容，因为它在许多其他应用中同样适用。例如，在矩阵乘法中就需要用到归约过程或归约操作。因此，理解归约的含义及其优化方法十分必要。

首先回顾向量加法。在之前的章节中，执行的操作是将两个向量的所有对应元素相加并将结果存入第三个向量。例如，将向量 A 的第 15 个元素与向量 B 的对应元素相加，结果存入向量 C 的相同位置。在该应用中，使用的线程数量等于元素数量，因为每个线程负责处理一个索引。作为练习，读者可尝试使用仅为一半数量的线程来完成向量加法：例如对于 512 个元素，不使用 512 个线程，而仅使用 256 个线程，让每个线程处理两个元素。

37.1 向量归约的定义

与向量加法不同，向量归约只涉及单个向量。假设有一个名为 input 的输入向量，其索引从 0、1、2 一直到 16。目标是将该向量中的所有元素相加，并将最终的总和存储在第一个位置（即索引 0 处）。如图所示，41 即为所有元素的总和。这就是向量归约的含义，它比向量加法复杂得多。

37.2 归约的执行流程

第一步，设定线程数量等于元素数量。假设有 16 个元素，则使用 16 个线程，索引为 0 到 15。这可以视为一个包含 16 个线程的线程块配置。关键点在于：一半的线程将处于空闲状态，不执行任何操作。

原因如下：注意区分索引和线程 ID，线程 ID 也从 0 到 15。以第一个线程（ID=0）为例，它的任务是将第一个元素与第二个元素相加，并将结果存回第一个位置。后续每对元素也是如此：线程 0 执行 input[0] = input[0] + input[1]，即处理元素 tid 和 tid+1。跳过线程 1，直接看线程 2：线程 2 执行 input[2] = input[2] + input[3]，同样处理 tid 和 tid+1。

线程 1 在此步骤中是不需要的，因此一半的线程会处于空闲状态。既然只需要一半的线程，为何不直接启动 8 个线程？因为向量归约并非一步完成，而是需要重复多次，直到所有元素累加为一个值。减少线程数会使代码变得复杂。

如图所示，”一半线程空闲”的原因在于：每个工作线程处理两个相邻元素，因此奇数 ID 的线程（如 1、3、5 等）在第一阶段无需工作。此步骤必须对新产生的值重复执行，因为在第一步之后，并未得到所有元素的总和，只是完成了相邻元素的配对相加。例如，11 和 7 相加的结果需要在下一步中继续参与运算。

第二步所需的线程数更少。第一步有 16 个元素，第二步仅剩 8 个有效元素（其余已在第一步中被合并）。将这 8 个元素两两相加仅需 4 个线程。编写代码时需要针对每一步进行相应的索引计算，因为每步使用的线程数都在递减。这也正是”归约”名称的由来：既指将所有元素归约为一个值，也指每步所需的操作数和线程数不断减半。

继续演示：将 11 与 7 相加，结果存入原位；-2 与 8 相加，结果存入原位，依此类推。第三步所需的有效元素进一步减少。第一步 16 个元素用 8 个活跃线程，第二步 8 个元素用 4 个线程，第三步 4 个元素仅需 2 个线程（线程 0 和线程 8）。线程 0 将前两个结果相加存入索引 0，线程 8 将后两个结果相加存入索引 8。最后一步仅需一个线程将索引 0 和索引 8 的值相加，最终结果存入 input[0]。这就是完整的归约流程。

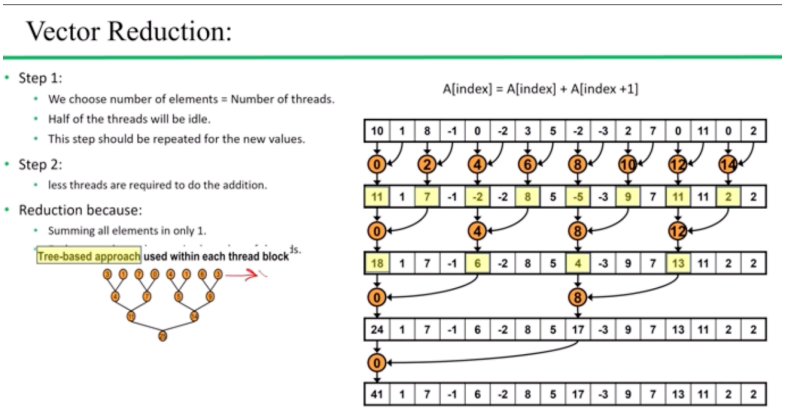

Vector Reduction:
Step 1:
We choose number of elements = Number of threads.
Half of the threads will be idle.
This step should be repeated for the new values.
Step 2:
less threads are required to do the addition.
Reduction because:
Summing all elements in only 1.
Tree-based approach used within each thread block
A[index] = A[index] + A[index +1]

图 37.2:

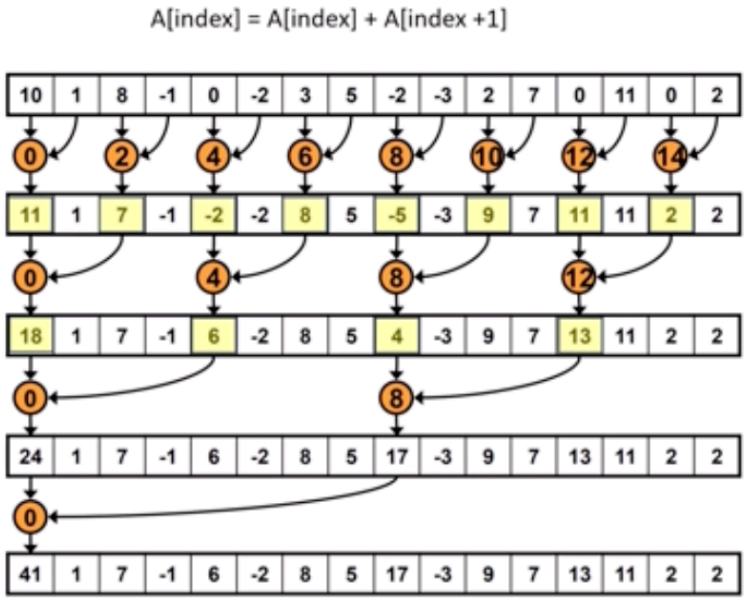

A[index] = A[index] + A[index +1]

图 37.3:

37.3 多块归约与跨块同步问题

在实际 CUDA 编程中，需要处理线程、块 ID、网格大小、块大小与向量大小之间的映射关系。这种模式被称为” 基于树的方法”（Tree-based Approach），因为其执行路径呈树状分支结构。从主向量开始，每经过一个阶段，有效元素数量减半，直至最终只剩一个包含总和的元素。

可以将整个向量划分为多个块,每个块负责处理其中一个数据块(Chunk)。例如，将向量分为两个块：块 1 处理前 8 个元素，块 2 处理后 8 个元素。每个块独立执行归约操作，最终各自产生一个结果。但这些结果并非最终总和，而是” 部分和”（Partial Sums）。

例如，块 1 的部分和为 24，块 2 的部分和为 17。要得到最终结果，必须将所有块的部分和再次相加。然而，无法简单地启动第三个块来同步并累加这些部分和，因为 CUDA 不支持跨线程块的全局同步。虽然存在线程束级、内核级甚至指令级的同步机制，但不存在块级同步。如果在所有前置块完成之前就开始执行汇总块，可能会读取到尚未更新的部分和，导致错误。NVIDIA 的文档也明确指出，由于硬件开销巨大，GPU 不提供跨块的全局同步。

37.4 多内核解决方案

解决方案是使用两个内核（Kernel）。第一个内核执行所有数据块的归约操作；利用 CUDA 的内核级同步机制，确保第一个内核完全执行完毕后，再启动第二个内核。第二个内核以第一个内核输出的部分和作为输入，计算最终总和。根据向量大小，可能需要三个或更多内核，但本例中两个即可。

两个内核的配置可能不同。例如，第一个内核可能有 8 个块，而第二个内核仅需 1 个块（因为只有 8 个部分和需要累加）。虽然本例中两个内核的块大小相同，但网格大小（Grid Size）是不同的。

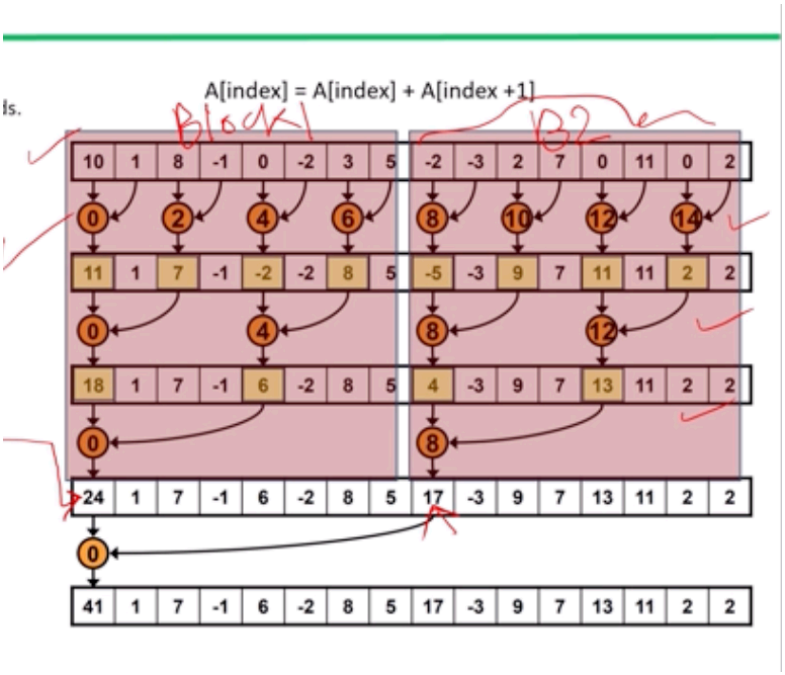

A[index] = A[index] + A[index +1]
Block1
B2
10 1 8 -1 0 -2 3 5 -2 -3 2 7 0 11 0 2
0 2 4 6 8 10 12 14
11 1 7 -1 -2 -2 8 5 -5 -3 9 7 11 11 2 2
0 4 8 12
18 1 7 -1 6 -2 8 5 4 -3 9 7 13 11 2 2
0 8
24 1 7 -1 6 -2 8 5 17 -3 9 7 13 11 2 2
0
41 1 7 -1 6 -2 8 5 17 -3 9 7 13 11 2 2

图 37.4:

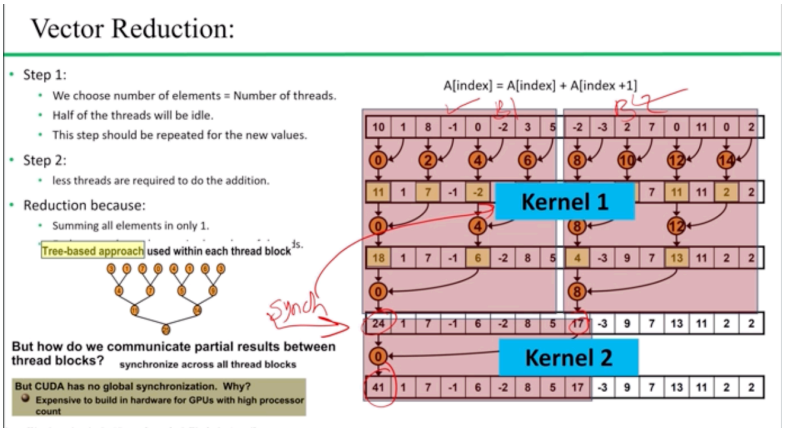

Vector Reduction:
Step 1:
We choose number of elements = Number of threads.
Half of the threads will be idle.
This step should be repeated for the new values.
Step 2:
less threads are required to do the addition.
Reduction because:
Summing all elements in only 1.
Tree-based approach used within each thread block
But how do we communicate partial results between thread blocks?
synchronize across all thread blocks
But CUDA has no global synchronization. Why?
Expensive to build in hardware for GPUs with high processor count
A[index] = A[index] + A[index +1]
Kernel 1
Kernel 2
Synch

图 37.5:

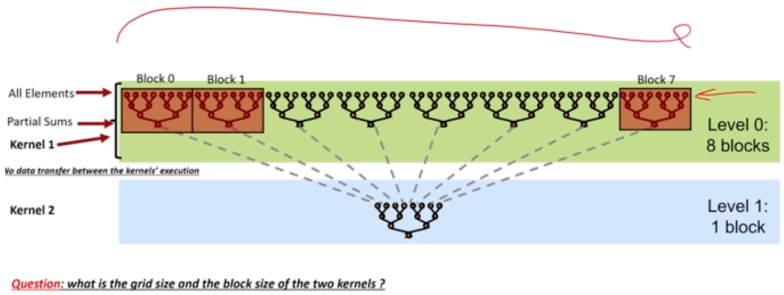


图 37.6:

37.5 内核代码实现

下面进入代码讲解。右侧展示了归约的所有步骤，左侧是一段简化的初始代码（并非最终正确版本），将逐步完善。首先，`__global__` 关键字定义了一个 GPU 内核。GPU 内核不能有返回值，若需将结果传回 CPU，必须使用 memcpy API。内核命名为 `reduce_in_place`，表示在原向量上进行归约。参数包括向量指针 input 和向量大小 n，传入 n 是为了防止越界访问，避免内存违规。

核心计算公式为 input[index] += input[index + 1]。假设 16 个线程，当 tid=0 时，执行 input[0] += input[1]；当 tid=1 时，执行 input[1] += input[2]。但在下一步中，索引 1 处的值已不再需要，只有索引 0、2、4 等偶数索引的值会被使用。因此 tid=1 的计算虽然是合法的，却是冗余的，浪费了资源。需要添加过滤器，仅允许偶数 ID 的线程（0, 2, 4, 6...）执行该指令。

```
1  __global__ void reduce_in_place(float* input, int n) {
2      int index = blockIdx.x * blockDim.x + threadIdx.x;
3
4      input[index] += input[index + 1];
5  }
```

过滤偶数 ID 的方法是使用取模运算：

```
1  if (tid % 2 == 0)。当 tid=0 时，0%2==0 成立，执行指令；当 tid=1
       时，1%2==1 不等于 0，跳过指令。
```

此外，还需要边界检查。当 tid=15 时，index+1 为 16，超出了大小为 16 的向量范围（最大索引为 15），会导致内存越界。因此必须添加条件 if (index + 1 < n)。综合两个过滤器，代码为：

```
1  __global__ void reduce_in_place(float* input, int n) {
2      int tid = threadIdx.x;
3      int index = blockIdx.x * blockDim.x + threadIdx.x;
4
5      if (tid % (2 * 1) == 0 && index + 1 < n) {
6          input[index] += input[index + 1];
7      }
8  }
```

然而，这段代码仅能完成第一步。为了完成整个归约过程，需要使用循环。观察发现，每一步的”步长”（Stride）在不断变化：第一步步长为 1（处理相邻元素），第二步步长为 2（间隔一个元素），第三步步长为 4，第四步步长为 8。通式为 stride = stride * 2，初始值为 1。循环终止条件为 stride < blockSize。同时，过滤器中的取模因子也应随步长变化，改为：

```
1  tid % (2 * stride) == 0。
```

最终的内核代码如下：

```
1  __global__ void reduce_in_place(float* input, int n) {
2      int tid = threadIdx.x;
3      int index = blockIdx.x * blockDim.x + threadIdx.x;
```

```
4
5       for (int stride = 1; stride < blockDim.x; stride *= 2) {
6           __syncthreads(); // Ensure all threads have completed
                the previous step
7
8           if (tid % (2 * stride) == 0 && index + stride < n) {
9               input[index] += input[index + stride];
10          }
11      }
12
13      // Store the block's reduced result in the first position
            of this block
14      if (tid == 0) {
15          input[blockIdx.x] = input[blockIdx.x * blockDim.x]; //
                Write the partial sum
16      }
17  }
```

37.6 部分和的收集

当有多个块时，每个块的归约结果位于该块数据的起始位置（如块 1 的结果在索引 0，块 2 的结果在索引 8）。为了在第二个内核中连续处理这些部分和，需要将它们移动到向量的前端连续位置。例如，将索引 8 处的值移到索引 1 处。这通过 input[blockIdx.x] = input[blockIdx.x * blockDim.x] 实现（假设块大小一致且紧凑排列）。第二个内核只需处理数量等于块数的元素。

本节讲解的内核仅使用了全局内存，未涉及共享内存或 Shuffle 指令等优化技术。这些高级优化将在后续章节中详细介绍。

38 CUDA GPU 并行编程：向量归约代码解析与性能剖析

本节将讲解向量归约（Vector Reduction）的完整代码。首先编写一个在 CPU 端执行向量归约的函数，用于验证 GPU kernel 输出的正确性。具体方法是将 GPU kernel 的输出与 CPU 函数的输出进行比较。由于 CPU 函数逻辑简单且结果可验证，因此可以作为基准。

38.1 CPU 端归约函数

向量归约的输入是一个包含多个元素的向量，输出是一个与输入类型相同的标量值，即所有元素的总和。因此 CPU 函数的返回类型为 float。函数命名为 `cpu_reduce`，参数与 GPU kernel 一致。函数通过 for 循环遍历所有元素并累加，最终返回 sum 变量。

```
1  float cpu_reduce(float* input, int n) {
2      float sum = 0.0;
3      for (int i = 0; i < n; i++) {
4          sum += input[i];
5      }
6      return sum;
7  }
```

38.2 CUDA 应用程序的七个步骤

接下来编写 main 函数。回顾之前章节中介绍的 CUDA 应用程序七个步骤：

第一步：在主机和设备上为向量分配空间。

本例仅有一个输入向量，需分别在 CPU 和 GPU 上为其分配空间。设

备端使用 d_input，主机端使用 h_input。主机端使用 C++ 的 new 操作符分配，元素数量设为 $n = 1024 \times 1024$。

```
float* d_input;
float* h_input = new float[n];
```

设备端使用 cudaMalloc 分配显存。需注意，分配的大小是字节数而非元素个数，因此需用 size_t size = n * sizeof(float) 计算。每个 float 占 4 字节，总字节数为元素数量乘以 4。

```
size_t size = n * sizeof(float);
cudaMalloc(&d_input, size);
```

第二步：在主机端初始化输入向量。

使用 for 循环将每个元素初始化为其索引值，并通过 static_cast<float>(i) 进行类型转换。

```
for (int i = 0; i < n; i++) {
    h_input[i] = static_cast<float>(i);
}
```

第三步：将主机数据复制到设备端。

使用 cudaMemcpy API，参数依次为目标地址（设备端 d_input）、源地址（主机端 h_input）、字节数（size）和传输方向（cudaMemcpyHostToDevice）。

```
cudaMemcpy(d_input, h_input, size, cudaMemcpyHostToDevice);
```

第四步：配置 kernel 启动参数。

设定 block size 为 256。grid size 需确保 block 数量乘以 block size 不小于 n。通用公式为：

```
int blocksize = 256;
int gridsize = (n + blocksize - 1) / blocksize;
```

当 $n = 1024 \times 1024$、blockSize = 256 时，gridSize = 4096。公式 `(n + blockSize - 1) / blockSize` 可解决 n 不是 blockSize 整数倍时的向上取整问题。

第五步：调用 CPU 函数获取基准结果。

在启动 GPU kernel 之前，先调用 CPU 函数获取归约结果，以便后续与 GPU 输出进行比较。

```
float sum = cpu_reduce(h_input, n);
std::cout << sum << std::endl;
```

第六步：启动 GPU kernel。

根据前一节的分析，向量归约需要多次启动 kernel。第一次 kernel 处理全部 n 个元素，每个 block 产生一个部分和。第一次执行时 grid size 为 4096，因此产生 4096 个部分和。

```
reduce_in_place<<<gridsize, blocksize>>>(d_input, n);
cudaDeviceSynchronize();
```

`cudaDeviceSynchronize()` 用于确保 CPU 等待 GPU kernel 执行完毕后再进入下一步。

第二次 kernel 以 4096 个部分和作为输入。block size 仍为 256，grid size 为 $4096/256 = 16$，即第二个 kernel 在 16 个 block 上执行，产生 16 个部分和。

```
reduce_in_place<<<4096 / blocksize, blocksize>>>(d_input, 4096)
    ;
cudaDeviceSynchronize();
```

第三次 kernel 仅需 1 个 block，因为待处理元素仅 16 个，不足一个 block（256 线程）。

```
reduce_in_place<<<1, blocksize>>>(d_input, 16);
cudaDeviceSynchronize();
```

第七步：将结果从设备复制回主机并释放资源。

最终结果存储在 `d_input[0]`。仅需复制一个 float 元素（4 字节），而非整个向量。

```
cudaMemcpy(h_input, d_input, sizeof(float),
    cudaMemcpyDeviceToHost);
std::cout << "GPU output: " << h_input[0] << std::endl;

cudaFree(d_input);
delete[] h_input;
```

完整的 main 函数代码如下：

```
#include <iostream>
#include <cuda_runtime.h>

__global__ void reduce_in_place(float* input, int n) {
    int tid = threadIdx.x;
    int index = blockIdx.x * blockDim.x + threadIdx.x;

    // Perform in-place reduction within each block
    for (int stride = 1; stride < blockDim.x; stride *= 2) {
        __syncthreads(); // Ensure all threads have completed
            the previous iteration

```

```
12          if (tid % (2 * stride) == 0 && index + stride < n) {
13              // Perform reduction and store the sum back in place
14              input[index] += input[index + stride];
15          }
16      }
17
18      // Store the block's reduced result in the first position
            of this block's portion
19      if (tid == 0) {
20          input[blockIdx.x] = input[blockIdx.x * blockDim.x]; //
                Write the reduced sum for this block
21      }
22  }
23
24  float cpu_reduce(float* input, int size) {
25      float sum = 0.0f;
26      for (int i = 0; i < size; ++i) {
27          sum += input[i];
28      }
29      return sum;
30  }
31
32  int main() {
33      int n = 1024 * 1024; // Number of elements
34      size_t bytes = n * sizeof(float);
35
36      // Host memory allocation
37      float* h_input = new float[n];
38      float* d_input;
```

```
39
40      // Initialize input array
41      for (int i = 0; i < n; i++) {
42          h_input[i] = static_cast<float>(i + 1); // Initialize
                from 1 to n
43      }
44
45      // Device memory allocation
46      cudaMalloc(&d_input, bytes);
47
48      // Copy data from host to device
49      cudaMemcpy(d_input, h_input, bytes, cudaMemcpyHostToDevice)
            ;
50
51      // Launch the reduction kernel multiple times
52      int blockSize = 256; // Number of threads per block
53      int gridSize = (n + blockSize - 1) / blockSize; // Number
            of blocks
54      // Calculate the sum using the CPU function for
            verification
55      float total_sum = cpu_reduce(h_input, 1024 * 1024);
56      std::cout << "Total sum (CPU): " << total_sum << std::endl;
57      // Perform iterative reduction until we have one block left
58      reduce_in_place << <gridSize, blockSize >> > (d_input, n);
59      cudaDeviceSynchronize(); // Ensure kernel execution
            completes
60      reduce_in_place << <16, blockSize >> > (d_input, 4096);
61      cudaDeviceSynchronize(); // Ensure kernel execution
            completes
```

```
62
63
64      // Final reduction when gridSize is 1
65      reduce_in_place << <1, blockSize >> > (d_input, 16);
66      cudaDeviceSynchronize();
67      // Copy the final result back to the host (the sum should
            be in h_input[0])
68      cudaMemcpy(h_input, d_input, sizeof(float),
            cudaMemcpyDeviceToHost);
69
70      // Print the result
71      std::cout << "Final sum (GPU): " << h_input[0] << std::endl
            ;
72
73
74
75      // Free memory
76      cudaFree(d_input);
77      delete[] h_input;
78
79      return 0;
80  }
```

38.3 精度差异说明

运行程序后，CPU sum 和 GPU sum 的值非常接近，但小数点后可能存在微小差异。这是由于 GPU 与 CPU 使用了不同的浮点精度。若将数据类型从 float 改为 double，两者的值将完全一致，因为 double 的精度高于 float。读者可自行验证。此外，可借助 C++ 模板（templates）机制实现数

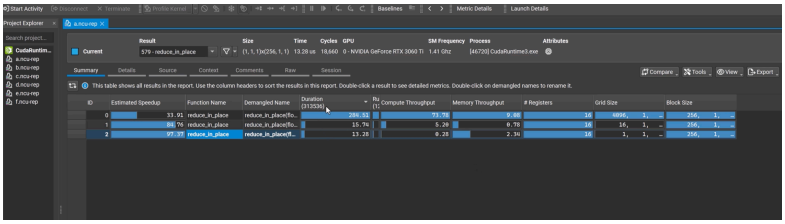


图 38.1:

据类型的灵活切换，无需在代码中逐一修改。

38.4 Nsight Compute 性能分析

接下来使用 Nsight Compute 对应用程序进行性能分析（profiling）。性能分析的细节在前面的章节中已有详细说明，建议读者回顾相关内容。

在 Nsight Compute 中创建一次新的分析运行，命名为"A" 作为基准线（Baseline），后续所有优化版本将以此为参考标准。

在 Summary 选项卡中，可以看到三个 kernel 的信息，名称均为 `reduce_in_place`，但性能指标各不相同：Duration（持续时间）、Compute Throughput（计算吞吐量）、Memory Throughput（内存吞吐量）、寄存器数量、Grid Size 和 Block Size。

观察 Grid Size：第一个 kernel 为 4096，第二个为 16，最后一个为 1，与代码中的配置完全吻合。Block Size 在所有 kernel 中保持恒定，均为 256 个线程。

重点关注 Duration：第一个 kernel 耗时 284 微秒，第二个约 15–16 微秒，最后一个约 13 微秒。第一个 kernel 耗时最长的原因在于其包含 4096 个 block。以 GPU 拥有 38 个 SM（Streaming Multiprocessor，流多处理器）为例，4096 除以 38 意味着每个 SM 需执行约 108 个 block（即多个波次）。而第二个 kernel 仅 16 个 block，超过一半的 SM 处于空闲状态，每个 SM 仅执行一个 block 即完成（一个波次）。

在 View 选项卡中展开各部分，可观察到以下关键指标：

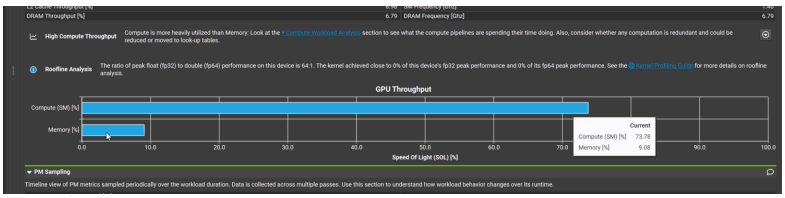


图 38.2:

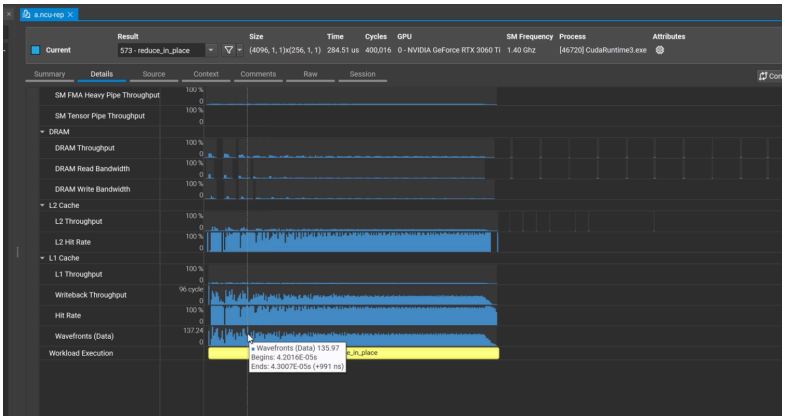


图 38.3:

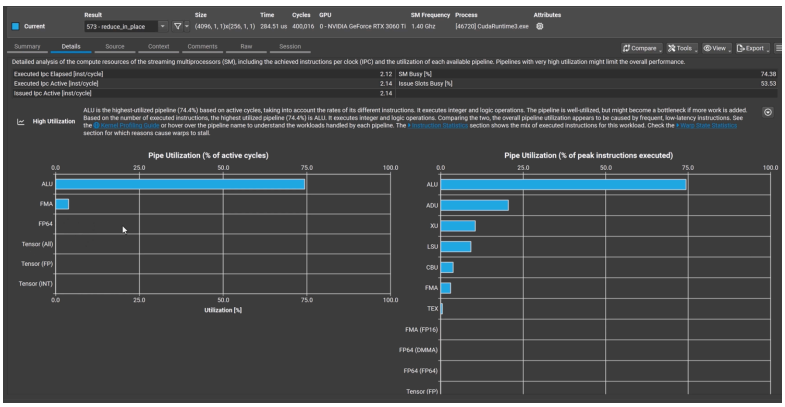


图 38.4:

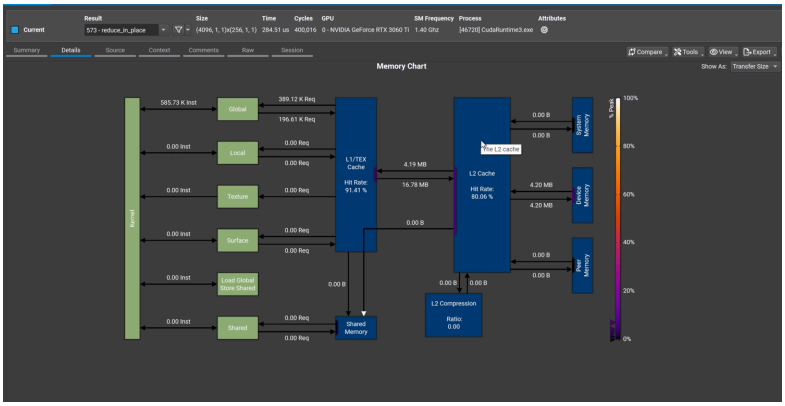


图 38.5:

- **SM 利用率**：约 74%，表明计算资源利用较为充分。
- **Memory Throughput**：仅约 9%，说明相比于内存，计算资源使用过多，存在计算受限的特征。
- **缓存命中率**：L1 命中率 91%，L2 命中率约 80%。这与向量加法（命中率接近 0%）形成鲜明对比。原因在于向量归约中每个元素在多个步骤中被重复使用，产生了数据重用（Data Reuse），从而带来缓存命中。
- **算术单元利用率**：ALU 和 FMA 有较高的利用率数值，而双精度利用率为零（因为使用的是 float 类型）。若将 float 改为 double，则会在双精度利用率一栏中看到相应数值。
- **时间轴图**：展示了执行过程中活跃 block 数量的变化，可用于识别瓶颈所在。

以上即为基准版本的性能分析结果。后续章节将在此基础上进行优化，并对比各版本的性能提升。

39 CUDA GPU 并行编程：向量归约优化

本节讲解向量归约（Vector Reduction）应用的第一项优化。

39.1 使用 while 循环自动化多步归约

在前面的讨论中提到，需要反复调用 GPU kernel，直到某一步中的 block 数量小于 block size，此时进入最后一步（仅需一个 block）。基于此逻辑，可设置 while 循环：若 grid size 大于 1，则继续迭代执行 kernel；若 grid size 等于 1，则执行最后一步即可。

当 grid size 大于 1 时，使用新的计算逻辑执行 kernel。例如，第一步的 grid size 为 4096，该值存储在变量 N 中，意味着第一步之后 N 等于 4096。下一步的新 grid size 计算公式为：

$$\text{new_grid_size} = (N + \text{block_size} - 1)/\text{block_size}$$

代入数值：$(4096 + 256 - 1)/256 = 16$，与之前的手动计算一致。while 循环检查 grid size 是否大于 1，当前为 16，因此继续执行 kernel。此时 N 从 1024×1024 变为 4096，block size 保持不变，grid size 为 16。

完成第二步后，N 变为 16。代入公式：$(16 + 256 - 1)/256 = 1$。因为仅 16 个元素不足一个完整 block（256 线程），但最少需要一个 block，所以 grid size 等于 1。while 循环再次检查：grid size 等于 1，不满足大于 1 的条件，因此跳出循环，执行最后一步函数。最后一步中 grid size 为 1，N 为 16。

整个流程经历了 $4096 \to 16 \to 1$ 三个阶段，与之前手动配置多次 kernel 的结果一致。编译并运行程序，对比 GPU 和 CPU 的输出结果，确认 while 循环工作正确。

39.2 移除取模过滤器的优化

接下来讨论内核代码中的过滤器（filter）。该过滤器的作用是筛选出每一步中实际需要工作的线程：第一步需要偶数编号线程 (0, 2, 4, ..., 14)，第二步需要线程 0, 4, 8, 12，第三步需要 0 和 8，最后一步仅需线程 0。

39.2.1 移除过滤器的可行性分析

如果移除过滤器，所有线程都会执行加法指令。以线程 1 为例，它会将索引 1 和索引 2 的值相加，并将结果存入索引 1。在第一步中，索引 1 处的值从 1 变为 9（即 $1+8$ 或其他值，取决于具体数据）。

关键问题在于：后续步骤是否会使用索引 1 处的值？答案是否定的。由于步长（stride）机制的存在，第二步的步长为 2，操作变为 `index = 0 + 2`，而非加 1。因此完全不会访问索引 1 的元素。其他所有步骤同理。

由此可见，移除过滤器不会影响最终输出，因为被”多余”线程写入的值在后续步骤中根本不会被读取。

```
1  #include <iostream>
2  #include <cuda_runtime.h>
3
4  __global__ void reduce_in_place(float* input, int n) {
5      int tid = threadIdx.x;
6      int index = blockIdx.x * blockDim.x + threadIdx.x;
7
8      // Perform in-place reduction within each block
9      for (int stride = 1; stride < blockDim.x; stride *= 2) {
10         __syncthreads(); // Ensure all threads have completed
               the previous iteration
11
12         if ( index + stride < n) {
13             // Perform reduction and store the sum back in place
```

```
14              input[index] += input[index + stride];
15          }
16      }
17
18      // Store the block's reduced result in the first position
            of this block's portion
19      if (tid == 0) {
20          input[blockIdx.x] = input[blockIdx.x * blockDim.x]; //
                Write the reduced sum for this block
21      }
22  }
23
24  float cpu_reduce(float* input, int size) {
25      float sum = 0.0f;
26      for (int i = 0; i < size; ++i) {
27          sum += input[i];
28      }
29      return sum;
30  }
31
32  int main() {
33      int n = 1024 * 1024; // Number of elements
34      size_t bytes = n * sizeof(float);
35
36      // Host memory allocation
37      float* h_input = new float[n];
38      float* d_input;
39
40      // Initialize input array
```

```
41      for (int i = 0; i < n; i++) {
42          h_input[i] = static_cast<float>(i + 1); // Initialize
                from 1 to n
43      }
44
45      // Device memory allocation
46      cudaMalloc(&d_input, bytes);
47
48      // Copy data from host to device
49      cudaMemcpy(d_input, h_input, bytes, cudaMemcpyHostToDevice)
            ;
50
51      // Launch the reduction kernel multiple times
52      int blockSize = 256; // Number of threads per block
53      int gridSize = (n + blockSize - 1) / blockSize; // Number
            of blocks
54      // Calculate the sum using the CPU function for
            verification
55      float total_sum = cpu_reduce(h_input, 1024 * 1024);
56      std::cout << "Total sum (CPU): " << total_sum << std::endl;
57      // Perform iterative reduction until we have one block left
58      while (gridSize > 1) {
59          reduce_in_place << <gridSize, blockSize >> > (d_input, n
                );
60          cudaDeviceSynchronize(); // Ensure kernel execution
                completes
61
62          // Update n to reflect the reduced number of elements
63          n = gridSize;
```

```
64          gridSize = (n + blockSize - 1) / blockSize; // Update
                gridSize for the next iteration
65      }
66
67      // Final reduction when gridSize == 1
68      reduce_in_place << <1, blockSize >> > (d_input, n);
69      cudaDeviceSynchronize(); // Ensure final kernel execution
            completes
70
71      // Copy the final result back to the host (the sum should
            be in h_input[0])
72      cudaMemcpy(h_input, d_input, sizeof(float),
            cudaMemcpyDeviceToHost);
73
74      // Print the result
75      std::cout << "Final sum (GPU): " << h_input[0] << std::endl
            ;
76
77
78
79      // Free memory
80      cudaFree(d_input);
81      delete[] h_input;
82
83      return 0;
84  }
```

39.2.2 取模运算的性能开销

移除过滤器的根本原因在于取模运算（modulus operation）在 GPU 上的开销极高。根据 NVIDIA GPU 指令延迟的研究数据：

- 加法运算：约 2–4 个时钟周期
- 乘法运算：约 2–9 个时钟周期
- 除法/取模运算：约 290 个时钟周期

取模运算的开销是加法的近 100 倍。除非在特定场景下不可替代，否则应尽量避免使用除法和取模运算，并以开销更小的操作替代。

39.2.3 性能对比验证

编译带过滤器的基准版本（报告 A），然后移除过滤器重新编译（报告 B）。在 Nsight Compute 中以 A 为基准线，对比 B 版本的性能。

结果显示：不带过滤器版本的执行时间为 176 微秒，而基准版本为 283 微秒，性能提升近 40%。第二和第三个 kernel 的性能提升接近 100%。（指标显示负值是因为衡量的是” 持续时间”，时间越短性能越好。）

运行程序验证 CPU 和 GPU 的输出一致性，确认移除过滤器后程序依然产生正确的归约结果。

39.3 汇编级分析

在 Nsight Compute 的源码视图中观察汇编代码，可以更深入地理解性能差异的原因。

不带过滤器的版本：函数的 if 语句仅包含约 6 条汇编指令，结构非常精简。尽管如此，该 if 语句仍然导致了约 60% 的流水线停顿（stalling）。其中的全局内存加载（load）操作单独造成了约 9% 的停顿。

带过滤器的版本：相同的加载操作位于源码第 131 行（不带过滤器版本在第 66 行），说明在此之前增加了大量额外指令。这些额外指令全部来

自取模运算所产生的 if 条件判断，包含五六十条指令——分支跳转、条件移动、同步操作等。相比之下，不带过滤器版本的同一个 if 语句仅有 6 条指令。

这正是执行时间从 283 微秒优化到 176 微秒的根本原因：取模运算消耗了过多的时钟周期，导致汇编代码膨胀。

39.4 调试器验证

通过 Visual Studio 的 CUDA 调试功能，可以直观地验证有无过滤器时的行为差异。

带过滤器的情况：在内存视图中观察 input 向量，单步执行完第一个 warp（前 32 个线程）后，可以看到：第 1 个元素被修改，第 2 个未变，第 3 个被修改，第 4 个未变……交替进行。这验证了过滤器的作用——仅特定位置的元素被更新。

不带过滤器的情况：移除过滤器后重新调试，执行第一个 warp 后可以观察到：所有元素均被修改——第 1 个、第 2 个、第 3 个、第 4 个……全部被更新。这印证了理论分析：移除过滤器后所有线程都参与了计算，但如前所述，”多余”的写入不会影响最终结果。

本节介绍了向量归约的第一项重要优化——通过移除取模过滤器减少指令数量，从而将第一个 kernel 的执行时间从 283 微秒降至 176 微秒。后续章节将探讨更多优化技术。

40 CUDA GPU 并行编程：竞态条件问题与调试方案

40.1 引言：调试选项引发的输出差异

在上一章中，通过移除取模过滤器提升了向量归约的性能。然而，使用完全相同的代码，在默认编译（`nvcc -o test2 02_vector_reduction_with_while`

.cu）下运行却得到了错误结果；而加上 **-G**（Generate Device Debug Information）选项重新编译后，输出反而正确。问题的根源在于编译选项的差异改变了线程的执行时序。

本章将深入剖析这一现象背后的核心概念——**竞态条件（Race Condition）**，并解释为什么本例中恰恰是“禁用优化的调试版本”得到了正确结果。

40.2 核心概念：竞态条件 (Race Condition)

40.3 什么是竞态条件

竞态条件是指程序的输出依赖于不可控的事件时序（如线程调度顺序）。当多个线程并发读写同一内存位置且缺乏同步机制时，执行顺序的微小变化就会导致截然不同的计算结果。

40.4 向量归约中的竞态场景

回顾移除过滤器后的归约代码（$stride = 1$ 时）：

```
1 // 所有线程均执行，无偶数ID过滤
2 if (tid + stride < blockDim.x) {
3     input[tid] += input[tid + stride];
4 }
```

以相邻的三个元素 $A[0], A[1], A[2]$ 为例：

- **线程 0**：读取 $A[0]$ 和 $A[1]$，计算 $A[0] + A[1]$ 并写回 $A[0]$。
- **线程 1**：读取 $A[1]$ 和 $A[2]$，计算 $A[1] + A[2]$ 并写回 $A[1]$。

数据竞争分析
线程 0 和线程 1 同时读取 $A[1]$。

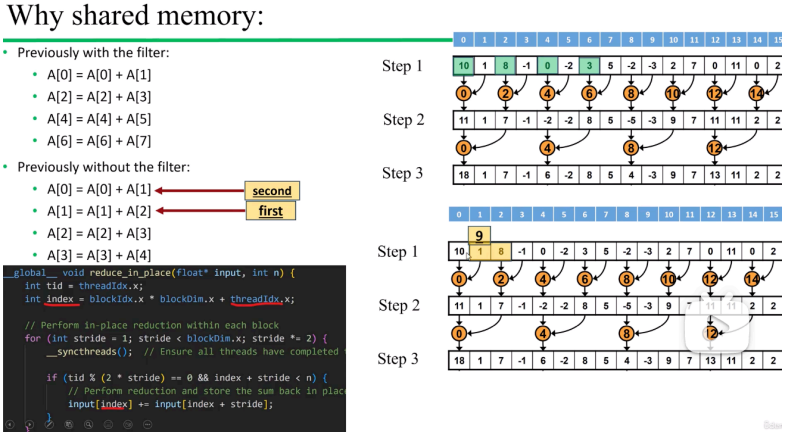


图 40.1:

1. **安全情况**：若两者在同一周期读取 $A[1]$ 的原始值（如 1），则各自计算正确。尽管 $A[1]$ 随后被线程 1 覆盖为 $1+8=9$，但线程 0 已持有旧值副本，$A[0]$ 的结果仍为正确的 $0+1=1$。

2. **危险情况**：若线程 1 先完成写入（$A[1]=9$），而线程 0 此时才读取 $A[1]$，则线程 0 读到的是新值 9。$A[0]$ 变为 $0+9=9$，而非预期的 $0+1=1$。最终归约结果将完全错误。

40.5 为什么调试版本反而正确?

在上一章的演示中，程序是在 Visual Studio 的 CUDA 调试会话（等价于 `-G`）中运行的，GPU 输出与 CPU 基准一致。这与本节开头的实测一致：加 `-G` 时结果正确，不加时结果错误。原因在于 `-G` 禁用了一切设备端优化：代码严格按照源码顺序执行，同一线程束（Warp）内的线程以锁步（Lock-step）方式同时读取 $A[1]$ 的旧值，随后才执行写入，竞争窗口被压到最小，因而“恰好”得到正确结果。

而默认优化（-O2/-O3）下，NVCC 会对指令进行重排、循环展开与寄存器重分配，打破了相邻线程原有的同步时序：某些线程的写入可能先于

其他线程的读取完成，竞争窗口被放大，错误结果稳定复现。因此，“Debug 正确、Release 错误”本身就是竞态条件存在的典型信号。

40.6 -G 选项与编译器优化的关系

40.7 -G 选项的作用

在 NVCC 编译器中，`-G` 选项用于生成设备端（GPU）调试信息。根据 NVIDIA 官方文档：

> *”Enabling this option disables all optimizations on device code.”*
> （启用此选项将禁用设备代码的所有优化。）

40.8 优化开关如何影响竞态表现

- **关闭优化 (-G/Debug)**：代码严格按照源码顺序逐条执行，同一线程束内的线程以锁步方式运行——所有线程先完成读取，再执行写入。因此在本例中，线程 0 读取的是 $A[1]$ 的旧值，恰好得到正确结果。
- **开启优化（默认/Release)**：编译器为了提升并行度，可能重排指令、展开循环、合并内存访问或将变量驻留寄存器。这种激进的调度打破了相邻线程之间的同步时序，使得“线程 1 先写、线程 0 后读”的竞争窗口被放大，从而稳定复现错误结果。

关键结论

调试模式下结果正确 $\neq$ 代码正确。

竞态条件的表现取决于线程时序，而时序又受编译优化影响。本例中恰好是 Debug 正确、Release 错误；在其他场景或硬件上，结果可能完全相反。无论哪种情况，只要不同编译选项下结果不一致，就几乎可以确定存在竞态条件。依赖“某个编译选项恰好正确”的代码是不可靠的。

40.9 跨平台编译配置对比

40.10 Linux 命令行 (NVCC)

Release 模式（默认优化，本例中稳定复现错误结果）

nvcc -O2 -arch=sm_80 reduce.cu -o reduce_release

Debug 模式（禁用优化，本例中恰好得到正确结果）

nvcc -G -g -arch=sm_80 reduce.cu -o reduce_debug

注意：`-g` 仅为主机端（CPU）生成调试信息，`-G`才是为设备端（GPU）生成调试信息并禁用优化的关键选项。

40.11 Windows Visual Studio

在 VS 中，这些选项隐藏在项目属性中，且默认值可能与实际需求不符：

1. **生成 GPU 调试信息**：属性 → CUDA C/C++ → Device → Generate Debug Information。选择 **Yes (-G)** 等同于命令行的 `-G`。

2. **代码生成架构**：属性 → CUDA C/C++ → Device → Code Generation。VS 默认可能为 `compute_52,sm_52`，务必修改为你实际 GPU 的架构（如 RTX 3060 对应 `compute_80,sm_80`），否则可能因 JIT 编译导致额外开销或行为差异。

40.12 实践任务与验证方法

40.13 四象限测试矩阵

请对向量归约代码执行以下四种组合测试，记录输出结果与执行时间：

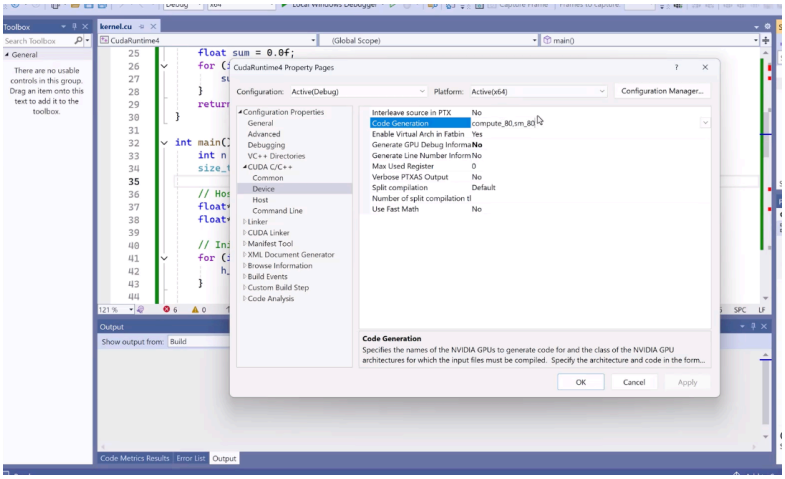


图 40.2:

配置	带过滤器（安全）	无过滤器（有竞态）
Release (-O2)	正确，速度快	错误（优化放大竞争窗口），速度快
Debug (-G)	正确，速度慢	恰好正确（时序固定），速度慢

40.14 预期观察

- **带过滤器版本**：无论 Release 还是 Debug，输出均应正确（因为从根本上消除了数据竞争）。
- **无过滤器版本**：本例实测中，Release（默认优化）下稳定输出错误值，Debug（`-G`）下恰好输出正确值。注意：竞态的表现与硬件、驱动及编译器版本相关，不同环境下可能观察到相反的结果，但"两种模式结果不一致"这一现象本身即竞态存在的证据。
- **性能差异**：Debug 模式的执行时间通常是 Release 模式的 2–5 倍，这反映了编译器优化对性能的巨大影响。

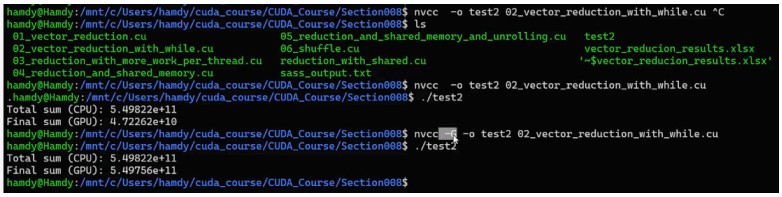


图 40.3:

40.15 总结

1. **竞态条件是并行编程的核心挑战**：移除取模过滤器虽减少了指令开销，但引入了读写冲突。上一章的性能提升是以牺牲正确性为代价的——本例中仅在 -G 调试模式下恰好得到正确结果，任何依赖特定编译选项的“正确性”都不可靠。

2. **理解编译器优化的双刃剑效应**：优化既能提升性能，也可能掩盖 bug；禁用优化虽便于调试，但会改变时序行为。

3. **正确性优先于性能**：在后续优化中，我们必须通过算法设计（如共享内存、Warp Shuffle）来 **消除**竞态，而非依赖编译器的“幸运调度”。

4. **浮点精度说明**：CPU 与 GPU 输出的微小尾数差异属于正常现象（IEEE 754 累加顺序不同），可通过改用 `double` 类型验证。但若出现数量级差异（如 10^{11} vs 10^{10}），则必为竞态条件所致。

41 CUDA GPU 并行编程：向量归约中的线程利用率优化

41.1 引言

在 CUDA 应用程序开发中，提升性能的核心策略之一是 **增加每个线程的工作量**（Work per Thread）。这一优化不仅适用于向量加法，对向量归

约（Vector Reduction）同样至关重要。在基础的向量归约实现中，往往存在严重的线程利用不足问题：在每一轮迭代中，仅有一半的线程处于活跃状态，另一半则处于空闲等待状态。这种资源浪费直接导致了 SM 的占用率（Occupancy）低下。本章将详细讲解如何通过让每个线程处理更多元素来消除空闲线程，从而显著提升硬件利用率与整体性能。

41.2 问题分析：线程利用不足

在之前的基准实现中，设定块大小（Block Size）为 256 个线程，每个块处理 256 个元素。此时，线程数量严格等于元素数量。然而，在归约操作的第一轮迭代中，只有偶数 ID 的线程执行加法操作，奇数 ID 的线程完全空闲。这意味着：

- 50% 的计算资源被浪费；
- SM 的实际占用率远低于理论峰值；
- 基准测试显示，包含分支发散（Warp Divergence）的内核执行时间为 283 微秒，即使移除条件分支优化后仍需 176 微秒，性能仍有巨大提升空间。

41.3 优化策略：增加每线程工作量

41.4 核心思想

将每个块处理的元素数量从 256 增加到 512，但保持块大小仍为 256 个线程。这意味着 **每个线程负责处理 2 个元素**。通过这种方式，所有 256 个线程在第一阶段都能满载工作，彻底消除了空闲线程。

41.5 两阶段执行流程

新的内核函数执行分为两个逻辑步骤：

1. **预归约（Pre-reduction）**：每个线程将分配给它的第二个元素（偏移量为 blockDim）加到第一个元素上。例如，线程 i 执行 $data[i] += data[i + blockDim]$。此步骤将 512 个元素归约为 256 个有效元素，且所有线程均参与计算。

2. **标准归约**：经过第一步处理后，数据规模变回 256 个元素，这与基准版本的输入完全一致。因此，后续可直接复用原有的归约逻辑，无需修改核心循环代码。

41.6 网格与索引调整

由于每个块处理的元素翻倍，启动配置需相应调整：

- **全局索引计算**：$index = (blockIdx.x \times blockDim.x + threadIdx.x) \times 2$。乘以 2 是因为每个线程现在跨越两个元素位置。

- **网格大小（Grid Size）**：变为原来的一半。因为总元素数不变，但每个块的处理能力翻倍，所需的块数量自然减半。这减少了 SM 调度的波次（Waves），有助于减少尾效应并提升整体吞吐。

41.7 性能分析与实测结果

41.8 执行时间对比

通过 Nsight Compute 等工具分析，优化效果显著：

- 基准版本（含分支）：283 μs

- 移除分支版本：176 μs

- **增加线程工作量版本：100 μs**

相比基准版本，性能提升接近 3 倍；相比仅移除分支的版本，仍有约 43% 的性能提升。

41.9 占用率与内存行为

- **占用率**：提升至接近 90%，证明 SM 资源得到了充分利用。
- **缓存流量**：L1 到 L2 缓存的数据写入量约为 19MB。虽然计算效率提升，但全局内存访问模式仍有优化空间，这为后续引入共享内存（Shared Memory）优化埋下伏笔。

41.10 进阶优化探讨

41.11 循环遍历顺序的影响

实验表明，将归约循环从”大步长递减”（Stride = BlockDim/2 →1）改为”小步长递增”（Stride = 1 → BlockDim/2）在本案例中反而导致性能下降（从 100μs 劣化至 150 μs）。尽管某些文献建议小步长优先有利于缓存局部性，但在未使用共享内存且数据跨度较大的场景下，大步长递减可能更符合当前的内存合并访问模式。**注意：最优策略高度依赖于具体硬件架构与数据规模，务必实测验证**。

41.12 位运算替代除法

在归约循环中，频繁使用除法和取模运算代价高昂。利用位运算可进一步微调性能：

- 右移一位（`>> 1`）等价于除以 2；
- 左移一位（`<< 1`）等价于乘以 2。

实测显示，将步长更新逻辑替换为位运算后，获得了约 2% 的额外性能提升。虽然幅度不大，但在大规模并行计算中，消除百万级线程的整数除法开销仍是良好的编程实践。

图 41.1:

41.13 总结与展望

本章证明了“增加每线程工作量”是解决向量归约中线程闲置问题的有效手段。通过将每块处理元素翻倍，实现了全线程活跃，大幅降低了执行时间并提升了 SM 占用率。然而，当前实现仍受限于全局内存带宽。下一阶段，将结合 **共享内存**技术，减少对 L2 缓存及全局内存的访问压力，进一步挖掘 GPU 性能潜力。建议学习者在掌握本章内容后，尝试将共享内存与线程工作量优化相结合，以获得最佳实践效果。

附录：关键代码片段

Listing 2: 优化后的向量归约内核（增加每线程工作量）

```
#include <iostream>
#include <cuda_runtime.h>

__global__ void reduce_in_place(float* input, int n) {
    int tid = threadIdx.x;
```

```
6       int index = 2 * blockIdx.x * blockDim.x + threadIdx.x;
7
8       // Each thread works on two elements initially, if within
            bounds
9       if (index + blockDim.x < n) {
10          input[index] += input[index + blockDim.x];
11      }
12
13      __syncthreads(); // Ensure all threads have completed the
            addition
14
15      // Perform in-place reduction within each block
16      for (int stride = blockDim.x >> 1; stride > 0; stride >>=
            1) {
17          if (index + stride < n) {
18              input[index] += input[index + stride];
19          }
20          __syncthreads(); // Ensure all threads have completed
                the current iteration
21      }
22
23      // Store the block's reduced result in the first position
            of this block's portion
24      if (tid == 0) {
25          input[blockIdx.x] = input[index]; // Write the reduced
                sum for this block
26      }
27  }
28
```

```
29 float cpu_reduce(float* input, int size) {
30     float sum = 0.0f;
31     for (int i = 0; i < size; ++i) {
32         sum += input[i];
33     }
34     return sum;
35 }
36
37 int main() {
38     int n = 1024 * 1024; // Number of elements
39     size_t bytes = n * sizeof(float);
40
41     // Host memory allocation
42     float* h_input = new float[n];
43     float* d_input;
44
45     // Initialize input array
46     for (int i = 0; i < n; i++) {
47         h_input[i] = static_cast<float>(i + 1); // Initialize
               from 1 to n
48     }
49
50     // Device memory allocation
51     cudaMalloc(&d_input, bytes);
52
53     // Copy data from host to device
54     cudaMemcpy(d_input, h_input, bytes, cudaMemcpyHostToDevice)
           ;
55
```

```
56      // Launch the reduction kernel multiple times
57      int blockSize = 256; // Number of threads per block
58      int gridSize = (n + 2 * blockSize - 1) / (2 * blockSize);
           // Number of blocks
59      // Calculate the sum using the CPU function for
           verification
60      float total_sum = cpu_reduce(h_input, n);
61      std::cout << "Total sum (CPU): " << total_sum << std::endl;
62
63      // Perform iterative reduction until we have one block left
64      while (gridSize > 1) {
65          reduce_in_place << <gridSize, blockSize >> > (d_input, n
               );
66          cudaDeviceSynchronize(); // Ensure kernel execution
               completes
67
68          // Update n to reflect the reduced number of elements
69          n = gridSize;
70          gridSize = (n + 2 * blockSize - 1) / (2 * blockSize); //
                Update gridSize for the next iteration
71      }
72
73      // Final reduction when gridSize == 1
74      reduce_in_place << <1, blockSize >> > (d_input, n);
75      cudaDeviceSynchronize(); // Ensure final kernel execution
           completes
76
77      // Copy the final result back to the host (the sum should
           be in h_input[0])
```

```
78      cudaMemcpy(h_input, d_input, sizeof(float),
            cudaMemcpyDeviceToHost);
79
80      // Print the result
81      std::cout << "Final sum (GPU): " << h_input[0] << std::endl
            ;
82
83      // Free memory
84      cudaFree(d_input);
85      delete[] h_input;
86
87      return 0;
88  }
```

42 NVIDIA GPU 内存层级回顾

在深入优化之前，必须清晰理解 GPU 内部的内存层级及其特性。从慢到快依次为：

1. **全局内存（Global Memory）**：容量最大但延迟最高（约 300 周期），是所有 SM 最终访问的存储层。
2. **L2 缓存**：所有 SM 共享，延迟约 180–200 周期。
3. **L1 缓存 / 共享内存**：位于 SM 内部，二者共用同一物理片上存储单元。L1 由硬件自动管理，而共享内存是**可编程**的显式存储器。访问 L1 需 Tag 查找（约 30 周期），而共享内存无此开销（约 23–25 周期），因此理论延迟更低。
4. **寄存器（Registers）**：速度最快，由编译器分配给每个线程私有使用。

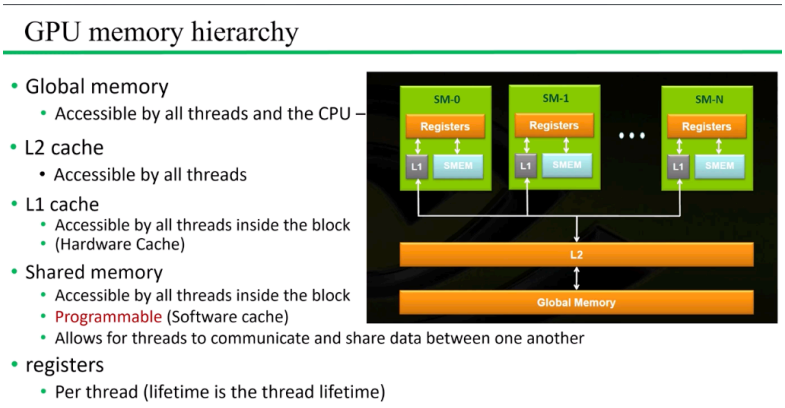


图 42.1:

重要提示：在之前的全局内存归约实现中，数据加载时已隐式经过 L1/L2 缓存。若仅简单将数据搬入共享内存而不改变访存模式，性能提升有限（通常仅 10%–20%）。共享内存的真正价值在于**配合算法重构**（如循环展开、消除 Bank 冲突）才能释放其低延迟优势。

42.1 共享内存优化实现

42.2 核心改动

在前一版“每线程处理 2 元素”的基础上，引入共享内存的关键修改如下：

- 声明块级共享数组：`__shared__ float sdata[BLOCK_SIZE];`，每个 Block 独立分配；
- 预归约阶段：将全局内存中后半段数据加到前半段，结果直接写入`sdata`而非全局内存；
- 同步屏障：`__syncthreads()`确保所有线程完成加载后再进入归约循环；

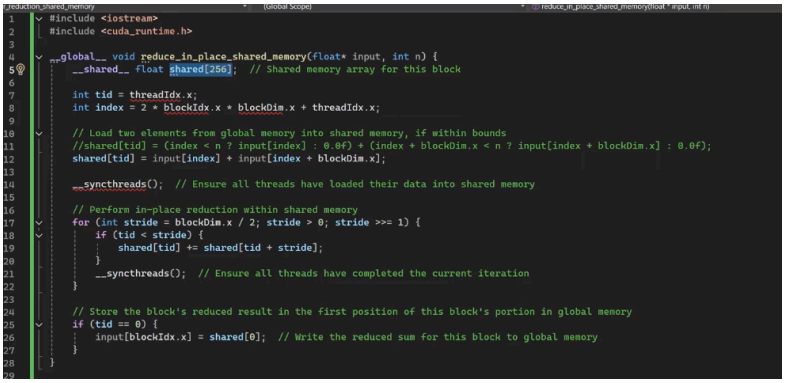


图 42.2:

- 归约循环：全程在sdata上进行读写，避免重复访问全局内存；
- 边界安全：使用三元运算符防止越界读取垃圾值。

42.3 边界检查的重要性

当向量长度不是块大小的整数倍时，必须对超出范围的索引返回 0 而非未定义值：

Listing 3: 安全的共享内存加载

```
// 安全加载：越界时补0，避免垃圾值污染归约结果
float val = (index < n) ? g_idata[index] : 0.0f;
float val2 = (index + blockDim.x < n) ? g_idata[index +
    blockDim.x] : 0.0f;
sdata[tid] = val + val2;
```

42.4 性能实测与瓶颈分析

单纯引入共享内存后性能反而略降（103 vs 100 μs），原因在于：

表 1: 不同优化阶段的执行时间对比

优化策略	执行时间 (μs)	相对基准提升
基准（含分支发散）	283	–
移除分支 + 每线程 2 元素	100	2.83×
+ 共享内存	103	2.75×
+ 循环展开	90	3.14×
+ 块大小调优（128 线程）	82	3.45×

- 前版已受益于 L1 缓存的隐式重用；
- 共享内存引入了额外的`__syncthreads()`同步开销；
- 未改变访存步长模式，未能消除潜在的 Bank Conflict 风险。

```
#include <iostream>
#include <cuda_runtime.h>

__global__ void reduce_in_place_shared_memory(float* input, int
    n) {
    __shared__ float shared[256]; // Shared memory array for
       this block

    int tid = threadIdx.x;
    int index = 2 * blockIdx.x * blockDim.x + threadIdx.x;

    // Load two elements from global memory into shared memory,
        if within bounds - shared[tid] = input[index] + input[
       index + blockDim.x];
```

```
11      shared[tid] = (index < n ? input[index] : 0.0f) + (index +
            blockDim.x < n ? input[index + blockDim.x] : 0.0f);
12      __syncthreads(); // Ensure all threads have loaded their
            data into shared memory
13
14      // Perform in-place reduction within shared memory
15      for (int stride = blockDim.x / 2; stride > 0; stride >>= 1)
             {
16          if (tid < stride) {
17              shared[tid] += shared[tid + stride];
18          }
19          __syncthreads(); // Ensure all threads have completed
                the current iteration
20      }
21
22      // Store the block's reduced result in the first position
            of this block's portion in global memory
23      if (tid == 0) {
24          input[blockIdx.x] = shared[0]; // Write the reduced sum
                for this block to global memory
25      }
26  }
27
28  float cpu_reduce(float* input, int size) {
29      float sum = 0.0f;
30      for (int i = 0; i < size; ++i) {
31          sum += input[i];
32      }
33      return sum;
```

```
}

int main() {
    int n = 1024 * 1024; // Number of elements
    size_t bytes = n * sizeof(float);

    // Host memory allocation
    float* h_input = new float[n];
    float* d_input;

    // Initialize input array
    for (int i = 0; i < n; i++) {
        h_input[i] = static_cast<float>(i + 1); // Initialize
            from 1 to n
    }

    // Device memory allocation
    cudaMalloc(&d_input, bytes);

    // Copy data from host to device
    cudaMemcpy(d_input, h_input, bytes, cudaMemcpyHostToDevice)
        ;

    // Launch the reduction kernel multiple times
    int blockSize = 256; // Number of threads per block
    int gridSize = (n + 2 * blockSize - 1) / (2 * blockSize);
        // Number of blocks
    size_t sharedMemSize = blockSize * sizeof(float); // Shared
         memory size per block
```

```
59
60      // Calculate the sum using the CPU function for
            verification
61      float total_sum = cpu_reduce(h_input, n);
62      std::cout << "Total sum (CPU): " << total_sum << std::endl;
63
64      // Perform iterative reduction until we have one block left
65      while (gridSize > 1) {
66          reduce_in_place_shared_memory << <gridSize, blockSize,
                sharedMemSize >> > (d_input, n);
67          cudaDeviceSynchronize(); // Ensure kernel execution
                completes
68
69          // Update n to reflect the reduced number of elements
70          n = gridSize;
71          gridSize = (n + 2 * blockSize - 1) / (2 * blockSize); //
                Update gridSize for the next iteration
72      }
73
74      // Final reduction when gridSize == 1
75      reduce_in_place_shared_memory << <1, blockSize,
            sharedMemSize >> > (d_input, n);
76      cudaDeviceSnchronize(); // Ensure final kernel execution
            completes
77
78      // Copy the final result back to the host (the sum should
            be in h_input[0])
79      cudaMemcpy(h_input, d_input, sizeof(float),
            cudaMemcpyDeviceToHost);
```

```
80
81      // Print the final result
82      std::cout << "Final sum (GPU): " << h_input[0] << std::endl
            ;
83
84      // Free memory
85      cudaFree(d_input);
86      delete[] h_input;
87
88      return 0;
89  }
```

42.5 循环展开（Loop Unrolling）优化

42.6 优化动机

标准归约循环中，每次迭代都包含一次`__syncthreads()`。当步长小于 Warp 大小（32）时，所有活跃线程属于同一 Warp，硬件保证 SIMT 锁步执行，**无需显式同步**。利用此特性可将最后 5 次迭代（stride=16,8,4,2,1）展开为无同步的顺序代码。

42.7 设备函数封装

使用`__device__`函数封装尾部展开逻辑，由主核函数在 stride<32 时调用：

Listing 4: Warp 级循环展开设备函数

```
1   #include <iostream>
2   #include <cuda_runtime.h>
3
```

```
__device__ void warpReduce(volatile float* sdata, int tid) {
    sdata[tid] += sdata[tid + 32];
    sdata[tid] += sdata[tid + 16];
    sdata[tid] += sdata[tid + 8];
    sdata[tid] += sdata[tid + 4];
    sdata[tid] += sdata[tid + 2];
    sdata[tid] += sdata[tid + 1];
}



__global__ void reduce_in_place_shared_memory(float* input, int
    n) {
    __shared__ float shared[256]; // Shared memory array for
       this block

    int tid = threadIdx.x;
    int index = 2 * blockIdx.x * blockDim.x + threadIdx.x;

    // Load two elements from global memory into shared memory,
        if within bounds
    shared[tid] = (index < n ? input[index] : 0.0f) + (index +
       blockDim.x < n ? input[index + blockDim.x] : 0.0f);
    __syncthreads(); // Ensure all threads have loaded their
       data into shared memory

    // Perform in-place reduction within shared memory
    for (int stride = blockDim.x / 2; stride > 32; stride >>=
       1) {
```

```
27          if (tid < stride) {
28              shared[tid] += shared[tid + stride];
29          }
30          __syncthreads(); // Ensure all threads have completed
                the current iteration
31      }
32      if (tid < 32) warpReduce(shared, tid);
33      // Store the block's reduced result in the first position
            of this block's portion in global memory
34      if (tid == 0) {
35          input[blockIdx.x] = shared[0]; // Write the reduced sum
                for this block to global memory
36      }
37  }
38
39  float cpu_reduce(float* input, int size) {
40      float sum = 0.0f;
41      for (int i = 0; i < size; ++i) {
42          sum += input[i];
43      }
44      return sum;
45  }
46
47  int main() {
48      int n = 1024 * 1024; // Number of elements
49      size_t bytes = n * sizeof(float);
50
51      // Host memory allocation
52      float* h_input = new float[n];
```

```
53      float* d_input;
54
55      // Initialize input array
56      for (int i = 0; i < n; i++) {
57          h_input[i] = static_cast<float>(i + 1); // Initialize
                from 1 to n
58      }
59
60      // Device memory allocation
61      cudaMalloc(&d_input, bytes);
62
63      // Copy data from host to device
64      cudaMemcpy(d_input, h_input, bytes, cudaMemcpyHostToDevice)
            ;
65
66      // Launch the reduction kernel multiple times
67      int blockSize = 256; // Number of threads per block
68      int gridSize = (n + 2 * blockSize - 1) / (2 * blockSize);
            // Number of blocks
69      size_t sharedMemSize = blockSize * sizeof(float); // Shared
             memory size per block
70
71      // Calculate the sum using the CPU function for
            verification
72      float total_sum = cpu_reduce(h_input, n);
73      std::cout << "Total sum (CPU): " << total_sum << std::endl;
74
75      // Perform iterative reduction until we have one block left
76      while (gridSize > 1) {
```

```
77          reduce_in_place_shared_memory << <gridSize, blockSize,
                sharedMemSize >> > (d_input, n);
78          cudaDeviceSynchronize(); // Ensure kernel execution
                completes
79
80          // Update n to reflect the reduced number of elements
81          n = gridSize;
82          gridSize = (n + 2 * blockSize - 1) / (2 * blockSize); //
                Update gridSize for the next iteration
83      }
84
85      // Final reduction when gridSize == 1
86      reduce_in_place_shared_memory << <1, blockSize,
            sharedMemSize >> > (d_input, n);
87      cudaDeviceSynchronize(); // Ensure final kernel execution
            completes
88
89      // Copy the final result back to the host (the sum should
            be in h_input[0])
90      cudaMemcpy(h_input, d_input, sizeof(float),
            cudaMemcpyDeviceToHost);
91
92      // Print the final result
93      std::cout << "Final sum (GPU): " << h_input[0] << std::endl
            ;
94
95      // Free memory
96      cudaFree(d_input);
97      delete[] h_input;
```

```
98
99      return 0;
100 }
```

关键技术点：

- 使用`volatile`关键字防止编译器将共享内存读写优化到寄存器，确保跨线程可见性；
- 展开后消除 5 次同步屏障，显著降低流水线停顿；
- 实测执行时间从 103 μs 降至 90 μs，提升约 12.6%。

42.8 块大小调优与综合优化

通过实验发现，将块大小从 256 调整为 128 后，结合共享内存与循环展开，执行时间进一步降至 82 μs。原因包括：

- 更小的块提高 SM 并发块数，改善占用率；
- 减少每块共享内存用量，允许更多块驻留；
- 更好的指令缓存局部性。

重要提醒：最优块大小高度依赖 GPU 架构、问题规模及访存模式，必须通过 Profiling 驱动的实验确定，不可盲目套用经验值。

42.9 总结与进阶方向

本章验证了以下核心结论：

1. 共享内存本身不是银弹，必须与算法重构结合才能体现价值；
2. 循环展开通过消除 Warp 级冗余同步获得稳定收益；

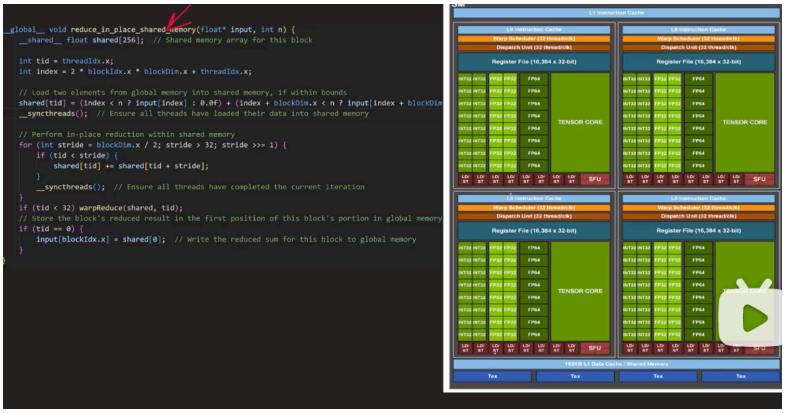


图 42.3:

3. 块大小是影响占用率与资源竞争的关键超参数，需实测调优；

4. Bank Conflict 检测（Nsight Compute Memory Workload Analysis）是共享内存优化的必要诊断步骤。

后续可探索的方向包括：模板元编程消除运行时分支、多级归约减少原子操作、以及使用 CUB/Thrust 库的生产级实现。建议读者结合 NVIDIA 官方《CUDA Best Practices Guide》第 5 章及本课程配套代码仓库进行动手实践。

43 CUDA GPU 并行编程：洗牌操作（Shuffle）优化技巧

43.1 引言与回顾

在本章中，将讲解另一种优化向量规约（Vector Reduction）应用的关键方法——**洗牌操作（Shuffle Operations）**。尽管此前已覆盖主要优化技术，但鉴于该主题对性能提升的重要性，特此补充本章内容。

在之前的讨论中，我们利用**共享内存（Shared Memory）**实现了约

10%–15% 的性能提升。共享内存之所以关键，是因为其访问速度远快于全局内存，且支持块内（Block-level）不同线程之间的通信。在规约操作中，由于同一元素可能被多次读取和计算，共享内存成为了必不可少的中间存储介质。结合循环展开（Loop Unrolling）技术，我们取得了一定的优化效果。然而，为了追求极致性能，我们需要进一步探索更底层的硬件特性。

43.2 GPU 内存层级与寄存器的局限性

43.3 内存层级回顾

为了理解洗牌操作的必要性，首先回顾 GPU 的内存层级结构（按访问延迟从低到高排列）：

1. **寄存器文件（Register File）**：位于 SM 内部，最靠近算术逻辑单元（ALU）。访问仅需几个时钟周期，是速度最快的存储单元。
2. **一级缓存（L1 Cache）/ 共享内存（Shared Memory）**：位于 SM 内部，访问延迟约为几十个周期。共享内存是“每块（Per-Block）”资源，块内所有线程均可读写。
3. **二级缓存（L2 Cache）**：在所有 SM 之间共享。
4. **全局内存（Global Memory）**：访问延迟最高，通常在数百个周期以上。

43.4 寄存器通信的瓶颈

虽然寄存器是最快的存储单元，但在向量规约中直接使用寄存器面临一个核心问题：**缺乏互通性**。

- **共享内存**是“每块”单元，块内 256 个线程可以共享同一块内存区域。
- **寄存器**是“每线程（Per-Thread）”单元。每个线程拥有私有的寄存器空间，线程 A 无法直接读取线程 B 寄存器中的值。

在传统规约中，当线程 0 需要与线程 1 的数据进行加法运算时，必须先将数据写回共享内存或全局内存，再由线程 0 读取。这种频繁的“寄存器 → 共享内存 → 寄存器”的数据搬运成为了性能瓶颈。**洗牌操作**正是为了解决这一问题而生，它允许线程束（Warp）内的线程直接交换寄存器数据，无需经过共享内存。

43.5 洗牌操作（Shuffle Operations）原理

43.6 基本概念

洗牌操作是一组用于在**线程束（Warp）**内直接交换变量的指令。根据 CUDA C 编程指南，主要有以下四种同步洗牌函数：

- `__shfl_sync(mask, var, srcLane)`：从指定通道（srcLane）直接复制数据。
- `__shfl_up_sync(mask, var, delta)`：从 ID 较低的通道复制数据（向上取值）。
- `__shfl_down_sync(mask, var, delta)`：从 ID 较高的通道复制数据（向下取值）。这是向量规约中最常用的操作。
- `__shfl_xor_sync(mask, var, laneMask)`：通过异或掩码交换数据（常用于蝶形规约）。

43.7 关键术语解析

通道（Lane）：指线程束内的线程索引（0–31），而非整个线程块的索引。无论线程块包含多少线程，洗牌操作始终限定在 32 个线程组成的 Warp 内进行。

增量/跨度（Delta/Offset）：即数据交换的距离。例如在 `__shfl_down_sync` 中，若 `delta=1`，则 Lane i 获取 Lane $i+1$ 的值；若 `delta=2`，则获

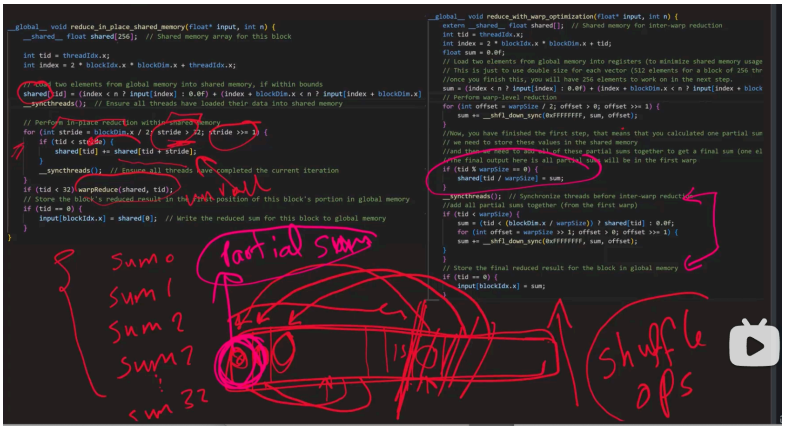


图 43.1:

取 Lane $i+2$ 的值。这对应了规约算法中步长（Stride）的变化。

掩码（Mask）：指定参与洗牌操作的线程集合。通常设为 `0xffffffff`（即所有 32 个线程均参与）。

43.8 规约中的工作流

在使用洗牌操作优化规约时，算法粒度从“线程块”细化为“线程束”：

1. **线程束内规约**：每个 Warp 独立对其负责的 32 个元素进行规约。通过循环改变 `delta`（16, 8, 4, 2, 1），使用 `__shfl_down_sync` 将高 ID 线程的值累加到低 ID 线程。最终，每个 Warp 的第 0 号线程（Lane 0）持有该 Warp 的部分和（Partial Sum）。

2. **跨 Warp 汇总**：由于一个线程块可能包含多个 Warp（如 256 线程 =8 个 Warp），我们需要将这 8 个部分和写入共享内存。

3. **最终规约**：通常由第一个 Warp（或特定线程）从共享内存读取所有部分和，再次利用洗牌操作完成最终的块级规约。

43.9 代码实现详解

43.10 核函数结构

以下为使用洗牌操作优化的规约核函数关键逻辑:

Listing 5: Shuffle-based Reduction Kernel

```
#include <iostream>
#include <cuda_runtime.h>

__inline__ __device__ float warpReduceSum(float val) {
    for (int offset = warpSize >> 1; offset > 0; offset >>= 1)
       {
        val += __shfl_down_sync(0xFFFFFFFF, val, offset);
    }
    return val;
}

__global__ void reduce_with_warp_optimization(float* input, int
    n) {
    extern __shared__ float shared[]; // Shared memory for
       inter-warp reduction
    int tid = threadIdx.x;
    int index = 2 * blockIdx.x * blockDim.x + tid;
    float sum = 0.0f;
    // Load two elements from global memory into registers (to
       minimize shared memory usage)
    // This is just to use double size for each vector (512
       elements for a block of 256 threads) (more work per
       thread optimization)
```

```
18      //once you finish this, you will have 256 elements to work
            on in the next step.
19      sum = (index < n ? input[index] : 0.0f) + (index + blockDim
            .x < n ? input[index + blockDim.x] : 0.0f);
20      // Perform warp-level reduction
21      for (int offset = warpSize >>1; offset > 0; offset >>= 1) {
22          sum += __shfl_down_sync(0xFFFFFFFF, sum, offset);
23      }
24      //Now, you have finished the first step, that means that
            you calculated one partial sum per warp.
25      // we need to store these values in the shared memory
26      //and then we need to add all of these partial sums
            together to get a final sum (one element per block From
            all warps)
27      //The final output here is all partial sums will be in the
            first warp
28      if (tid % warpSize == 0) {
29          shared[tid / warpSize] = sum;
30      }
31      __syncthreads(); // Synchronize threads before inter-warp
            reduction
32      //add all partial sums together (from the first warp)
33      if (tid < warpSize) {
34          sum = (tid < (blockDim.x / warpSize)) ? shared[tid] :
                0.0f;
35          for (int offset = warpSize >> 1; offset > 0; offset >>=
                1) {
36          sum += __shfl_down_sync(0xFFFFFFFF, sum, offset);
37      }
```

```
38      }
39      // Store the final reduced result for the block in global
            memory
40      if (tid == 0) {
41          input[blockIdx.x] = sum;
42      }
43  }
44
45  float cpu_reduce(float* input, int size) {
46      float sum = 0.0f;
47      for (int i = 0; i < size; ++i) {
48          sum += input[i];
49      }
50      return sum;
51  }
52
53  int main() {
54      int n = 1024 * 1024; // Number of elements
55      size_t bytes = n * sizeof(float);
56
57      // Host memory allocation
58      float* h_input = new float[n];
59      float* d_input;
60
61      // Initialize input array
62      for (int i = 0; i < n; i++) {
63          h_input[i] = static_cast<float>(i + 1); // Initialize
                from 1 to n
64      }
```

```
65
66      // Device memory allocation
67      cudaMalloc(&d_input, bytes);
68
69      // Copy data from host to device
70      cudaMemcpy(d_input, h_input, bytes, cudaMemcpyHostToDevice)
            ;
71
72      // Launch the reduction kernel multiple times
73      int blockSize = 256; // Number of threads per block
74      int gridSize = (n + 2 * blockSize - 1) / (2 * blockSize);
           // Number of blocks
75      size_t sharedMemSize = (blockSize / 32) * sizeof(float); //
            Using 32 instead of warpSize
76
77      // Calculate the sum using the CPU function for
           verification
78      float total_sum = cpu_reduce(h_input, n);
79      std::cout << "Total sum (CPU): " << total_sum << std::endl;
80
81      // Perform iterative reduction until we have one block left
82      while (gridSize > 1) {
83          reduce_with_warp_optimization<<<gridSize, blockSize,
               sharedMemSize>>>(d_input, n);
84          cudaDeviceSynchronize(); // Ensure kernel execution
               completes
85
86          // Update n to reflect the reduced number of elements
87          n = gridSize;
```

```
88          gridSize = (n + 2 * blockSize - 1) / (2 * blockSize); //
                Update gridSize for the next iteration
89      }
90
91      // Final reduction when gridSize == 1
92      reduce_with_warp_optimization<<<1, blockSize, sharedMemSize
            >>>(d_input, n);
93      cudaDeviceSynchronize(); // Ensure final kernel execution
            completes
94
95      // Copy the final result back to the host (the sum should
            be in h_input[0])
96      cudaMemcpy(h_input, d_input, sizeof(float),
            cudaMemcpyDeviceToHost);
97
98      // Print the final result
99      std::cout << "Final sum (GPU): " << h_input[0] << std::endl
            ;
100
101     // Free memory
102     cudaFree(d_input);
103     delete[] h_input;
104
105     return 0;
106 }
```

43.11 关键点说明

- **寄存器变量 `sum`**：在核函数内部定义的局部变量默认存储在寄存器文件中。对于 256 线程的块，编译器会分配 256 个独立的寄存器实例。
- **为何仍需共享内存?**：虽然 Warp 内通信消除了共享内存依赖，但跨 *Warp* 通信仍需共享内存作为桥梁。上述代码中 `sdata` 数组仅用于存储各 Warp 的部分和，大小仅为 32 个整数，远小于传统实现。
- **位运算优化**：代码中使用右移操作（`>>=`）代替除法，使用取模和整除运算确定 Lane ID 和 Warp ID，以减少指令开销。

43.12 性能分析与实测结果

43.13 编译与环境配置

- **编译器**：推荐使用 Linux 下的 `nvcc` 命令行编译。Visual Studio 用户需在项目属性中正确配置“代码生成（Code Generation）”选项，确保匹配目标 GPU 的计算能力（Compute Capability）。例如 RTX 3060（Ampere 架构）需指定`-arch=sm_86` 或 `compute_80`。
- **验证**：务必对比 CPU 与 GPU 计算结果以确保正确性。

43.14 性能对比

在 RTX 3060 平台上对相同数据集进行测试：

- **共享内存优化版**：执行时间约 **82** μ**s**。
- **洗牌操作优化版**：执行时间约 **17–18** μ**s**。

结论：引入洗牌操作后，性能提升了约 **4 倍**。Nsight Compute 分析显示，该优化显著降低了 L1/Shared Memory 吞吐压力，并提高了计算单元的利用率。这是因为大量中间数据的交换直接在寄存器层面完成，避免了昂贵的内存读写操作。

43.15 总结

洗牌操作是 CUDA 高性能编程中至关重要的优化手段。通过将规约粒度下沉至 Warp 级别，并利用寄存器级数据交换替代共享内存通信，我们能够突破内存带宽瓶颈。尽管代码复杂度略有增加，但其带来的数倍性能收益使其成为向量规约等并行算法的标准优化范式。建议开发者在实际应用中，结合 Nsight Compute 等工具深入分析内存访问模式，充分挖掘硬件潜力。

44 CUDA GPU 并行编程：屋顶线模型分析

44.1 引言：性能评估的核心目标

本章将深入讲解**屋顶线模型 (Roofline Model)**。这是 GPU 高性能计算中至关重要的性能分析工具，其核心目的在于：

1. 科学评估应用程序的性能瓶颈；
2. 判定应用属于**计算受限(Compute-Bound)**还是**内存受限(Memory-Bound)**；
3. 基于分析结果选择正确的优化路径。

在进行屋顶线分析之前，建议先掌握 GPU 硬件架构参数（如总线速度、显存带宽、指令吞吐量等）的基础知识。只有理解了这些底层指标，才能准确解读分析结果。例如，在之前的向量规约（Vector Reduction）案例中，我们采用了共享内存、合并访问和 Shuffle 操作等优化手段，正是因为通过屋顶线分析确认了该应用是典型的*内存受限*应用。

44.2 关键性能指标与算术强度

44.3 峰值性能参数

评估性能需对比实测值与硬件理论峰值。以 NVIDIA Ampere 架构（A100）为例：

- **显存带宽峰值(Peak Memory Bandwidth)**:约 1.5 TB/s(即 1500 GB/s)。
- **计算吞吐量峰值（Peak Compute Throughput）**:
  - FP32（非 Tensor Core）：19.5 TFLOPS
  - FP64：9.7 TFLOPS
  - INT32：19.5 TOPS

注意：仅凭单一指标（如达到 1 TB/s 带宽）无法判断性能优劣。高带宽可能源于过多的内存操作而缺乏有效计算。我们需要一个综合指标来平衡二者。

44.4 算术强度（Arithmetic Intensity, AI）

算术强度是屋顶线模型的核心度量，定义为每字节内存访问所执行的浮点运算次数：

$$\text{AI} = \frac{\text{指令吞吐量 (FLOPS/s)}}{\text{内存带宽 (Bytes/s)}} \quad \text{[单位: FLOP/Byte]} \tag{2}$$

- **高 AI**：计算量远大于数据搬运量 → 倾向于计算受限。
- **低 AI**：数据搬运量远大于计算量 → 倾向于内存受限。

44.5 可达性能公式

应用程序的可达性能（Attainable Performance）受限于硬件峰值和当前算术强度下的带宽上限，取二者最小值：

$$\text{Performance} = \min\left(\text{Peak Compute}, \quad \text{AI} \times \text{Peak Bandwidth}\right) \tag{3}$$

该公式构成了屋顶线图的数学基础：当 AI × Bandwidth < Peak Compute 时，性能随 AI 线性增长（内存墙）；反之则达到平台期（计算墙）。

44.6 屋顶线图解读与转折点

44.7 坐标轴定义

- **X 轴（对数坐标）**：算术强度（AI），单位 FLOP/Byte。
- **Y 轴（对数坐标）**：性能，单位 GFLOPS 或 TFLOPS。

44.8 转折点（Ridge Point）

转折点是内存受限区与计算受限区的分界点，对应硬件同时达到带宽峰值和计算峰值的状态：

$$\mathrm{AI}_{\mathrm{ridge}} = \frac{\text{Peak Compute(FLOPS/s)}}{\text{Peak Bandwidth (Bytes/s)}} \tag{4}$$

以 A100 FP32 为例：$\mathrm{AI}_{\mathrm{ridge}} = 19500/1500 = 13\,\mathrm{FLOP/Byte}$。

- **左侧区域**（$\mathrm{AI} < \mathrm{AI}_{\mathrm{ridge}}$）：**内存受限区**。优化重点：合并访问、共享内存、L1/L2 缓存利用率、预取。
- **右侧区域**（$\mathrm{AI} > \mathrm{AI}_{\mathrm{ridge}}$）：**计算受限区**。优化重点：指令级并行、消除依赖、Fast Math、Tensor Core、寄存器优化。

44.9 多级存储屋顶线

实际分析中需区分不同存储层级，因其带宽差异巨大：

- **DRAM Roofline**：使用全局显存峰值带宽（最常用基准）。
- **L2 Cache Roofline**：使用 L2 缓存峰值带宽（转折点右移）。

- **L1/Shared Mem Roofline**：使用 L1/SMEM 峰值带宽（转折点进一步右移）。

同一应用在 DRAM 层面可能是内存受限，但在 L1 层面可能已进入计算受限区。优化目标是将数据点向右上方推移，逼近或超越转折点。

44.10 实战案例：向量规约的屋顶线分析

44.11 基准版本分析

使用 Nsight Compute 对向量规约基准版本（01_reduce）进行分析：

```
nvcc -O3 -o reduce_01 reduce_01.cu
ncu --launch-mode profile ./reduce_01
```

在 Nsight Compute 的 “Roofline” 面板中选择 “FP32(Non-Tensor)” 视图。对于 RTX 3060 等消费级卡，转折点 AI 约为 30FLOP/Byte。基准版本的实测点位于转折点**左侧远处**（AI≈0.12），确认为严重内存受限。

44.12 优化版本对比

对优化版本（03_reduce_shared_unroll）进行相同分析：

- **基准版**：AI = 0.12 FLOP/B, 性能 ≈ 15 GFLOPS
- **优化版**：AI = 0.66 FLOP/B, 性能 ≈ 177 GFLOPS

分析结论：

1. 算术强度提升约 **5.5 倍**（0.66/0.12）；
2. 绝对性能提升约 **11.8 倍**（177/15）；
3. 数据点在屋顶线图中显著向右上方移动，但仍处于转折点左侧，说明仍有内存优化空间（如后续 Shuffle 优化可进一步提升 AI）。

44.13 总结与优化策略指引

屋顶线模型为 GPU 性能优化提供了量化导航：

表 2: 基于屋顶线位置的优化策略

区域	特征	推荐优化手段
内存受限区	AI < Ridge Point	合并访存、Shared Memory、向量化加载、L1/L2 Cache 复用、减少冗余读写
计算受限区	AI > Ridge Point	ILP/TLP 提升、消除分支发散、Fast Math Intrinsics、Tensor Core、算法复杂度优化
转折点附近	AI ≈ Ridge Point	硬件已充分利用，考虑算法级重构

核心原则：优化的终极目标是使应用的数据点在屋顶线图中持续向右上方迁移，直至逼近硬件物理极限构成的 “屋顶”。每次优化后都应重新运行 Nsight Compute 验证数据点位移，形成 “测量 → 分析 → 优化 → 验证” 的闭环迭代。

45 CUDA GPU 并行编程：矩阵乘法基础实现

45.1 引言：矩阵乘法的重要性

矩阵乘法（GEMM, General Matrix Multiply）是计算科学与工程领域的基石操作。无论是在大规模仿真、优化问题，还是在当今热门的机器学习与人工智能领域，矩阵乘法都扮演着核心角色。

- **深度学习与 AI**：神经网络的层运算本质上可表示为矩阵运算。加速矩阵乘法可直接提升训练与推理速度。
- **科学仿真**：流体力学、量子力学等领域的数值模拟严重依赖矩阵运算。

- **图像与信号处理**：图像滤波、变换及信号分析均可分解为矩阵-向量或矩阵-矩阵乘法。例如，应用滤波器即为将图像矩阵与滤波器矩阵相乘。

45.2 软件生态：专用库与硬件单元

由于矩阵乘法的普适性，主流硬件厂商均提供了高度优化的软件库和专用硬件单元。

45.3 软件库

- **NVIDIA cuBLAS**：CUDA 基础线性代数子程序库，提供标准且高度优化的 GEMM 内核。
- **NVIDIA cuTlass**：相比 cuBLAS 更具灵活性，允许用户自定义和优化 GEMM 内核模板。
- **AMD Tensile**：基于配置的库，通过修改配置选项自动生成针对特定 GEMM 操作优化的可执行文件，无需手动编写底层内核。

45.4 专用硬件单元

现代 GPU 引入了专门用于加速矩阵运算的硬件单元，其吞吐量远超通用计算核心：

- **NVIDIA Tensor Cores**：专为混合精度矩阵乘法设计。例如在 A100 上，Tensor Core 的 FP16 吞吐量可达 300+ TFLOPS，而普通 CUDA Core 仅约 20 TFLOPS。一条 Tensor Core 指令可在数个周期内完成一个小矩阵块（如 4×4 或 16×16）的乘加运算。
- **AMD Matrix Cores**：功能类似于 Tensor Core，专用于加速矩阵运算。

45.5 算法原理：从 CPU 到 GPU

45.6 数学定义

标准矩阵乘法定义为 $C = A \times B$，维度遵循 $M \times K$ 与 $K \times N$ 规则：

$$C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} \cdot B_{k,j}$$

其中 A 为 $M \times K$，B 为 $K \times N$，结果 C 为 $M \times N$。

45.7 CPU 串行实现

CPU 端通常需要三重嵌套循环遍历行 (i)、列 (j) 和累加维度 (k)。为提高效率，应使用局部变量 `sum` 进行累加，避免频繁访问全局内存。

45.8 GPU 朴素实现（Naive GEMM）

GPU 并行化的核心思想是用线程索引替代外层循环：

- **线程映射**：每个线程负责计算输出矩阵 C 中的一个元素 $C_{i,j}$。
- **索引计算**：利用二维网格和二维线程块，通过 `blockIdx` 和 `threadIdx` 计算全局行号 row 和列号 col。
- **内层循环**：k 维度的累加仍需在线程内部通过循环完成。

注意：此版本称为“朴素”实现，仅使用全局内存，未利用共享内存或分块（Tiling）技术，存在大量冗余内存访问，性能远未达到硬件上限。

45.9 CUDA 代码实现步骤

编写 CUDA 矩阵乘法程序需遵循以下七个标准步骤：

1. **分配内存**：在主机（Host）和设备（Device）分别分配矩阵 A, B, C 的存储空间。

2. **初始化数据**：在主机端初始化矩阵 A, B（随机值）及 C（零值）。

3. **数据传输 H2D**：使用 `cudaMemcpy` 将 A, B 从主机复制到设备。

4. **定义内核**：编写 `__global__` 函数实现朴素矩阵乘法。

5. **启动内核**：配置二维网格与线程块尺寸，传入设备指针及矩阵维度。

6. **数据传输 D2H**：将计算结果 C 从设备复制回主机。

7. **释放内存**：使用 `delete[]` 和 `cudaFree` 释放主机与设备内存。

45.10 关键代码片段

Listing 6: 朴素矩阵乘法 CUDA 内核

```
/*naive implementation of MM*/
/*created for cuda couese on udemy*/
#include <iostream>
#include <cuda_runtime.h>
#include <cstdlib>

__global__ void mm_naive(float*A, float*B , float*C, int N){

    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < N && col < N) {
        int sum = 0;
        for (int k = 0; k < N; k++) {
            sum += A[row * N + k] * B[k * N + col];
        }
```

```
17          C[row * N + col] = sum;
18      }
19  }
20
21
22  void cpu_mm_naive(const float* A, const float* B, float* C, int
        N)
23  {
24      for (int i = 0; i < N; i++) {
25          for (int j = 0; j < N; j++) {
26              float sum = 0;
27              for (int k = 0; k < N; k++) {
28                  sum += A[i * N + k] * B[k * N + j];
29              }
30              C[i * N + j] = sum;
31          }
32      }
33  }
34
35
36  int main()
37  {
38      const int N=512;
39
40      float* h_A = new float[N*N];
41      float* h_B = new float[N*N];
42      float* h_C = new float[N*N];
43      float* h_C_gpu = new float[N*N];
44
```

```
45      float *d_A,*d_B,*d_C;
46      cudaMalloc((void**)&d_A,N*N*sizeof(float));
47      cudaMalloc((void**)&d_B,N*N*sizeof(float));
48      cudaMalloc((void**)&d_C,N*N*sizeof(float));
49
50      for(int i=0;i<N*N;i++){
51          h_A[i]=std::rand() % 10;
52          h_B[i]=std::rand() % 10;
53          h_C[i]=0;
54          h_C_gpu[i]=0;
55      }
56
57
58      cpu_mm_naive(h_A,h_B,h_C,N);
59
60      cudaMemcpy(d_A,h_A,N*N*sizeof(float),cudaMemcpyHostToDevice
            );
61      cudaMemcpy(d_B,h_B,N*N*sizeof(float),cudaMemcpyHostToDevice
            );
62
63      int block_size=16;
64      dim3 threadsperblock(block_size,block_size);
65      dim3 numBlocks((N+block_size-1)/block_size,(N+block_size-1)
            /block_size);
66
67      mm_naive<<<numBlocks,threadsperblock>>>(d_A,d_B,d_C,N);
68
69      cudaMemcpy(h_C_gpu,d_C,N*N*sizeof(float),
            cudaMemcpyDeviceToHost);
```

```
70
71      long temp=0;
72      for(int i=0;i<N*N;i++){
73          temp += std::abs(h_C_gpu[i]-h_C[i]);
74      }
75      if(temp==0)
76       std::cout<<"results are correct\n";
77      else
78      std::cout<<"wrong results\n";
79
80
81
82
83
84      delete[] h_A;
85      delete[] h_B;
86      delete[] h_C;
87      delete[] h_C_gpu;
88
89      cudaFree(d_A);
90      cudaFree(d_B);
91      cudaFree(d_C);
92
93  }
```

45.11 验证与性能分析

- **正确性验证**：必须保留一份 CPU 计算结果作为基准。比较时计算 $\sum |C_{GPU} - C_{CPU}|$，若使用浮点数，允许极小误差；若为整数则应严

格为 0。

- **性能考量**：GPU 存在数据拷贝开销，仅在矩阵规模较大（如 $N \geq 1024$）时才能体现加速优势。例如 $N = 1024$ 时，CPU 可能耗时 7000ms，而 GPU 仅需 28ms。

- **Roofline 分析**：使用 Nsight Compute 进行分析。朴素实现通常受限于内存带宽（Memory Bound）。后续优化（如 Shared Memory Tiling）旨在减少全局内存访问，使性能向计算受限（Compute Bound）区域移动。

45.12 总结与展望

本章实现了矩阵乘法的朴素 CUDA 版本，建立了性能基准。尽管该实现逻辑简单，但效率较低。下一章将介绍**分块（Tiling）技术**与**共享内存（Shared Memory）**的使用，这是优化 GEMM 性能的核心手段，也是所有高性能矩阵乘法库的基础。

46 使用共享内存优化矩阵乘法：分块技术

46.1 引言：全局内存的重复读取问题

考察矩阵乘法 $C = A \times B$，其中 A、B、C 均为方阵。为了计算矩阵 C 中的全部元素，矩阵 A 的第一行需要从全局内存中读取多少次?

回顾矩阵乘法的计算规则：计算 $C_{1,1}$ 需要将 A 的第一行与 B 的第一列做内积；计算 $C_{1,2}$ 需要将 A 的第一行与 B 的第二列做内积；计算 $C_{1,3}$ 需要将 A 的第一行与 B 的第三列做内积。因此，若 B 有三列，则 A 的第一行需要被读取三次。推广到一般情形：若 B 有 100 列，则 A 的第一行需要被读取 100 次。

同一行数据被反复从全局内存加载，构成了矩阵乘法中最典型的性能瓶颈之一。

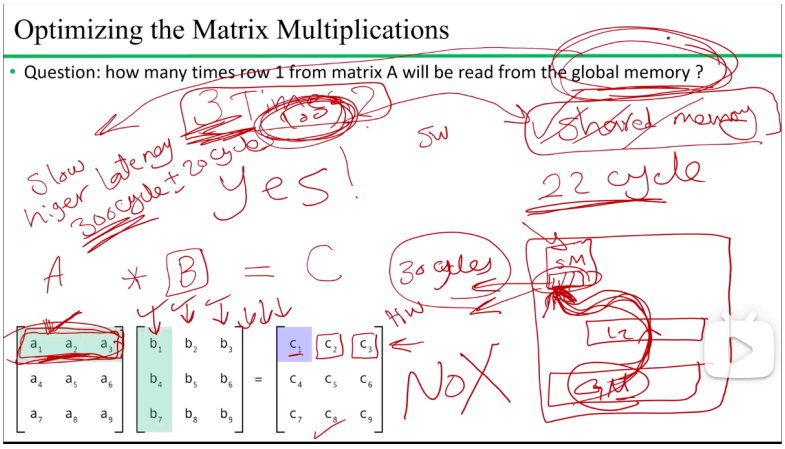

Optimizing the Matrix Multiplications
Question: how many times row 1 from matrix A will be read from the global memory ?
3 Times
Slow
higher latency
200cycle+
yes!
SW
shared memory
22 cycle
A * B = C
30 cycles
HW
NoX
SM
L2
3rd

图 46.1:

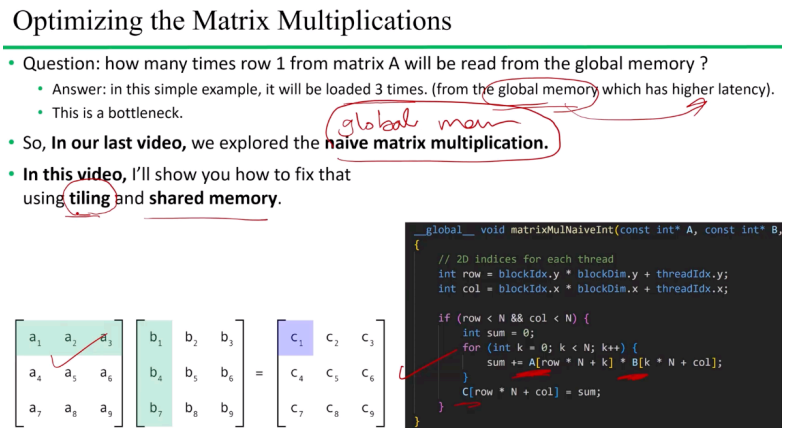

Optimizing the Matrix Multiplications
Question: how many times row 1 from matrix A will be read from the global memory ?
Answer: in this simple example, it will be loaded 3 times. (from the global memory which has higher latency).
This is a bottleneck.
global mem
So, In our last video, we explored the naive matrix multiplication.
In this video, I'll show you how to fix that using tiling and shared memory.
__global__ void matrixMulNaiveInt(const int* A, const int* B,
{
// 2D indices for each thread
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;
if (row < N && col < N) {
int sum = 0;
for (int k = 0; k < N; k++) {
sum += A[row * N + k] * B[k * N + col];
}
C[row * N + col] = sum;
}
}

图 46.2:

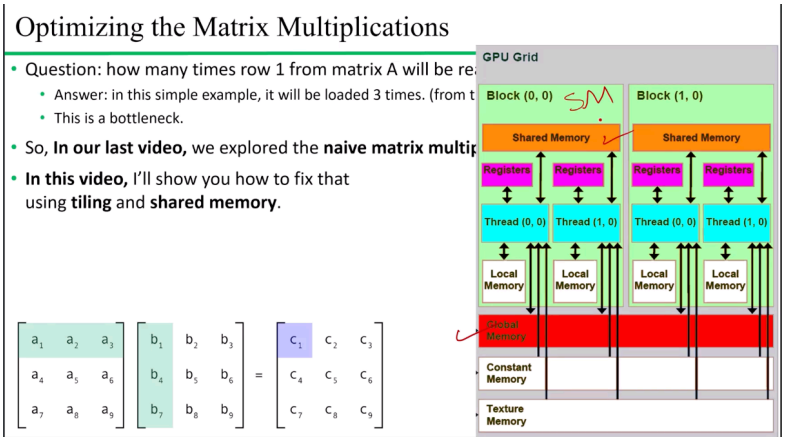


图 46.3:

46.2 全局内存与共享内存的延迟对比

根据第一部分硬件章节中引用的研究数据，各层级存储器的访问延迟大致如下：

存储器类型	访问延迟（时钟周期）
全局内存（Global Memory）	约 300（浮动 ±10–20）
共享内存（Shared Memory）	约 20–22

共享内存的读写延迟比全局内存低一个数量级。当矩阵的某一列包含 K 个元素时，同一行数据将被重复读取 K 次；若数据始终驻留于全局内存，则每次访问均须承担约 300 个周期的延迟，成为整个应用的性能瓶颈。

46.3 L1 缓存与共享内存的本质差异

一个自然的问题是：数据从全局内存被加载后，通常会经由 L2、L1 缓存驻留在 SM 附近，L1 缓存的延迟同样较低，为何仍需要使用共享内存？

关键在于**数据驻留的确定性**。L1 缓存由硬件管理，程序员无法控制其换入换出策略。当多个线程块竞争同一 SM 的 L1 容量时，硬件可能随时驱

逐其中的数据。若矩阵 A 的第一行被逐出 L1，后续访问将被迫再次从全局内存加载，重复承担高延迟开销。

共享内存则不同，它属于**软件控制的内存**：数据的加载与驻留完全由程序显式管理。只要程序员不主动覆盖，数据在整个内核执行期间都保证驻留在共享内存中。因此，对于需要多次重用的数据，使用共享内存可以在**确定性**的前提下获得低延迟访问。

综上所述，本应用的瓶颈在于：同一行数据被多次从高延迟的全局内存读取；解决方案是将需要重用的数据显式加载到共享内存中，该优化技术称为**分块（Tiling）**。

46.4 分块与瓦片的基本概念

46.4.1 瓦片的定义

在矩阵乘法中，矩阵 C 的每个元素都依赖于 A 的**一整行**与 B 的**一整列**（区别于向量加法中一个元素对应一个元素）。因此，GPU 并行化的基本策略是：将矩阵 C 划分为若干子区域，每个子区域称为一个**瓦片（Tile）**，并将每个瓦片分配给一个线程块独立计算。

瓦片大小与瓦片数量是两个不同的概念：例如一个 3×3 的矩阵可划分为 9 个瓦片（瓦片大小为 1）；而在实际应用中，通常将 C 划分为大小为 16×16 的瓦片。瓦片大小的选取受两个因素约束：

- **共享内存容量**：瓦片对应的 A 行片段与 B 列片段必须能够放入共享内存。
- **线程数上限**：每个瓦片由 $TILE_SIZE \times TILE_SIZE$ 个线程计算，不能超过每个块的线程数上限（1024）。

46.4.2 网格与线程块配置

以 $N = 512$、$TILE_SIZE = 16$ 为例：

- **块大小（Block Size）**：$16 \times 16 = 256$ 个线程，即 X、Y 维度的线程 ID 均从 0 到 15。
- **网格大小（Grid Size）**：X、Y 维度的块数量均为

$$\frac{N + TILE_SIZE - 1}{TILE_SIZE} = \frac{512 + 16 - 1}{16} = 32$$

  即网格为 32×32 个线程块。
- **总线程数**：32×32 块 $\times$ 256 线程/块 $= 262144 = 512 \times 512$，恰好等于矩阵 C 的元素数，与向量加法中”线程数等于元素数”的原则一致。

46.4.3 K 维循环累加

单个瓦片（16×16 区域）的计算需要 A 中对应的 16 个整行与 B 中对应的 16 个整列，每行/列包含 512 个元素。将其按 16 个元素一段划分为 32 个片段（块），则瓦片的计算结果为：

$$C_{tile} = \sum_{t=0}^{31} A_{tile,t} \times B_{t,tile}$$

其中 t 为片段索引（K 维迭代变量）。以第一个瓦片（块索引 $(0, 0)$）为例：

- $t = 0$：A 的第 0–15 列片段与 B 的第 0–15 行片段相乘，结果累加到 sum；
- $t = 1$：A 的第 16–31 列片段与 B 的第 16–31 行片段相乘，继续累加；
- 依此类推，直至 $t = 31$，完成全部 512 个元素的乘加运算。

注意 K 维步长：对 A 而言，相邻片段在 X 维度偏移 16；对 B 而言，相邻片段在 Y 维度偏移 16（对应 X 维度偏移 16×512）。因此，需要两层循环：外层遍历 K 维片段（t），内层遍历片段内的 16 个元素（k）。

46.5 共享内存实现

46.5.1 共享内存数组的声明

在 CUDA 内核中，共享内存变量使用 `__shared__` 关键字声明。矩阵 A、B 的元素类型为 float，因此声明两个 $TILE_SIZE \times TILE_SIZE$ 的二维数组：

```
__shared__ float s_a[TILE_SIZE][TILE_SIZE];
__shared__ float s_b[TILE_SIZE][TILE_SIZE];
```

46.5.2 共享内存容量核算

以 $TILE_SIZE = 16$ 为例，共享内存占用为：

$$16 \times 16 \times 4 \text{ 字节} \times 2 \text{ 个数组}/1024 = 2 \text{ KB}$$

该值远小于各架构的共享内存上限（如 RTX 3060 约 96–100 KB，A100 可达 160 KB）。调整 $TILE_SIZE$ 时须重新核算，确保不超过硬件容量。

46.5.3 数据加载与索引计算

在加载共享内存之前，A、B 的数据驻留于全局内存（由 `cudaMemcpy` 从主机复制而来）。每个线程将全局内存中的一个元素复制到共享内存的对应位置。

- **行索引与列索引**：

```
int row = blockIdx.y * TILE_SIZE + threadIdx.y;
int col = blockIdx.x * TILE_SIZE + threadIdx.x;
```

行索引使用 Y 维度（块的 Y 索引与线程的 Y ID），列索引使用 X 维度。

- **加载 A 的片段**:

```
s_a[threadIdx.y][threadIdx.x] = a[row * N + t * TILE_SIZE
    + threadIdx.x];
```

其中 `row * N` 定位到矩阵 A 的第 row 行（每行 N 个元素）；`t * TILE_SIZE` 定位到 K 维第 t 个片段在行内的起始列；`threadIdx.x` 为片段内偏移（0–15）。

- **加载 B 的片段**:

```
s_b[threadIdx.y][threadIdx.x] = b[(t * TILE_SIZE +
    threadIdx.y) * N + col];
```

B 的片段沿 Y 维度推进：`t * TILE_SIZE + threadIdx.y` 给出片段起始行加行内偏移，乘以 N 后加上列索引 col 完成定位。

46.5.4 同步与计算

- **第一次同步**：加载 A、B 的当前片段后，必须调用 `__syncthreads()`，确保块内所有线程完成加载后才能开始计算，否则部分线程可能读取到尚未写入的共享内存数据。

- **片段内乘加**：对片段内的 16 个元素执行乘加运算：

```
for (int k = 0; k < TILE_SIZE; k++) {
    sum += s_a[threadIdx.y][k] * s_b[k][threadIdx.x];
}
```

- **第二次同步**:在进入下一个片段的加载之前再次调用 `__syncthreads()`，防止某些线程在其余线程完成计算前覆盖共享内存。

全部片段累加完毕后，将结果写回全局内存：

```
c[row * N + col] = sum;
```

46.5.5 完整内核代码

```
#include <iostream>
#include <cuda_runtime.h>

#define N 1024         // Assume N is a multiple of TILE_SIZE
#define TILE_SIZE 16  // Could also use 32 for some GPUs

// GPU Kernel: Tiled MM (assuming N % TILE_SIZE == 0)
__global__ void matrixMulTiled(const float* A, const float* B,
    float* C)
{
    __shared__ float sA[TILE_SIZE][TILE_SIZE];
    __shared__ float sB[TILE_SIZE][TILE_SIZE];

    // Calculate this thread's row and column
    int row = blockIdx.y * TILE_SIZE + threadIdx.y;
    int col = blockIdx.x * TILE_SIZE + threadIdx.x;

    // Each thread will accumulate the result in 'sum'
    float sum = 0.0f;

    // Number of tiles we need to iterate over the k dimension
    // Since N is multiple of TILE_SIZE, this is N / TILE_SIZE
    int numTiles = N / TILE_SIZE;
```

```
23
24      for (int t = 0; t < numTiles; t++) {
25          // Load one tile of A into shared memory
26          sA[threadIdx.y][threadIdx.x] = A[row * N + (t *
                TILE_SIZE + threadIdx.x)];
27          // Load one tile of B into shared memory
28          sB[threadIdx.y][threadIdx.x] = B[(t * TILE_SIZE +
                threadIdx.y) * N + col];
29
30          // Synchronize to make sure the tile is loaded
31          __syncthreads();
32
33          // Compute partial sums for this tile
34          for (int k = 0; k < TILE_SIZE; k++) {
35              sum += sA[threadIdx.y][k] * sB[k][threadIdx.x];
36          }
37
38          // Synchronize before loading the next tile
39          __syncthreads();
40      }
41
42      // Write the result to global memory
43      C[row * N + col] = sum;
44  }
45  void cpu_mm_naive( float* A, float* B, float* C)
46  {
47      for (int i = 0; i < N; i++) {
48          for (int j = 0; j < N; j++) {
49              float sum = 0;
```

```
50              for (int k = 0; k < N; k++) {
51                  sum += A[i * N + k] * B[k * N + j];
52              }
53              C[i * N + j] = sum;
54          }
55      }
56  }
57  int main()
58  {
59
60      // 1. Allocate Host Memory (simple example)
61      float *h_A = new float[N*N];
62      float *h_B = new float[N*N];
63      float *h_C = new float[N*N];
64      float *h_C_gpu = new float[N*N];
65
66      // 2. Initialize Input Matrices
67      for (int i = 0; i < N*N; i++) {
68          h_A[i] = static_cast<float>(i % 10); // arbitrary
69          h_B[i] = static_cast<float>((i * 2) % 10);
70      }
71
72      // 3. Allocate Device Memory
73      float *d_A, *d_B, *d_C;
74      cudaMalloc(&d_A, N*N*sizeof(float));
75      cudaMalloc(&d_B, N*N*sizeof(float));
76      cudaMalloc(&d_C, N*N*sizeof(float));
77
78      // 4. Copy Data from Host to Device
```

```
79      cudaMemcpy(d_A, h_A, N*N*sizeof(float),
            cudaMemcpyHostToDevice);
80      cudaMemcpy(d_B, h_B, N*N*sizeof(float),
            cudaMemcpyHostToDevice);
81
82      // 5. Launch Tiled Kernel
83      dim3 threads(TILE_SIZE, TILE_SIZE);
84      dim3 blocks(N / TILE_SIZE, N / TILE_SIZE);
85      cpu_mm_naive(h_A,h_B,h_C);
86
87      matrixMulTiled<<<blocks, threads>>>(d_A, d_B, d_C);
88
89      // 6. Copy Result Back
90      cudaMemcpy(h_C_gpu, d_C, N*N*sizeof(float),
            cudaMemcpyDeviceToHost);
91
92      // 7. (Optional) Verify or Print Some Results
93      long temp=0;
94      for(int i=0;i<N*N;i++){
95          temp += std::abs(h_C_gpu[i]-h_C[i]);
96      }
97      if(temp==0)
98       std::cout<<"results are correct\n";
99      else
100     std::cout<<"wrong results\n";
101     // 8. Cleanup
102     delete[] h_A;
103     delete[] h_B;
104     delete[] h_C;
```

```
105     cudaFree(d_A);
106     cudaFree(d_B);
107     cudaFree(d_C);
108
109     return 0;
110 }
```

46.6 常见错误与调试

在实现过程中容易出现的错误包括：

1. **语法错误**：语句末尾缺少分号、变量（如 `numTiles`）未声明、常量名大小写不一致（如 `TILE_SIZE` 误写为 `tile_size`）等。

2. **逻辑错误（关键）**：将片段乘加循环（内层 for 循环）错误地放在片段遍历循环（外层 for 循环）之外。此时程序会先加载完所有片段，再仅对**最后一个片段**执行乘加，计算结果严重错误。正确的控制流应当是：**加载一个片段** → **同步** → **乘加累加** → **同步**，如此循环往复，直至遍历完 K 维的所有片段。

46.7 性能评估

46.7.1 与朴素实现的对比

在 $N = 512$ 时，使用 Nsight Compute 命令行工具测得：

实现	执行时间（N=512）	执行时间（N=1024）
朴素矩阵乘法（全局内存）	360 微秒	约 3 毫秒
分块矩阵乘法（共享内存）	约 251 微秒	约 2 毫秒

使用共享内存后，执行时间由 360 微秒降至约 251 微秒，性能提升约 30%–40%。当矩阵规模增大至 $N = 1024$ 时，同样取得约 33% 的提升，说明共享内存优化在较大规模下依然有效。

46.7.2 瓦片大小的选择

进一步调整 $TILE_SIZE$ 观察其影响：

TILE_SIZE	每块线程数	共享内存占用	结果（N=1024）
16	256	2 KB	约 2 毫秒（最优）
32	1024	8 KB	约 2.1 毫秒（略有下降）
64	4096	32 KB	内核无法执行

- $TILE_SIZE = 32$ 时，每块线程数为 1024，已达硬件规定的每块线程数上限。由于线程规模过大、占用率受限，执行时间反而略有增加。
- $TILE_SIZE = 64$ 时，每块线程数为 $64 \times 64 = 4096$，超过 1024 的上限，内核无法启动（Nsight Compute 提示"No kernels were profiled"）。注意：此限制源于线程数上限，而非共享内存容量（64 瓦片仅需 32 KB，未超出硬件共享内存上限）。

46.8 总结

本节通过分块技术将矩阵乘法中被反复读取的行/列数据显式加载到共享内存，将每元素访问延迟从约 300 个周期降至约 20 个周期，在 $N = 512$ 与 $N = 1024$ 规模下均获得约 30%–40% 的性能提升。要点如下：

1. 矩阵乘法的数据重用特性（同一行/列被读取 K 次）使其天然适合共享内存优化。
2. 共享内存属于软件控制的内存，数据驻留具有确定性，这是 L1 缓存无法保证的。

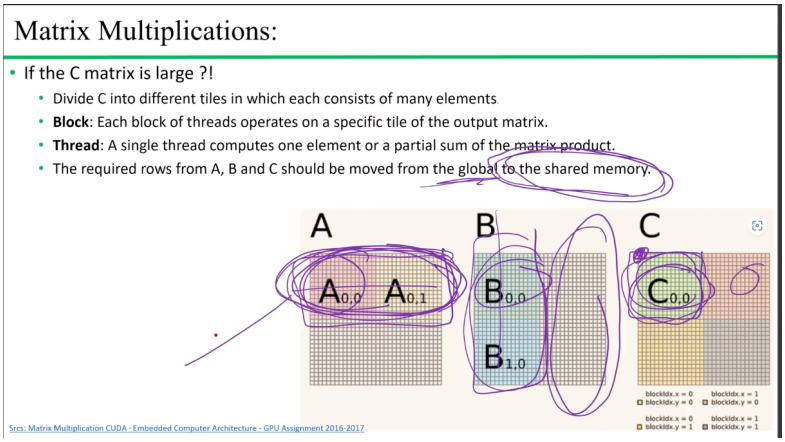


图 46.4:

3. 瓦片大小的选取须同时满足共享内存容量与每块线程数上限（1024）两个约束。

4. 分块实现的控制流必须严格遵循” 加载 → 同步 → 计算 → 同步” 的顺序，任何一环缺失都将导致错误结果。

47 CUDA GPU 并行编程：Float4 向量化优化矩阵乘法

47.1 前言与背景回顾

在前序章节中，我们讨论了矩阵乘法（GEMM）的分块（Tiling）策略。由于矩阵规模通常远超单个 SM 的存储容量，必须将矩阵分割为若干“小块”（Tile）。若不分块，仅在一个 SM 上执行计算会导致其他 SM 闲置，造成硬件资源浪费。

47.2 核心概念辨析

- **小块（Tile）**：指矩阵数据被分割后的逻辑子矩阵。
- **线程块（Thread Block）**：指 CUDA 编程模型中的执行单元集合。

关键映射关系：C 矩阵的一个小块对应一个 CUDA 线程块。若 C 矩阵被分为 N 个小块，则需要 N 个线程块。每个线程块负责计算 C 矩阵中对应小块的所有元素。需要注意的是，计算 C 的一个小块需要循环遍历 K 维度上所有对应的 A 和 B 的小块，累加部分和（Partial Sum）才能得到最终结果。

47.3 优化动机：计算与内存受限

朴素版本及基础共享内存版本的矩阵乘法同时面临两类瓶颈：

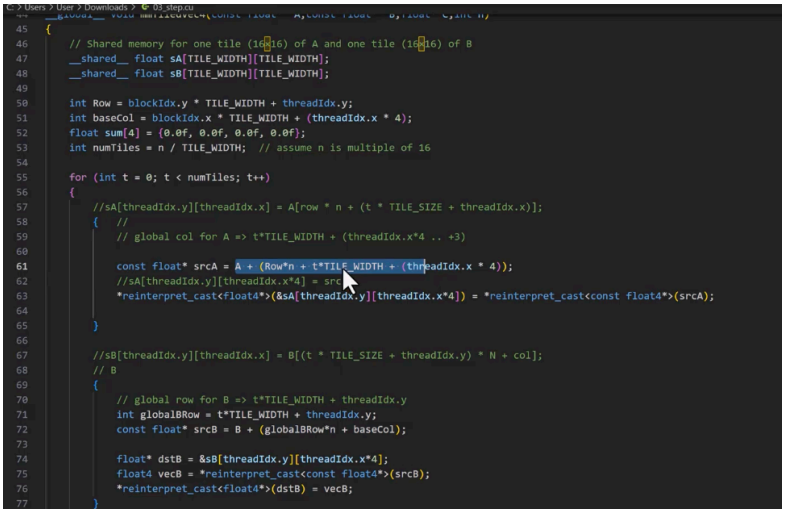
1. **计算受限（Compute-bound）**：算术运算指令过多，包括循环控制开销。
2. **内存受限（Memory-bound）**：全局内存带宽不足，访存延迟高。

本章介绍的 `float4` 向量化优化旨在同时改善上述两个问题，使屋顶线模型（Roofline Model）中的性能点向更高算力利用率方向移动。

47.4 优化原理

47.5 循环展开（Loop Unrolling）

循环展开通过减少循环控制指令（如计数器递增、条件跳转）来降低非计算指令的比例。例如，将大小为 16 的循环完全展开，可消除 1 6 次循环判断开销，直接提升计算密集型任务的性能。



```
{
    // Shared memory for one tile (16×16) of A and one tile (16×16) of B
    __shared__ float sA[TILE_WIDTH][TILE_WIDTH];
    __shared__ float sB[TILE_WIDTH][TILE_WIDTH];

    int Row = blockIdx.y * TILE_WIDTH + threadIdx.y;
    int baseCol = blockIdx.x * TILE_WIDTH + (threadIdx.x * 4);
    float sum[4] = {0.0f, 0.0f, 0.0f, 0.0f};
    int numTiles = n / TILE_WIDTH;  // assume n is multiple of 16

    for (int t = 0; t < numTiles; t++)
    {
        //sA[threadIdx.y][threadIdx.x] = A[row * n + (t * TILE_SIZE + threadIdx.x)];
        {   //
            // global col for A => t*TILE_WIDTH + (threadIdx.x*4 .. +3)

            const float* srcA = A + (Row*n + t*TILE_WIDTH + (threadIdx.x * 4));
            //sA[threadIdx.y][threadIdx.x*4] = src
            *reinterpret_cast<float4*>(&sA[threadIdx.y][threadIdx.x*4]) = *reinterpret_cast<const float4*>(srcA);


        }

        //sB[threadIdx.y][threadIdx.x] = B[(t * TILE_SIZE + threadIdx.y) * N + col];
        // B
        {
            // global row for B => t*TILE_WIDTH + threadIdx.y
            int globalBRow = t*TILE_WIDTH + threadIdx.y;
            const float* srcB = B + (globalBRow*n + baseCol);

            float* dstB = &sB[threadIdx.y][threadIdx.x*4];
            float4 vecB = *reinterpret_cast<const float4*>(srcB);
            *reinterpret_cast<float4*>(dstB) = vecB;
        }
```

图 47.1:

47.6 向量化加载与预取 (Vectorized Load & Prefetching)

利用 `float4` 数据类型，单条指令即可加载 4 个连续的浮点数。这相当于一种“内存级循环展开”:

- **带宽利用率提升**：一次 128-bit 内存事务替代四次 32-bit 事务。
- **指令数减少**：加载相同数量元素所需的指令数降为原来的 1/4。

47.7 线程配置调整

由于采用 `float4` 加载，X 维度的线程需求发生变化：

- **原方案**：16 × 16 小块，X 维度需 16 个线程，每线程加载 1 个元素。
- **新方案**：16 × 16 小块，X 维度仅需 4 个线程，每线程加载 4 个元素（`float4`）。
- **Y 维度**：保持不变，仍需 16 个线程处理 16 行。

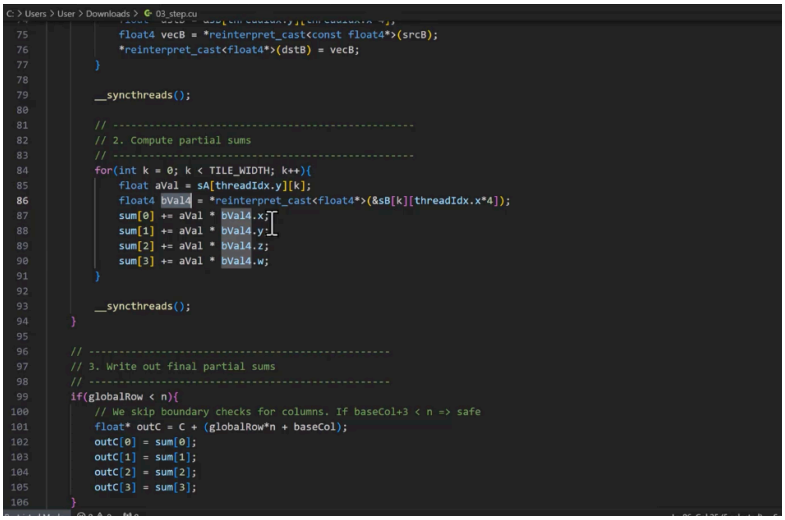


图 47.2:

因此，每个线程块处理的总工作量不变，但活跃线程数减少，单线程工作负载增加。

47.8 内存布局约束

`float4` 要求内存地址连续且对齐。由于矩阵采用行主序（Row-major）存储，同一行的元素在物理内存中连续，而跨行元素不连续。因此，**向量化加载必须沿 X 维度（列方向）进行**，Y 维度保持标量迭代。

47.9 代码实现要点

47.10 累加器声明

由于每个线程现在负责 C 矩阵中的 4 个元素（同一行相邻 4 列），需声明 4 个独立的累加器以消除数据依赖，便于流水线执行：

```
float sum0 = 0.0f, sum1 = 0.0f, sum2 = 0.0f, sum3 = 0.0f;
// 或使用数组 float sum[4] = {0};
```

47.11 全局内存到共享内存的向量化加载

加载矩阵 A 和 B 时，使用 `float4` 指针进行类型转换与读取：

注意：全局内存与共享内存的地址空间不同，使用 `reinterpret_cast` 是为了满足编译器对 `float4` 对齐的要求，避免警告并确保生成正确的 128-bit 加载指令（如 `LDG.E.128`）。

47.12 计算阶段的向量化访问

在共享内存中进行乘加运算时，利用 `float4` 的成员访问语法：

```
/*****************************************************************************

 * File: mm_float4_only.cu
 *
 * Demonstrates:
 *   - Tiled matrix multiplication using shared memory
 *   - Vectorized float4 loads (assuming alignment is correct)
 *   - No pinned memory usage (uses normal new[]/delete[] +
    cudaMalloc/cudaMemcpy)
 *
 * Compile:
 *   nvcc mm_float4_only.cu -o mm_float4_only -std=c++11
 *****************************************************************************/


#include <iostream>
#include <cuda_runtime.h>
```

```
15 #include <cstdlib>
16 #include <cmath>
17
18 // For simplicity, we pick N=512 (a multiple of 16)
19 static const int N = 512;
20 // We'll keep tile size "16×16" conceptually, but blockDim.x=4
      => each thread loads 4 columns.
21 static const int TILE_WIDTH = 16; // tile is 16 wide, 16 tall
22
23 //---------------------------------------------------------------------

24 // CPU naive mm
25 //---------------------------------------------------------------------

26 void cpu_mm_naive(const float* A, const float* B, float* C, int
       n)
27 {
28     for (int i = 0; i < n; i++){
29         for (int j = 0; j < n; j++){
30             float sum = 0.0f;
31             for (int k = 0; k < n; k++){
32                 sum += A[i*n + k] * B[k*n + j];
33             }
34             C[i*n + j] = sum;
35         }
36     }
37 }
38
39 //---------------------------------------------------------------------
```

```
40 // Kernel: Tiled + float4 loads
41 //  blockDim: (4,16) => 4 threads in x, each loading 4 columns
       => 16 columns total
42 //                      16 threads in y => 16 rows
43 //---------------------------------------------------------------------------

44 __global__ void mmTiledVec4(const float* __restrict__ A,
45                          const float* __restrict__ B,
46                          float*     __restrict__ C,
47                          int n)
48 {
49     // Shared memory for one tile (16×16) of A and one tile (16
          ×16) of B
50     __shared__ float sA[TILE_WIDTH][TILE_WIDTH];
51     __shared__ float sB[TILE_WIDTH][TILE_WIDTH];
52
53     // Identify which row & column(s) this thread will handle
54     // blockIdx.x, blockIdx.y => which tile
55     // threadIdx.x in [0..3], threadIdx.y in [0..15]
56     int Row = blockIdx.y * TILE_WIDTH + threadIdx.y;
57     // Each thread covers 4 columns:
58     //  baseCol = blockIdx.x*16 + threadIdx.x*4
59     int baseCol = blockIdx.x * TILE_WIDTH + (threadIdx.x * 4);
60
61     // We'll accumulate partial sums for these 4 columns in a
          small array
62     float sum[4] = {0.0f, 0.0f, 0.0f, 0.0f};
63
```

```
64      // how many 16×16 tiles we must iterate over in the K
           dimension
65      int numTiles = n / TILE_WIDTH; // assume n is multiple of
           16
66
67      // For each tile in [0..numTiles-1]
68      for (int t = 0; t < numTiles; t++)
69      {
70          // ------------------------------------------------
71          // 1. Load sub-tile of A and B into shared memory
72          //   Each thread loads a float4 from A, and a float4
               from B
73          // ------------------------------------------------
74
75          // A
76          //sA[threadIdx.y][threadIdx.x] = A[row * n + (t *
               TILE_SIZE + threadIdx.x)];
77          {  //
78              // global col for A => t*TILE_WIDTH + (threadIdx.x*4
                   .. +3)
79              int globalACol = t*TILE_WIDTH + (threadIdx.x * 4);
80              const float* srcA = A + (Row*n + globalACol);
81              float* dstA = &sA[threadIdx.y][threadIdx.x*4];
82
83              float4 vecA = *reinterpret_cast<const float4*>(srcA)
                   ;
84              *reinterpret_cast<float4*>(dstA) = vecA;
85          }
86
```

```
87          //sB[threadIdx.y][threadIdx.x] = B[(t * TILE_SIZE +
                threadIdx.y) * N + col];
88          // B
89          {
90              // global row for B => t*TILE_WIDTH + threadIdx.y
91              int globalBRow = t*TILE_WIDTH + threadIdx.y;
92              // columns => baseCol..baseCol+3
93              const float* srcB = B + (globalBRow*n + baseCol);
94              float* dstB = &sB[threadIdx.y][threadIdx.x*4];
95
96              float4 vecB = *reinterpret_cast<const float4*>(srcB)
                    ;
97              *reinterpret_cast<float4*>(dstB) = vecB;
98          }
99
100         __syncthreads();
101
102         // ---------------------------------------------------
103         // 2. Compute partial sums
104         // ---------------------------------------------------
105         for(int k = 0; k < TILE_WIDTH; k++){
106             float aVal = sA[threadIdx.y][k];
107             float4 bVal4 = *reinterpret_cast<float4*>(&sB[k][
                    threadIdx.x*4]);
108             sum[0] += aVal * bVal4.x;
109             sum[1] += aVal * bVal4.y;
110             sum[2] += aVal * bVal4.z;
111             sum[3] += aVal * bVal4.w;
112         }
```

```
113
114         __syncthreads();
115     }
116
117     // ------------------------------------------------
118     // 3. Write out final partial sums
119     // ------------------------------------------------
120     if(Row < n){
121         // We skip boundary checks for columns. If baseCol+3 < n
                => safe
122         float* outC = C + (Row*n + baseCol);
123         outC[0] = sum[0];
124         outC[1] = sum[1];
125         outC[2] = sum[2];
126         outC[3] = sum[3];
127     }
128 }
129
130 //-----------------------------------------------------------------------

131 // Main
132 //-----------------------------------------------------------------------

133 int main()
134 {
135     // 1. Allocate host memory (regular/paged)
136     float* h_A = new float[N*N];
137     float* h_B = new float[N*N];
138     float* h_C = new float[N*N];
```

```
139     float* h_Cgpu = new float[N*N];
140
141     // 2. Initialize
142     srand(0);
143     for(int i=0; i<N*N; i++){
144         h_A[i] = float(rand()%10);
145         h_B[i] = float(rand()%10);
146         h_C[i] = 0.0f;
147         h_Cgpu[i] = 0.0f;
148     }
149
150     // CPU reference
151     cpu_mm_naive(h_A, h_B, h_C, N);
152
153     // 3. Allocate device memory
154     float *d_A, *d_B, *d_C;
155     size_t nBytes = N*N*sizeof(float);
156     cudaMalloc((void**)&d_A, nBytes);
157     cudaMalloc((void**)&d_B, nBytes);
158     cudaMalloc((void**)&d_C, nBytes);
159
160     // 4. Copy data from Host to Device (synchronous)
161     cudaMemcpy(d_A, h_A, nBytes, cudaMemcpyHostToDevice);
162     cudaMemcpy(d_B, h_B, nBytes, cudaMemcpyHostToDevice);
163
164     // 5. Launch the vectorized kernel
165     // blockDim => (4,16). => 4 in x => 4 columns per thread,
            16 in y => 16 rows
166     dim3 block(4,16);
```

```
167     dim3 grid(N/TILE_WIDTH, N/TILE_WIDTH);
168     mmTiledVec4<<<grid, block>>>(d_A, d_B, d_C, N);
169     cudaDeviceSynchronize();
170 
171     // 6. Copy result back
172     cudaMemcpy(h_Cgpu, d_C, nBytes, cudaMemcpyDeviceToHost);
173 
174     // 7. Check correctness
175     long long diff = 0;
176     for(int i=0; i<N*N; i++){
177         diff += (long long)fabs(h_Cgpu[i] - h_C[i]);
178     }
179     if(diff==0) {
180         std::cout << "Results match!\n";
181     } else {
182         std::cout << "Mismatch, diff=" << diff << "\n";
183     }
184 
185     // Cleanup
186     delete[] h_A;
187     delete[] h_B;
188     delete[] h_C;
189     delete[] h_Cgpu;
190     cudaFree(d_A);
191     cudaFree(d_B);
192     cudaFree(d_C);
193 
194     return 0;
195 }
```

这种显式展开写法避免了循环依赖，有利于编译器调度指令。

47.13 性能评估

在相同硬件环境下，三种实现的执行时间对比如下：

表 3: 矩阵乘法优化性能对比

优化版本	执行时间 (μs)	相对加速比
Naive（朴素版）	360	1.0x
Shared Memory（共享内存版）	250	1.44x
Float4 Vectorized（本课优化）	**148**	**2.43x**

结果分析：相较于共享内存版本，`float4` 优化进一步减少了约 100μs 的执行时间。屋顶线分析表明，该优化有效缓解了内存带宽压力，同时将性能点推向了计算与内存平衡的过渡区域（Transition Point），验证了向量化加载对双重瓶颈的改善效果。

47.14 总结

1. `float4` 优化的核心是**向量化内存访问**，而非改变矩阵的存储分配方式。
2. 必须严格遵循**行主序**的内存连续性要求，仅在列方向（X 维）应用向量化。
3. 线程配置需相应调整：X 维度线程数除以 4，Y 维度不变。
4. 配合循环展开与多累加器设计，可同时缓解计算与内存瓶颈。
5. 实际编码中需注意类型转换（`reinterpret_cast`）以满足对齐要求。

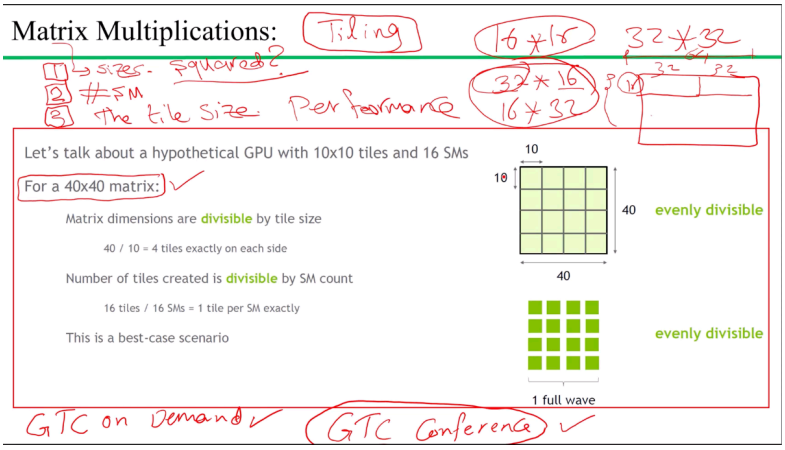


图 48.1:

48 CUDA GPU 并行编程：矩阵乘法中的分块技术

48.1 引言

在矩阵乘法的 GPU 优化中，**分块（Tiling）**是提升性能的核心技术。本章将深入探讨如何为矩阵乘法应用选择最佳的分块策略，重点分析矩阵尺寸、SM 数量与分块尺寸三者之间的制约关系，特别是针对非方阵的处理方法。

48.2 关键参数与约束

在选择分块策略时，必须综合考虑以下三个关键参数：

1. **矩阵尺寸（Matrix Dimensions）**：包括方阵与非方阵。对于非方阵，需在 X 和 Y 维度上采用不同的分块尺寸（如 32×16），前提是矩阵维度必须是分块尺寸的整数倍。

2. **SM 数量（Number of SMs）**：直接决定 GPU 利用率。若分块总数

少于 SM 数量，部分 SM 将处于空闲状态；若分块总数不是 SM 数量的整数倍，最后一轮计算将导致 SM 资源浪费。

3. **分块尺寸（Tile Size）**：直接影响共享内存占用与执行时间。通常需要通过实验（Profiling）确定最优值，而非盲目选择。

核心原则：为了最大化 GPU 占用率（Occupancy），**总分块数应尽可能为 SM 数量的整数倍**，且矩阵维度应能被分块尺寸整除。

48.3 分块策略案例分析

以下案例假设 GPU 拥有 **16 个 SM**，旨在说明不同参数组合对性能的影响。

48.4 案例 1：理想方阵分块

- **矩阵尺寸**：40×40
- **分块尺寸**：4×4
- **分析**：

$$\text{X 方向分块数} = \frac{40}{4} = 10, \quad \text{Y 方向分块数} = \frac{40}{4} = 10$$

$$\text{总分块数} = 10 \times 10 = 100$$

虽然 100 不是 16 的倍数，但 $100 \div 16 = 6 \ldots 4$，最后一轮仅有 4 个分块，利用率尚可。

- **更优解**：若选择分块尺寸使总分块数恰好为 16 或 32 等 SM 倍数，则利用率达到 100%。例如，若矩阵为 64×64，分块为 16×16，总分块数 $4 \times 4 = 16$，完美匹配 16 个 SM。

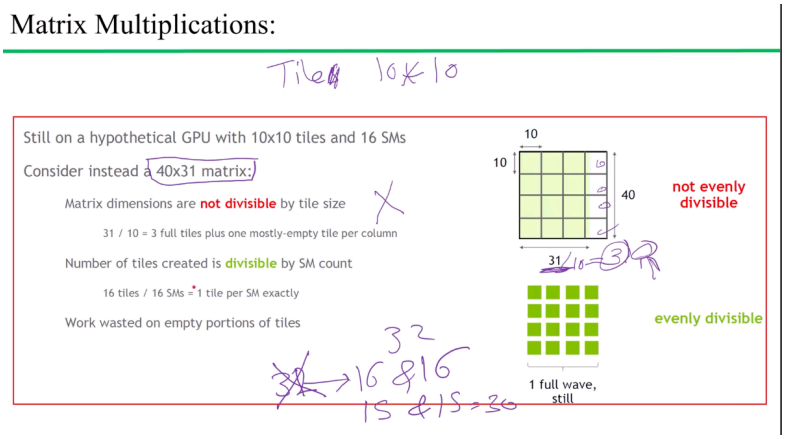


图 48.2:

48.5 案例 2：非方阵与边界填充

- **矩阵尺寸**：$31 \times N$
- **问题**：31 无法被常见分块尺寸（如 16、32）整除。
- **解决方案**：**零填充（Zero Padding）**。将矩阵扩展至 $32 \times N$，新增列/行补零。这虽引入少量冗余计算，但保证了内存访问的对齐与分块的规整性，整体收益远大于代价。

48.6 案例 3：分块数与 SM 不匹配

- **矩阵尺寸**：40×50，**分块尺寸**：10×10
- **分析**：

$$\text{总分块数} = \left(\frac{40}{10}\right) \times \left(\frac{50}{10}\right) = 4 \times 5 = 20$$

20 不是 16 的倍数。第一轮 16 个 SM 满载，第二轮仅剩 4 个分块，导致 12 个 SM 空闲，利用率骤降至 25%。

- **教训**：即使矩阵维度可被整除，若总分块数不是 SM 倍数，仍会造成严重的**尾部效应（Tail Effect）**。

48.7 案例 4：最差场景

- **矩阵尺寸**：40×41，**分块尺寸**：10×10
- **问题叠加**：

  1. 41 无法被 10 整除，产生不规则边界块（仅 1 列有效数据）。
  2. 总分块数 $4 \times 5 = 20$，非 SM 倍数，SM 利用率低。

  此场景同时存在**内存访问不对齐**与**计算资源浪费**双重问题，应极力避免。

48.8 非方形分块的灵活性

分块不必是正方形。对于 $M \times K$ 与 $K \times N$ 的非方阵，可采用矩形分块以适配维度：

- 示例：64×32 矩阵，可采用 16×8 分块。
- X 方向：$64/16 = 4$ 块
- Y 方向：$32/8 = 4$ 块
- 总分块数：$4 \times 4 = 16$，完美匹配 16 SM。

关键：只要满足“维度可整除”且“总分块数为 SM 倍数”两个条件，任意矩形分块均为合法且潜在最优的选择。

48.9 学习资源推荐

建议关注 **NVIDIA GTC (GPU Technology Conference)** 大会。该会议每年 3 月举办，涵盖科学计算、AI、大模型、Omniverse 及光线追踪等数百场专题讲座。可通过搜索引擎访问 **GTC On-Demand** 平台，免费注册后即可观看历年讲座文章与资料，是深入学习 GPU 并行编程的权威资源。

48.10 总结与展望

本章明确了分块策略选择的黄金法则：

$$\textbf{最优分块} \iff \begin{cases} \text{矩阵维度} \mod \text{分块尺寸} = 0 \\ \text{总分块数} \mod \text{SM 数量} = 0 \end{cases}$$

下一章将转向 **Tensor Cores（张量核心）**。在 Ampere 及更新架构中，Tensor Core 执行矩阵乘法的性能可达普通 CUDA Core 的 **9 倍以上**，已成为现代深度学习与高性能计算中 GEMM 操作的唯一推荐路径。理解普通核心的分块原理，是掌握 Tensor Core 编程的重要基础。

49 CUDA GPU 并行编程：张量核心（Tensor Core）技术详解

49.1 引言：张量核心的重要性

本章将深入讨论**张量核心（Tensor Core）**。张量核心是当今 NVIDIA GPU 中最核心的计算单元。自 2017 年首次被引入 NVIDIA GPU 以来，它已被广泛应用于深度学习、广义机器学习乃至科学计算等领域。其核心原因在于：张量核心是专为加速**矩阵乘法**而设计的专用硬件单元。

深度学习极度依赖矩阵乘法运算。正如本系列首章所述，神经网络（例如卷积神经网络）的本质就是矩阵运算。以卷积神经网络为例，输入图像被

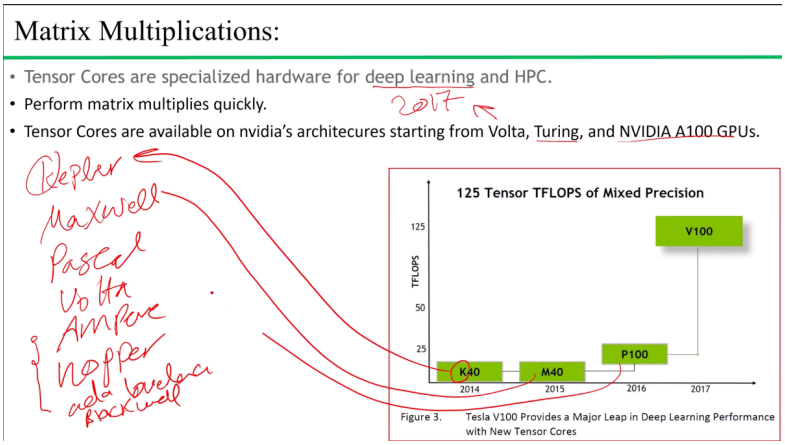


图 49.1:

表示为矩阵，滤波器（Filter）本身也是矩阵，通过将滤波器与图像矩阵相乘来实现特征提取。因此，我们需要一个专门用于加速矩阵乘法的硬件单元，这就是张量核心存在的意义。值得一提的是，AMD GPU 中对应的单元称为**矩阵核心（Matrix Core）**，其功能与张量核心完全相同。

49.2 张量核心的架构演进与性能飞跃

张量核心是专为深度学习和高性能计算（HPC）设计的专用硬件，能够快速执行混合精度矩阵乘法。它最早出现在 2017 年发布的 **Volta** 架构中，随后历经 Turing、Ampere、Hopper、Ada Lovelace 以及最新的 Blackwell 等架构迭代。

49.3 代际性能对比

图 1 展示了不同架构 GPU 的性能差异（单位：TFLOPS）。从 Kepler、Maxwell、Pascal 到 Volta，性能出现了质的飞跃：

- **Pascal (P100)**：仅配备普通 CUDA 核心，峰值性能约为 15 TFLOPS。
- **Volta (V100)**：首次引入张量核心，峰值性能跃升至约 125 TFLOPS。

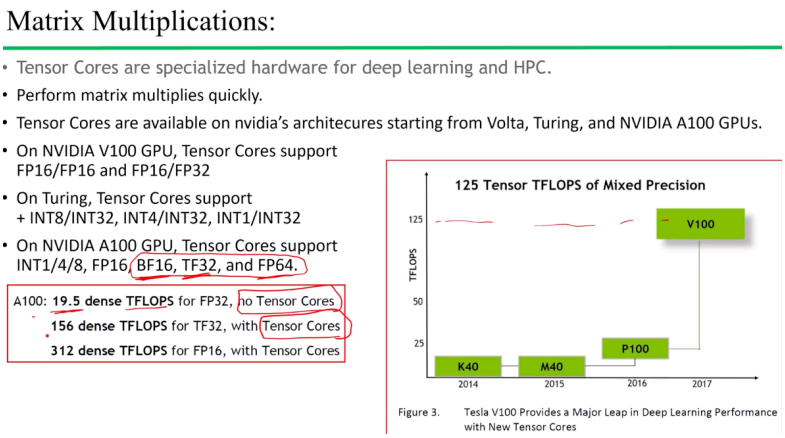


图 49.2:

相比 Pascal，Volta 在使用张量核心时实现了约 **9 倍**的性能提升。这种巨大提升源于张量核心的特殊设计：每个张量核心内部由多个子单元组成，使其每个时钟周期可执行 64 次融合乘加（FMA）运算，而普通 CUDA 核心每周期仅能执行 1 次运算。

49.4 数据类型与精度支持

不同架构的张量核心支持的数据类型逐步丰富：

Volta 架构 支持 FP16 × FP16 → FP16 以及 FP16 × FP16 → FP32。当输入矩阵数值较大时，为避免累加溢出，通常选择 FP32 作为输出精度。

Turing 架构 在 Volta 基础上增加了整数支持，包括 INT8 × INT8 → INT32 等格式。

Ampere 架构 进一步扩展了数据类型：

- **INT1**：适用于二值神经网络（Binary Neural Network），输入为 1-bit，输出至少为 4-bit 或 8-bit（因为累加结果无法用 1-bit 表

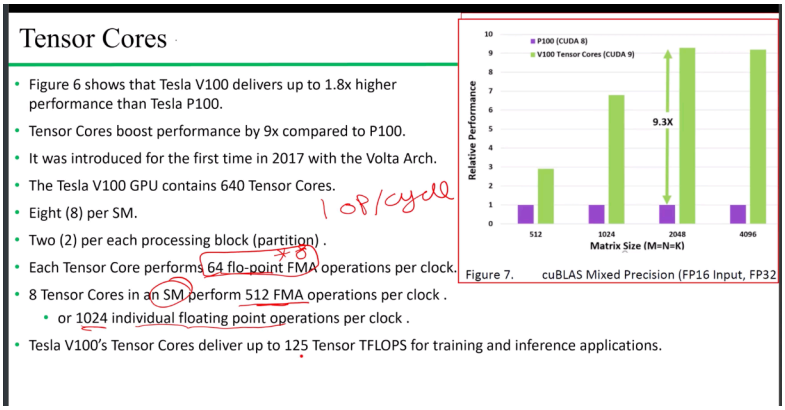


图 49.3:

示)。

- **BF16 与 TF32**：新增 BFloat16 和 TensorFloat-32 格式，兼顾训练稳定性与计算速度。
- **FP64**：支持双精度张量运算，拓展了科学计算应用场景。

以 Ampere (A100) 为例，使用普通核心时性能约为 19–20 TFLOPS，而启用张量核心后，根据数据类型不同，性能可达 156 至 312 TFLOPS。

49.5 软件生态与编程接口

硬件的革新推动了软件栈的全面更新。NVIDIA 对以下核心库进行了升级以支持张量核心：

- **cuBLAS**：基础线性代数库
- **cuDNN**：深度学习原语库
- **cuSPARSE**：稀疏矩阵运算库
- **cuFFT**：快速傅里叶变换库

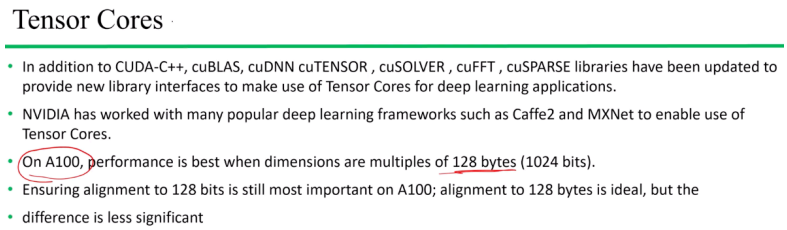


图 49.4:

- **NCCL**：多 GPU 通信库

主流深度学习框架（如 PyTorch、TensorFlow、Caffe2、MXNet）均已在后端集成张量核心支持。用户在 GPU 上执行矩阵乘法时，框架会自动调用张量核心，无需手动干预。

49.6 课程预告：实测与编程实践

在下一章中，我们将从理论转向实践：

1. **性能实测**：对比相同尺寸矩阵乘法在普通核心与张量核心上的实际执行时间，验证理论性能差距。
2. **编程方法**：介绍两种使用张量核心的编程方式：
   - **WMMA API**（Warp Matrix Multiply-Accumulate）：底层 Warp 级矩阵运算接口。
   - **cuBLAS / cuDNN**：高层库函数接口。
3. **对齐要求**：讨论矩阵维度对齐（如 128 字节倍数）对张量核心性能的影响。

50 CUDA GPU 并行编程：张量核心实战评估

50.1 引言

在本章中，我们将通过实际基准测试展示使用张量核心（Tensor Cores）相比普通 CUDA 核心所能获得的性能提升。为了聚焦核心对比，本次演示仅选取两个代表性应用程序进行测试，而非此前预告的四个。后续章节将详细讲解其他应用场景及张量核心的多种编程方法。本章旨在直观呈现硬件加速带来的收益，暂不涉及应用本身的算法原理。

50.2 实验环境与配置

50.3 矩阵参数与精度

两个测试程序均执行 1024×1024 方阵乘法运算，且统一采用半精度浮点数（FP16）。两者唯一的区别在于计算单元的选择：朴素版本使用标准 CUDA 核心，而加速版本调用张量核心 API。

50.4 编译注意事项

在编译过程中需特别注意架构参数的指定。

50.5 朴素版本编译

朴素版本的编译命令如下：

```
nvcc -o naive naive.cu
```

该命令可正常生成可执行文件。

```
// naive.cu
#include <stdio.h>
```

```
#include <cuda.h>
#include <cuda_fp16.h>

#define N 1024

__global__ void matMulCUDA(half *A, half *B, half *C) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < N && col < N) {
        half sum = __float2half(0.0f);
        for (int k = 0; k < N; k++) {
            sum = __hadd(sum, __hmul(A[row * N + k], B[k * N +
                col]));
        }
        C[row * N + col] = sum;
    }
}

void randomMatrix(half *mat, int size) {
    for (int i = 0; i < size; i++) {
        mat[i] = __float2half(static_cast<float>(rand()) /
            RAND_MAX);
    }
}

int main() {
    size_t bytes = N * N * sizeof(half);

```

```
30      half *h_A, *h_B, *h_C;
31      half *d_A, *d_B, *d_C;
32
33      h_A = (half*)malloc(bytes);
34      h_B = (half*)malloc(bytes);
35      h_C = (half*)malloc(bytes);
36
37      randomMatrix(h_A, N * N);
38      randomMatrix(h_B, N * N);
39
40      cudaMalloc(&d_A, bytes);
41      cudaMalloc(&d_B, bytes);
42      cudaMalloc(&d_C, bytes);
43
44      cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
45      cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);
46
47      dim3 blockSize(16, 16);
48      dim3 gridSize(N / blockSize.x, N / blockSize.y);
49
50      matMulCUDA<<<gridSize, blockSize>>>(d_A, d_B, d_C);
51
52      cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost);
53
54      printf("Matrix multiplication completed using CUDA Cores (
            FP16).\n");
55
56      cudaFree(d_A);
57      cudaFree(d_B);
```

```
    cudaFree(d_C);
    free(h_A);
    free(h_B);
    free(h_C);

    return 0;
}
```

```

// tensor.cu
#include <stdio.h>
#include <cuda.h>
#include <cuda_fp16.h>
#include <mma.h>

#define N 2048
#define TILE_SIZE 16 // Using 16x16 matrix tiles

using namespace nvcuda;

__global__ void matMulTensorCores(half *A, half *B, half *C) {
    int warpM = (blockIdx.x * blockDim.x + threadIdx.x) /
       warpSize;
    int warpN = (blockIdx.y * blockDim.y + threadIdx.y);

    if (warpM < N / TILE_SIZE && warpN < N / TILE_SIZE) {
        wmma::fragment<wmma::matrix_a, TILE_SIZE, TILE_SIZE,
           TILE_SIZE, half, wmma::row_major> fragA;
```

```
19          wmma::fragment<wmma::matrix_b, TILE_SIZE, TILE_SIZE,
               TILE_SIZE, half, wmma::col_major> fragB;
20          wmma::fragment<wmma::accumulator, TILE_SIZE, TILE_SIZE,
               TILE_SIZE, half> fragC;
21
22          wmma::fill_fragment(fragC, __float2half(0.0f));
23
24          for (int k = 0; k < N / TILE_SIZE; k++) {
25              wmma::load_matrix_sync(fragA, A + warpM * TILE_SIZE
                   * N + k * TILE_SIZE, N);
26              wmma::load_matrix_sync(fragB, B + k * TILE_SIZE * N
                   + warpN * TILE_SIZE, N);
27              wmma::mma_sync(fragC, fragA, fragB, fragC);
28          }
29
30          wmma::store_matrix_sync(C + warpM * TILE_SIZE * N +
               warpN * TILE_SIZE, fragC, N, wmma::mem_row_major);
31      }
32  }
33
34  void randomMatrix(half *mat, int size) {
35      for (int i = 0; i < size; i++) {
36          mat[i] = __float2half(static_cast<float>(rand()) /
               RAND_MAX);
37      }
38  }
39
40  int main() {
41      size_t bytes = N * N * sizeof(half);
```

```
42
43      half *h_A, *h_B, *h_C;
44      half *d_A, *d_B, *d_C;
45
46      h_A = (half*)malloc(bytes);
47      h_B = (half*)malloc(bytes);
48      h_C = (half*)malloc(bytes);
49
50      randomMatrix(h_A, N * N);
51      randomMatrix(h_B, N * N);
52
53      cudaMalloc(&d_A, bytes);
54      cudaMalloc(&d_B, bytes);
55      cudaMalloc(&d_C, bytes);
56
57      cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
58      cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);
59
60      dim3 blockSize(32, 32);
61      dim3 gridSize(N / TILE_SIZE, N / TILE_SIZE);
62
63      matMulTensorCores<<<gridSize, blockSize>>>(d_A, d_B, d_C);
64
65      cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost);
66
67      printf("Matrix multiplication completed using Tensor Cores
            (FP16).\n");
68
69      cudaFree(d_A);
```

```
    cudaFree(d_B);
    cudaFree(d_C);
    free(h_A);
    free(h_B);
    free(h_C);

    return 0;
}
```

50.6 张量核心版本编译陷阱

若直接使用默认参数编译张量核心版本：

```
nvcc -o tensor tensor.cu
```

编译器将报错，提示“名称后跟‘::’必须是类或命名空间”。这是因为未指定目标架构时，`nvcc` 可能默认回退至较旧的计算能力（如 Compute Capability 5.x），而该架构不支持张量核心相关指令。

50.7 解决方案：指定计算能力

必须显式添加 `-arch` 参数以匹配目标 GPU 架构。例如，针对 Ampere 架构（RTX30 系列/A100 等），应使用 `sm_80`：

```
nvcc -o tensor tensor.cu -arch=sm_80
```

- **背景知识**：张量核心于 2017 年随 Volta 架构（Compute Capability 7.x）首次引入。若使用 CC 5.x 编译，等同于在张量核心诞生前的架构上构建代码，必然导致符号缺失。

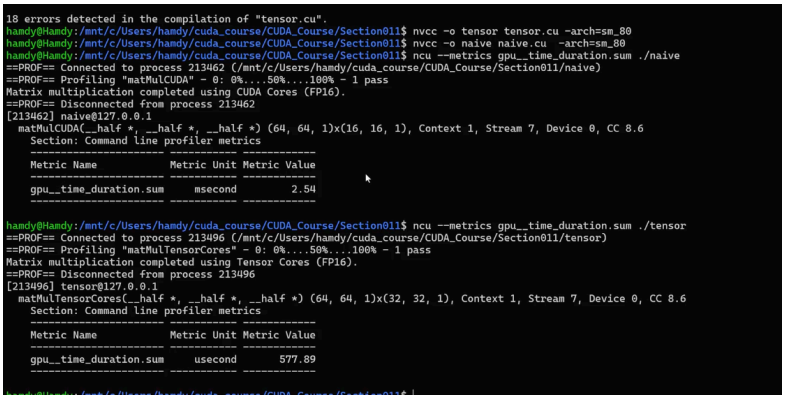


图 50.1:

- **最佳实践**：建议在所有涉及新特性的编译中始终明确指定 `-arch` 参数，确保编译器启用正确的指令集与内建函数。

50.8 性能评估结果

使用 NVIDIA Nsight Compute (`ncu`) 工具测量内核执行时间，对比结果如下：

50.9 基准测试 (1024×1024)

- **朴素版本**：2.454 ms
- **张量核心版本**：577 μs (0.577 ms)
- **加速比**：$\frac{2454}{577} \approx 4.25\times$

注：本测试基于消费级 RTX 3060 显卡。若使用 HPC 专用卡（如 A100），预期加速比可达 7–9 倍。

50.10 规模扩展测试 (2048×2048)

增大矩阵尺寸以观察性能差异的变化趋势：

- **朴素版本**：20.17 ms
- **张量核心版本**：2.73 ms
- **加速比**：$\frac{20.17}{2.73} \approx 7.39\times$

结果表明，随着问题规模增大，张量核心的吞吐优势更加显著。

50.11 讨论与总结

50.12 无优化下的纯硬件收益

上述性能提升完全源于硬件差异，未对代码进行任何额外优化（如内存访问模式调整或共享内存分块优化）。两个版本均已使用共享内存，但仅此而已。若在张量核心版本基础上进一步实施软件层面优化，性能仍有巨大提升空间。

50.13 后续内容预告

下一章将系统介绍张量核心的编程工具链。不同于多数教程仅关注 cuBLAS/cuDNN 等高层库，本课程将深入讲解底层 WMMA (Warp Matrix Multiply-Accumulate)API，这是实现精细化张量核心控制的关键技术，也是本课程的独家重点内容。

51 CUDA GPU 并行编程：使用 WMMA API 编程张量核心

51.1 引言

本章将深入讲解专用于张量核心（Tensor Cores）编程的 **WMMA API**（Warp Matrix Multiply-Accumulate）。在正式介绍 API 之前，我们首先回顾 GPU 执行矩阵乘法的四个通用步骤，以便理解 WMMA 如何映射到这些基础操作中。

51.2 矩阵乘法的核心步骤回顾

无论是使用普通 CUDA 核心还是张量核心，矩阵乘法内核均遵循以下四个阶段：

1. **分块 (Tiling/Partitioning)**：根据矩阵维度 N 和块大小（Block Size）计算网格与线程块的划分。

2. **加载 (Loading)**：将子矩阵 A 和 B 从全局内存加载至共享内存（Shared Memory），以减少访存延迟。注意：输出矩阵 C 通常不需要预加载，除非进行累加操作。

3. **计算 (Execution)**：执行乘加运算。在传统实现中，结果暂存于寄存器变量（如 `sum`）中；在 WMMA 中，则存储于寄存器碎片（Register Fragments）中。

4. **存储 (Storing)**：将寄存器中的最终结果写回全局内存中的矩阵 C。

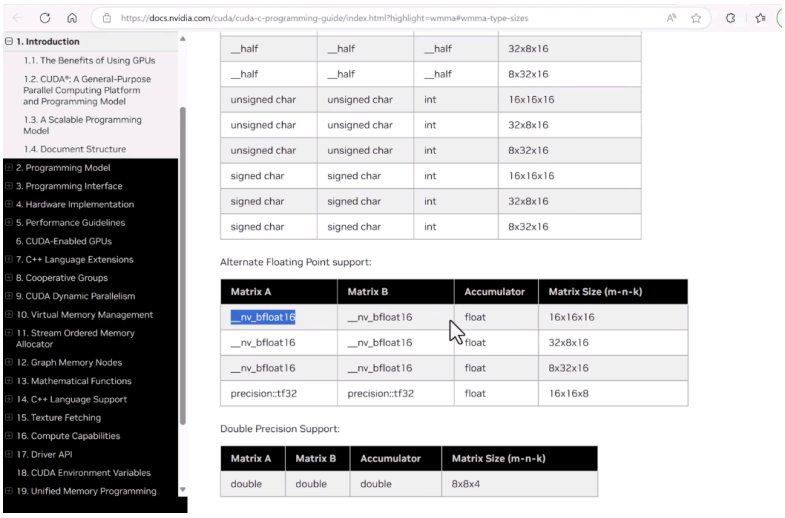


图 51.1:

51.3 WMMA API 核心概念

51.4 定义与适用架构

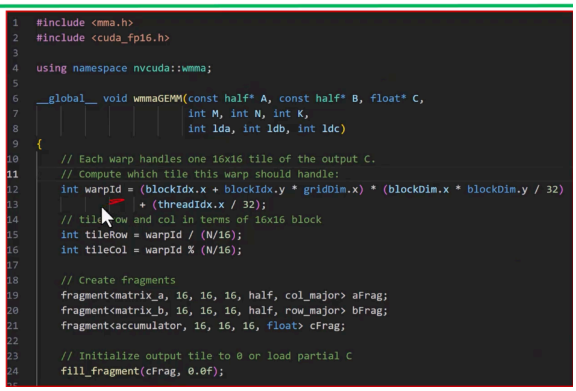
WMMA 全称为 **Warp Synchronous Matrix Multiply-Accumulate**。与普通 CUDA 核心以线程块（Thread Block）为调度单位不同，WMMA 是以 **Warp（线程束）**为基本执行单元的矩阵运算接口。

- **硬件要求**：仅适用于计算能力（Compute Capability）≥ 7.0 的设备。
- **架构背景**：张量核心首次引入于 2017 年的 Volta 架构（CC 7.x）。后续的 Turing、Ampere、Hopper 等架构均支持并扩展了该特性。Volta 之前的架构因缺乏物理张量核心，无法使用此 API。

51.5 碎片（Fragments）与数据布局

WMMA 引入了“碎片”（Fragment）的概念，即分布在 Warp 内 32 个线程寄存器中的矩阵子块。

Programming Tensor Cores:



```cpp
#include <mma.h>
#include <cuda_fp16.h>

using namespace nvcuda::wmma;

__global__ void wmmaGEMM(const half* A, const half* B, float* C,
                         int M, int N, int K,
                         int lda, int ldb, int ldc)
{
    // Each warp handles one 16x16 tile of the output C.
    // Compute which tile this warp should handle:
    int warpId = (blockIdx.x + blockIdx.y * gridDim.x) * (blockDim.x * blockDim.y / 32)
               + (threadIdx.x / 32);
    // tileRow and col in terms of 16x16 block
    int tileRow = warpId / (N/16);
    int tileCol = warpId % (N/16);

    // Create fragments
    fragment<matrix_a, 16, 16, 16, half, col_major> aFrag;
    fragment<matrix_b, 16, 16, 16, half, row_major> bFrag;
    fragment<accumulator, 16, 16, 16, float> cFrag;

    // Initialize output tile to 0 or load partial C
    fill_fragment(cFrag, 0.0f);
```

图 51.2:

51.6 支持的维度与类型

WMMA 不支持任意尺寸的分块，必须使用预定义的形状。常见组合包括：

- $16 \times 16 \times 16$
- $32 \times 8 \times 16$ / $8 \times 32 \times 16$
- 数据类型组合需严格遵循官方文档表格（例如：输入 FP16 + 累加器 FP32，或输入 FP16 + 累加器 FP16）。
- 支持的新类型包括 BF16、TF32、INT8 等，其中 TF32 具有与 FP32 相同的动态范围但精度较低（10 位尾数）。

51.7 内存布局：行主序 vs 列主序

- **Row-Major**：按行连续存储。
- **Col-Major**：按列连续存储。

优化提示：为最大化内存合并访问效率，通常建议矩阵 A 使用 Col-Major（便于按列加载），矩阵 B 使用 Row-Major（便于按行加载），具体取决于内核的遍历策略。错误的布局会导致非合并访问，显著增加缓存未命中率和内存请求次数。

51.8 WMMA 关键指令

```
// 1. 声明碎片
wmma::fragment<wmma::matrix_a, M, N, K, half, wmma::col_major>
   frag_a;
wmma::fragment<wmma::matrix_b, M, N, K, half, wmma::row_major>
   frag_b;
wmma::fragment<wmma::accumulator, M, N, K, float> frag_c;

// 2. 初始化累加器
wmma::fill_fragment(frag_c, 0.0f);

// 3. 从全局/共享内存加载到寄存器碎片
wmma::load_matrix_sync(frag_a, ptr_a, lda);
wmma::load_matrix_sync(frag_b, ptr_b, ldb);

// 4. 执行矩阵乘加: D = A * B + Cwmma::mma_sync(frag_c, frag_a,
    frag_b, frag_c);

// 5. 将结果存回内存
wmma::store_matrix_sync(ptr_c, frag_c, ldc, wmma::mem_row_major
   );
```

51.9 Warp ID 的计算方法

CUDA 提供了 `threadIdx` 和 `blockIdx`，但**没有**预定义的 `warpId` 变量。在使用 WMMA 时，必须手动计算当前线程所属的 Warp ID。

51.10 一维网格/线程块

若线程块为一维组织，Warp ID 计算公式为：

$$\text{warpId} = \frac{\text{threadIdx.x}}{32} + \left(\frac{\text{blockDim.x}}{32}\right) \times \text{blockIdx.x} \tag{5}$$

解析：

- threadIdx.x/32：计算块内 Warp 偏移（整数除法）。
- (blockDim.x/32) × blockIdx.x：计算当前块之前所有块包含的 Warp 总数。

51.11 二维网格扩展

对于二维网格，需先将二维 blockIdx 展平为一维线性索引：

$$\text{linearBlockId} = \text{blockIdx.y} \times \text{gridDim.x} + \text{blockIdx.x} \tag{6}$$

随后代入上述公式即可。注意：Warp 的划分始终基于线程的线性索引，与网格维度无关。

51.12 WMMA 矩阵乘法执行流程

结合 Warp ID 计算，完整的 WMMA GEMM 内核逻辑如下：

1. **计算 Warp 坐标**：根据 `warpId` 确定当前 Warp 负责处理的全局矩阵分块位置（Row, Col）。
2. **初始化**：使用 `fill_fragment` 将累加器碎片置零。

3. **K 维循环**：沿 K 维度以步长 16（或其他碎片 K 维度）遍历：
    - 计算子矩阵 A 和 B 在当前 K 步的指针偏移。偏移量依赖于 `Leading Dimension (LDA/LDB)` 而非简单的矩阵宽度 N。
    - 使用 `load_matrix_sync` 加载数据到寄存器碎片。
    - 调用 `mma_sync` 执行乘加运算。
4. **写回结果**：循环结束后，使用 `store_matrix_sync` 将累加器碎片写入全局内存矩阵 C。

51.13 重要提示

- **Leading Dimension (LD)**：在计算内存偏移时，务必使用 LD 参数（通常为矩阵的实际分配行数/列数），而非逻辑维度。这是避免内存越界和数据错位的关键。
- **文档查阅**：WMMA 对数据类型、布局和尺寸的组合有严格限制。开发时请务必查阅 CUDA Programming Guide 中的"Warp Matrix Functions" 章节，确认所用组合受硬件支持。
- **性能调优**：WMMA 仅是基础原语。实际高性能实现还需结合共享内存分块、向量化加载、流水线重叠等优化技术。

52 CUDA GPU 并行编程：WMMA API 实战案例

52.1 概述

本章节详细讲解如何使用 **WMMA (Warp Matrix Multiply-Accumulate) API** 对 NVIDIA GPU 的张量核心（Tensor Cores）进行编程。我们将通过

一个 1024×1024 方阵乘法的实际案例，演示从核函数编写、编译配置到性能对比的完整流程。

52.2 核函数实现细节

```
#include <cstdio>
#include <cstdlib>
#include <cuda_fp16.h>
#include <mma.h>
#include <iostream>

using namespace nvcuda::wmma;

// Define matrix dimension N (must be multiple of 16)
#ifndef N
#define N 1024
#endif

// We will use 16×16 WMMA tiles
#define TILE_SIZE 16

// Kernel: half-precision inputs, float accumulation/output
//  1D block of 128 threads => 4 warps per block
//  Each warp computes one 16×16 tile in the output
__global__ void matMulTensorCoresFp32(const half* A, const half
    * B, float* C, int n)
{
    // warpsPerBlock = 4 if blockDim.x=128
    int warpsPerBlock = blockDim.x / 32;
```

```
24      // global warp ID
25      int warpId = blockIdx.x * warpsPerBlock + (threadIdx.x /
            32);
26
27      int tilesPerDim = n / TILE_SIZE;         // e.g. N=256 =>
            16
28      int tileCount = tilesPerDim * tilesPerDim; // e.g.
            16*16=256
29
30      if (warpId >= tileCount) return;
31
32      // Identify which 16×16 tile we're responsible for
33      int tileRow = warpId / tilesPerDim;
34      int tileCol = warpId % tilesPerDim;
35
36      // Create WMMA fragments
37      // matrix_a uses half in col_major
38      fragment<matrix_a, TILE_SIZE, TILE_SIZE, TILE_SIZE, half,
            col_major> aFrag;
39      // matrix_b uses half in row_major
40      fragment<matrix_b, TILE_SIZE, TILE_SIZE, TILE_SIZE, half,
            row_major> bFrag;
41      // accumulator uses float
42      fragment<accumulator, TILE_SIZE, TILE_SIZE, TILE_SIZE,
            float> cFrag;
43
44      // Initialize accumulator to 0.f
45      fill_fragment(cFrag, 0.0f);
46
```

```
47      // Loop over the K dimension in steps of 16
48      for (int k0 = 0; k0 < n; k0 += TILE_SIZE)
49      {
50          const half* Ablock = A + (tileRow * TILE_SIZE * n) + k0;
51          const half* Bblock = B + (k0 * n) + (tileCol * TILE_SIZE
                );
52
53          load_matrix_sync(aFrag, Ablock, n);
54          load_matrix_sync(bFrag, Bblock, n);
55
56          // Multiply-accumulate
57          mma_sync(cFrag, aFrag, bFrag, cFrag);
58      }
59
60      // Write the 16×16 tile of results to the float matrix C
61      float* Csub = C + (tileRow * TILE_SIZE * n) + (tileCol *
            TILE_SIZE);
62      store_matrix_sync(Csub, cFrag, n, mem_row_major);
63  }
64
65
66  // Simple helper: fill a half-precision matrix with random
        values [0..1]
67  void randomMatrix(half *mat, int size) {
68      for (int i = 0; i < size; i++) {
69          float r = static_cast<float>(rand()) / RAND_MAX;
70          mat[i] = __float2half(r);
71      }
72  }
```

```
73
74  // CPU reference multiply, but store result in float
75  //  C[i,j] = sum_{k} ( float(A[i,k]) * float(B[k,j]) )
76  void cpuMultiplyHalfFloat(const half* A, const half* B, float*
        C, int n) {
77      for (int i = 0; i < n; i++) {
78          for (int j = 0; j < n; j++) {
79              float sum = 0.0f;
80              for (int k = 0; k < n; k++) {
81                  float aVal = __half2float(A[i * n + k]);
82                  float bVal = __half2float(B[k * n + j]);
83                  sum += aVal * bVal;
84              }
85              C[i * n + j] = sum;
86          }
87      }
88  }
89
90
91
92  int main()
93  {
94      // N must be multiple of 16
95      if (N % TILE_SIZE != 0) {
96          std::cerr << "Error: N must be a multiple of 16.\n";
97          return 1;
98      }
99
100     srand(0);
```

```
101
102     // Host memory: half inputs, float output
103     size_t totalElements = N * N;
104     size_t bytesAorB  = totalElements * sizeof(half);
105     size_t bytesC     = totalElements * sizeof(float);
106
107     half* h_A = (half*) malloc(bytesAorB);
108     half* h_B = (half*) malloc(bytesAorB);
109     float* h_C = (float*) malloc(bytesC); // GPU result
110     float* h_Ccpu = (float*) malloc(bytesC); // CPU reference
111
112     // Random initialization of A, B in half
113     randomMatrix(h_A, N * N);
114     randomMatrix(h_B, N * N);
115
116     // Allocate device memory
117     half* d_A = nullptr;
118     half* d_B = nullptr;
119     float* d_C = nullptr;
120     cudaMalloc(&d_A, bytesAorB);
121     cudaMalloc(&d_B, bytesAorB);
122     cudaMalloc(&d_C, bytesC);
123
124     // Copy A, B to device
125     cudaMemcpy(d_A, h_A, bytesAorB, cudaMemcpyHostToDevice);
126     cudaMemcpy(d_B, h_B, bytesAorB, cudaMemcpyHostToDevice);
127     cudaMemset(d_C, 0, bytesC);
128
129     // Compute grid/block configuration
```

```
130     // - 128 threads => 4 warps per block
131     // - total 16×16 tiles = (N/16)*(N/16)
132     // - each block handles 4 tiles
133     int tilesPerDim = N / TILE_SIZE;
134     int tileCount  = tilesPerDim * tilesPerDim;
135     int warpsPerBlk = 128 / 32; // 4
136     int blocksNeeded = (tileCount + warpsPerBlk - 1) /
            warpsPerBlk;
137
138     dim3 blockSize(128);
139     dim3 gridSize(blocksNeeded);
140
141     // Launch kernel
142     matMulTensorCoresFp32<<<gridSize, blockSize>>>(d_A, d_B,
            d_C, N);
143     cudaDeviceSynchronize();
144
145     // Copy result back to host
146     cudaMemcpy(h_C, d_C, bytesC, cudaMemcpyDeviceToHost);
147
148     // CPU reference multiply (in float)
149     cpuMultiplyHalfFloat(h_A, h_B, h_Ccpu, N);
150
151     // Compare first 10 elements
152     std::cout << "Matrix multiplication (FP16 inputs -> FP32
            output) completed.\n";
153     std::cout << "Compare first 10 elements (CPU vs. GPU):\n";
154     for (int i = 0; i < 10; i++) {
155         float cpuVal = h_Ccpu[i];
```

```
156         float gpuVal = h_C[i];
157         std::cout << "C[" << i << "]: CPU=" << cpuVal
158                   << ", GPU=" << gpuVal << std::endl;
159     }
160
161     // Cleanup
162     cudaFree(d_A);
163     cudaFree(d_B);
164     cudaFree(d_C);
165     free(h_A);
166     free(h_B);
167     free(h_C);
168     free(h_Ccpu);
169
170     return 0;
171 }
```

52.3 基本参数与 Warp 划分

在本案例中，矩阵 A、B、C 的尺寸均为 $N = 1024$，分块尺寸（Tile Size）设为 16。核函数定义为 `matrixMultiplyFP32`，输出为单精度浮点数（FP32），输入为半精度浮点数（FP16）。

Warp ID 的获取与大矩阵的分块映射逻辑如下：

- 将矩阵维度 N 除以 Tile Size 得到每个维度的分块数。
- 确保参与计算的 Warp 总数等于 Tile 总数。若不相等，需提前返回以避免越界访问或段错误（Segmentation Fault）。
- Fragment 初始化：将累加器 Fragment C 填充为零。

52.4 数据加载与计算流程

1. **初始化**：使用 `randomMatrix` 函数生成随机矩阵 A 和 B。
2. **内存分配**：在主机端（Host）和设备端（Device）分别分配指针 `hA`，`hB`，`hC` 与 `dA`，`dB`，`dC`。
3. **数据传输**：使用 `cudaMemcpy` 将数据从主机拷贝至设备；使用 `cudaMemset` 将设备端矩阵 C 初始化为零。
4. **执行计算**：加载 Fragment A 和 B，调用 WMMA API 执行矩阵乘法，并将结果存储回全局内存中的矩阵 C。

52.5 主机端验证与同步

- **尺寸校验**：必须确保 N 是 Tile Size 的整数倍（例如 Tile=16 时，N 不能为 15）。
- **同步机制**：调用 `cudaDeviceSynchronize()` 确保 CPU 等待 GPU 核函数执行完毕后再进行后续操作。
- **结果验证**：实现一个 CPU 版本的朴素矩阵乘法，对比 GPU 输出的前 10 个元素以验证正确性。

52.6 编译配置的关键注意事项

52.7 架构指定（至关重要）

WMMA API 专为张量核心设计，而张量核心首次引入于 **Volta** 架构。若未指定架构，NVCC 默认使用较旧的计算能力（如 SM_52），导致编译报错（提示 `nvcuda::wmma` 未定义）。

正确编译命令示例（以 RTX 3060 Ti / Ampere 架构为例）：

```
nvcc -o tensor2 tensor2.cu -arch=sm_80
```

注意: 无论是否使用张量核心, 都应始终显式指定目标架构 (`-arch=sm_XX`), 以充分利用特定硬件的特性与优化。

52.8 性能分析与对比

52.9 Profiling 工具使用

使用 `ncu` (Nsight Compute) 测量核函数执行时间:

```
ncu --metrics sm__cycles_elapsed.avg ./tensor2
```

52.10 性能对比结果

针对 1024×1024 矩阵乘法，实测数据如下:

实现版本	执行时间	相对加速比
朴素版本（Naive）	2.5 ms (2500 μs)	1.0x
WMMA API（张量核心）	388 μs	$\approx$ 6.5x

表 4: WMMA API 与朴素矩阵乘法性能对比

结论: 仅通过使用 WMMA API 调用张量核心，相较于无优化的朴素版本，性能提升约 **6.5 倍**（节省约 85% 的执行时间）。建议进一步将 WMMA 版本与共享内存优化版本进行对比，以全面评估不同优化策略的收益。

52.11 总结与展望

本章完整展示了 WMMA API 的基本用法及编译要求。张量核心编程不仅适用于矩阵乘法，还可扩展至 cuBLAS、cuDNN 等库的底层加速。掌

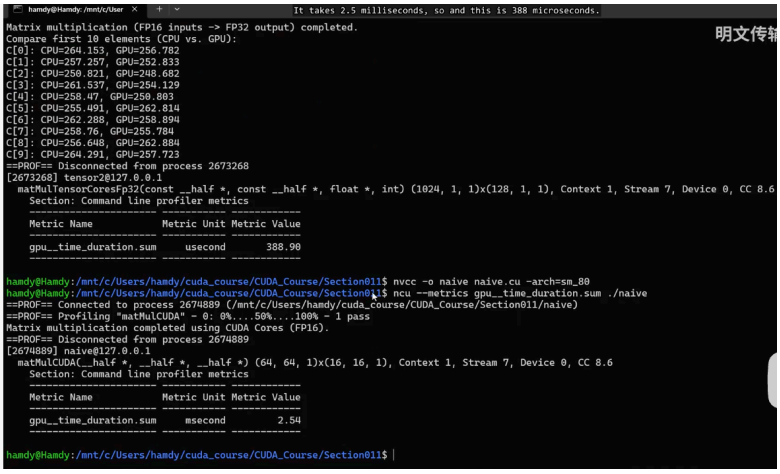


图 52.1:

握架构感知的编译习惯与性能度量方法，是进行高效 GPU 并行编程的基础。

53 CUDA GPU 并行编程：使用 cuBLAS 库调用张量核心

53.1 概述：cuBLAS 与 WMMA API 的区别

本章主要讨论 **cuBLAS** 库及其在张量核心（Tensor Core）矩阵乘法编程中的应用。cuBLAS 是一个高度优化的线性代数库，其功能与前两章中介绍的 WMMA API 类似，但二者存在本质区别：

- **WMMA API**：需要开发者手动编写 GPU 核函数，自行管理数据从全局内存到寄存器的加载、MMA Sync 指令调用以及结果存储等底层细节，具有高度可定制性。
- **cuBLAS**：是**不可定制**的封装库。开发者无需编写矩阵乘法的 GPU 核函数，只需配置规格参数并调用库函数即可。NVIDIA 已在库内部

实现了针对张量核心的深度优化。

53.2 环境配置与安装

在使用 cuBLAS 之前，需确保正确安装并链接库文件。

53.3 头文件引用

除了常规的 CUDA Runtime 头文件外，还需包含 cuBLAS v2 头文件以支持半精度与张量核心运算：

```
1 #include <cublas_v2.h>
```

53.4 库的安装

cuBLAS 通常随 CUDA Toolkit 一同发布。若需单独安装或更新，请前往 NVIDIA 官网下载对应 CUDA 版本的 cuBLAS 库。在 Linux 环境下，需根据系统架构和 CUDA 版本选择正确的安装包。

53.5 代码实现详解

53.6 基本流程

与自定义 Kernel 不同，使用 cuBLAS 的主函数流程如下：

1. 分配主机端与设备端内存（`cudaMalloc`）。
2. 初始化矩阵 A、B 并将数据传输至设备。
3. **创建 cuBLAS 句柄（Handle）**：这是 cuBLAS 的核心概念，类似于上下文管理器，所有库函数调用均需通过此句柄进行。

```
cublasHandle_t handle;
cublasCreate(&handle);
```

4. 设置标量参数 α 和 β。

5. 调用矩阵乘法函数。

6. 销毁句柄并释放资源。

53.7 GEMM 公式

cuBLAS 执行的通用矩阵乘法（GEMM）公式为:

$$C = \alpha \cdot A \times B + \beta \cdot C \tag{7}$$

在本示例中，为执行纯粹的矩阵乘法 $C = A \times B$，我们将 α 设为 1，β 设为 0。

53.8 函数调用解析：cublasGemmEx

为了利用张量核心并混合精度计算，我们使用 `cublasGemmEx` 而非传统的 `cublasSgemm`。

```
#include <iostream>
#include <cuda.h>
#include <cublas_v2.h>
#include <cuda_fp16.h>
#include <cstdlib>

#define N 1024

```

```
9  void cpu_matrix_multiply(half* A, half* B, float* C) {
10     for (int i = 0; i < N; i++) {
11         for (int j = 0; j < N; j++) {
12             float sum = 0.0f;
13             for (int k = 0; k < N; k++) {
14                 sum += __half2float(A[i * N + k]) * __half2float(
                       B[k * N + j]);
15             }
16             C[i * N + j] = sum;
17         }
18     }
19 }
20
21 int main() {
22     cublasHandle_t handle;
23     cublasCreate(&handle);
24
25     half* h_A = new half[N * N];
26     half* h_B = new half[N * N];
27     float* h_C = new float[N * N];
28     float* h_C_cpu = new float[N * N];
29
30     for (int i = 0; i < N * N; i++) {
31         h_A[i] = __float2half(static_cast<float>(rand()) /
                   RAND_MAX);
32         h_B[i] = __float2half(static_cast<float>(rand()) /
                   RAND_MAX);
33     }
34
```

```
35      half *d_A, *d_B;
36      float *d_C;
37      cudaMalloc(&d_A, N * N * sizeof(half));
38      cudaMalloc(&d_B, N * N * sizeof(half));
39      cudaMalloc(&d_C, N * N * sizeof(float));
40
41      cudaMemcpy(d_A, h_A, N * N * sizeof(half),
           cudaMemcpyHostToDevice);
42      cudaMemcpy(d_B, h_B, N * N * sizeof(half),
           cudaMemcpyHostToDevice);
43
44      float alpha = 1.0f, beta = 0.0f;
45      cublasGemmEx(handle, CUBLAS_OP_N, CUBLAS_OP_N,
46                   N, N, N,
47                   &alpha,
48                   d_A, CUDA_R_16F, N,
49                   d_B, CUDA_R_16F, N,
50                   &beta,
51                   d_C, CUDA_R_32F, N,
52                   CUBLAS_COMPUTE_32F,
                        CUBLAS_GEMM_DEFAULT_TENSOR_OP);
53
54      cudaMemcpy(h_C, d_C, N * N * sizeof(float),
           cudaMemcpyDeviceToHost);
55
56      cpu_matrix_multiply(h_A, h_B, h_C_cpu);
57
58      std::cout << "First 10 elements of GPU result:" << std::
           endl;
```

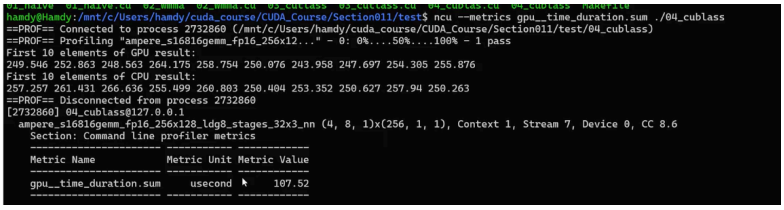


图 53.1:

```
59      for (int i = 0; i < 10; i++) std::cout << h_C[i] << " ";
60      std::cout << "\n";
61
62      std::cout << "First 10 elements of CPU result:" << std::
            endl;
63      for (int i = 0; i < 10; i++) std::cout << h_C_cpu[i] << "
            ";
64      std::cout << "\n";
65
66      delete[] h_A;
67      delete[] h_B;
68      delete[] h_C;
69      delete[] h_C_cpu;
70
71      cudaFree(d_A);
72      cudaFree(d_B);
73      cudaFree(d_C);
74      cublasDestroy(handle);
75      return 0;
76  }
```

关键参数说明:

- **转置选项**：CUBLAS_OP_N 表示不转置，CUBLAS_OP_T 表示转置。合理设置转置可改善内存访问模式（如将列主序转为行主序），减少 Cache Miss。
- **数据类型**：示例中输入 A、B 为半精度（CUDA_R_16F），输出 C 为单精度（CUDA_R_32F）。这种混合精度模式是张量核心的典型用法。
- **计算类型**：指定为 CUDA_R_32F，确保即使输入为 FP16，累加运算仍以 FP32 进行，保证数值稳定性。
- **算法选择**：CUBLAS_GEMM_DEFAULT_TENSOR_OP 明确指示库优先使用张量核心算法。

53.9 编译与性能对比

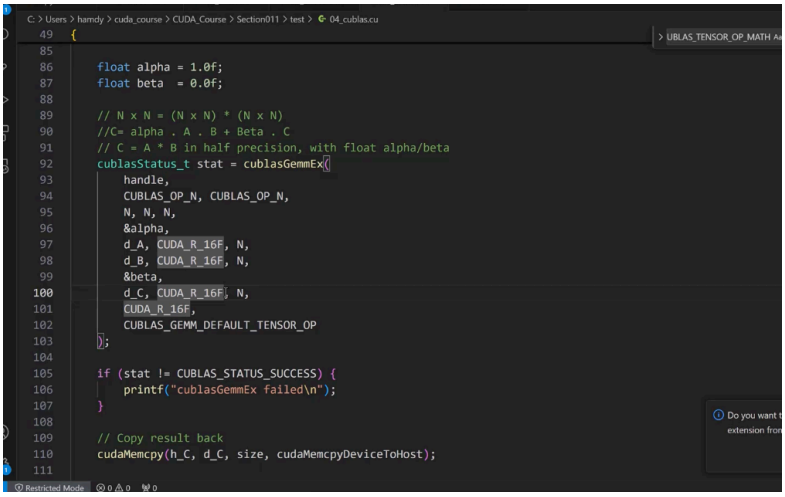
53.10 Makefile 编译配置

建议使用 Makefile 简化编译流程。编译 cuBLAS 程序时，除常规优化标志外，**必须链接 cuBLAS 库**：

```
# 通用编译标志
COMMON_FLAGS = -arch=sm_80 -O3

# cuBLAS 目标编译规则
cublas: cublas.cu
    nvcc $(COMMON_FLAGS) -o cublas cublas.cu -lcublas
```

注意：架构标志 -arch=sm_80 需根据实际 GPU 型号调整；-O3 用于开启最高级别编译器优化；-lcublas 是链接 cuBLAS 库的关键，缺失会导致链接错误。



```
C: > Users > hamdy > cuda_course > CUDA_Course > Section011 > test > 04_cublas.cu
49  {
85
86      float alpha = 1.0f;
87      float beta  = 0.0f;
88
89      // N x N = (N x N) * (N x N)
90      //C= alpha . A . B + Beta . C
91      // C = A * B in half precision, with float alpha/beta
92      cublasStatus_t stat = cublasGemmEx(
93          handle,
94          CUBLAS_OP_N, CUBLAS_OP_N,
95          N, N, N,
96          &alpha,
97          d_A, CUDA_R_16F, N,
98          d_B, CUDA_R_16F, N,
99          &beta,
100         d_C, CUDA_R_16F, N,
101         CUDA_R_16F,
102         CUBLAS_GEMM_DEFAULT_TENSOR_OP
103     );
104
105     if (stat != CUBLAS_STATUS_SUCCESS) {
106         printf("cublasGemmEx failed\n");
107     }
108
109     // Copy result back
110     cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
111
> UBLAS_TENSOR_OP_MATH
Do you want t
extension fron
Restricted Mode
```

图 53.2:

53.11 性能测试结果

在矩阵尺寸 $N = 1024$ 的测试中：

- **WMMA API 手写 Kernel**：约 388 μs
- **cuBLAS (Tensor Core)**：约 107 μs

结论：cuBLAS 提供了远超手写 WMMA 核函数的性能。这得益于 NVIDIA 对库内核函数的极致优化（包括寄存器分块、流水线调度等）。对于生产环境中的矩阵乘法需求，强烈建议优先使用 cuBLAS 而非自行实现。

53.12 学习建议

本章仅作为入门引导。cuBLAS 提供了丰富的 API 变体（如 `cublasSgemm` ,`cublasDgemm`, `cublasHgemm` 等），建议课后深入查阅 NVIDIA cuBLAS 官方文档，以掌握不同精度、布局及批处理操作的详细用法。同时，建议学习 Makefile 的基本语法，以提升 CUDA 项目的构建效率。